

—— // REWRITING THE DESKTOP // ——

Rewriting the Desktop in Rust

First Edition

THE COMPANION

the port · the core · the shell

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTAPP

Contents

Before You Begin	4
Chapter 1 — The Thesis: Engine Core, Thin Shell	6
Chapter 2 — The <code>*-core</code> Pattern	11
Chapter 3 — The Shared Shell: Tauri v2 and <code>zgui-core</code>	16
Chapter 4 — The Command Bridge and the IPC Contract	21
Chapter 5 — <code>zreq</code> : Rebuilding Postman	26
Chapter 6 — <code>zcite</code> : Rebuilding Zotero	32
Chapter 7 — <code>ztunnel</code> : Rebuilding Tunnelblick	37
Chapter 8 — <code>zgo</code> : Rebuilding Alfred	42
Chapter 9 — <code>zftp</code> : Rebuilding Cyberduck	47
Chapter 10 — <code>zcontainer</code> : Rebuilding Docker Desktop and Lens	52
Chapter 11 — The Document Suite: <code>zoffice</code> , <code>zemail</code> , <code>zpdf</code> , <code>zphoto</code>	58
Chapter 12 — <code>zgui-core</code> as a Design System	65
Chapter 13 — The Embed Matrix	70
Chapter 14 — The Embed Contract and Cross-App Composition	75
Chapter 15 — Doing It Solo	79
Chapter 16 — The Failure Contract: How Ten Engines Fail	83
Chapter 17 — Persistence: The <code>store</code> Module and the Data Directory	87
Chapter 18 — The Shared Substrate: Terminal, File Browser, and the Commands Every App Registers	91
Chapter 19 — Testing a Fleet Without a Window	94
Chapter 20 — Speaking Every Language: The Shared <code>i18n</code> Catalogue	97
Chapter 21 — The Security Boundary: What the WebView May Ask For	100
Chapter 22 — Concurrency: One Lock, Three Hosts, and the Long Operation	103
Chapter 23 — The Cost of JSON: Where the Uniform Boundary Gets Expensive	106
Chapter 24 — The Mountable Webui and the Two Substrates	109

Chapter 25 — Reading a Predecessor as a Specification	112
Chapter 26 — Bringing Up the Eleventh Engine	115
Chapter 27 — The Import Surface: Reading the Predecessor's Files	119
Chapter 28 — Finding Things: Retrieval Models Under One Highlight	122
Chapter 29 — Time, Undo, and the Verb <code>clear</code>	125
Chapter 30 — The Engine as Its Own Diagnostic	128
Chapter 31 — The Session Engine: An Unnamed Archetype	131
Chapter 32 — Feature Gates and the Dependency Ledger	134
Chapter 33 — The Palette as the Second Command Surface	137
Chapter 34 — The Process Model: One Instance, Two Windows, One Engine	140
Chapter 35 — Cold Start: The Cost of the First Frame	143
Chapter 36 — The Idle Fleet: What Runs When You Are Not Looking	146
Chapter 37 — Durability: The Save That Is Not Fenced	149
Chapter 38 — Synchronous by Construction: Where the Fleet's Async Actually Lives	152
Chapter 39 — Accessibility Below the Effects Layer	155
Chapter 40 — Distribution: Signing, Notarization, and the Update Path	158
Appendix A — The Engine Command Surfaces	161
Appendix B — The C-ABI Reference	164
Appendix C — The Shared Component Surface	166
Appendix D — Build, Packaging, and Versioning	168
Appendix E — Glossary	170
Appendix F — A Source Map	172
Appendix G — The <code>zpdf</code> Operation Catalogue	173
Appendix H — The <code>EngineEvent</code> Catalogue	174
Appendix I — Predecessors, Dependencies, and the Claim	175
Appendix J — The Feature-Gate Map	176
Appendix K — The Honesty Ledger	177
Appendix L — The Failure and Persistence Surfaces	178
Appendix M — The Shared Substrate Commands	180
Appendix N — The Localization Surface	181

Appendix O — The Boundary Contract in Full	182
Appendix P — The Import and Export Surface	184

Before You Begin

This is the book about a decision repeated a dozen times: to take a desktop program everyone already uses — Postman, Zotero, Tunnelblick, Alfred, Cyberduck, Docker Desktop, Microsoft Office, Adobe Acrobat, GIMP — and rebuild it from scratch in Rust, behind one cyberpunk HUD, on one shared architecture. It is not a manual for any single app; each app has its own. This is the book about the *pattern* — the “engine core, thin shell” thesis — and about what happens when you apply it to ten very different products and demand they all share the same UI toolkit, the same embeddable engines, and the same build.

The thesis fits in one sentence: **every app is a thin Tauri v2 shell over a pure-Rust `*-core` engine that exposes both a native Rust API and a C ABI, so the same engine that powers the standalone app can be embedded inside every other app.** `zpdf-core` is the PDF engine behind `zpdf` and the PDF viewer inside `zcite`, `zreq`, `zoffice`, `zemail`, and the rest. `zoffice-core` opens a spreadsheet in `zoffice` and previews an attachment in `zemail`. The apps are not ten islands; they are ten shells over a shared fleet of engines, wired together by one component library, `zgui-core`.

This book is honest about what is inherited and what is new. The *products* being replaced are prior art — Postman defined the API-client workflow, Zotero the reference manager, Cyberduck the transfer client — and this book credits them by name in every chapter, because the point of a port is fidelity to a workflow people already know. The *original* work is the architecture: doing all of it in one compiled-native stack, with engines that compose, from one author. Where an app merely matches its predecessor, the chapter says so; where the stack does something the predecessor cannot — `zcontainer` being the first compiled-native desktop GUI for both Docker and Kubernetes, against an all-Electron field — the chapter makes the claim and cites its basis.

Everything here is grounded in the repositories: the `*-core` crates, their C ABI headers, the Tauri command surface, the `zgui-core window.ZGui.*` API, and the `COMPONENTS.md` embed matrix that records which engine each app consumes. This is the work of MenkeTechnologies.

How to read this book

The book is built in three movements. The first four chapters are the architecture in the abstract: the engine-core/thin-shell thesis (Chapter 1), the `-core` contract (Chapter 2), the shared shell and design system at a glance (Chapter 3), and the command bridge that connects them (Chapter 4). If you read only those, you have the whole pattern. The middle chapters (5 through 11) are the pattern applied, one product at a time — `zreq`, `zcite`, `ztunnel`, `zgo`, `zftp`, `zcontainer`, and the four-engine document suite — and each can be read on its own if a particular app is why you opened the book. The last four chapters step back out: the design system as a system (Chapter 12), the embed matrix that records who consumes what (Chapter 13), the embed contract that makes live composition safe (Chapter 14), and the economics of doing all of it solo (Chapter 15). After that the book turns to the cross-cutting subsystems that no single app owns: how the engines fail (Chapter 16), how they persist (Chapter 17), the shared terminal and file browser every app registers (Chapter 18), how a fleet with no windows tests itself (Chapter 19), and the one localization catalogue they all draw from (Chapter 20). The final six chapters are the questions a senior reader asks after the architecture is clear — where the trust boundary actually is (Chapter 21), what one global mutex implies for long operations (Chapter 22), where the uniform JSON boundary gets expensive (Chapter 23), how an engine ships its own mountable view across two substrates (Chapter 24), how a shipping predecessor is turned into a specification you can be scored against (Chapter 25), and the order of operations for standing up an eleventh engine (Chapter 26). A final run of chapters reads the fleet sideways, looking for structure that no single app owns and that only shows up when all ten command surfaces are laid next to each other: why every engine reads its predecessor’s files and what the import surface measures (Chapter 27), the several different retrieval architectures hiding under one fuzzy

highlight (Chapter 28), what the fleet remembers, what it can take back, and what the verb `clear` is really for (Chapter 29), how you diagnose an engine you cannot look at (Chapter 30), the session-engine archetype two cores converged on without sharing a line (Chapter 31), the feature-gate tables read as a design surface rather than build configuration (Chapter 32), and the command palette as a second, hand-maintained projection of a vocabulary no engine can enumerate (Chapter 33). A closing run takes up the operational questions the architecture chapters set aside — how many processes and windows the design actually assumes (Chapter 34), what an app does between launch and first frame (Chapter 35), what keeps running when nobody is looking (Chapter 36), what a whole-file save promises if the power fails mid-write (Chapter 37), why every engine's domain surface is synchronous and where the async went instead (Chapter 38), what accessibility means below the effects toggles (Chapter 39), and what it costs to sign, notarize, and update a fleet rather than an app (Chapter 40). The appendices are reference: the command surfaces, the C-ABI, the component surface, the build and versioning, a source map, and catalogues of operations, events, and formats.

The honesty compact runs through every chapter and is worth stating once up front, because it is the difference between a book about a stack and a brochure for one. Every claim in this book is meant to be checkable against the repositories, and where the code is provisional the book says so rather than rounding up: `zpdf-core` ships no C ABI, and the book flags it every time PDF embedding comes up; the port coverage numbers (`zreq`'s 92.2%, `zcite`'s 95.0%) are conservative and named with their denominators; `zftp`'s network backends report `needs_transport` and `ztunnel`'s report `needs_privilege` rather than faking a live connection; the version skew is partial, not uniform; and where a crate's own header comment lags its code, the book notes the drift. Predecessors are credited by name as prior art for the workflow, dependencies are credited as prior art for the plumbing, and what is claimed as original — the composable engines, the document plane, the fleet one person can hold in one checkout — is claimed only after both credits are paid. Appendix K collects those honesty notes in one place.

Let's start.

Chapter 1 — The Thesis: Engine Core, Thin Shell

Before any one app, the shape they all share. A desktop app, in this stack, is a web front end rendered in a Tauri v2 WebView, talking over a command bridge to a Rust backend whose real work lives in a standalone `*-core` crate with no GUI dependencies at all. This chapter argues why that boundary is the whole game: it is what makes an engine testable without a window, embeddable inside a sibling app, and reusable behind a C ABI.

The shape is the same in every app. The engine is a plain crate; the shell is a Tauri command that calls it; the C ABI is a thin `extern "C"` skin over the same Rust API, so a sibling app — or any non-Rust host — embeds the identical engine:

```
// zpdf-core: one engine, two front doors.
// 1) the native Rust API the zpdf shell calls over the Tauri bridge
pub fn render_page(doc: &Document, page: usize, scale: f32) -> RgbaImage { /* ... */ }

// 2) the C ABI other apps embed to preview a PDF inside their own window
#[no_mangle]
pub extern "C" fn zpdf_render_page(doc: *const Document, page: usize, scale: f32) -> *mut
    ~ ZpdfBitmap {
    // same render_page(), wrapped for FFI
}
```

The three layers: WebView shell, Tauri command bridge, `*-core` engine

Every app in the fleet is three layers stacked in the same order. At the top is a WebView — a browser surface, but not a browser tab: it is a native window whose content happens to be HTML, CSS, and JavaScript, served either from a static `frontend/` directory or from assets the Rust side hands it. The middle layer is the Tauri command bridge: a small set of Rust functions annotated `#[tauri::command]` that the JavaScript calls by name through `window.__TAURI__`. The bottom layer is the engine — a `*-core` crate that knows how to parse a Postman collection, resolve a citation, spawn an OpenVPN process, or diff two Docker images, and knows *nothing* about windows, WebViews, or JavaScript. The whole book is about keeping that bottom layer clean.

What is striking, once you have read the repositories, is how little lives in the middle layer. In `zreq`, the entire API-client surface — collections, requests, environments, variables, auth, history, code generation, import and export — flows through exactly one Tauri command, `zrq_invoke(cmd, args)`. In `zcite` it is `zct_invoke`; in `ztunnel` it is `ztn_invoke`; in `zgo` it is `zgo_invoke`. The bridge does not explode the engine's hundred operations into a hundred Tauri commands. It multiplexes all of them through one string-dispatched entry point, so the command bridge is a dozen lines and the interesting code is all in the engine, where it can be tested, embedded, and versioned on its own.

The WebView layer is deliberately dumb about *domain* logic and deliberately smart about *chrome*. It does not know how an APA citation is formatted; it asks the engine and renders the string. But it owns the command palette, the tables, the tabs, the toasts, the settings panel, the file browser, and the embedded

terminal — and all of that comes from the shared `zgui-core` toolkit on `window.ZGui.*`, so the chrome is identical across apps and the app-specific JavaScript is just the glue between the engine's JSON and ZGui's widgets. Open `zreq/frontend/main.js` and you find it mounting `ZGui.appShell(...)`; open `zcite/frontend/zcite.js` and the header comment says outright that “ALL UI chrome in this view is built from `window.ZGui.*`.”

The reason to draw the boundary here, and not somewhere more convenient, is that each of the three layers has a different rate of change and a different reuse story. The engine changes when the domain changes — a new citation style, a new transfer protocol — and it is reused by every app that embeds it. The chrome changes when the design system changes, and it is reused by *all* apps at once, through one submodule. The bridge changes almost never, because it is ten lines of `invoke` plumbing. Putting domain logic in the bridge, or chrome logic in the engine, would couple things that change at different rates and destroy the reuse. The three-layer split is not architecture for its own sake; it is the shape that lets one author maintain ten products.

Why the engine must have zero GUI dependencies

The single hardest rule in the stack is the one that sounds like a nicety: a `*-core` crate may not depend on anything graphical. No Tauri (except behind an optional feature), no WebView, no windowing library, no rendering toolkit in its default build. `zreq-core`'s dependency list is `serde`, `serde_json`, `thiserror`, and `dirs`, with `reqwest` behind a `net` feature and `tauri` behind a `tauri` feature — and the README makes the discipline explicit as a “durable-deps rule.” `zcite-core`, `ztunnel-core`, `zgo-core`, and the document cores all read the same way: a tiny mandatory core, everything optional gated behind a Cargo feature.

The reason is not asceticism, it is testability. An engine with zero GUI dependencies can be exercised by a plain `cargo test` with no display server, no WebView, no event loop — you construct an `Engine`, call `invoke("collection.add", args)`, and assert on the returned JSON. `zcite`'s README claims 57 unit plus 13 integration tests; `zreq-core` ships a feature matrix of 83 enumerated Postman behaviours scored by tests. None of those tests open a window, because there is no window to open — the thing under test is a data structure and a function, and that is exactly what a test wants. Chapter 2 returns to this; the point here is that the zero-GUI rule is what *makes* the testing possible.

The second reason is embedding. The moment an engine drags in a WebView or a windowing dependency, it can no longer live *inside* another app's window, because you cannot nest one app's event loop inside another's. A pure engine, by contrast, is just a library: `zpdf-core` linked into `zcite` is the same crate linked into `zpdf`, running in the host's process, drawing into the host's surface through the host's `zgui-core` chrome. The zero-GUI rule is the precondition for the entire embed matrix of Chapter 13. An engine that needed its own window could power its own app and nothing else.

The third reason is the C ABI. A crate that is pure data-and-logic can expose a handful of `extern "C"` functions over JSON strings and be embedded by any host in any language. A crate entangled with a Rust GUI framework cannot — you cannot hand a Tauri `AppHandle` across an FFI boundary to a C++ host. Keeping the engine pure is what keeps the C ABI thin, which is what keeps the “embed anywhere” promise real rather than aspirational. The rule pays for itself three times over: once in tests, once in embeds, once in the C ABI.

There is a cost, and the book is honest about it: the engine cannot *do* the platform-specific, privileged, or streaming work itself. `ztunnel-core` can build an OpenVPN command line and parse the management socket, but bringing the tunnel up needs root and a privileged helper the *shell* installs. `zcontainer-core` can speak the Docker and Kubernetes APIs, but managing a `dockerd` guest VM is the app's job. The engine draws the line at “everything that is pure computation over data” and hands the rest to the shell across a narrow interface — an event sink, a platform trait, a subscription channel. That line is where the next section lives.

Native Rust API and C ABI: two front doors to one engine

Each engine has one body of logic and two front doors into it. The first door is the native Rust API: a `struct Engine` with a method `invoke(&mut self, cmd: &str, args: &Value) -> Result<Value>`

that dispatches a dotted command string — `"request.send"`, `"bib.bibliography"`, `"vpn.connect"` — to the right internal function and returns JSON. A Rust host (the app's own Tauri shell, or a sibling app that vendors the crate) constructs an `Engine`, holds it behind a `Mutex`, and calls `invoke` directly. This is the fast path, a normal Rust function call with no serialization across a process boundary.

The second door is the C ABI, and it is deliberately a thin skin over the first. Look in any core's `src/ffi.rs` and you find the same four-function shape: an `_init` that constructs a process-global `Engine` behind a `OnceLock<Mutex<Engine>>`, an `_invoke` that takes a command string and a JSON argument string and returns a heap-allocated JSON result string, a `_string_free` to release that string, and a `_set_event_callback` to register a C function pointer for asynchronous events. The prefixes are per-engine — `zrq_` for `zreq-core`, `zct_` for `zcite-core`, `ztn_` for `ztunnel-core`, `zgo_` for `zgo-core`, `zml_` for `zemail-core`, `zftp_` for `zftp-core`, `zoffice_`, `zphoto_`, `zcontainer_` — but the contract is identical, because they are all the same pattern.

The C header is hand-written and lives in the crate's `include/` directory, not generated by `cbindgen`; several cores ship a C++ RAII facade alongside it (for example `zreq_core.hpp` exposing `zreq::Core::invoke("request.send", json)`). The memory contract is uniform and load-bearing: every `char*` the ABI returns is a heap-allocated, NUL-terminated UTF-8 buffer the caller must hand back to `<prefix>_string_free`, and the engine itself is a single process-global instance behind a `mutex`, so the C side does not juggle handles. It is the least surprising FFI surface that could work, which is the point — the surprises should be in the domain logic, never in the boundary.

One engine in the fleet deliberately omits the C ABI, and the exception proves the rule: `zpdf-core` ships no `include/` directory and no `extern "C"` functions at all. It exposes only the native Rust API and the Tauri plugin. The reason is practical rather than principled — the PDF engine's surface is enormous (its command dispatch alone runs to dozens of operations like `apply_redactions`, `certify`, `add_qr_code`, `linearize`, `accessibility_check`) and every consumer of it in the fleet is itself a Rust/Tauri app that can link it natively, so the C door was never needed. When a non-Rust host eventually wants to embed PDF preview, the four-function skin gets added; until then, the crate does not carry an ABI it has no caller for. The book flags this rather than pretending the pattern is perfectly uniform.

There is, in truth, a *third* door, though it is really a specialization of the first: the Tauri plugin. Each core's `src/tauri_plugin.rs` (behind the `tauri` feature) defines one `#[tauri::command] <prefix>_invoke` and a `setup` helper that wires the engine's event sink to emit a `WebView` event — `"zrq-event"`, `"ztn-event"`, `"zgo-event"`. This is not a separate implementation; it is the native Rust API with a Tauri-shaped adapter, so the same `Engine::invoke` that a C host reaches through `zrq_invoke` a JavaScript host reaches through `invoke("zrq_invoke", { cmd, args })`. Two front doors to one engine, and a hallway between them.

“From scratch”: what we build and what we depend on

“Rebuilt from scratch in Rust” is a strong claim, and the stack earns it by being precise about the phrase. From scratch means the *product* — the workflow, the data model, the feature surface — is rebuilt in owned Rust, not that every byte of every dependency is hand-rolled. `zreq` builds its own Postman collection model, its own variable resolver, its own auth signers, its own code generators, and its own importers; it does *not* reimplement TLS, and it reaches for `reqwest` (behind the `net` feature) to actually put bytes on the wire. The line is drawn at “the part that is Postman” versus “the part that is plumbing every HTTP client shares.”

The cores are unusually disciplined about where that line falls, and it shows in their dependency lists. The auth signers in `zreq-core` — HTTP Digest, OAuth 1.0a, AWS Signature v4, Hawk — need MD5, SHA-1, SHA-256, and HMAC, and rather than pull a crypto crate the engine hand-rolls them in `src/crypto.rs`, because the durable-deps rule prefers a few hundred lines of owned hashing to a supply-chain dependency for four signature schemes. `zcite-core` parses BibTeX, RIS, CSL-JSON, EndNote XML, MODS, MARCXML, and Zotero RDF, and hand-writes its CSL 1.0 citation processor rather than binding a C library. Where a dependency *is* taken, it is a load-bearing one that would be perverse to rewrite: `lopdf` for the PDF object

model in `zpdf`, `bollard` for the Docker API and `kube` for Kubernetes in `zcontainer`, `boringtun` for the WireGuard Noise handshake in `ztunnel`, `opendal` and `russh` for transfer protocols in `zftp`.

The honest framing, then, is the one this book applies in every chapter: the *predecessor* is credited as prior art for the workflow, and the *dependency* is credited as prior art for the plumbing, and what is claimed as original is the Rust rebuild of the product and the architecture that lets ten such rebuilds share one toolkit and embed one another. Postman defined the collection-and-environment model; `zreq` reimplements it faithfully in Rust and adds nothing false to the claim. `request` sends the request; `zreq` does not pretend to have written a TLS stack. Both credits are given plainly, and what remains after both is genuinely new.

This is also why the stack's own project bar — the one that kills “X but faster” and “X but in Rust” as goals — does not kill these apps. A faster Postman clone with nothing new would be dead on arrival by that rule. What survives the rule is the *composition*: a Postman that shares a PDF engine with a Zotero that shares a citation engine with an office suite, all behind one command palette, all embedding one another's document views, all built by one person from one monorepo. No app in the fleet is pitched as “the fast one.” Each is pitched as one faithful rebuild in a set that composes, and the composition is the part no predecessor has.

The two doors, in real signatures

The “two front doors” is easy to state and worth seeing in the actual code, because the discipline of the pattern lives in how little the second door adds over the first. The native door is `Engine::invoke`; the C door is four `extern "C"` functions that wrap it. Here is the whole of `zreq-core`'s `src/ffi.rs` shape, which is byte-for-byte the shape of `zcore-core`, `ztunnel-core`, `zgo-core`, `zftp-core`, and `zemail-core` but for the prefix:

```
// zreq-core/src/ffi.rs — the C door, four functions over one engine.
pub type EventCb = extern "C" fn(json: *const c_char, user: *mut c_void);

#[no_mangle] pub unsafe extern "C"
fn zrq_set_event_callback(cb: EventCb, user: *mut c_void) { /* register sink */ }

#[no_mangle] pub extern "C"
fn zrq_init() -> *mut c_char { /* build OnceLock<Mutex<Engine>>, return {"ok":true} */ }

#[no_mangle] pub unsafe extern "C"
fn zrq_invoke(cmd: *const c_char, args_json: *const c_char) -> *mut c_char {
    // parse cmd + args, lock the global Engine, call invoke, return heap JSON
}

#[no_mangle] pub unsafe extern "C"
fn zrq_string_free(s: *mut c_char) { /* reclaim any string the ABI returned */ }
```

Set that against the native signature — `pub fn invoke(&mut self, cmd: &str, args: &Value) -> Result<Value>` — and the relationship is exact: `zrq_invoke` is `invoke` with its `&str/&Value` marshalled from C strings and its `Result<Value>` marshalled to a heap `char*`. Nothing in the C door decides anything; it translates types and locks the mutex. The `EventCb` typedef is the same idea for the reverse direction: a C function pointer the engine's `EventSink` calls, so the async narration that a Tauri host receives as a “`zrq-event`” a C host receives as a call to `cb(json, user)`. Two doors, one body, and the second door is a translator, not a second implementation — which is exactly why keeping the engine pure (Chapter 1's rule) is what keeps the C door four functions long instead of a parallel port.

Three doors, one hallway: the Tauri plugin as adapter

The two front doors get most of the attention, but there is a third, and calling it out precisely matters because it is not a third implementation — it is the native door wearing a Tauri-shaped hat. Each core's `src/tauri_`

`plugin.rs`, compiled only behind the `tauri` feature, defines one `#[tauri::command] <prefix>_invoke` plus a `setup` helper. The command is four lines: take the `cmd` string and `args` value the `WebView` sent, lock the process-global engine, call `invoke`, return the JSON. The setup helper does the one interesting thing — it installs an `EventSink` that forwards each `EngineEvent` to `app_handle.emit("<prefix>-event", event)` — so the same `EngineEvent` a C host receives through a registered callback a JavaScript host receives as a `WebView` event it can `listen` for.

That symmetry is the whole point of calling the plugin a third door rather than a third surface. The engine has exactly one dispatch (`invoke`) and one narration mechanism (the sink); the C ABI adapts them to function pointers and heap strings, and the Tauri plugin adapts them to a registered command and emitted events. Neither adapter decides anything the engine does not; both are hallways between the same body of logic and a different host. This is why a bug fixed in `invoke` is fixed for all three hosts at once, and why the coverage a `cargo test` measures against the native `invoke` is coverage of what the C host and the JavaScript host will see too. The “two front doors” of Chapter 1 is really “one body, three hallways,” and the third hallway — the Tauri plugin — is the one every app in this book actually walks through.

The shell's half of the bargain

The zero-GUI rule is usually stated as a constraint on the engine, but it is just as much a description of the shell, because everything the engine refuses is something the shell must own. If the engine is pure computation over data, then the impure world — pixels, processes, privilege, the event loop, the file system's dialogs, the menu bar, the tray — is the shell's territory by elimination, and it is worth naming that territory precisely, because “thin shell” does not mean “empty shell.” The shell is thin in *domain* logic and thick in exactly the things a domain-pure engine cannot touch.

Four responsibilities recur in every app's shell. The first is the window and its chrome: creating the `WebView`, sizing and positioning it, wiring the tray or menu-bar presence, handling the platform's close/hide/reveal gestures — the reason `zgo` registers `zgo_hide` and `zgo_reveal` is that a launcher's window behaviour is a platform gesture the engine has no business knowing about. The second is privileged and platform work: `ztunnel`'s shell installs and talks to the privileged helper that actually brings a tunnel up, because `ztunnel-core` can compute the command line and parse the management socket but cannot hold root; `zcontainer`'s shell manages the `dockerd` guest VM the engine only speaks to over an API. The third is streaming and long-lived channels — the PTY behind the embedded terminal, the subscription that follows a container's logs — which do not fit a pure `invoke(cmd, args) -> Value` and live in the shell's async runtime. The fourth is the bridge itself: registering the engine's one command, wiring the event sink to a `WebView` emit, and forwarding the handful of platform commands.

The discipline that makes this clean is that the interface between the two halves is narrow and mostly flows one way. The engine hands the shell *data* — a `PrivilegePlan`, a `FirewallPlan`, a `rename_plan`, a rendered bitmap, a JSON result — and the shell performs the effect the data describes. The engine does not call the shell; it returns a value and, for asynchronous narration, pushes an `EngineEvent` into a sink the shell installed. That asymmetry — the engine computes and returns, the shell observes and acts — is what keeps the engine testable (Chapter 2) and embeddable (Chapter 13): a test or a host app can supply its own trivial sink and drive the same engine that the real shell drives, because the engine never reaches back into a particular shell's world. The bargain has two halves, and the engine's half is only clean because the shell's half absorbs everything impure.

Chapter 2 – The *-core Pattern

The `-core` suffix is a contract, not a naming habit. This chapter specifies it: a pure-Rust crate that parses, edits, and serializes its domain; exposes a stable native API; and re-exports that API through a C ABI so any other app — or any non-Rust host — can embed it. `zpdf-core`, `zphoto-core`, `zoffice-core`, `zemail-core`, `zreq-core`, `zcite-core`, `ztunnel-core`, `zgo-core`, `zftp-core`, `zcontainer-core`: ten engines, one shape.

What belongs in `-core` and what belongs in the shell

The dividing question is simple to state and subtle to apply: does this code compute a result from data, or does it touch the platform, the window, or the user? Everything on the first side belongs in `-core`; everything on the second belongs in the shell. Parsing a `.ovpn` file, building the `openvpn` command line, and interpreting the management-socket stream are pure computation, and they live in `ztunnel-core`'s `config`, `openvpn`, and `supervisor` modules. Actually spawning the process with elevated privilege, installing a privileged helper via `SMApService` or `polkit`, and drawing the connection list are platform and window concerns, and they live in the shell.

The Cargo manifests make the contract legible. Every core declares `crate-type = ["rlib", "staticlib", "cdylib"]` — an `rlib` for Rust hosts, a `staticlib` and `cdylib` for C hosts — and a `[lib] name` that underscores the crate name (`zreq_core`, `zcite_core`, `ztunnel_core`). Every core's default features are minimal and its platform reach is gated: `net` turns on the HTTP or socket transport, `tauri` turns on the plugin adapter, `wireguard` turns on the `boringtun` data path, `keychain` turns on OS-keychain credential storage. Build a core with `--no-default-features` and you get a pure, network-free, GUI-free library that still parses, edits, and serializes its whole domain — which is exactly the configuration the tests run in.

The shell, correspondingly, is thin by construction. A Tauri app's `src-tauri/src/main.rs` in this fleet registers the engine's one `invoke` command, four terminal commands (`terminal_spawn`, `terminal_write`, `terminal_resize`, `terminal_kill`) for the shared embedded PTY, a set of `fs_*` commands from the shared `zpw-file-browser` crate, and perhaps a handful of genuinely platform-specific commands — `zgo`'s `zgo_hide/zgo_reveal/zgo_trash` for its menu-bar launcher, `zreq`'s `open_hooks_editor` for the Monaco window. That is the whole backend. Everything a user would call a “feature” lives one layer down, in the engine, reachable identically from the app and from any sibling that embeds it.

The discipline occasionally forces a genuinely interesting design decision at the boundary. `zcontainer-core` needs to stream — container logs follow, `kubectl exec` sessions, port-forwards — and streaming does not fit the request/response shape of `invoke`. Rather than smuggle a socket into the engine, the core exposes a subscription API (`subscribe`, `unsubscribe`, `stream_send`) and a matching pair of Tauri commands, so the *shape* of streaming lives in the engine's contract while the *transport* stays in the shell. The boundary bent to accommodate a real requirement without breaking the rule, and that bend is documented in the crate rather than hidden.

The C ABI surface and the embedding contract

The embedding contract is four functions and a memory rule, and it is worth stating exactly because its uniformity is what makes the fleet composable. For a core with prefix `P`, the C header in `include/P_core.h` declares:

`P_init()` returns a JSON status string after constructing the process-global engine; `P_invoke(cmd, args_json)` runs one command and returns a JSON result string; `P_string_free(s)` releases any string the ABI returned; and `P_set_event_callback(cb, user)` registers a `void (*)(const char *json, void *user)` for asynchronous events the engine emits. Nine of the ten cores implement exactly this; `zpdf-core` is the documented exception, Rust-and-Tauri only.

The events deserve attention because they are how an engine talks *back* across the boundary without blocking. Each core defines an `EngineEvent` enum and an `EventSink = Box<dyn Fn(EngineEvent) + Send + Sync>`. `zreq-core` emits `RequestStarted`, `RequestFinished { status, time_ms }`, and `Error`; `ztunnel-core` emits `Status`, `Stats { bytes_in, bytes_out }`, and `Log`. Over the C ABI these become JSON strings handed to the registered callback; over the Tauri plugin they become WebView events ("`zrq-event`", "`ztn-event`") the JavaScript listens for. The engine does not know or care which; it calls its sink, and the host decides what a "sink" means in its world.

The memory contract is the part a C programmer must get right, and it is uniform so there is only one rule to learn: strings returned by `P_invoke` and `P_init` are heap-allocated and owned by the caller, freed with `P_string_free`; the engine is a single global behind a mutex, so there are no handles to track and no lifetime to reason about beyond the strings. This is a deliberately conservative design — one global engine per process is a limitation if you wanted two isolated libraries at once — but it buys a boundary a C or C++ host can use correctly on the first try, which for an "embed anywhere" promise matters more than theoretical multi-instance flexibility.

Consuming the ABI from a non-Rust host looks like this, and the uniformity means the sketch is the same for every engine save the prefix:

```
#include "zreq_core.h"
#include <stdio.h>

static void on_event(const char *json, void *user) {
    /* {"type":"request_finished","status":200,"time_ms":42} */
    printf("event: %s\n", json);
}

int main(void) {
    zrreq_set_event_callback(on_event, NULL);
    char *status = zrreq_init();          /* {"ok":true} */
    zrreq_string_free(status);

    char *res = zrreq_invoke("request.send", "{\"id\":\"r1\"}");
    printf("result: %s\n", res);
    zrreq_string_free(res);
    return 0;
}
```

Testing an engine with no app around it

A GUI-free engine is a gift to a test suite, because the thing under test is a value and a function. There is no need for a WebView driver, a screenshot differ, or a synthetic event loop; a test constructs an `Engine`, feeds it a command and JSON arguments, and asserts on the JSON that comes back. `zcite-core` routes its entire ~100-command surface through `Engine::invoke`, so a single test harness — build an engine on a temp profile path, call `invoke`, compare — exercises the whole library. The engines even ship JavaScript test files (`zcite.test.cjs`) alongside the Rust tests, because the mountable webui can be driven against a mock transport without a backend at all.

This is why the coverage claims in the READMEs are believable rather than aspirational. `zreq-core` keeps a machine-readable feature matrix, `scripts/port_features.tsv`, enumerating 83 Postman behaviours and scoring each `full`, `partial`, or `missing`, rendered to a `PORT_REPORT.md` — and the scoring is conservative,

marking a feature `full` only when both behaviour and GUI match Postman. `zcite` reports “95.0% weighted, 0 overclaim flags” against its own `PORT_REPORT.md`. Those numbers are only meaningful because the engine can be tested exhaustively in isolation; a feature matrix over an untestable GUI would be marketing, but a feature matrix over a pure engine is a test plan.

The pattern also makes the network the easy part to fake. Because transport lives behind the `net` feature, a test build with `--no-default-features` still runs the whole *request-preparation* path — `zreq-core`'s `http::prepare` builds the full `PreparedRequest` (method, URL, resolved headers, encoded body, applied auth) without a `net` feature in sight, and only `http::send` needs the wire. So the code generators, which render a `PreparedRequest` into curl or Python or Go, can be tested with no network at all: prepare a request, generate the snippet, assert on the string. The feature gate is not just a dependency-trimming trick; it is a seam the tests exploit to cover the interesting logic without touching a socket.

Where an engine genuinely needs a live peer, it isolates that need behind a small, testable protocol layer. `ztunnel-core`'s OpenVPN management-socket handling is split so that `parse_management_line` — the pure function turning a `>STATE:` or `>BYTECOUNT:` line into a `MgmtEvent` — is tested directly on strings, while the `run_session` loop that actually reads the socket is generic over any `BufRead + Write` and is exercised over a loopback pair in tests. The lesson repeats across the fleet: find the pure function inside the impure operation, put it in the core, test it on data, and let the shell own the few genuinely impure edges.

Versioning engines independently of their shells

Because an engine is its own crate — often its own git repository, mounted into the app as a submodule — it carries its own version, and that version moves on the engine's schedule, not the app's. The split is visible right now in the fleet's numbers: `zgo-core` sits at `0.1.0` while the `zgo` app that wraps it is at `0.2.0`; `zpdf-core` is `0.1.0` while the `zpdf` app is `0.2.0`; `zcontainer-core` is `0.2.0` while its app has moved to `0.3.0`. The engine and the shell are two versioned artifacts that happen to ship together, and the version skew is not a mistake — it is the pattern working, the shell iterating on chrome and packaging while the engine holds steady, or the reverse.

The mechanism is git submodules. `zreq-core` lives at `github.com/MenkeTechnologies/zreq-core.git` and is mounted into the `zreq` superproject at `crates/zreq-core`; the same crate, byte for byte, is the standalone repository and the app's vendored copy. A sibling app that wants to embed the engine adds the same submodule and pins it to a commit. Bumping the engine everywhere is a submodule-pointer bump in each consumer, which is a commit that records exactly which engine revision each app shipped — the monorepo becomes a ledger of who embeds what at which version.

This independence is the precondition for the embed matrix of Chapter 13. If the engine's version were welded to the app's, then embedding `zpdf-core` into `zcite` would mean `zcite` inherits `zpdf`'s release cadence, and a fleet of ten apps would seize up trying to release in lockstep. Because the engine versions independently, `zcite` pins the `zpdf-core` revision it was tested against and upgrades on its own schedule, and `zpdf` the app does the same from the other side. Ten shells can pin ten different revisions of the same engine and none of them blocks another's release.

The cost is the cost of any submodule fleet: a change that must reach every app is a cascade of pointer bumps, and a stale pointer is an app running an old engine until someone bumps it. The stack accepts this deliberately, because the alternative — one giant versioned monolith — trades the cascade for a worse problem, a release train where the slowest car holds the whole train. Chapter 15 returns to the economics of the cascade for a solo author; the point here is that independent versioning is what the `-core` pattern *buys*, and the pointer cascade is what it *costs*, and the trade is made with eyes open.

The C-ABI surface is a spectrum, not a single stamp

The four-function skin is the pattern's headline, and it is accurate for most of the fleet — but a book that promises honesty about its own uniformity should read the ten manifests and admit that the C ABI is a spectrum, not a

stamp pressed identically onto every crate. What varies is exactly what the domain requires, and the variation is legible in a way that vindicates the pattern rather than embarrassing it. Six engines carry the canonical four: `zrq_`, `zct_`, `ztn_`, `zgo_`, `zftp_`, and `zml_` each export `<prefix>_set_event_callback`, `<prefix>_init`, `<prefix>_invoke`, and `<prefix>_string_free`. These are the engines with an asynchronous life — requests, lookups, tunnels, launches, transfers, mail — so they need the event callback and the init that constructs a stateful global.

At the two ends of the spectrum sit engines whose domains push the count up or down. `zcontainer-core` carries *eight* functions, adding `zcontainer_subscribe`, `zcontainer_unsubscribe`, and `zcontainer_stream_send` to the four, because its domain has genuine streams (Chapter 4). `zphoto-core` carries *three* — `zphoto_init`, `zphoto_invoke`, `zphoto_string_free` — and no event callback, because an image editor computes on request and has nothing to narrate asynchronously. `zoffice-core` carries *two*, `zoffice_invoke` and `zoffice_string_free`, with no `init` and no callback: its `Engine` is a stateless unit struct, so there is no global to construct and no event to emit, and the crate is written on the 2024 edition with `#[unsafe(no_mangle)]` to match. And `zpdf-core` carries *zero*, with no `ffi.rs`, no `include/`, and a `crate-type` that defaults to `rlib` only — the documented exception, driven by having no non-Rust caller yet.

The lesson is that “four functions” was never the invariant; the invariant is “the smallest C door the domain needs, and no more.” An engine that emits events has `set_event_callback`; one that does not, omits it. An engine that holds process state has `init`; a stateless one skips it. An engine that streams has the subscription trio; the eight others do not carry a streaming API they would never call. The `string_free/invoke` pair is the true universal — every engine that has a C ABI at all has those two, because every engine returns heap JSON strings that a C caller must release. Read that way, the spectrum is the durable-deps rule applied to the ABI itself: carry the functions a caller will actually use, hand-fit to the domain, rather than stamping a uniform surface onto crates that have no use for half of it. The uniformity is in the *contract* — JSON in, heap JSON out, free what you get — not in a fixed function count.

The manifest, read line by line

The whole discipline of the `-core` pattern is legible in a single Cargo.toml, and `zftp-core`'s is the richest example because its domain forces the most feature seams. The `[lib]` line declares `name = "zftp_core"` and `crate-type = ["rlib", "staticlib", "cdylib"]` — the `rlib` for Rust hosts, the `staticlib` and `cdylib` for C hosts — which is the same three-target declaration on every engine but `zpdf-core`. The mandatory dependencies are the fleet's durable four: `serde`, `serde_json`, `thiserror`, `dirs`. Everything with weight is optional and gated, and reading the gates tells you exactly what each feature buys.

The `[features]` table is the map. `default = ["net", "discovery", "keychain"]` is the app build; `net = ["dep:opendal", "dep:tokio", "dep:russh", "dep:russh-sftp", "dep:encoding_rs"]` is the one gate that pulls the entire network stack — the `opendal` data-access layer with its long `services-*` feature list, the `russh` pure-Rust SSH transport, and the `encoding_rs` charset handling — so a build with `--no-default-features` gets the bookmark parser, the session model, the transfer queue, and the `.duck` importer with every transport stubbed. `discovery = ["dep:mdns-sd"]` adds LAN server discovery; `keychain = ["dep:keyring"]` adds OS-secret credential storage; `tauri = ["dep:tauri"]` adds the plugin. The manifest is not bookkeeping — it is the contract Chapter 2 keeps describing, written as Cargo directives: a tiny mandatory core, every platform reach behind a flag, and a `--no-default-features` build that still parses, edits, and serializes its whole domain, which is exactly the configuration the tests run in.

Finding the pure function inside the impure operation

The single technique that recurs most across the fleet is worth naming as a method: locate the pure function hiding inside an impure operation, put *it* in the core, test it on data, and leave only the genuinely impure edge to the shell. Bringing up a VPN is impure — it needs root, a real interface, a live socket — but the decision of *what* interface, address, route, and DNS to configure is pure, so `ztunnel-core` computes a `PrivilegePlan` of `NetOps` the shell applies. Reading an OpenVPN management socket is impure, but turning a `>STATE`: or

>BYTECOUNT: line into a typed `MgmtEvent` is pure, so `parse_management_line` is a function tested directly on captured strings while the socket read stays in the shell.

The same cut appears in every engine. Sending an HTTP request is impure, but building the `PreparedRequest` is pure, so `zreq-core`'s `http::prepare` is tested — and reused by the code generators — with no socket. Fetching a DOI is impure, but classifying and normalizing an identifier is pure, so `zcite-core`'s `identifier::detect` is tested on strings while `lookup` is gated behind `net`. Filtering a Zotero library against a smart search is pure, so `search::saved_matches` is a predicate a test exercises directly. Watching the pasteboard is impure, but holding and searching clipboard history is pure, so `zgo`'s daemon lives in the shell and the `Clipboard` lives in the engine. Over and over the boundary is drawn by asking “what part of this computes a result from data?” — and that question, applied relentlessly, is what fills the cores with testable logic and leaves the shells thin. The zero-GUI rule of Chapter 1 is the constraint; finding the pure function inside the impure operation is the technique that satisfies it.

Three link targets, one crate

The `crate-type = ["rlib", "staticlib", "cdylib"]` line on every engine but `zpdf-core` is worth unpacking, because those three targets are what let one crate serve a Rust host, a static-linking C host, and a dynamic-linking C host from a single build. The `rlib` is the Rust library a sibling app links when it vendors the engine as a submodule — the fast path, a normal Rust dependency with no FFI. The `staticlib` produces a `lib<name>_core.a` a C or C++ program links directly into its binary, and the `cdylib` produces a `lib<name>_core.{so,dylib,dll}` a program loads dynamically or links against. All three expose the same `<prefix>_*` symbols over the same `Engine::invoke`, so which one a host chooses is a packaging decision, not a capability one.

The C++ facade most cores ship alongside the `.h` makes the C-linking path pleasant without adding a second implementation. `zreq_core.hpp` declares `namespace zreq { class Core }` with static `init()`, `invoke(cmd, argsJson = "{}")`, and `onEvent(handler)`, and it is header-only: it wraps the C functions, owns the returned `char*` with a private `take()` that frees through `zrq_string_free`, and installs a `trampoline` that adapts the C callback to a `std::function`. So a C++ host writes `zreq::Core::invoke("request.send", json)` and never touches a raw pointer, while the crate still ships only the four `extern "C"` functions — the facade is a convenience in a header the crate hands over, not a second surface it maintains. Three link targets and an optional C++ ergonomics layer, all over one `Engine`, is how “embed anywhere” reaches hosts in three linking styles from a single crate.

Chapter 3 – The Shared Shell: Tauri v2 and `zgui-core`

Ten apps cannot look like ten apps. This chapter covers the front-end half of the stack: the Tauri v2 runtime, the app shell every GUI app mounts, and `zgui-core` — the single source of truth for every control, dialog, table, toast, modal, command palette, and chrome element, consumed as a shared submodule on `window.ZGui.*`. The rule is strict: if a widget isn't in `zgui-core`, you add it there first and consume it — you never hand-roll a one-off.

The app shell and `GUI_APP_REQUIREMENTS`

The family has a conformance spec, `GUI_APP_REQUIREMENTS.md`, and it opens with a sentence that is the whole design philosophy in one line: “Divergence is a bug.” Ten requirements, R1 through R10, are a MUST for every app in the roster, and each names the canonical shared source an app must consume rather than reimplement. R1 is a fuzzy command palette on Cmd/Ctrl+K. R2 is the shared Monaco hooks editor. R3 is a real PTY-backed embedded terminal on Ctrl+backtick and Cmd+T. R4 is the shared cyberpunk design tokens. R5 is a tile dashboard plus tab bar. R6 is the logo top-left. R7 is that every filter is fuzzy with highlighted matched characters. R8 is that every table is sortable and resizable. R9 is the embedded DAW arrangement grid. R10 is the ported multi-pane file browser. An app is not “done” until it is green on all ten.

The keystone that delivers most of this is `ZGui.appShell(root, opts)`, in `zgui-core/webui/app-shell.js`. It is a single-instance component — it throws if mounted twice, guarded by a `data-zgui-shell-mounted` flag on the document element — and mounting it gives an app its entire chrome in one call: the header strip with the brand glyph top-left, the fuzzy filter input, the Cmd+K command palette, the settings panel, the shortcuts overlay, the colorscheme switcher, the CRT-effect layer, and the OS-native menu bar. The call returns a handle whose `body` is where the app puts its own content and whose `setPaletteItems(items)` is how the app feeds commands into the one palette. An app's `main.js` is, in large part, one `ZGui.appShell(...)` call plus the wiring of its engine into that handle.

You can read the contract in a consuming app's mount. `zreq/frontend/main.js` mounts the shell with a brand of `{ glyph: "⚡", title: "ZREQ", subtitle: "api client" }`, a filter placeholder, an empty initial palette it fills later, and a `settingsExtra` callback that injects `zreq`'s own request options (SSL verify, redirects, timeout, proxy) into the shell's single settings panel. `ztunnel`'s `main.js` does the same with brand `ZTUNNEL/vpn`, relocating its `#ztunnelRoot` into `shell.body`. The apps do not build headers, palettes, or settings panels; they configure the one shell and hand it their content. That is what “divergence is a bug” means in practice — the chrome is not per-app code to keep in sync, it is one component every app calls.

The spec is candid about where conformance is still being reached, and the book follows it. `GUI_APP_REQUIREMENTS.md` keeps a “known conformance gaps” section: the command palette is not yet single-source across the older Audio apps, the Tauri apps and JUCE apps still split their stylesheet source, the arrangement grid (R9) is embedded only in the DAW so far. What is fully converged is the newer Tauri roster this book is about — `zreq` and `zcite` are called out by name as routing every table, filter, and tab through the shared `ZGui.table`, `ZGui.fzf`, and `ZGui.tabs`. The conformance spec is a live checklist, not a finished monument, and the honest state is “green where it matters most, closing the rest.”

window.ZGui.*: the canonical component surface

`zgui-core` is roughly 240 component files under `webui/`, each an IIFE that hangs one namespace off the single global `window.ZGui`. There is no build step to import a widget and no per-app copy to drift; the app loads the submodule's files directly and calls `ZGui.<name>`. The surface is large but it is not sprawling — it is the accumulated set of “the generic kernel of something one or more apps had already built,” extracted so the N apps stop maintaining N divergent versions of the same table. The README lists the lineage explicitly: the toolkit was pulled out of Audio-Haxor, `zreq`, and `zpwir-patch-core`, which is why so many of its components carry the exact markup those apps had already settled on.

The pieces an app reaches for constantly are a small subset of the 240. `ZGui.appShell` is the shell. `ZGui.palette` is the command palette, with a host singleton (`register`, `open`, `bindHotkey`) and a scoped `create({ items, scope })` for embedded views. `ZGui.fzf` is the fuzzy matcher — `fzfMatch(query, text)` returning a score and matched indices, plus `highlightMatch` wrapping matched characters in `<mark class="fzf-hl">` — extracted verbatim from Audio-Haxor so every filter in every app highlights identically. `ZGui.tabs`, `ZGui.toast`, `ZGui.modal` (with `confirm()` and `prompt()` returning promises), `ZGui.settings`, and the two table components — `ZGui.table` (a resize/sort/filter toolkit) and `ZGui.dataTable` (a simpler self-drawing table) — round out the daily set.

The naming has a few sharp edges worth knowing, because the book uses the real names throughout. The command palette is `ZGui.palette`, not `ZGui.commandPalette`. The preferences window shell is `ZGui.prefsShell`, distilled from `zgo`'s Alfred-style prefs, not `ZGui.prefs`. The HUD telemetry helpers — `statStrip`, `headlineStats`, `playingPill`, `heatColor` — are top-level functions on `ZGui`, not grouped under a `ZGui.hud`. Both `ZGui.table` and `ZGui.dataTable` exist and are different things. And `ZGui.embed.view()` exists exactly as named, which Chapter 3's last section is about. Getting these names right matters because the whole point of a single surface is that an app author reaches for the one real name and finds the one real widget.

A mount looks like this, and it is the same call in every app but for the brand and the callbacks — the baseline every conforming app must reproduce, straight from `zgui-core`'s `CONSUMERS.md`:

```
const shell = ZGui.appShell(document.getElementById("app"), {
  brand: { glyph: "◆", title: "MYAPP", subtitle: "..." },
  filterPlaceholder: "Filter...",
  onFilter: (q) => myList.filter(q),
  palette: [{ label: "Reload", run: reload }],
  onOpenLog: () => invoke("open_log_file"),
  onOpenLogDir: () => invoke("open_log_dir"),
  onOpenDataDir: () => invoke("open_data_dir"),
});
// app content goes in shell.body; commands go in via shell.setPaletteItems(...)
```

The cyberpunk HUD as a design system, not a skin

The look is not a stylesheet an app can override; it is a token system every surface reads from, and that distinction is the difference between a design system and a skin. `zgui-core/webui/cyberpunk.css` defines the palette as CSS custom properties on `:root` — `--accent: #ff2a6d` (the hot magenta-pink), `--cyan: #05d9e8`, `--magenta: #d300c5`, `--green: #39ff14`, over a near-black stack of `--bg-primary: #05050a`, `--bg-secondary`, `--bg-card`, `--bg-hover`, with text in `--text: #e0f0ff` and dimmed variants — plus a glassmorphism group (`--glass-fill`, `--glass-blur: blur(16px) saturate(165%)`, `--glass-sheen`) for the frosted panels. New surfaces are required (R4) to use the token variables, not hardcoded colours, so that an element moved from one app to another looks the same without touching it.

The type is part of the system, not decoration. Two fonts, both bundled as `@font-face` from vendored files rather than depending on a system install: **Orbitron** (a variable-weight display face) for headings and the big HUD numbers, and **Share Tech Mono** for terminal and body text. The bundling is load-bearing, and the

requirements spec has a whole pitfall about it: a `font-family` that resolves only against a user-installed font renders tofu on a clean machine while looking fine on the dev box that happens to have the font. So the fonts ship inside the toolkit, `font-display: block`, and every app inherits the same two typefaces without a system dependency.

Because it is a token system, theming is a first-class, runtime operation rather than a rebuild. `ZGui.colorscheme` ships eight built-in schemes — `cyberpunk`, `midnight`, `matrix`, `ember`, `arctic`, `crimson`, `toxic`, `vapor`, each with a light variant — and applying one is writing 24 known custom properties (`SCHEME_VAR_KEYS: --accent, --cyan, --magenta, --green`, the backgrounds, the borders) onto `:root`. A user picks a scheme from the shell's switcher and every component recolours at once because every component draws from the same 24 tokens. There is also a custom-scheme editor, preset save/load, and a light/dark toggle, all built on the same key set. The HUD is not a paint job; it is a small vocabulary of tokens that the entire toolkit and every app speak.

The effects layer — CRT scanlines, a bezel vignette, neon glow — is likewise part of the system and likewise controllable, disabled per-body with classes `no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim` for accessibility or taste. The visual language is coherent because it is centralized: one token file, one font file, one colorscheme engine, one effects layer, consumed by every app. When the design evolves, it evolves in `zgui-core` and reaches every app on the next submodule bump, which is the same reuse story as the engines, applied to the front end. The design system is `zgui-core`'s answer to “ten apps cannot look like ten apps,” and it answers it by making the look a shared dependency rather than a shared aesthetic that each app re-implements.

Cross-app embeds: mounting a sibling engine's view live, not as a placeholder

The hardest requirement in the whole architecture is that a cross-app embed be *real* — a live, working sibling engine mounted in the host's window, not a screenshot, a stub, or a “coming soon” panel. The stack's own architecture notes are blunt that scaffold placeholders do not count; an embed must make the core genuinely embeddable and mount it live. This is only possible because the engines are GUI-free (Chapter 1) and the chrome is shared (this chapter), but it introduces one real hazard: two components on one document fighting over the same globals.

The concrete failure is the keyboard. If a host app binds `Cmd+K` for its command palette and an embedded core also binds `Cmd+K` for *its* palette, the two bindings fight on one `document` and the app reads as broken. The same collision lurks for the settings panel, the global shortcut map, the `:root` colorscheme, and the full-document CRT and splash overlays. An embedded core that innocently calls `ZGui.appShell` or `ZGui.palette.bindHotkey()` or `ZGui.colorscheme.apply()` assumes it is the whole app, and inside another app that assumption corrupts the host.

`zgui-core` enforces the rule mechanically rather than by convention, through `ZGui.embed.view(fn)` in `webui/embed.js`. For the duration of `fn`, a hard-coded `FORBIDDEN` list of `window.ZGui` paths is swapped for throwing stubs — `appShell`, `palette.bindHotkey`, `shortcuts.init`, the document-writing `colorscheme` methods (`apply`, `applyVars`, `setLight`, `load`, `buildSwitcher`, `buildEditor`), the `splash` methods, `crt`, `neonGlow.set/toggle`, and the top-level `baseline` stamp — with the originals restored in a `finally`. An embedded view that reaches for a document-global singleton throws at `init` instead of silently corrupting the host. The error text even points the way out: “Open it from a button with the scoped API (`buttonBar / palette.create({scope}) / colorscheme.scope`).”

The scoped APIs that remain callable are the whole trick. A core embeds by contributing *through* the host rather than seizing globals: it opens its own palette from a button with `ZGui.palette.create({ items, scope: paneEl })` instead of grabbing `Cmd+K`, it themes only its own pane with `ZGui.colorscheme.scope(paneEl, "matrix")` instead of writing `:root`, and it feeds its commands and settings to the host shell via the shell handle's `setPaletteItems(...)` and the `settingsExtra` callback. The host shell owns the one palette, the one settings panel, and the global keyboard; the embedded

core is a well-behaved guest that adds to them. The embed side reads like this:

```
ZGui.embed.view(() => {
  // ZGui.palette.bindHotkey() here would THROW - the FORBIDDEN stub fires.
  const pal = ZGui.palette.create({ items: coreCommands, scope: paneEl });
  coreButton.onclick = pal.open; // button-opened, no global ⌘K
  ZGui.colorscheme.scope(paneEl, "matrix"); // theme only this pane, not :root
});
```

That mechanism — GUI-free engines, a shared shell, and a hard boundary that turns “seize a global” into an immediate exception — is what makes the embed matrix of Chapter 11 a set of live features rather than a wall of placeholders. The rest of this book’s app chapters can each end with “and here is what this engine looks like embedded in its siblings,” and mean it literally, because `ZGui.embed.view` is what keeps a live embed from breaking the app it lives in.

Two substrates, one frontend contract

There is a subtlety in “the chrome is shared” that the requirements spec is careful about and this book should be too: the shared web modules run on *two* native substrates, not one. The desktop apps this book is about are Tauri v2, but the same `zgui-core` toolkit and the same `zpwir-embed-terminal` also run inside the fleet’s JUCE audio plugins, and `GUI_APP_REQUIREMENTS.md` states the rule plainly — “two substrates, one frontend contract. The shared web modules must run unchanged in both.” That constraint is what forces the toolkit to be honest about its dependencies on the host: it may not assume Tauri, because half its hosts are not Tauri.

The two substrates differ in exactly two places, and a thin shim hides both. Under Tauri, assets are served from a static directory and the JavaScript reaches the backend through `window.__TAURI__` — `invoke(cmd, args)` for calls and `listen(name, handler)` for events. Under JUCE, assets are embedded via `juce_add_binary_data` and served by a resource provider that resolves a request URL to a `BinaryData` blob by *basename*, and the JavaScript reaches the backend through `window.Juce (getNativeFunction)` plus `window.__JUCE__.backend` events. A mountable webui detects which world it is in and picks the transport accordingly; the reference implementation the spec points at is `zpwir-embed-terminal/webui/terminal.js`, which speaks PTY over whichever door exists. Everything above that shim — the tables, the palette, the tokens, the fuzzy filter — is identical bytes in both.

This is why the mountable webui’s transport auto-detection, which Chapter 5 shows in `zreq`’s view falling back to an in-memory mock in a plain browser, is not a testing convenience bolted on; it is the same seam that lets the view run under JUCE. A component that assumed `window.__TAURI__` existed would render tofu in a plugin and fail in a browser preview; a component that asks “which host am I in?” runs unchanged in three places — Tauri app, JUCE plugin, and mock browser. The “divergence is a bug” rule has a corollary the spec makes explicit: divergence across *substrates* is a bug too, and the frontend contract is single not only across ten apps but across the two runtimes those apps and plugins are built on.

What a consuming app’s `main.js` actually does

The clearest way to feel how thin the app layer is, is to read what an app’s `main.js` does with the shell, because it is mostly configuration and wiring rather than construction. It mounts `ZGui.appShell(root, opts)` with its brand and its callbacks, relocates its engine’s mountable view into the returned `shell.body`, and feeds the engine’s commands and settings into the one shell — and that is the bulk of it. `zreq`’s `main.js` mounts the shell with a brand of `{ glyph: "⚡", title: "ZREQ", subtitle: "api client" }`, a filter placeholder, an initially empty palette it fills later via `setPaletteItems`, and a `settingsExtra` callback that injects `zreq`’s own request options — SSL verify, redirects, timeout, proxy — into the shell’s single settings panel. `ztunnel`’s `main.js` does the same with brand `ZTUNNEL/vpn`, relocating its `#ztunnelRoot` into `shell.body`.

Neither app builds a header, a palette, or a settings panel; each configures the one shell and hands it content, which is what “divergence is a bug” means in practice — the chrome is not per-app code to keep in sync,

it is one component every app calls. The app-specific JavaScript that remains is the glue between the engine's JSON and ZGui's widgets: take the `Value` a `zrq_invoke` returned, render it into a `ZGui.table` or a `ZGui.tiles` grid; take a `"zrq-event"` and clear a spinner; take a click and issue the next `invoke`. Open `zreq/frontend/main.js` and you find exactly that shape, and open `zcite/frontend/zcite.js` and its header comment says outright that all UI chrome in the view is built from `window.ZGui.*`. The interesting code in a new app is its engine; the shell is a call and some wiring, and that asymmetry is the whole reason a solo author can carry ten of them.

One settings panel, and `settingsExtra`

A subtle expression of “divergence is a bug” is that there is one settings panel in the whole shell, not a settings window each app builds. `ZGui.appShell` owns the panel — general preferences, the colorscheme switcher, the effects toggles, the diagnostics buttons — and an app injects its *own* rows through a single `settingsExtra(bodyEl)` callback rather than mounting a parallel dialog. `zreq` uses it to add its request options — SSL verify, redirects, timeout, proxy — into the shell's panel; `ztunnel` adds its VPN preferences the same way. The app does not decide where settings live or how they look; it hands the shell a function that appends its rows to the one panel, and the panel renders them in the shared style.

That single-panel design is also what makes embedding tractable, because it gives an embedded core a place to contribute settings without opening its own dialog. An embedded engine cannot mount `appShell` (Chapter 14 makes that throw), so it has no settings window of its own; instead it feeds the host's `settingsExtra` the rows it needs, and the host's one panel grows a section for the guest. The diagnostics buttons the panel carries — `onOpenLog`, `onOpenLogDir`, `onOpenDataDir` — are the same idea for the platform edge: the shell renders the buttons, the app wires them to the Tauri commands that open the right directory, and every app's “reveal my logs” affordance looks and behaves identically because it is one panel's buttons calling per-app commands. The settings panel is a microcosm of the whole shell: one component, one look, per-app content injected through one callback.

Chapter 4 – The Command Bridge and the IPC Contract

Between the WebView and the engine sits a layer the last three chapters kept calling “thin,” and this chapter is what “thin” actually means. The command bridge is the Tauri v2 IPC surface — the set of `#[tauri::command]` functions the front end reaches through `window.__TAURI__` — and its whole job is to hand a JSON string to `Engine::invoke` and hand the JSON result back. It is worth a chapter of its own because its shape is the same in every app, because the choices it makes (one command, not a hundred; events over a sink, not polling; a memory rule at the boundary) are the choices that keep the engine pure, and because the one place the bridge grows a second door — `zcontainer`’s streaming trio — is the exception that shows why the common case is one door.

One command per engine, not one per feature

The instinct, coming from most desktop frameworks, is to expose each backend operation as its own named IPC entry point: a `send_request` command, a `list_collections` command, an `add_environment` command. The bridge in this fleet refuses that. It exposes exactly one command for the engine — `zrq_invoke` in `zreq`, `zct_invoke` in `zcite`, `ztn_invoke`, `zgo_invoke`, `zftp_invoke`, `zml_invoke`, `zcontainer_invoke`, `zpdf_invoke` — and multiplexes the engine’s whole surface through it as a `(cmd, args)` pair. The Tauri command is a five-line adapter: take a command string and a JSON `Value`, lock the process-global engine, call `invoke`, map the `Result` to Tauri’s `Result<Value, String>`, return. The hundred operations are strings the one command dispatches, not a hundred commands the framework has to register, permission, and keep in sync with the front end.

The payoff is that the IPC surface stops being a maintenance burden that grows with the feature set. Add a citation style to `zcite-core`, a new `bib.*` command, and the bridge does not change at all — the new capability is a new string the existing `zct_invoke` already forwards. The front end learns the string; the backend learns the match arm; the boundary between them is untouched. A fleet whose IPC grew one command per feature would have thousands of registered commands across ten apps to version and secure; a fleet whose IPC is one command per engine has ten, and the churn all happens one layer down where it is a Rust `match` and a JavaScript call, both trivially testable.

There is a second, quieter reason the single-command shape matters: it makes the three doors of Chapter 1 genuinely the same door. The native Rust caller invokes `engine.invoke("request.send", &args)`; the C host calls `zrq_invoke("request.send", "{...}")`; the JavaScript host calls `invoke("zrq_invoke", { cmd: "request.send", args })`. Three hosts, three transports, one dispatch, one command vocabulary. If the Tauri bridge had exploded the surface into per-feature commands, the JavaScript door would speak a different language than the C door, and the “one engine, many front doors” promise would be a promise about the Rust API only. Keeping the bridge to one command keeps the command vocabulary identical across all three, which is what lets the mountable webui drive the engine over Tauri in the app and over a mock in a plain browser without changing a line of dispatch.

The args and result contract: JSON in, JSON out, errors as strings

The bridge's data contract is deliberately boring. Arguments cross as a JSON object — `serde_json::Value` on the Rust side, a plain object on the JavaScript side — and the result crosses as JSON too. The engine's `invoke(&mut self, cmd: &str, args: &Value) -> Result<Value>` is the whole contract: a command names an operation, the `args` object carries its parameters, and the returned `Value` is whatever that operation produces — an item, a list, a formatted string, a status object. There is no bespoke serialization, no binary framing, no schema negotiation; there is `serde_json` on both ends and a dotted string in the middle.

Errors ride the same rail without a parallel channel. Every engine defines a crate `Error` type (a `thiserror` enum), `invoke` returns `Result<Value, Error>`, and the Tauri command maps the error to a string so the front end gets Tauri's ordinary `Result<Value, String>` rejection. An unknown command is not a panic and not a silent empty result; it is `Error::UnknownCommand(cmd)` from the `match`'s catch-all arm, surfaced to the caller as a rejected promise. The uniformity is the point: a front end handles every failure from every engine the same way — a rejected `invoke` — because every engine funnels every failure through the one command's `Result`. The `zpdf` bridge even spells its catch-all out in the returned string, `"zpdf_invoke: unknown cmd \{other\}"`, so a typo in a command name fails loudly with the name that was not recognized.

Because the args are a JSON object rather than positional parameters, the contract is forgiving in exactly the way a fast-moving engine wants. A command can gain an optional field without breaking existing callers, drop a field it stopped needing, or accept a union of shapes, all without touching the bridge, because the bridge never inspects the object — it forwards it. The engine's `invoke` reads the fields it needs out of the `Value` and ignores the rest, which means the front end and the engine can evolve the shape of a command's arguments between releases as long as they agree on the fields that matter, with the bridge a pane of glass they both see through rather than a wall they both have to cut a matching hole in.

The event bridge: a sink, two transports, no polling

Request/response covers most of an engine's surface, but not the parts that happen *to* the app rather than *because* the app asked: a request finishes, mail arrives, a VPN reconnects, a container log emits a line. For those the engine holds an `EventSink = Box<dyn Fn(EngineEvent) + Send + Sync>` and calls it whenever something narratable happens, and the bridge's job is to route that sink to whatever the host understands. Under Tauri the setup helper wires the sink to `app_handle.emit(...)` a `WebView` event; under the C ABI the sink is a registered `extern "C"` callback. The engine calls one `emit`; the host decides what a sink means.

The event names follow the prefix. `zreq-core` emits over `"zrq-event"`, `ztunnel-core` over `"ztn-event"`, `zemail-core` over `"zml-event"`, `zcontainer-core` over its own event channel — and the JavaScript listens with Tauri's `listen(name, handler)`. The payload is the serialized `EngineEvent`, which each crate tags for the wire: `zreq-core`'s enum is `#[serde(rename_all = "snake_case", tag = "type")]`, so `RequestFinished { status, time_ms }` crosses as `{"type":"request_finished","status":200,"time_ms":42}`. The front end switches on `type` and updates the relevant view — a spinner clears, a timing appears, an inbox row lights up. Nothing polls; the engine narrates and the UI reacts, and the same `emit` call inside the engine feeds a C callback in a non-Tauri host with no change to the engine.

The set of events an engine emits is a small, honest catalogue of “what can happen without being asked,” and it differs by engine because the domains differ. `zreq-core` emits `RequestStarted { method, url }`, `RequestFinished { status, time_ms }`, and `Error { tag, message }` — the life of a request. `ztunnel-core` emits `Status { config_id, status }`, `Stats { config_id, bytes_in, bytes_out }`, and `Log { config_id, level, message }` — the life of a tunnel. `zemail-core` emits `SyncStarted`, `SyncFinished { fetched }`, `MessageSent { recipients }`, and `Error` — the life of a mailbox. And three engines emit *nothing*: `zoffice-core`, `zphoto-core`, and `zpdf-core` have no `EngineEvent` enum

at all, because a document, an image, and a PDF are computed on request and have no asynchronous life of their own. The presence or absence of an event channel is itself a fact about the domain, and the bridge carries it only where the domain has something to say.

The shared commands every shell also registers

The one engine command is not quite the whole bridge, because a conforming app owes its user more than its engine's domain: an embedded terminal (R3), a multi-pane file browser (R10), and a few genuinely platform-specific gestures. Those arrive as additional `#[tauri::command]`s that are *the same in every app*, because they come from shared crates rather than app code. The embedded terminal contributes `terminal_spawn`, `terminal_write`, `terminal_resize`, and `terminal_kill` — a real PTY behind four commands, from `zpwir-embed-terminal`. The file browser contributes the `fs_*` family from `zpwir-file-browser`, and it is large: `fs_list_dir`, `fs_list_subdirs`, `fs_get_info`, `fs_chmod`, `fs_grep`, `fs_hash`, `fs_compare_dirs`, `fs_find_duplicates`, `fs_git_status`, `fs_xattrs`, `fs_compress`, `fs_extract`, `fs_secure_delete`, `fs_read_file_base64`, `fs_open_terminal`, `fs_open_in_editor`, `fs_run_program`, and a dozen more, plus a `fb_watcher_set` for live directory watching. An app that embeds the browser registers the family; an app that does not, does not.

You can read the whole bridge of a real app in one `invoke_handler`. `zgo`'s `src-tauri/main.rs` registers `zgo_core::tauri_plugin::zgo_invoke` (the engine), then `zgo_hide`, `zgo_reveal`, and `zgo_trash` (the launcher's menu-bar gestures), then the four `terminal_*` commands, then the whole `fs_*` family — and that is the entire backend surface. The interesting thing is the ratio: one command is the engine, three are `zgo`-specific platform gestures, and the rest are shared infrastructure every app registers identically. The app-specific IPC of a launcher is *three commands*; everything else is either the engine (one command) or the family baseline (terminal plus file browser). The bridge is thin not as an aspiration but as a measurable fact — the backend of an app is one engine command, a handful of platform commands, and a block of shared registrations.

The app-specific commands are exactly the platform edges the engine may not touch, and each app has only the few it truly needs. `zgo` needs to hide and reveal its menu-bar window and to move a file to the trash, so it registers `zgo_hide`, `zgo_reveal`, `zgo_trash`; another app that opens a separate Monaco window for its hooks editor registers a command to spawn it; an app with a diagnostics panel registers the `open_log_file/open_log_dir/open_data_dir` trio the shell's settings panel calls. These are the genuinely unshareable operations — a window gesture, a trash call, an OS “reveal in finder” — and the bridge is where they live precisely because they are the residue left after everything computable has been pushed down into the engine and everything chrome-shaped has been pushed up into `zgui-core`.

The one exception: streaming through a second door

The single-command shape assumes every operation fits request/response, and one engine has operations that do not. `zcontainer` follows container logs, holds a `kubectl exec` session, and forwards a port — long-lived, often bidirectional channels that cannot be a `Value` returned once. Rather than distort `invoke` into something that could return a stream, `zcontainer-core` grows a *second* door: a subscription trio, `subscribe(cmd, args) -> u64`, `unsubscribe(id)`, and `stream_send(id, data)`, mirrored over the C ABI as `zcontainer_subscribe`, `zcontainer_unsubscribe`, `zcontainer_stream_send` and over Tauri as three more commands. `subscribe` opens a stream and returns a numeric handle; `stream_send` pushes input into it — the keystrokes of an exec, say; `unsubscribe` tears it down.

The stream's *output* does not need a new door, because the event sink already exists. `zcontainer-core`'s `EngineEvent` carries `Stream { id, data }`, `StreamEnd { id }`, and `StreamError { id, tag, message }` alongside the ordinary `Error`, so a log line or an exec output chunk flows back through the same sink and the same WebView-event/C-callback machinery every other engine's events use, keyed by the subscription id. The front end listening for `zcontainer` events routes a `stream` event to the panel that owns that id — a terminal for an exec, a log view for a follow — with the same listener it uses for a container-stopped

notification. The subscription trio added a way to *open* and *feed* a stream; delivering it reused the event channel that was already part of the bridge.

That asymmetry — one new door for opening and feeding streams, no new door for delivering them — is the bridge's design in miniature. The common case is one command and one event channel, and it covers nine engines completely. The one engine with genuinely streaming operations adds the smallest thing that makes streaming possible (a way to name a stream and push into it) and reuses the largest thing that already existed (the sink). The bridge did not become a general-purpose streaming framework; it grew exactly three functions on exactly the one engine that needed them, and left the other nine — and the whole request/response contract — untouched. That restraint is why “thin command bridge” survived contact with the hardest app in the fleet.

The memory rule and the one global engine

The bridge's memory story is deliberately the least surprising thing that could work, because a boundary is the wrong place for cleverness. On the C side, every `char*` the ABI hands back — from `<prefix>_invoke` and `<prefix>_init` — is a heap-allocated, NUL-terminated UTF-8 buffer the caller owns and must return to `<prefix>_string_free`; the engine never frees a string the caller still holds and the caller never frees a string with `free()` instead of the ABI's own deallocator. That single rule is the whole memory contract, and it is uniform across all nine C-ABI engines so a host learns it once and applies it everywhere. On the Tauri side the rule is invisible, because `serde_json` owns and drops the values, but it is the same discipline: the boundary hands ownership across explicitly and the receiver is responsible for release.

The engine being a single process-global instance is the other half of the simplicity, and it is a deliberate trade with eyes open. Each core constructs its `Engine` behind a `OnceLock<Mutex<Engine>>`, so `<prefix>_init` builds the one instance and every `<prefix>_invoke` locks it — which means a C or C++ host juggles no handles, tracks no lifetimes beyond the strings, and cannot accidentally hold two divergent engines. The cost is that you cannot run two isolated instances of the same engine in one process, which would matter if a host wanted two independent workspaces side by side; the benefit is a boundary a host uses correctly on the first try. For an “embed anywhere” promise, first-try correctness beats theoretical multi-instance flexibility, and the mutex-guarded global is the shape that buys it. The interesting surprises live in the domain logic, one layer down; the boundary is kept boring on purpose.

The dotted string as an API, and why it is not stringly-typed cheating

A reader steeped in type-driven design will flinch at the bridge's central move: every operation is a *string*, dispatched by a `match` on a dotted name like `"request.send"` or `"docker.container.inspect"`, with arguments as an untyped JSON `Value`. Is this not exactly the stringly-typed anti-pattern that a Rust program should refuse — trading the compiler's guarantees for a runtime lookup that fails on a typo? The objection is worth taking seriously, because the answer is what makes the single-command bridge defensible rather than lazy.

The dotted string is not the *implementation* of an operation; it is the *name* of one at the boundary, and the boundary is a place where a name is genuinely the right currency. The three hosts an engine serves — a Rust caller, a C caller, a JavaScript caller — do not share a type system, so there is no `enum Command` that all three could name; a C host has no Rust enum and a `WebView` has no Rust type at all. What all three *do* share is the ability to send a string and a blob of JSON, so the boundary speaks the one language common to every host. Inside the engine, the moment the string is matched, it is immediately turned into typed work: `request.send` deserializes its args into a typed request struct with `serde`, and from that point the code is as typed as any Rust program. The string lives for exactly one `match` arm and then vanishes into types. It is stringly-typed only at the one seam where every host is string-typed anyway.

There is a real cost, and the bridge pays it deliberately rather than pretending it away: a mistyped command

name is a runtime error, not a compile error, so the engine must return a clean “unknown command” result rather than panic, and the front end must be tested against the real dispatch rather than trust the compiler to have checked the names. The mitigations are concrete. The command names are enumerated in one place — the `match` in `Engine::invoke` — so the surface is auditable in a single file (Appendix A is that audit); the tests drive real command strings through real dispatch (Chapter 19), so a renamed command breaks a test rather than shipping; and the C++ RAII facades and the front-end wrappers keep the string literals in a small, reviewed set rather than scattered through call sites. The trade is the same one every network protocol makes — a wire format is stringly-typed at the wire and richly-typed at both ends — and the bridge is, in the end, a wire protocol between three hosts and one engine, so it makes the protocol's trade for the protocol's reasons.

Evolving a command vocabulary you cannot recall

The bridge is stable, but the vocabulary it forwards is not, and the question the single-command shape raises and does not answer is how a command name changes once consumers exist. This matters more here than in a typical library, because of a combination Chapter 13 makes explicit: the command names are the engine's public API, consumers are separate repositories that pin the engine by commit, and there is no package registry to yank a version from. A renamed command is a break that lands whenever each consumer chooses to bump.

The pointer cascade turns out to make this easier rather than harder, in a way that is worth noticing because so much else about the cascade is friction. Nothing breaks at the moment of the rename; a consumer sits on the pointer it tested against and keeps working indefinitely. The break happens at the bump, which is a deliberate act performed by someone who is already testing that app. That is the opposite of a registry-published library, where a rename is a semver decision that lands on consumers you cannot see and cannot coordinate with. Here the consumer set is enumerated in the matrix, the adoption is manual, and the cost of a rename is paid as a known number of bumps rather than as an unknown number of surprised downstreams.

What that argues for, operationally, is a specific shape of change. Additive changes are free: a new command arm is a new string old callers never send, so an engine can grow its surface without any consumer noticing, which is why the fleet's engines have accumulated large dispatches without breaking anything. Renames are cheap-but-not-free, and the cheapest form is the one that keeps the old arm dispatching to the same implementation alongside the new one — two strings, one body — so a consumer can bump the pointer and migrate its call sites in separate steps rather than in the same commit. Removals are the expensive form, and the pattern that makes them survivable is that the removal is a separate, later change, after the matrix's consumers have all been bumped past the overlap.

Argument shapes are the quieter half of the same problem and are handled by the contract rather than by process. Because arguments cross as a JSON object the bridge never inspects, and the engine reads the fields it needs and ignores the rest, a command can gain an optional field without breaking a caller that does not send it, or stop needing a field without breaking a caller that still does. That forgiveness is what lets the front end and the engine evolve between pointer bumps at all — the two artifacts are versioned separately (Appendix D's skew table is the evidence), so any pair of adjacent versions has to interoperate, and an argument contract that broke on an unexpected field would make that impossible. The looseness the type-driven reader flinches at in Chapter 4 is precisely the property that makes independent versioning survivable.

The one place the fleet has no mechanism and relies on discipline is *semantic* change: a command that keeps its name and its argument shape and changes what it does. Nothing in the contract detects that — not the compiler, which never saw the string; not the bridge, which forwards blindly; not the consumer's tests, which exercise the consumer's own expectations against whichever engine revision it pinned. The honest position is that this is the residual risk of a string-dispatched boundary, that the fleet's mitigation is the same enumeration that makes the surface auditable (the one `match`, Appendix A's map), and that a semantic change to a shared engine's command is the kind of edit that warrants being treated as a rename even when the name could technically stay. Where a boundary cannot detect a break, the only remaining instrument is to make the break visible in the one place every consumer already reads.

Chapter 5 – zreq: Rebuilding Postman

The API client. Workspaces, collections, and requests; environments, variables, and auth; HTTP execution with history; code generation; import/export. This chapter walks the port and the `zreq-core` request engine that other apps embed.

The Postman workflow, faithfully

Postman's mental model is a nested tree — a workspace holds collections, a collection holds folders and requests, a request holds a method, a URL, query params, headers, an auth block, and a body — and layered over that tree is a set of environments whose variables get substituted into requests at send time. `zreq` rebuilds exactly this model, and you can read it straight out of `zreq-core`'s `model.rs`: a `Workspace` holding `collections`, `environments`, `globals`, an `active_environment`, and `cookies`; a `Collection` holding `variables`, an `auth`, and `items`; an `Item` that is a tagged enum of `Folder` or `Request`. The names are Postman's names because the workflow is Postman's workflow.

The fidelity extends into the parts users actually fiddle with. A `Request` carries `query` and `path_variables` as parameter lists, `headers`, an `auth`, a `body`, and — crucially — `pre_request` and `tests` script fields, because Postman users script. The `Body` is a tagged enum over exactly Postman's body modes: `None`, `Raw` with a language, `FormData` with typed fields (text or file), `UrlEncoded`, `GraphQL` with a query and variables, and `Binary` with a path. A `zreq` user who knows Postman finds every affordance where they expect it, because the data model was built to match the predecessor field for field rather than to be a cleaner rethink of it.

The pitch, though, is not “Postman again.” `zreq`'s README leads with “no cloud account” — everything runs locally, the workspace is a file on disk, there is no sign-in and no server round-trip to open your own collections. That is a genuine difference from Postman's account-centric direction, and it is a difference the architecture makes free: because the engine is a local Rust library with a file-backed workspace (`store::default_workspace_path`, `load_workspace`, `save_workspace`), “local-first” is not a feature that had to be built, it is what you get when the engine is a library instead of a client for someone's cloud.

The port's coverage is measured rather than asserted. `zreq-core`'s `scripts/port_features.tsv` enumerates 83 Postman behaviours and scores the port at 92.2% — 72 full, 9 partial, 2 missing — with `full` reserved for features whose behaviour *and* GUI match. The two missing rows are honest and specific: a full JavaScript test sandbox (deliberately excluded, because embedding a heavyweight JS engine would violate the durable-deps rule) and per-host client TLS certificates. `zreq` does not claim to be a superset of Postman; it claims a precisely measured, conservatively scored 92.2% of it, minus two features it names, and that honesty is the same honesty the whole fleet applies to its predecessors.

zreq-core: the request/executing engine

The engine is a single `struct Engine` owning the workspace, the on-disk paths, the history, the settings, and an optional event sink, and its entire surface is one method: `invoke(&mut self, cmd: &str, args: &Value) -> Result<Value>`, a large match that dispatches dotted commands. The command list reads like Postman's verbs made explicit: `workspace.get/save/list/create/switch`, `collection.list/add/get/update/remove`, `request.get/add/update/remove/send`, `environment.list/add/remove/activate`, `history.list/replay/clear`, `codegen.generate`, `import.curl/postman/openapi/har`, `export.postman`. Every app feature is one of these strings, and the Tauri shell forwards all of them through the single `zrq_invoke` command.

Execution is split into a pure preparation phase and an impure send phase, and the split is the engine's cleanest idea. `http::prepare(request, effective_auth, scope)` builds a `PreparedRequest` — resolved URL, headers, encoded body, applied auth — with no network involved and no `net` feature required; `http::send(prepared, opts)` is the only part gated behind `net`, and it is what actually reaches `request`. Everything interesting about a request — how variables resolve, how auth signs, what the body serializes to — is decided in `prepare`, which means it is all testable without a socket and all reusable by the code generators, which render a `PreparedRequest` rather than re-deriving one.

History is a first-class part of the engine, not a UI convenience. Each send is recorded as a `HistoryEntry` — method, URL, status, `time_ms`, size, timestamp, and a snapshot of the request — into a `History` persisted alongside the workspace, and `history.replay` re-sends a past entry. This is why the `EngineEvent` enum carries `RequestStarted` and `RequestFinished { status, time_ms }`: the engine narrates its own execution so the shell can show a live spinner and a timing without polling. The narration crosses whichever boundary the host uses — a C callback or a `"zrq-event"` WebView event — from the same `emit` call inside the engine.

The engine also carries the parts of Postman that are not strictly HTTP: a `runner` that flattens a collection into a run list and can drive it over CSV or JSON data rows (`runner::flatten, parse_csv, json_rows`), a `script` module that implements a native subset of Postman's `pm.*` test API without a JS sandbox (`script::run` returning `TestResults`, `script::extract_sets` pulling out variable writes), and optional `ws` and `grpc` features for WebSocket and gRPC requests behind their own gates. The core is “the request/execution engine” in the full Postman sense — collection runner, test assertions, and protocol reach included — kept behind feature flags so a host embeds only what it needs.

Environments, variables, and auth resolution

Variable resolution is where Postman's model gets subtle, and `zreq-core`'s `vars.rs` matches the subtlety. A `Scope` is built from globals, the active environment, and the collection (`Scope::build(globals, env, collection)`), with precedence collection over environment over global, and `Scope::render` substitutes `{{name}}` tokens in any string. The precedence is Postman's precedence, so a collection-level variable shadows an environment variable of the same name exactly as a Postman user would expect, and the whole resolution is a pure function over three inputs, which is why it is trivially testable.

The dynamic variables — Postman's `{{$....}}` generators — are implemented too, and the list is faithful: `{{$guid}}` and `{{$randomUUID}}` produce a UUID v4, `{{$timestamp}}` and `{{$isoTimestamp}}` the time, `{{$randomInt}}` a bounded integer, and a catalog of `{{$random*}}` fakers (`randomFirstName`, `randomEmail`, `randomIP`, `randomHexColor`, `randomMACAddress`, and more) backed by small static word lists in the crate. None of this needs a dependency; it is a few hundred lines of owned generators, which is the durable-deps rule choosing to hand-roll the small thing rather than import the large one.

Auth resolution is the richest corner, because Postman's auth is inheritable and some schemes need the fully resolved request to sign. `zreq-core`'s `Auth` enum defaults to `Inherit`, and the resolution walks request → folder → collection until it finds a concrete scheme (`resolve_auth, resolve_in`). The simple schemes — `None`, `Basic`, `Bearer`, `ApiKey`, `OAuth2` token-attach — apply in `auth::apply(auth, scope)` before the request is even fully built. The signature schemes need more: `auth::sign(auth, scope, ctx)` dispatches on the `Other { scheme }` variant to `sign_digest` (RFC 2617), `sign_oauth1` (RFC 5849), `sign_aws_v4` (AWS Signature v4), and `sign_hawk`, each of which needs the resolved method, URL, and body to compute its signature.

Those signers are the durable-deps rule at its most striking. HTTP Digest, OAuth 1.0a, AWS Signature v4, and Hawk all need MD5, SHA-1, SHA-256, and HMAC — and rather than pull a crypto crate, `zreq-core` hand-rolls those primitives in `src/crypto.rs` and builds the four signers on top. It is a deliberate trade: a few hundred lines of owned, audited hashing instead of a supply-chain dependency for four auth schemes. The OAuth2 token flow is likewise owned — `auth::oauth2_token_form` builds the token-endpoint request for the client-credentials, password, and refresh-token grants, driven by the `oauth2.token` command. Postman's auth surface, rebuilt to the RFCs, with its cryptography written rather than imported.

Code generation and collection import/export

Code generation is a pure transform from a `PreparedRequest` to a snippet, and `codegen::generate(target, prepared)` fans out to a dozen real targets: `curl`, `http` (raw), `fetch`, `python` (requests), `httpie`, `go`, `java`, `csharp`, `php`, `ruby`, and `node`. Because it consumes a `PreparedRequest` — the same one send consumes — the snippets are guaranteed to match what `zreq` would actually send: the resolved URL, the applied auth header, the encoded body. And because preparation needs no network, code generation is a no-`net` operation, testable by preparing a request and asserting on the generated string, which is exactly how it is tested.

Import is the mirror image — foreign formats parsed into `zreq`'s own model — and the format list is Postman's interoperability surface: `import::import_curl` turns a curl command line into a `Request`, `import_postman` reads a Postman Collection v2.1 into a `Collection`, `import_openapi` turns an OpenAPI 3 document into a collection of requests, and `import_har` reads an HAR 1.2 capture. Each returns `zreq`'s native types with fresh ids assigned, so an imported collection is indistinguishable from one built by hand. Export closes the loop: `import::export_postman` writes a full Postman v2.1 collection back out, bodies, auth, and scripts included, so a `zreq` collection round-trips to and from the tool it was ported from.

The interoperability is the quiet proof that the model is faithful. You can only round-trip a Postman collection through `zreq` and back if `zreq`'s model is a superset of the information in that collection — every folder, every body mode, every auth scheme, every script — and the fact that `export_postman` can reconstruct a valid v2.1 document means the port did not lose anything on import. Fidelity is not a claim here; it is a testable property, and the import/export pair is the test.

The frontend for all of this is the mountable webui `zreq-core` ships in its own `webui/` directory, `zreq.{html,css,js}`, copied into the app's `frontend/` at build time. It auto-detects its transport — under Tauri it calls `invoke("zrq_invoke", { cmd, args })` and listens for `"zrq_event"`; in a plain browser it falls back to an in-memory mock so the view renders in preview mode with no backend. The view is built entirely from `ZGui.*` chrome — `ZGui.tabs.init` for the independent sidebar/editor/response strips, `ZGui.tiles` for the dashboard, `ZGui.formatBadge` for the HTTP verb chips, `ZGui.fzf.fzfMatch` for the fuzzy sidebar filter, `ZGui.modal` and `ZGui.toast` for dialogs — with CSP-safe fallbacks for each when `ZGui` is somehow absent. It is the reference example of the whole front-end contract: engine JSON in, `ZGui` widgets out, no bespoke chrome.

The crypto, written not imported, walked signer by signer

The durable-deps decision to hand-roll cryptography is the kind of claim that deserves the receipts, so here they are. `zreq-core/src/crypto.rs` exports exactly six functions and nothing more: `md5(input) -> [u8; 16]`, `sha1(input) -> [u8; 20]`, `sha256(input) -> [u8; 32]`, `hmac_sha1(key, msg) -> [u8; 20]`, `hmac_sha256(key, msg) -> [u8; 32]`, and `hex(bytes) -> String`. Three digests, two HMACs over those digests, and a hex encoder — the complete set of primitives the four auth signers need, owned in a few hundred lines rather than pulled from a crypto crate. This is the whole cryptographic dependency surface of an app that signs requests four different ways: zero external crates.

The four signers consume those primitives in `auth.rs`, and each maps to a real specification. `sign_digest` implements HTTP Digest access authentication (RFC 2617) — it needs `md5` for the `HA1/HA2`/response hashes and the `hex` encoder for the wire form. `sign_oauth1` implements OAuth 1.0a (RFC 5849) — it needs `hmac_sha1` over the signature base string. `sign_aws4` implements AWS Signature Version 4 — it needs `sha256` for the canonical-request and string-to-sign hashes and a chain of `hmac_sha256` calls to derive the signing key from the secret, date, region, and service. `sign_hawk` implements Hawk — `hmac_sha256` over the request MAC. Each signer takes the fully resolved request context (`SignContext` carrying method, URL, and body) plus a timestamp, because a signature is over the concrete request, which is why signing happens after `http::prepare` has built the `PreparedRequest` and not before.

The dispatch is worth seeing because it shows how the inheritable-auth model meets the signers. Auth

inheritance — request to folder to collection — is resolved inside the engine before signing; the resolved `Auth` is then handed to `auth::apply` for the simple schemes (`None`, `Basic`, `Bearer`, `ApiKey`, `OAuth2` token-attach, applied by writing a header) and to `auth::sign` for the signature schemes, which matches the lowercased scheme name — `"digest"`, `"oauth1"`, `"awsv4"`, `"hawk"` — and calls the corresponding function, returning `Error::FeatureDisabled` for anything it does not recognize. The OAuth2 token flow rounds it out: `auth::oauth2_token_form` builds the token-endpoint form for the client-credentials, password, and refresh grants, driven by the `oauth2.token` command. Postman's entire auth surface, rebuilt to the RFCs, standing on six hand-written primitives — that is the durable-deps rule at its most literal, and the tests exercise every signer without a network because signing is pure computation over a prepared request.

The command surface, read as a map of Postman

Because the engine's public API is its command strings, the cleanest way to see how completely `zreq` covers Postman is to read the dispatch. The workspace verbs are the tree operations — `workspace.get`, `workspace.save`, `workspace.list`, `workspace.current`, `workspace.create`, `workspace.switch`, `workspace.rename`, `workspace.delete` — and the collection and request verbs walk down it: `collection.list/add/get/update/remove`, `request.get/add/update/remove/send`. The environment and variable layer is `environment.list/add/remove/activate`, `globals.get/set`, and `vars.resolve`, which renders `{{name}}` tokens and the `{{$.}}` dynamic generators against a resolved scope. History is `history.list/replay/clear`, cookies are `cookie.list/set/clear`, settings are `settings.get/update/reset/path`, and the collection runner is `runner.run`. Every one of these is a `match` arm the single `zrq_invoke` forwards.

The interoperability commands are where the fidelity claim becomes testable. `codegen.generate` renders a `PreparedRequest` to a snippet, dispatching on eleven exact target strings — `"curl"`, `"http"`, `"fetch"`, `"python"`, `"httpie"`, `"go"`, `"java"`, `"csharp"` (also `"c#"`), `"php"`, `"ruby"`, and `"node"` (also `"nodejs"`). Import is four foreign formats parsed into `zreq`'s own model — `import.curl`, `import.postman`, `import.openapi`, `import.har`, backed by `import.curl`, `import.postman`, `import.openapi`, and `import.har` — and `export.postman` writes a full Postman v2.1 collection back out via `export_postman`. Protocol reach is behind feature gates as `oauth2.token`, `ws.exchange` (the `ws` feature), and `grpc.call` (the `grpc` feature). The reason this list is worth printing is that it is the port report in another form: 83 enumerated Postman behaviours, 72 scored `full`, and the commands above are where each of those behaviours lives, one dotted string at a time.

Prepare, send, and the runner: the pure/impure seam in the request path

The engine's cleanest idea is that a request has a pure half and an impure half, and the module boundary falls exactly between them. `http::prepare(request, effective_auth, scope)` builds a `PreparedRequest` — the resolved URL, the rendered headers, the encoded body, the applied auth — with no network and no `net` feature required, leaning on the crate's own `percent_encode/percent_decode` and `raw_content_type` helpers to serialize a body faithfully. Only `http::send(prepared, opts)` is gated behind `net`, and it is the sole function that touches `request`. Everything a user would call “how the request behaves” is decided in `prepare`, which is why the whole preparation path — variables, auth signing, body encoding — is testable with `--no-default-features` and no socket in sight.

That seam is also why code generation is guaranteed faithful. `codegen::generate` consumes the *same* `PreparedRequest` that `send` consumes, so a curl or Python or Go snippet renders the resolved URL, the applied auth header, and the encoded body that `zreq` would actually put on the wire — never a re-derived approximation. Prepare a request, generate the snippet, assert on the string: the test needs no network because the interesting computation already happened in the no-`net` half. The same property makes `import/export` a testable fidelity check rather than a claim — a Postman collection that round-trips through `import_postman`

and `export_postman` and comes back intact proves the model lost nothing, and the round-trip runs with no socket.

The engine also carries the parts of Postman that are not strictly one request. The `runner` module flattens a collection into a run list and drives it over CSV or JSON data rows (`runner::flatten`, `parse_csv`, `json_rows`), exposed as `runner.run`. The `script` module implements a native subset of Postman's `pm.*` test API without embedding a JavaScript engine — `script::run` returns `TestResults` and `script::extract_sets` pulls out the variable writes a test performs — which is the durable-deps rule declining to bundle a heavyweight JS sandbox for the test surface (the one Postman feature `zreq` marks deliberately out of scope). And the protocol reach beyond HTTP is feature-gated to keep the default build lean: `ws.exchange` behind the `ws` feature (`tungstenite`) and `grpc.call` behind `grpc` (`proton`, `prost`, `prost-reflect`). A host embeds only the protocols it needs, and the tests run the whole preparation and generation surface with none of them.

History as a first-class part of the engine

History in `zreq` is not a UI convenience layered over the request view; it is part of the engine, which is why `history.list`, `history.replay`, and `history.clear` are `invoke` commands rather than front-end state. Each send records a `HistoryEntry` — the method, URL, status, `time_ms`, size, timestamp, and a snapshot of the request — into a `History` persisted alongside the workspace, so history survives a restart and travels with the file the way the collections do. `history.replay` re-sends a past entry from its snapshot, which means “run that request again” is an engine operation over stored data, not a reconstruction the UI has to attempt.

Putting history in the engine is what makes the request narration coherent. The `EngineEvent` enum carries `RequestStarted { method, url }` and `RequestFinished { status, time_ms }` precisely because the engine is the thing that knows a request began and ended — it recorded the `HistoryEntry`, so it emits the matching events, and the shell shows a live spinner and a timing without polling. A design that kept history in the front end would have to invent a parallel notion of “a request finished” for the UI while the engine stayed silent; keeping history in the engine means one source of truth narrates its own execution, and the same `emit` that updates the app's history view feeds a C host's callback. It is a small feature that demonstrates the whole boundary: the interesting state lives one layer down, where it is persisted, testable, and embeddable, and the shell renders what the engine remembers.

Dynamic variables and the faker catalogue

Postman's `{{$....}}` dynamic variables are a small feature that shows the durable-deps rule choosing to hand-roll rather than import, and `zreq-core`'s `vars.rs` implements a generous catalogue of them in `Scope::dynamic`. The identity and time generators are there — `{{$guid}}` and `{{$randomUUID}}` produce a UUID v4, `{{$timestamp}}` and `{{$isoTimestamp}}` the time, `{{$randomInt}}`, `{{$randomBoolean}}`, `{{$randomAlphaNumeric}}` — and beyond them a faker catalogue: people (`randomFirstName`, `randomLastName`, `randomFullName`, `randomUserName`, `randomJobTitle`), internet (`randomEmail`, `randomUserAgent`, `randomDomainName`, `randomUrl`, `randomIP`, `randomMACAddress`, `randomPort`), location (`randomCity`, `randomCountry`, `randomCountryCode`), commerce (`randomPrice`, `randomCurrencyCode`, `randomColor`, `randomHexColor`, `randomPhoneNumber`), and text (`randomLoremWord`, `randomLoremWords`, `randomLoremSentence`). An unknown generator returns an empty string, leaving no half-substituted token on the wire.

None of that needs a dependency. The random values come from a small xorshift64 RNG (`next_rand`) and the fakers draw from static word lists compiled into the crate, so a few hundred lines of owned generators replace a faker dependency for a feature that is, after all, just “produce a plausible fake value.” It sits inside the same `Scope::render` that substitutes ordinary `{{name}}` variables, so dynamic and static variables resolve in one pass with the same precedence rules, and the whole thing is pure — a `Scope` rendered against inputs, testable by asserting that `{{$randomEmail}}` looks like an email and `{{$guid}}` parses as a UUID. It is the

durable-deps rule at its smallest and most characteristic: the feature is real and complete, and the dependency count it added is zero.

Chapter 6 – zcite: Rebuilding Zotero

The reference manager. Library, collections, tags, and saved searches; citations and bibliographies in APA / MLA / Chicago / IEEE; BibTeX / RIS / CSL-JSON import-export; DOI / ISBN / PMID lookup; duplicate detection.

The library model and saved searches

Zotero's library is a typed catalog: every item is one of dozens of reference types — journal article, book, book section, conference paper, thesis, report, preprint, patent, dataset, and on — and each type has its own fields with a base-field system that renames shared fields per type. `zcite-core` rebuilds this precisely. Its `schema.rs` defines all 37 Zotero reference types with their field sets and creator roles (author, editor, translator, contributor, series editor, book author), and functions like `item_type`, `base_field_name`, and `base_field_to_csl` implement the base-field renames and the mapping into CSL variables. An `Item` in `model.rs` carries its `item_type`, its `fields`, its `creators`, its notes and attachments, and a deleted flag for the trash.

The container structure is Zotero's too. A `Library` holds items and is partitioned into My Library plus group libraries (`LibraryKind`), `Collections` nest, tags carry colours (`TagColor`), items relate to one another, and there is a trash with restore. Creators are modeled with the full Zotero nuance — a `Creator` is either a person or an institution, with `display_last_first` and `display_first_last` renderings and an `is_institutional` flag — because a citation formatter that cannot tell “Smith, J.” from “World Health Organization” gets bibliographies wrong. The model is built to carry everything a Zotero library carries, because the citation engine downstream needs all of it.

Saved searches are Zotero's condition-based smart collections, and `zcite-core` models them as data rather than code. A `SavedSearch` is a set of `Conditions`, each a field, a `ConditionOp` (contains, is, before, after, and so on), and a value, with a `MatchMode` for all-versus-any. `search.rs` evaluates them: `condition_matches` tests one condition against one item, `saved_matches` combines them per the match mode, and `saved_run` filters the library. Because a saved search is data, it persists in the library file and re-runs on every change, exactly like Zotero's smart collections, and exactly like `zreq`'s environments it is a pure predicate over items that a test can exercise directly.

Quick search — the omnipresent filter box — is the same machinery pointed at every field at once. `search::haystack(item)` flattens an item into searchable text (including indexed attachment text and HTML), `quick_matches` tests a query against it, and `quick_run` filters. In the frontend this is wired to `ZGui.fzf` so the library filter is fuzzy with highlighted matches like every other filter in the fleet (R7), but the underlying “what text does this item expose to search” decision is the engine's, so the same haystack backs both the quick filter and the condition operators that test “title contains” or “any field contains.”

Citation styles and the CSL engine

Citations are `zcite`'s reason to exist, and the engine implements them at two levels. The first is a hand-written formatter in `bib.rs` for the four styles users ask for by name: a `Style` enum of `Apa`, `Mla`, `Chicago`, `ChicagoNote`, and `Ieee`, with `bibliography(item, style)` producing a reference-list entry, `in_text(item, style, index)` producing an in-text citation, and `cluster_ref` for grouped citations. The labels are the real ones — “APA 7th edition”, “MLA 9th edition” — because the whole value of a reference manager is getting the style exactly right, and hand-writing the four most-used styles is how you guarantee they match without depending on a stylesheet being present.

The second level is a general CSL 1.0 processor in `cs1.rs`, and this is the part that makes `zcite` a real Zotero rather than a four-style toy. `Style::parse(src)` reads an arbitrary `.cs1` stylesheet — its macros, its citation and bibliography layouts, its terms, its sort keys — and `render_bibliography`, `render_citation`, and `render_document` format items against it. Because the base-field-to-CSL mapping lives in the schema, any of the thousands of published CSL styles can be loaded and applied, which is exactly Zotero's model: the four built-ins are the fast path, and the CSL engine is the escape hatch to every other journal's house style.

BibTeX cite-key generation is a small but telling piece of the same module. `bib.rs` carries `cite_key(item)`, which builds the `author-year` style key LaTeX users expect, because a reference manager that exports to BibTeX without sensible keys is useless to the LaTeX users who are half its audience. It is the kind of detail that only exists if the port took the predecessor's *workflow* seriously and not just its data model — the difference between “we can emit BibTeX” and “we emit BibTeX the way someone writing a paper needs it.”

The whole citation surface is reachable through the same `invoke` dispatch as everything else: `bib.bibliography`, `bib.citation`, `bib.cite_key`, `bib.cs1`, `bib.cs1_document`, `cite.document`. A host — `zcite` itself, or a sibling app that embeds `zcite-core` to add “cite this” to its own documents — calls one of those strings with an item and a style and gets back a formatted string. The engine is, in the embed-matrix sense, the fleet's citation service, and Chapter 13 returns to what it means for `zoffice` or `zemail` to embed a citation formatter the same way they embed a PDF viewer.

Metadata lookup: DOI, ISBN, PMID

Nobody types bibliographic metadata by hand if they can help it, and Zotero's signature move is “paste an identifier, get a fully populated item.” `zcite-core`'s `identifier.rs` implements it. An `IdKind` enum classifies a raw string as `Doi`, `Isbn`, `Pmid`, `Arxiv`, or `Unknown`, and `detect(raw)` both classifies and normalizes — stripping `arXiv:` prefixes and version suffixes, recognizing the `10.x/` DOI shape, handling ISBN-10 and ISBN-13, spotting a bare numeric PMID. The detection is pure and testable; the fetch is the part that needs the network.

`lookup(raw)` does the fetch, and it is feature-gated exactly like every other network operation in the fleet: behind the `net` feature it resolves the identifier against the right service — CrossRef for DOIs, Open Library for ISBNs, NCBI for PMIDs — and returns a populated `Item`; with `--no-default-features` the same function is a stub that returns an error. So the classification and normalization logic is tested without a network, and only the actual HTTP call needs a live service. The commands `identifier.detect`, `identifier.lookup`, and `identifier.add` expose all three steps — classify, fetch, insert — so a UI can show what it detected before it commits to fetching.

There is a second acquisition path that is pure cleverness over the same machinery: retrieve metadata from a PDF. `pdf.rs` extracts text from an attached PDF (`extract_text`) and scans it for a recognizable identifier (`find_identifier`) — a DOI, an arXiv id, an ISBN printed in the text — and then runs the normal lookup. A user drops in a paper's PDF and `zcite` fills in the citation by reading the identifier off the page and resolving it, which is one of Zotero's most-loved features rebuilt out of the text extractor plus the identifier detector the engine already had.

Beyond identifier lookup, the engine carries the rest of Zotero's acquisition surface: pulling from a Zotero account over the Web API (`zotero.rs`, with `pull/push/verify` and an `item_to_zotero` mapper) and syncing attachments over WebDAV (`webdav.rs`, a `WebDav` client with `put/get/delete`). Both are `net`-gated with no-net stubs, so a `zcite` built for tests or for pure local use compiles and runs without them, and a `zcite` built for a user who lives in the Zotero ecosystem gets full account and attachment sync. The pattern is the fleet's pattern: the domain logic mandatory, the network optional, the seam a feature flag.

zcite-core as the citation engine other apps embed

`zcite-core` is a first-class member of the embed fleet, and its shape is the fleet shape: crate `zcite-core` (lib `zcite_core`) building `rlib`, `staticlib`, and `cdylib`; a `struct Engine` with `invoke`; a four-function C ABI

in `src/ffi.rs` with prefix `zct_` (`zct_init`, `zct_invoke`, `zct_string_free`, `zct_set_event_callback`) and a hand-written header in `include/zcite_core.h` plus a C++ facade; and a Tauri plugin exposing the single `zct_invoke` command. The whole ~100-command surface — library, schema, items, collections, tags, saved searches, citations, imports, exports, lookups, duplicates — funnels through that one entry point.

What makes `zcite-core` interesting as an *embedded* engine is that its output is text every other document app wants. A citation is a formatted string; a bibliography is a list of formatted strings; a CSL render is a document fragment. `zoffice` writing a paper wants to insert a formatted citation; `zemail` composing a message wants to drop in a reference; `zpdf` annotating a paper wants to cite it. In principle any of them can embed `zcite-core` and call `bib.citation` or `cite.document` to get exactly the string Zotero would produce, mounting `zcite`'s own view live through `ZGui.embed.view` so the citation picker reads as part of the host. The engine was built GUI-free precisely so it can be the fleet's citation service, not just `zcite`'s backend.

The import/export breadth reinforces the point. `zcite-core`'s `import.rs` and `export.rs` handle BibTeX, RIS, and CSL-JSON, and `xml_formats.rs` adds the XML interchange formats — EndNote XML, MODS, MARCXML, Zotero RDF — as paired parse/serialize functions built on `quick-xml`. An app embedding `zcite-core` inherits all of that: not just “format a citation” but “read this library's worth of references in any of seven formats.” The engine is a complete bibliographic toolkit behind one `invoke`, which is what “embeddable citation engine” has to mean to be worth embedding.

Duplicate detection rounds out the toolkit, and it is a small model of the whole engine's testable purity. `duplicates.rs`'s `duplicate_sets(items)` buckets items by a priority key — shared normalized DOI first, then shared ISBN, then normalized title plus first-author last name — excluding trashed items and returning only buckets of two or more. It is a pure function from a list of items to a list of duplicate groups, with no network, no UI, and no state, and the `duplicates.list` and `duplicates.merge` commands expose detection and resolution. Like the saved-search evaluator and the variable resolver in `zreq`, it is the kind of self-contained, data-in-data-out logic that the zero-GUI rule is designed to make trivial to get right and trivial to test.

The 37 types and the base-field bridge into CSL

The claim that `zcite` ports Zotero's type system is exact, and the crate asserts it with a test. `schema.rs` defines `ITEM_TYPES` as thirty-seven reference types, and a unit test named `all_reference_types_ported` fails if the count drifts from 37. The list is Zotero's: `journalArticle`, `book`, `bookSection`, `conferencePaper`, `thesis`, `report`, `magazineArticle`, `newspaperArticle`, `webpage`, `manuscript`, `preprint`, `encyclopediaArticle`, `dictionaryEntry`, `document`, `patent`, `dataset`, `computerProgram`, `blogPost`, `audioRecording`, `videoRecording`, `radioBroadcast`, `tvBroadcast`, `podcast`, `film`, `artwork`, `map`, `letter`, `interview`, `presentation`, `email`, `instantMessage`, `forumPost`, `standard`, `case`, `statute`, `bill`, and `hearing`. Note (attachment, annotation) live on the item rather than as reference types, exactly as in Zotero.

The subtle part is the base-field system, and `schema.rs` carries the two functions that make citations correct across types. `base_field_name(item_type, base)` implements Zotero's per-type field renames — the field a book calls `publisher` a thesis calls `university` and a patent calls `assignee`, all the same base field under the hood — and `base_field_to_csl(field)` maps a base field to its CSL variable. Together with `is_field_valid`, `ItemTypeDef::all_fields`, and `primary_creator`, they are the bridge that lets the citation layer treat 37 kinds of thing uniformly: the formatter asks for a CSL variable, the schema resolves it through the base-field renames for that specific type, and the right string comes out whether the item is a journal article or a court case. A citation engine that skipped the base-field layer would format half the types wrong; `zcite` carries it because getting “World Health Organization” and “Smith, J.” and a statute's fields each right is the entire job.

The bibliographic toolkit: two citation levels and seven interchange formats

The command surface makes plain that `zcite-core` is a complete bibliographic toolkit rather than a citation formatter with a library attached. Citations come at two levels through `bib.bibliography`, `bib.citation`, `bib.cite_key`, `bib.csl`, `bib.csl_document`, and `cite.document`: the hand-written formatter behind the first few (the `Style` enum of `Apa`, `Mla`, `Chicago`, `ChicagoNote`, `Ieee`, with `bibliography`, `in_text`, `cluster_ref`, and `cite_key`) and the general CSL 1.0 processor behind `bib.csl/bib.csl_document` (a separate parsed-stylesheet `Style` with `parse`, `render_bibliography`, `render_citation`, `render_document`). The two share the schema's base-field-to-CSL mapping, so the four built-ins are the fast path and any of the thousands of published `.csl` styles is the escape hatch.

Interchange is where the “real Zotero” claim is cashed, and the format list is seven wide in each direction. Import parses `import.bibtex`, `import.ris`, `import.csl_json` (via `import::parse_bibtex/parse_ris/parse_csl_json`) plus the four XML formats `import.endnote_xml`, `import.mods`, `import.marxml`, `import.zotero_rdf` (via `xml_formats::parse_endnote/parse_mods/parse_marxml/parse_zotero_rdf`), and export mirrors all seven — `export.bibtex`, `export.ris`, `export.csl_json`, `export.endnote_xml`, `export.mods`, `export.marxml`, `export.zotero_rdf`. Around that sit the library and item verbs (`library.*`, `libraries.*`, `item.*` with its `set_field/set_rating/trash/restore/relate` operations, `collection.*`, `tag.*`), the acquisition path (`identifier.detect/lookup/add`, `pdf.extract_text/recognize`, `zotero.pull/push`, `webdav.upload/download`), annotations (`annotation.add/list/update/remove`), search (`search.quick`, `search.saved.*`), and `duplicates.list/merge`. The whole ~100-command surface funnels through the one `zct_invoke`, and every one of those strings is a Zotero affordance rebuilt as a testable, embeddable operation.

Acquisition and dedup: the pure detector, the gated fetch

Zotero's signature move is “paste an identifier, get a populated item,” and `zcite-core` splits it into a pure detector and a gated fetch so the smart part is testable without a network. `identifier::detect(raw) -> (IdKind, String)` classifies and normalizes in one step — recognizing the `10.x/` DOI shape, ISBN-10 and ISBN-13, a bare numeric PMID, an arXiv id with its `arXiv:` prefix and version suffix stripped, or `Unknown` — and it is pure, so the classification is tested directly on strings. `identifier::lookup(raw)` is the fetch, and behind the `net` feature it dispatches on the detected kind — `Doi` to `lookup_doi` (CrossRef), `Isbn` to `lookup_isbn` (Open Library), `Pmid` to `lookup_pmid` (NCBI), `Arxiv` to `lookup_arxiv` — returning a populated `Item`; with `--no-default-features` the same function is a stub. Detect, fetch, insert are the three commands `identifier.detect/lookup/add`, so a UI can show what it recognized before it commits to the network.

There is a second acquisition path that is pure cleverness over machinery the engine already has: `pdf.rs` extracts text from an attached PDF (`extract_text`, behind the `pdf/lopdf` feature) and scans it for a recognizable identifier, then runs the normal lookup — so dropping in a paper's PDF fills the citation by reading the DOI off the page. Beyond identifiers, the engine carries the rest of Zotero's acquisition surface behind the same `net` gate: pulling from a Zotero account over the Web API (`zotero.rs`, `zotero.pull/push/verify`) and syncing attachments over WebDAV (`webdav.rs`, `webdav.upload/download/delete`), each with a no-net stub so a local-only or test build compiles without them.

Duplicate detection closes the toolkit and is a model of the engine's testable purity. `duplicates::duplicate_sets(items) -> Vec<Vec<String>>` buckets items by a priority key — shared normalized DOI first, then shared ISBN, then normalized title plus first-author last name — excluding trashed items and returning only groups of two or more. It is a pure function from a list of items to a list of id-groups, with no network, no UI, and no state, exposed as `duplicates.list` and resolved by `duplicates.merge`. Like the saved-search predicate (`search::saved_matches`) and the quick-search haystack (`search::haystack/quick_matches`), it is exactly the data-in-data-out shape the zero-GUI rule is designed to make trivial to test — and the reason `zcite`'s 95.0% coverage over 60 scored features is a test plan rather than a brochure.

Items, notes, annotations, and libraries: the CRUD surface

Underneath the citation formatter and the interchange formats is the ordinary library CRUD that a reference manager lives in day to day, and reading the `item.*` verbs shows how much Zotero nuance the model carries. An item is created and edited through `item.add`, `item.update`, and the granular `item.set_field`, and it carries the reading-workflow state a modern manager tracks — `item.set_reading`, `item.set_rating`, `item.set_favorite` — plus the trash lifecycle `item.trash`, `item.restore`, `item.delete` that keeps a deleted item recoverable until it is purged. Tags attach and detach with `item.add_tag/item.remove_tag` (and tags carry colours through `tag.set_color`), items relate to one another with `item.relate`, and notes ride on the item via `item.add_note/update_note/remove_note` as well as the standalone `note.*` family.

Attachments and annotations are first-class, because a real library is as much PDFs and highlights as it is metadata. `attachment.store_file`, `attachment.snapshot`, and `attachment.index` bring a file into the library and index its text (which is what the quick-search haystack later searches), and `annotation.add/list/update/remove` model the highlights and notes a user makes on an attached PDF. Libraries themselves are plural and partitioned — `libraries.list/active/create/rename/switch/remove` manage My Library and group libraries as `LibraryKinds` — and collections nest with `collection.add/rename/remove/add_item/remove_item`. Every one of these is a `match` arm behind the one `zct_invoke`, and every one operates on a plain data model that a test constructs on a temp profile path and asserts on directly, which is why the whole ~100-command surface is exercised by a single build-invoke-compare harness with no window in sight.

The frontend contract, seen in `zcite`'s view

`zcite`'s frontend is the reference example of the whole front-end contract, and its own source says so: `zcite.js`'s header comment states outright that all UI chrome in the view is built from `window.ZGui.*`. The view is engine JSON in, ZGui widgets out — the library is a `ZGui.table` with sortable, resizable columns (R8) and a fuzzy filter wired to `ZGui.fzf` so matches highlight (R7); the item editor is ZGui form controls; dialogs are `ZGui.modal` and `ZGui.toast`; tabs are `ZGui.tabs`. The app-specific JavaScript is the glue between what `zct_invoke` returns and what those widgets render, and nothing more — no bespoke table, no hand-rolled filter, no parallel toast.

The transport underneath the view is auto-detected, which is the seam Chapter 3's two-substrates rule depends on. Under Tauri the view calls `invoke("zct_invoke", { cmd, args })` and listens for the engine's events; in a plain browser it falls back to an in-memory mock so the view renders in a preview with no backend at all; and under JUCE it would speak the same commands over `window.Juce`. That detection is why the mountable `webui/` a core ships can be developed in a browser, tested against a mock, embedded in a sibling app, and shipped in the standalone shell without changing a line of view code — one view, one command vocabulary, three hosts. `zcite`'s view is the clearest single answer to “what does an app's frontend actually contain”: a mount of ZGui chrome, a transport shim, and the glue that turns the citation engine's JSON into the widgets the whole fleet shares.

Chapter 7 — ztunnel: Rebuilding Tunnelblick

The VPN tunnel manager. OpenVPN and WireGuard connections, configs, process control, logs, stats, and credentials — one owned, cross-platform desktop app.

OpenVPN and WireGuard under one model

Tunnelblick is an OpenVPN GUI, and a faithful port would stop there — but `ztunnel` takes Tunnelblick's model and widens it, and the widening is the chapter's story. The connection model in `ztunnel-core`'s `model.rs` is protocol-agnostic: a `Config` carries a `backend`, a `source`, the `raw` text, parsed `fields`, and per-config `options`, and the `Backend` enum spans fourteen variants — `OpenVpn`, `WireGuard`, `IkeV2`, `L2tp`, `Pptp`, `Sstp`, `OpenConnect`, `GlobalProtect`, `Shadowsocks`, `V2Ray`, `Trojan`, `SoftEther`, `Tailscale`, `ZeroTier`. One model, many protocols, where Tunnelblick had one.

The honesty about *how* each backend runs is built into the type system, which is unusual and worth dwelling on. `Backend::launch_kind()` returns a `LaunchKind` — `Native`, `ManagedProcess`, `OsNative`, or `Coordinated` — that names the provenance of each protocol's data path. WireGuard is `Native`: `ztunnel` owns the data path in pure Rust. OpenVPN is `ManagedProcess`: `ztunnel` drives the system `openvpn` binary, exactly as Tunnelblick does. IKEv2 and L2TP are `OsNative`: they lean on the OS's own VPN stack. Tailscale and ZeroTier are `Coordinated`: they are managed alongside their own daemons. The enum does not pretend every protocol is implemented the same way; it records, in the type, which are owned and which are driven, which is the same inherited-versus-original honesty the whole book insists on, encoded as a method.

OpenVPN is the faithful Tunnelblick port, and it is done Tunnelblick's way: drive the system `openvpn` binary and observe it over its management interface. `openvpn.rs` builds the command line with `build_argv`, wiring `--management`, `--management-client`, `--management-up-down`, and `--management-query-passwords`, and then `parse_management_line` interprets the socket's `>STATE:`, `>BYTECOUNT:`, `>PASSWORD:`, and `>LOG:` lines into a `MgmtEvent`. This is exactly Tunnelblick's architecture — the GUI does not reimplement OpenVPN, it supervises the real one and reads its management channel — rebuilt in Rust with the parser as a pure, tested function.

WireGuard is the part Tunnelblick never had, and it is genuinely native. Behind the `wireguard` feature, `wireguard_tunnel.rs` builds a `boringtun Tunn` from the config's x25519 keys (`make_tunn`) and runs a userspace data path — a real `tun/utun` device, a UDP socket, and three pump threads (`spawn_tun_to_net`, `spawn_net_to_tun`, `spawn_timers`) moving packets through the Noise protocol. The config side is pure Rust too: `wireguard.rs` parses a `.conf` into a `WgConfig` of interface and peers, validates that keys decode to 32 bytes, and round-trips back to text with `to_conf`. This is `ztunnel`'s superset over Tunnelblick — a native WireGuard client — and the book claims it as original because the code owns the data path rather than shelling out to `wg-quick`.

Process control, logs, and live stats

Supervising an external process correctly is fussier than it looks, and `ztunnel-core`'s `supervisor.rs` does the fussing. An `OpenVpnProcess::start` writes a temp config, spawns `openvpn` with the management socket wired, and its `Drop` implementation kills the child, reaps it, and removes the temp file — so a dropped

supervisor never leaks a running VPN process or a config file with credentials in it. `is_running` and `stop` give the shell explicit control, and `spawn_raw` is the escape hatch for a raw argument vector. The lifecycle is owned end to end, because a VPN manager that leaks `openvpn` processes is worse than no VPN manager.

The connection state machine lives in `manager.rs`, and it enforces legality rather than trusting callers. A `Manager` holds the connections, the logs, and the throughput, and its methods — `begin_connect`, `begin_disconnect`, `mark_disconnected`, `mark_error`, `set_status`, `update_bytes` — police the state transitions: connecting twice, or disconnecting something already disconnected, is an `InvalidState` error, not a silent no-op. The `ConnectionStatus` enum (`Disconnected`, `Connecting`, `Connected`, `Reconnecting`, `Disconnecting`, `Error`) is the vocabulary, and the manager is the referee, so the shell can drive the UI off a state it can trust.

Logs are a bounded ring, deliberately. `log.rs`'s `LogBuffer` is a fixed-capacity buffer with `record`, `entries`, and `clear`, so a chatty OpenVPN connection cannot grow the log without bound and exhaust memory over a long session — old lines fall off the end. The logs are per-connection, tagged with a level, and exposed through `vpn.logs` and `vpn.clear_logs`, and they are also pushed live: the `EngineEvent::Log` variant carries a line to the sink as it arrives, so the shell's log view updates in real time rather than by polling.

Live stats are derived, not stored, which is the right way to turn a VPN's cumulative byte counters into a throughput readout. `stats.rs`'s `Throughput` holds the last byte counts and timestamp, and `sample(bytes_in, bytes_out, ts)` returns a `Rate { in_bps, out_bps }` computed from the delta since the last sample. OpenVPN's `>BYTECOUNT:` lines and WireGuard's counters both feed the same `update_bytes`, so the throughput math is one implementation regardless of backend, and the `EngineEvent::Stats { bytes_in, bytes_out }` events stream the numbers to the shell for the HUD's live meters. Cumulative counters in, bits-per-second out, one function, both protocols.

Credential storage

VPN credentials are the part you must not get wrong, and `ztunnel-core` gets it right by making storage a pluggable trait with a safe default. `creds.rs` defines a `Credential` (username, password, optional key passphrase) and a `CredStore` trait with `get`, `set`, and `remove`; the default implementation is `SessionCreds`, an in-memory map that never touches disk, so the safe behaviour is the default behaviour — credentials live in memory for the session and vanish when the process exits. An app has to opt *in* to persistence, not opt out of it.

Persistence, when wanted, goes to the OS keychain, not a file. Behind the `keychain` feature, `KeychainCreds` stores each credential as a single JSON secret through `keyring::Entry`, so a saved credential lives in the platform's real secret store — Keychain on macOS, Secret Service on Linux — with the OS's protection, rather than in a config file `ztunnel` would have to protect itself. The store is chosen at runtime via `set_cred_store`, so the same engine can run session-only in a test and keychain-backed in production without a code change. The persistence decision is the shell's, injected into the engine, which keeps the engine pure and the secret handling in the platform's hands.

The engine is equally careful about what it *writes* elsewhere. The profile store (`store.rs`, `save_profile`) persists configs, settings, and servers as JSON — but secret-free, so a stolen profile file yields configs and preferences but not passwords. And the temp config the supervisor writes for `openvpn` is removed on drop, so credentials that must be on disk for the length of a connection do not outlive it. The whole credential story is a series of these small, deliberate choices — default to memory, persist to the keychain, never write secrets to the profile, clean up the temp file — and together they are why a VPN manager can be trusted with the thing it exists to hold.

ztunnel-core: the embeddable VPN engine

`ztunnel-core` follows the fleet shape exactly — `rlib/staticlib/cdylib`, a `struct Engine` with `invoke`, the `ztn_`-prefixed four-function C ABI in `src/ffi.rs` (`ztn_init`, `ztn_invoke`, `ztn_`

`string_free`, `ztn_set_event_callback`) with a hand-written `include/ztunnel_core.h`, and a Tauri plugin exposing `ztn_invoke`. The command surface spans config management (`config.list/add/import/remove/rename/duplicate`, `config.parse`, `config.wireguard_format`), connection lifecycle (`vpn.connect/disconnect/status/connections`), live data (`vpn.logs`, `vpn.clear_logs`), credentials (`creds.set/clear`), and server selection (`servers.list/fastest/ping`).

One detail the book flags in the interest of the honesty it keeps promising: the C header's inline command-name comment has drifted behind the engine. `Engine::invoke` implements newer commands — `config.rename`, `config.duplicate`, `config.wireguard_format`, `vpn.autoconnect`, `vpn.network_changed`, `feature.matrix`, `vpn.killswitch_plan`, the `servers.*` family — that the header's comment does not yet list. The functions work; the documentation comment lags. It is a small thing, but a book that credits its predecessors honestly and marks its own provisional numbers should also note where a crate's own docs trail its code, and this is one such place.

The engine carries more of a full VPN client than Tunnelblick did, and the extras are where “embeddable engine” pays off. There is a kill switch (`killswitch.rs`, a `FirewallPlan` that renders pf or nft rules to block traffic outside the tunnel), a network monitor (`netmonitor.rs`, watching for the path change that should trigger a reconnect), a server list with latency probing (`servers.rs`, `fastest_first` and `probe_latency`), and a privileged-operations planner (`privileged.rs`, a `PrivilegePlan` of `NetOps` — create interface, assign address, add route, set DNS — applied through a per-OS `PlatformNet` trait). Each is a pure planner in the engine whose application is the shell's privileged job.

That split is the chapter's closing point about the whole `-core` pattern under a hard constraint: bringing a tunnel up needs root, and the engine cannot have root. So `ztunnel-core` computes everything that can be computed without privilege — parse the config, build the argv, plan the interface and routes and firewall rules, classify the backend — and hands the shell a plan to *apply* with the privilege the shell holds (a `SMApService` helper on macOS, polkit on Linux). If root is not held, `can_apply()` says so and `connect` records a `needs_privilege` state rather than faking a live tunnel. The engine stays pure, the privilege stays in the shell, and the boundary between them is a data structure — which is the `-core` pattern surviving contact with the most privileged thing a desktop app does.

Privilege and the firewall, computed as data the shell applies

The cleanest way to see the pure-engine/privileged-shell split is to read the two planners `ztunnel-core` hands the shell, because both are data structures the engine computes without any privilege and the shell applies with the privilege it holds. `privileged.rs` builds a `PrivilegePlan` — `PrivilegePlan::bring_up(iface, fields, route_all, use_dns)` returns a plan of `NetOps`, and each `NetOp` is a concrete, inspectable step: `CreateInterface { name }`, `AssignAddress { iface, cidr }`, `AddRoute { dest, iface }`, `SetDns { servers }`. Applying the plan is `PrivilegePlan::apply(&self, platform: &dyn PlatformNet)`, and `PlatformNet` is the per-OS trait whose three methods — `name`, `can_apply`, and `apply(op)` — the shell implements against the real network stack. If `can_apply()` is false because root is not held, `connect` records a `needs_privilege` state rather than faking a tunnel. The engine decided *what* to do; the shell, holding privilege, decides *whether it can* and *how*.

The kill switch is the same shape pointed at the firewall. `killswitch.rs` computes a `FirewallPlan { platform, rules }` where each `FirewallRule` is an `allow` flag over a `RuleTarget` — `All`, `Loopback`, `Iface(name)`, `Cidr(cidr)`, `Dns`, `Ipv6` — plus a human note, and the module carries the exact set of local networks it must never leak, `LAN_CIDRS: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16, 224.0.0.0/4, and 255.255.255.255/32`. The engine renders the plan — allow loopback and the tunnel interface, allow the LAN ranges, block everything else — as data; the shell translates that plan into pf rules on macOS or nft rules on Linux and installs them with privilege. A user who reads the plan sees exactly which traffic the kill switch will block before a single rule is applied, which is the same “plan, present, apply” trust model as `zftp`'s sync and `zoffice`'s diff.

Supervising the real OpenVPN, line by line

The OpenVPN path is a faithful Tunnelblick port, and its two pure functions are where the fidelity lives. `openvpn.rs::build_argv(config, config_path, mgmt_host, mgmt_port)` constructs the exact command line — `--config`, `--management`, `--management-client`, `--management-up-down`, `--verb 3`, and, only when the config declares it needs credentials, `--management-query-passwords` — so the engine drives the system `openvpn` binary over its management interface precisely as Tunnelblick does. The observing half is `parse_management_line(line) -> Option<MgmtEvent>`, a pure string parser that turns the socket's chatter into a typed event: `State { status, raw, local_ip, remote_ip }` from a `>STATE:` line, `ByteCount { bytes_in, bytes_out }` from `>BYTECOUNT:`, `NeedAuth { realm }` from a password prompt, and `Log { message }` from `>LOG:`. Because it is pure, it is tested directly on captured management lines with no process in sight.

Everything the engine exposes about a connection is one of the `config.*`, `vpn.*`, `creds.*`, or `servers.*` commands, and reading them shows how much more than Tunnelblick the engine carries. Config management is `config.list/get/add/import/remove/set_options/rename/duplicate`, plus `config.parse` and `config.wireguard_format` for the two config dialects. The connection lifecycle is `vpn.connect/disconnect/status/connections`, with `vpn.autoconnect`, `vpn.network_changed`, `vpn.logs`, `vpn.clear_logs`, and `vpn.killswitch_plan`. Credentials are `creds.set/clear`, backed by the `CredStore` trait whose default `SessionCreds` never touches disk and whose `keychain`-gated `KeychainCreds` stores through `keyring::Entry`. Server selection is `servers.list/add/remove/favorite/fastest/ping`. The connection state machine in `manager.rs` polices the transitions with `begin_connect`, `begin_disconnect`, `mark_disconnected`, `mark_error`, and `set_status` over the `ConnectionStatus` vocabulary (`Disconnected`, `Connecting`, `Connected`, `Reconnecting`, `Disconnecting`, `Error`), so an illegal transition is an `InvalidState` error the shell can trust rather than a silent corruption of the HUD.

WireGuard, owned end to end

The part of `ztunnel` that Tunnelblick never had is a native WireGuard client, and the crate owns both the config and the data path rather than shelling out to `wg-quick`. The config side is pure Rust in `wireguard.rs`: it parses a `.conf` into a `WgConfig` of an interface and its peers, validates that the x25519 keys decode to 32 bytes, and round-trips back to text with `to_conf`, all with no privilege and no network — so a malformed key is a parse error, not a failed connection. The `config.wireguard_format` command exposes the round-trip, and the `Backend::WireGuard` variant's `launch_kind()` returns `Native`, recording in the type system that this is the one protocol whose data path `ztunnel` owns outright.

The data path lives behind the `wireguard` feature in `wireguard_tunnel.rs`, and it is a genuine userspace implementation. `make_tunn` builds a `boringtun Tunn` from the config's keys — the Noise-protocol state machine that does the WireGuard handshake and transport encryption — and the session runs three pump threads that move packets: `spawn_tun_to_net` reads plaintext from the tun/utun device and hands it to `Tunn` to encrypt and send over the UDP socket, `spawn_net_to_tun` reads ciphertext from the socket and hands it to `Tunn` to decrypt and write to the device, and `spawn_timers` drives `Tunn`'s periodic handshake and keepalive timers. That is a real WireGuard data path — a tun device, a UDP socket, and the Noise state machine — running in `ztunnel`'s own process, which is why the book claims it as original work rather than a wrapper. The feature gate (`wireguard = ["dep:boringtun", "boringtun/device", "dep:libc", "net"]`) means a build that only needs OpenVPN never pulls `boringtun` or `libc`.

Both backends feed one throughput and one log implementation, which is the fleet's habit of solving the same shape once. OpenVPN's `>BYTECOUNT:` lines and WireGuard's counters both call `manager.update_bytes`, which feeds `stats::Throughput::sample` — a function holding the last byte counts and timestamp and returning a `Rate { in_bps, out_bps }` from the delta — so the HUD's live meters are one implementation regardless of protocol. Logs are a bounded `LogBuffer` ring per connection, so a chatty tunnel cannot grow the log without bound over a long session; old lines fall off the end, and each new line is also pushed live

through `EngineEvent::Log`. One throughput math, one log ring, two protocols — cumulative counters in, bits per second and a bounded scrollbar out.

Netmonitor, servers, and a fuller VPN client than Tunnelblick

The extras `ztunnel-core` carries beyond a faithful OpenVPN GUI are where “embeddable engine” pays off, and each is a pure planner or observer the shell drives. The network monitor (`netmonitor.rs`) watches for the path change that should trigger a reconnect — a laptop moving from Wi-Fi to Ethernet, a network dropping — so a tunnel can survive a connectivity change rather than silently dying, exposed through the `vpn.network_changed` command the shell calls when the OS reports a path change. The server list (`servers.rs`) turns a set of endpoints into a ranked choice with latency probing: `probe_latency` measures each server and `fastest_first` orders them, behind the `servers.list/fastest/ping` commands, so a user picks “the fastest server” and the engine has actually measured it.

Together with the kill switch and the privilege planner from earlier in the chapter, these make `ztunnel-core` a fuller VPN client than Tunnelblick was, and they all obey the same discipline: the engine computes without privilege and the shell applies with it. The book keeps one honesty note here that a project crediting its predecessors should also apply to itself — the C header’s inline command-name comment has drifted behind the engine. `Engine::invoke` implements `config.rename`, `config.duplicate`, `config.wireguard_format`, `vpn.autoconnect`, `vpn.network_changed`, `feature.matrix`, `vpn.killswitch_plan`, and the `servers.*` family that the header’s comment does not yet list. The functions work; the documentation comment lags, and a book that marks its provisional numbers should mark where a crate’s own docs trail its code, because that drift is the ordinary cost of a surface that grows faster than its hand-written header.

The supervisor’s Drop: no leaked processes, no orphaned secrets

Supervising an external process correctly is fussier than it looks, and `ztunnel-core`’s `supervisor.rs` does the fussing with Rust’s `Drop` rather than hoping a caller remembers to clean up. `OpenVpnProcess::start` writes a temp config, spawns `openvpn` with the management socket wired, and holds the child; its `Drop` implementation kills the child, reaps it, and removes the temp file. So a supervisor that goes out of scope — because a connection failed, or the app is shutting down, or an error unwound the stack — never leaks a running VPN process and never leaves a temp config with credentials in it behind on disk. `is_running` and `stop` give the shell explicit control, and `spawn_raw` is the escape hatch for a raw argument vector, but the safety net is the destructor: the lifecycle is owned end to end by the type system.

That `Drop` is a small thing with an outsized correctness payoff, because the two failure modes it prevents are exactly the ones a VPN manager must never hit. A leaked `openvpn` process is a tunnel the user thinks is down but is up — a security problem. A temp config left on disk is a credentials file outliving the connection it was written for — also a security problem. By tying both to the supervisor’s lifetime, the engine makes “the connection is gone” and “the process and its secrets are gone” the same event, enforced by the compiler’s insertion of the destructor rather than by a cleanup path a programmer has to route every error through. It is the same instinct as the memory rule at the C boundary and the `InvalidState` guards in the manager: make the correct thing automatic and the incorrect thing hard, and let the type system carry the discipline a comment would only request.

Chapter 8 – zgo: Rebuilding Alfred

The launcher. Workflows as objects and connections, Script Filter feedback, fuzzy matching, variable/token expansion, web searches, clipboard history, and snippets with auto-expansion.

The workflow graph: objects and connections

Alfred's power-user feature is its workflow editor, and its data model is a graph: objects (triggers, inputs, actions, outputs) wired by connections, so a keyword trigger flows into a Script Filter flows into a Run Script flows into a Copy to Clipboard. `zgo-core`'s `model.rs` rebuilds that graph exactly: a `Workflow` holds `objects: Vec<Object>` and `connections: BTreeMap<String, Vec<Connection>>`, an `Object` carries a `uid`, a `type_`, a `version`, and a `config`, and `Workflow::downstream(uid)` walks the connections to find what a fired object flows into. It is Alfred's node graph as a Rust data structure.

Firing the graph is `dispatch.rs`, and it enumerates Alfred's trigger kinds as entry points: `keyword_trigger`, `external_trigger`, `hotkey_trigger`, `snippet_trigger`, `fallback_trigger`, `file_action_triggers`. Each finds the matching object and returns a `Fired { object, downstream }` — the object that matched plus the objects it connects to — so the shell can run the object and then follow the graph. The dispatch is the runtime half of the workflow model: the graph is the static structure, the triggers are how execution enters it, and `downstream` is how execution flows through it.

Interoperability with Alfred is real, not aspirational. `import.rs`'s `import_workflow` reads an Alfred `info.plist` (converted to JSON) into a `Workflow`, `export_workflow` writes one back, and `import_snippet` reads Alfred snippet files. Because `zgo`'s graph is a superset of the information in an Alfred workflow, an existing Alfred workflow imports and runs — which is the same “round-trip the predecessor's format” fidelity test that `zreq` passes with Postman collections and `zcite` passes with Zotero libraries. A launcher that could not run existing Alfred workflows would be a different product; `zgo` runs them.

The workflow editor in the UI is one of the few places `zgo` needs a genuinely graphical component, and it uses the shared one: `prefs.js` builds the lane/node/bezier editor with `ZGui.nodeGraph`, so even the node editor is a `zgui-core` widget rather than a bespoke canvas. The whole preferences window is `ZGui.*` — `dataTable`, `palette`, `settings`, `shortcuts` (rebindable), `modal`, `statistic` — mounted in a `ZGui.prefsShell`, which is itself the component distilled from `zgo`'s own Alfred-style preferences and promoted into the toolkit. `zgo` gave the fleet its preferences shell and then consumed it back, which is the reuse loop working in both directions.

Script Filter feedback and fuzzy matching

The Script Filter is Alfred's extensibility mechanism — a workflow object runs a script that emits a list of result items as JSON (or, historically, XML), and Alfred renders them — and `zgo-core`'s `scriptfilter.rs` speaks it fluently. `parse(stdout)` auto-detects the format by first character: a leading `{` is Alfred's JSON feedback, a leading `<` is the Alfred 2 legacy XML, parsed by `parse_xml`. The `Feedback` and `Item` structs mirror every documented Script Filter field — `uid`, `title`, `subtitle`, `arg`, `icon`, `valid`, `match`, `autocomplete`, `type`, `mods`, `text`, `quicklookurl`, `variables`, `action` — so a script written for Alfred emits feedback `zgo` renders identically.

Running the script is `scriptfilter.rs` too, and it sets up Alfred's environment faithfully. `run_env(workflow)` builds the `alfred_workflow_*` environment variables a Script Filter script expects, and `run_script`, `run_command`, and `run_stryke` execute the script — the last one an inline `stryke -e` runner,

a nod to the stack's own parallel-Perl tool, dispatched on an `@` prefix. All of this is behind the `exec` feature, so a `zgo-core` built `--no-default-features` is a feedback-only library that can parse and render Script Filter output without the ability to spawn a process, which is exactly the safe configuration for a test or a sandboxed embed.

Fuzzy matching is the launcher's soul — the thing that turns “gogl” into “Google Chrome” — and `matcher.rs` implements it as a scored subsequence match. `score(query, candidate)` returns `Some(i32)` when the query is a subsequence of the candidate and `None` when it is not, with bonuses for word-start matches, consecutive characters, and case matches, and penalties for leading gaps and interior gaps (the constants `BONUS_WORD_START=12`, `BONUS_CONSECUTIVE=8`, `BONUS_CASE_MATCH=2`, and the gap penalties are right there in the source). `filter(query, items)` ranks best-first, stable on ties, scoring against an item's `match` field or its title.

The book is careful about what that matcher claims. Its comments say outright that it re-implements the *shape* of Alfred's matching — subsequence with word-start and consecutive bonuses — not Alfred's private, unpublished constants. This is the same honesty applied to a smaller thing: `zgo` matches Alfred's *behaviour*, a launcher that ranks the obvious result first, without pretending to have reverse-engineered Alfred's exact scoring. And it is worth noting the fleet has two fuzzy matchers for two jobs — `zgo-core`'s Rust `matcher::score` ranks launcher results in the engine, while `ZGui.fzf.fzfMatch` highlights filter matches in every app's UI — because one runs in Rust over launch items and the other runs in JavaScript over list rows, and they are tuned for their different jobs.

Clipboard history and snippet auto-expansion

Clipboard history is a launcher staple, and `clipboard.rs` models it plainly: a `ClipEntry` tagged by `ClipKind` (`Text`, `Image`, `File`), held in a `Clipboard` with `push`, `search`, and `clear`. The interesting part is not the data structure but how it fills, and that is a shell job the engine is designed to accept: the `zgo` app runs a background clipboard daemon (`spawn_clipboard_daemon`) that polls the system pasteboard and feeds new entries into the engine through `zgo_invoke`. The engine holds and searches the history; the shell watches the pasteboard. The pure part is in the core, the platform part is in the shell, exactly as the pattern demands.

Snippet auto-expansion is the fiddliest feature in the chapter, because it means watching what the user types and replacing a trigger the instant it completes. `snippet.rs`'s `auto_expand_match(collections, typed)` is the matcher: it returns the snippet and the trigger length when the typed buffer ends with a collection's prefix plus a snippet's keyword plus its suffix and the snippet is marked `auto_expand`, with the longest trigger winning so `;;addr` beats `;addr`. The expansion itself is `expand_in(body, clipboard, collections)`, which resolves the snippet's placeholders — `{clipboard}`, `{cursor}` (returning a cursor offset so the shell can reposition the caret), `{date}` — into the final `Expansion`.

Variable and token expansion is shared across snippets, actions, and Script Filters, and it lives in `vars.rs`. A `Scope` built from variables, the current query, and the clipboard has a `render(input)` that substitutes Alfred's token vocabulary: `{query}`, `{clipboard}` and `{clipboard:N}` for history depth, `{var:name}`, `{date}`, `{time}`, `{datetime}` with optional formats, `{uuid}`, and `{allvars}`, leaving unknown tokens verbatim. This is the same idea as `zreq`'s `{{name}}` resolver and `zcite`'s CSL variable mapping — a pure string-substitution scope — and it is another instance of the fleet solving the same shaped problem the same way in different domains.

Web search rounds out the launcher's built-ins, and `websearch.rs` is small and faithful: a `Search` of keyword, title, and URL template, a `BUILTINS` table (`google`, `ddg`, `amazon`, `wiki`, `youtube`, `github`, `maps`, `images`, `twitter`, `imdb`), and `url_for(keyword, query)` that expands the template with the query percent-encoded. A user types `gh rust` and `zgo` opens the GitHub search for “rust,” which is Alfred's web-search feature rebuilt as a table plus a URL expander — the kind of feature that is ten lines when the surrounding architecture is right, and the architecture is right because everything else in this chapter already exists.

zgo-core: the launcher engine

`zgo-core` is the same shape as its siblings — `rlib/staticlib/cdylib`, a `struct Engine` with `invoke`, the `zgo_`-prefixed four-function C ABI in `src/ffi.rs` with `include/zgo_core.h`, and a Tauri plugin exposing `zgo_invoke` — but it has the broadest command surface in the fleet, because a launcher is a hub for everything. Beyond the workflow, Script Filter, matcher, clipboard, snippet, vars, and websearch commands, `invoke` dispatches a small operating system of features: `calc.eval` (an inline calculator), `system.list/run` (system commands), `index.scan/search` (an app index), `runningapps.search`, `bookmarks.search`, `music.nowplaying/command`, `contacts.search`, `dictionary.define`, `spotlight.search`, `onpassword.search`, and a `utility.*` family (conditionals, split/join, transforms, counters).

That breadth is the point of a launcher, and it is only tractable because each feature is a small module behind the one `invoke` — the engine is a coat rack of narrow, testable capabilities rather than one tangled program. The `learn.record/rank` commands even give the launcher a frequency memory (the `frequency.rs` module), so `zgo` learns that “term” means the terminal for this user and floats it up the results, which is Alfred’s usage-learning rebuilt as a rankable event log. A launcher is judged by whether the right result is first, and frequency plus fuzzy matching plus learning is how `zgo` earns that.

The shell around the engine is the most “desktop-integrated” of the fleet, because a launcher has to be. `zgo`’s app is a macOS Accessory — menu-bar, no Dock icon — toggled by a global `⌘Space` hotkey through `tauri-plugin-global-shortcut`, hiding on focus loss, with its own `zgo_hide/zgo_reveal/zgo_trash` commands for the platform gestures a launcher needs. The launcher window itself is `ZGui.launcher` chrome with `zgo`’s logic behind it, `ZGui.colorscheme.load()` for theming, and a Large Type overlay (`⌘L`) built from `ZGui.modal`. The engine is a hub of pure capabilities; the shell is the thin, deeply OS-integrated skin that makes those capabilities feel like Alfred — instant, keyboard-first, always a hotkey away.

The version skew here is the pattern’s independent versioning made visible: `zgo-core` is `0.1.0`, the `zgo` app is `0.2.0`. The engine — the workflow graph, the matcher, the snippet expander — has been stable while the app iterated on its menu-bar integration, its global hotkey, and its window behaviour. That is the `-core` split doing exactly what Chapter 2 promised: the domain logic holding steady at its own version while the shell moves faster at its own, two artifacts shipping together, neither one holding the other back.

A launcher is an operating system of small modules

`zgo-core` has the broadest module list in the fleet, and the breadth is the whole point of a launcher — Alfred is a hub, so its engine is a coat rack. The modules read like a directory of capabilities: `dispatch`, `scriptfilter`, `matcher`, `snippet`, `keystroke`, `vars`, `websearch`, `frequency`, `calc`, `clipboard`, `conditional`, `utilities`, `fallback`, `filesearch`, `queryhistory`, plus the provider modules `appindex`, `bookmarks`, `contacts`, `dictionary`, `music`, `onpassword`, `runningapps`, `spotlight`, `syscommands`, and `actions`. Each is a narrow, testable thing behind the one `zgo_invoke`, which is why a launcher’s enormous surface stays tractable — it is thirty small programs sharing a dispatch, not one tangled one.

The command surface reflects that. Beyond the workflow verbs (`workflow.list/get/add/remove`, `workflow.import/export/run`), Script Filter (`scriptfilter.run`, `feedback.parse/render`), matching (`match.filter`), snippets (`snippet.list/expand/match/import/add/remove`, `keystroke.type`), variables (`var.render`), and web search (`websearch.list/url`, `search.add/remove/fallback`), the engine dispatches a small operating system: `calc.eval` (an inline calculator), `learn.record/learn.rank` (the frequency memory), `clipboard.list/add/search/clear`, the trigger family (`trigger.keyword/external/fileaction/snippet/fallback/hotkey`), `query.record/recent/list/clear`, `file.search/browse/filter`, and a whole `utility.*` family — `conditional`, `split`, `join`, `transform`, `replace`, `random`, `counter`, `file_conditional`, `json_config`, `junction`, `delay`, `expression`. The provider commands round it out: `runningapps.search`, `bookmarks.search`, `music.nowplaying/music.command`, `contacts.search`, `dictionary.define`,

`spotlight.search`, `onpassword.search`, `system.list/run`, and `index.scan/index.search`. That is Alfred's entire feature set, one module and one dotted string at a time.

The two matchers and the ten searches, exactly

Two details are worth pinning to their constants because the book earlier promised to use the real ones. The launcher's fuzzy matcher, `matcher::score(query, candidate) -> Option<i32>`, is a scored subsequence match with the exact tuning in the source: `BONUS_WORD_START = 12`, `BONUS_CONSECUTIVE = 8`, `BONUS_CASE_MATCH = 2`, a per-character `PENALTY_LEADING = -2` capped at `PENALTY_LEADING_CAP = -12` before the first match, and `PENALTY_GAP = -1` for interior gaps. Its comments say outright that it re-implements the *shape* of Alfred's ranking — subsequence with word-start and consecutive bonuses — not Alfred's private constants, which is the honesty the whole book insists on applied to a scoring function. And this is the launcher's Rust matcher, distinct from `ZGui.fzf.fzfMatch`, the JavaScript matcher that highlights list rows in every app's UI; two matchers, two jobs.

Web search is a table plus a URL expander, and the table is ten entries wide. `websearch::BUILTINS` carries `google` (`https://www.google.com/search?q={query}`), `ddg` (`https://duckduckgo.com/` with `?q={query}`), `amazon`, `wiki` (`https://en.wikipedia.org/w/index.php?search={query}`), `youtube`, `github` (`https://github.com/search?q={query}`), `maps`, `images`, `twitter`, and `imdb`, each a key-word/title/URL-template triple, and `url_for(keyword, query)` expands the template with the query percent-encoded. A user types `gh rust` and `zgo` opens the GitHub search — ten lines of table and one expander, because the surrounding architecture (the matcher, the dispatch, the variable scope) already exists and the feature is only the last small module snapped onto it.

The workflow graph and Script Filter, in the data

Alfred's power-user surface is a node graph, and `zgo-core`'s `model.rs` is that graph as Rust data. A `Workflow` carries a `bundleid`, `name`, `version`, `variables`, an `objects: Vec<Object>`, and `connections: BTreeMap<String, Vec<Connection>>`; an `Object` is a `uid`, a `type_` (serialized as "type"), a `version`, and an opaque `config: Value`; a `Connection` is a `destinationuid`, `modifiers`, and a `vitoclose` flag. `Workflow::downstream(uid)` walks the connections to return the objects a fired object flows into. Firing is `dispatch.rs`, which enumerates Alfred's trigger kinds as entry points — `keyword_trigger`, `external_trigger`, `hotkey_trigger`, `snippet_trigger`, `fallback_trigger`, and `file_action_triggers` — each returning a `Fired` wrapper of the matched object and its downstream. The graph is the static structure, the triggers are how execution enters it, and `downstream` is how it flows; `import_workflow/export_workflow` read and write Alfred's `info.plist` so an existing workflow runs unchanged.

The Script Filter is Alfred's extensibility mechanism, and `scriptfilter.rs` speaks it by the letter. `parse(stdout)` auto-detects the format by first character — a leading `{` is Alfred's JSON feedback, a leading `<` is the Alfred 2 legacy XML routed to the private `parse_xml` — and the `Feedback/Item` structs in `model.rs` mirror every documented field: an `Item` carries `uid`, `title`, `subtitle`, `arg`, `icon`, `valid`, `match_` (serialized "match"), `autocomplete`, `type_`, `mods`, `text`, `quicklookurl`, `variables`, and `action`. Running a filter script is faithful too: `run_env(workflow)` builds the `alfred_workflow_*` environment a script expects, and `run_script/run_command/run_stryke` execute it behind the `exec` feature — the last an inline `stryke -e` runner, a nod to the stack's own parallel-Perl tool. Built `--no-default-features`, `zgo-core` is a feedback-only library that can parse and render Script Filter output without the ability to spawn a process, the safe shape for a test or a sandboxed embed.

Snippet auto-expansion is the fiddliest feature, because it means watching what the user types and replacing a trigger the instant it completes. `snippet::auto_expand_match(collections, typed)` returns the snippet and the trigger length when the typed buffer ends with a collection prefix plus a snippet keyword plus a suffix and the snippet is marked `auto_expand`, with the *longest* trigger winning so `;;addr` beats `;;addr.expand_in(body, clipboard, collections)` then resolves the placeholders — `{clipboard}`, `{cursor}` (stripped, returning a caret offset the shell uses to reposition), `{date}/{date:FMT}`,

and `{snippet:keyword}` for nested snippets — into an `Expansion { text, cursor }`. The same substitution idea powers `vars.rs`, whose `Scope::render` handles Alfred’s token vocabulary — `{query}`, `{clipboard}/{clipboard:N}`, `{var:NAME}`, `{date}/{time}/{datetime}` with optional formats, `{uuid}`, `{allvars}` — leaving unknown tokens verbatim. Three different features, one pure string-substitution scope, the same shape as `zreq`’s `{{name}}` resolver.

Frecency: the launcher’s memory

A launcher is judged by whether the right result is first, and fuzzy matching alone does not get there — the same query should float different results for different users, because “term” means the terminal to one person and a dictionary lookup to another. `zgo-core` earns that with a frecency memory in `frecency.rs`, driven by the `learn.record` and `learn.rank` commands. Each time a user picks a result, the shell records the choice; `learn.rank` then biases future rankings toward the frequently-and-recently chosen, so `zgo` learns a user’s habits as a rankable event log rather than a fixed table. It is Alfred’s usage-learning rebuilt as pure data: an append-only record the engine ranks, testable by recording a sequence of picks and asserting on the resulting order.

The interaction between the three ranking inputs is the launcher’s whole feel. `matcher::score` decides *whether* a candidate matches and how well by subsequence shape; frecency decides *which* of several good matches this user means; and the provider modules decide *what* is a candidate in the first place — running apps, bookmarks, contacts, the app index, Spotlight. A query flows through all three: the providers propose candidates, the matcher scores them, frecency reweights them, and the best-first, stable-on-ties list comes out. That layered ranking is why a launcher that is “just fuzzy match” feels worse than one that is fuzzy match plus learning, and it is tractable only because each layer is a small module behind the one `zgo_invoke`, tested on data with no UI. The learning is the difference between a launcher that ranks the obvious result first for everyone and one that ranks the obvious result first for *you*.

The workflow editor is the shared node graph

`zgo` needs one genuinely graphical component that most apps do not — a node editor for the workflow graph — and rather than hand-roll a canvas it uses the shared one. `prefs.js` builds the lane/node/bezier editor with `ZGui.NodeGraph`, so even the most app-specific surface in the launcher is a `zgui-core` widget consumed like every other. The whole preferences window is `ZGui.*: dataTable, palette, settings`, rebindable `shortcuts`, `modal`, and `statistic`, mounted inside a `ZGui.prefsShell`.

The interesting twist is that `prefsShell` is itself distilled *from* `zgo`. The Alfred-style preferences window `zgo` built was lifted into `zgui-core` as the shared `prefsShell`, and `zgo` now consumes it back — the reuse loop of Chapter 12 running in both directions, an app contributing a component to the toolkit and then depending on the toolkit’s version of it. That is the design system’s answer to “even the bespoke surface should be shared”: if a widget is specific enough that only one app needs it today, it still goes into `zgui-core` first and is consumed from there, so the day a second app needs a node editor or an Alfred-style prefs window, it is a call rather than a rebuild. The launcher’s most graphical corner is the clearest proof that “if a widget isn’t in `zgui-core`, you add it there and consume it” is a rule the fleet actually follows, not a slogan.

Chapter 9 — zftp: Rebuilding Cyberduck

The file-transfer client. FTP / FTPS / SFTP / WebDAV plus S3, Google Cloud Storage, Azure, Backblaze B2, Swift, Google Drive, Dropbox, OneDrive, and Box; `.duck` bookmark import; a session manager; a download / upload / sync queue; per-session logs and throughput; stored credentials.

One transfer abstraction over many protocols

Cyberduck's trick is that FTP, SFTP, WebDAV, and a dozen cloud object stores all look the same in its UI — a remote file browser, a bookmark, a transfer — and `zftp-core` earns the same uniformity with a transport abstraction. The `transport/` module defines the operations a browser needs — list, mkdir, delete, rename, chmod, peek — over a `local`, `remote`, and `scp` implementation, and the `protocol` module carries the protocol matrix. Above that, the `Engine's fs.*` commands (`fs.list/mkdir/delete/rename/rename_plan/chmod/peek`) are protocol-agnostic: the UI lists a remote directory the same way whether it is SFTP or S3, because the transport layer hides the difference.

The dependency choices are the load-bearing prior art, and `zftp` picks them carefully. `opendal` provides the unified data-access layer over FTP, SFTP, WebDAV, S3, and the cloud stores (GCS, Azure, B2, Swift, Drive, Dropbox, OneDrive, Box), and `russh` with `russh-sftp` provides a pure-Rust SFTP and SCP path with the `ring` crypto backend. This is the durable-deps rule applied to a domain where rewriting the plumbing would be madness — nobody should hand-roll an S3 signer or an SSH transport — so `zftp` owns the *product* (the bookmark model, the session manager, the transfer queue, the browser) and depends on `opendal` and `russh` for the *protocols*.

The honesty note the fleet always insists on applies here too: the network backends are phased. The local transport is fully wired, and a session against a network backend that has not been connected yet reports a `needs_transport` state rather than pretending to be live — the same pattern as `ztunnel's needs_privilege`. `zftp` does not claim every one of the dozen protocols is production-complete; it claims a uniform abstraction with the local path done and the network paths coming online behind it, and it says so in the state it reports rather than in a footnote. A transfer client that faked a connection would be worse than one that admitted the backend was not wired.

Encoding is the unglamorous detail that separates a real transfer client from a toy, and `zftp-core` has an `encoding` module built on `encoding_rs` for it — because FTP servers serve filenames in whatever charset they please, and a client that assumes UTF-8 mangles half the world's filenames. Handling the charset of a remote listing is exactly the kind of correctness Cyberduck spent years getting right, and porting the *product* faithfully means porting that correctness, not just the happy-path UTF-8 case. The module's existence is a small sign that the port took the predecessor's hard-won robustness seriously.

The transfer queue: download, upload, sync

A transfer client is, underneath the browser, a queue: you drag files and they line up to download, upload, or sync, with progress and the ability to cancel. `zftp-core` models this directly in its `transfer` module, exposed through `transfer.*` commands — `transfer.add`, `transfer.list`, `transfer.status`, `transfer.cancel`, `transfer.clear`. A transfer is a queued unit with a status the engine advances, and

because the engine has an event sink, progress and completion stream to the shell as `EngineEvents` rather than requiring the UI to poll, so the queue view updates live the way Cyberduck's does.

Sync is the queue's most interesting mode, because "make this local folder match that remote one" is a diff problem before it is a transfer problem — you have to decide what to upload, what to download, and what to leave alone, which is why the engine carries a `rename_plan` alongside `rename` and computes what a sync would do before it does it. Planning before acting is the same shape as `ztunnel`'s `PrivilegePlan` and `zoffice`'s `DiffOp`: compute the plan as pure data, let the shell present it, then apply it. A sync that showed the user what it was about to do — and let them cancel — is a sync the user can trust with a folder that matters.

Per-session logs and throughput are the operational readout, and they mirror `ztunnel`'s exactly, because a transfer session and a VPN session have the same monitoring needs. The `log` and `stats` modules give each session a bounded log and a throughput sample, exposed through `session.logs` and streamed as events, so a user watching a slow transfer sees the bytes-per-second and the protocol chatter in real time. The fleet's habit of solving the same shaped problem the same way pays off: the throughput math and the bounded-log discipline are conceptually the same code as `ztunnel-core`'s, applied to transfers instead of tunnels.

Credentials, likewise, follow the fleet pattern. `zftp-core`'s `creds` module handles `creds.set/store/load/delete`, storing transfer credentials the same careful way `ztunnel` stores VPN credentials — the safe default in memory, persistence to the platform's secret store when asked — because an FTP password and a VPN password deserve the same protection. A file-transfer client accumulates a lot of credentials (one per bookmark, potentially dozens), so getting the storage right is not optional, and the engine gets it right by reusing the pattern the VPN engine already proved.

`.duck` bookmark import and the session manager

Cyberduck's bookmarks live in `.duck` files, and a Cyberduck refugee's first question is "will my bookmarks come with me." `zftp-core` answers yes: the `bookmark` and `protocol` modules implement `.duck` bookmark import (`bookmark.import`), reading Cyberduck's own bookmark format into `zftp`'s `Bookmark` model, alongside `bookmark.import_filezilla` for the other big client's format. A `Bookmark` carries the host, protocol, path, and per-bookmark options (`bookmark.set_options`), so an imported bookmark reconnects exactly where Cyberduck left it. This is the same round-trip-the-predecessor's-format fidelity as every other chapter, applied to the one file a transfer-client user most wants to bring along.

The session manager is the runtime layer over the bookmarks, and it is where a transfer client differs from a one-shot uploader. `session.connect/disconnect/status/logs/list` manage live connections: a session is opened from a bookmark, holds the transport, and can be listed, inspected, and torn down, so a user browsing three remote servers at once has three sessions the manager tracks independently. This is Cyberduck's model — a browser window per connection, each with its own state — rebuilt as a set of engine sessions the shell renders as tabs or panes.

Discovery is a small nicety that the engine includes because Cyberduck users expect it: `discovery.scan` (the `discovery` module) finds local network servers to connect to, so a user on a LAN sees the SFTP and WebDAV servers around them without typing a hostname. It is a minor feature, but its presence is another data point that the port aimed at the *whole* Cyberduck workflow — browse, bookmark, discover, transfer — rather than just the transfer core, because a faithful port is faithful to the affordances users actually reach for, not only the marquee ones.

The frontend, per the fleet contract, is the mountable webui over the one `zftp_invoke` command, built from `ZGui.*` chrome — the remote browser is a `ZGui` table with sortable, resizable columns (R8) and a fuzzy filter (R7), the transfer queue is a live-updating list fed by session events, the bookmarks are a tile grid or tree. A Cyberduck user opening `zftp` finds the same three-panel shape — bookmarks, browser, queue — rendered in the cyberpunk HUD, which is the whole fleet thesis applied to file transfer: a faithful workflow, a familiar shape, one shared toolkit, one embeddable engine underneath.

zftp-core: the embeddable transfer engine

`zftp-core` completes the fleet shape with the full four-function C ABI — prefix `zftp_` (`zftp_init`, `zftp_invoke`, `zftp_set_event_callback`, `zftp_string_free`) in `src/ffi.rs`, headers in `include/zftp_core.h` and a C++ `.hpp` facade — and the Tauri plugin registering `zftp_core::tauri_plugin::zftp_invoke`. Like `zcite-core`'s citation service and `zpdf-core`'s PDF viewer, the transfer engine is a capability the rest of the fleet can embed: any app that needs “put this file on a remote server” or “open a file from S3” can, in principle, embed `zftp-core` and drive it through the same `invoke` surface the standalone app uses.

The value of `zftp-core` as an embedded engine is that “transfer a file to somewhere remote” is a need many apps have and none should re-solve. `zoffice` exporting a document to a WebDAV server, `zphoto` publishing an image to S3, `zcite` syncing attachments (which `zcite-core` already does over its own WebDAV client, but could delegate to the shared transfer engine) — all of these are `zftp-core`'s job. The engine's `opendal`-backed protocol breadth means embedding it once gives a host every cloud store at once, which is exactly the compounding-reuse argument the embed matrix of Chapter 11 is built on: solve remote transfer once, in one GUI-free engine, and every app that needs it embeds it rather than reinventing it.

The `net` feature gate makes the engine embed cleanly even where the network is not wanted. A host that embeds `zftp-core` for its local-transfer capabilities, or a test that exercises the bookmark parser and the transfer queue without touching a network, builds `--no-default-features` and gets the pure model — bookmarks, sessions, the queue, the `.duck` importer — with the transports stubbed. The same seam that lets the engine be tested without a network lets it be embedded without dragging `opendal` and `russh` into a host that only wants the local path.

The chapter closes on the same note as `ztunnel`'s: `zftp` is a case where the `-core` pattern meets genuinely impure requirements — live network sockets, cloud credentials, long-running transfers — and holds the line anyway. The engine owns the model, the queue, the bookmarks, the plans, and the protocol abstraction; the transports are behind a feature; the credentials default to memory and persist to the platform keychain; the streaming progress goes through the event sink. Cyberduck rebuilt in Rust, as an engine any app can embed, without the pure core ever needing to know it is inside a window — which is the sentence this book keeps earning, one app at a time.

The Transport trait and the sixteen-protocol matrix

The abstraction that makes FTP and S3 look the same in the browser is a single Rust trait, and reading it shows exactly how much a “protocol” has to provide to slot into `zftp`. `transport::Transport` declares the operations a remote browser needs — `connect`, `list`, `make_dir`, `delete`, `rename`, `stat`, `chmod`, `peek`, `download`, `upload` — plus the operations a serious transfer client needs on top: `download_resume(remote, local, offset)` for resuming an interrupted transfer, and `download_throttled/upload_throttled` for rate-limited transfers. It also carries `name` and `is_wired`, the second of which is the honesty seam — a transport that has not been connected reports `is_wired() == false`, and a session against it returns `needs_transport` rather than pretending to be live, the same pattern as `ztunnel`'s `needs_privilege`.

Three implementations satisfy the trait, and `transport::for_protocol(protocol)` routes to them: `LocalTransport` for the local filesystem, `ScpTransport` (pure-Rust `russh`, behind the `net` feature) for SCP, and `RemoteTransport` (OpenDAL-backed) for everything else. The `Protocol` enum is sixteen variants wide — `Ftp`, `Ftps`, `Sftp`, `Scp`, `Webdav`, `Webdavs`, `S3`, `Gcs`, `Azure`, `B2`, `Swift`, `GoogleDrive`, `Dropbox`, `OneDrive`, `Box`, and `Local` — and the reason a single `RemoteTransport` covers most of them is that OpenDAL is enabled with a long feature list (`services-ftp`, `services-sftp`, `services-webdav`, `services-s3`, `services-gcs`, `services-azblob`, `services-b2`, `services-swift`, `services-gdrive`, `services-dropbox`, `services-onedrive`). One trait, three implementations, sixteen protocols — and the whole `fs.*` command family (`fs.list/mkdir/delete/rename/rename_plan/chmod/peek`) is written once against the trait, blind to which of the sixteen is underneath.

The unglamorous correctness lives in `encoding`. `PathCodec::new(label)` builds a codec over `encoding_`

`rs`, with `decode/encode` and the path-aware `decode_name/encode_path`, because an FTP server serves filenames in whatever charset it pleases and a client that assumes UTF-8 mangles half the world's names. And the Cyberduck-refugee promise is kept by `bookmark.rs`: `import_path` sniffs a file and dispatches to `parse_duck` for Cyberduck's `.duck` plist (keyed on `Protocol`) or `parse_filezilla` for FileZilla's XML (importing every `<Server>`), exposed as `bookmark.import` and `bookmark.import_filezilla`. Around the transport sit the session manager (`session.connect/disconnect/status/list/logs`), the queue (`transfer.add/list/status/cancel/clear`), credentials (`creds.set/clear/store/load/delete`), and `discovery.scan` — and the event sink streams `Status`, `Stats`, and `Transfer { transfer_id, status, transferred_bytes, total_bytes }` so the queue view fills live. Cyberduck's three-panel workflow, rebuilt over one trait and sixteen protocols.

The queue, the session manager, and the honest states

Underneath the browser a transfer client is a queue, and `zftp-core`'s `transfer` module models it directly, exposed as `transfer.add/list/status/cancel/clear`. A transfer advances through a `TransferStatus` — `Queued`, `Running`, `Complete`, `Error`, `Cancelled` — and because the engine holds an event sink, progress streams to the shell as `EngineEvent::Transfer { transfer_id, status, transferred_bytes, total_bytes }` rather than requiring the UI to poll, so the queue view updates live the way Cyberduck's does. Sync is the queue's most interesting mode because “make this local folder match that remote one” is a diff before it is a transfer, which is why the engine carries `fs.rename_plan` alongside `fs.rename` and computes what a sync would do as inspectable data before it does it — plan, present, apply, the same trust model as `ztunnel`'s privilege plan.

The session manager is the runtime layer over the bookmarks and the place a transfer client differs from a one-shot uploader. `session.connect/disconnect/status/list/logs` manage live connections: a session is opened from a bookmark, holds a `Transport`, and can be listed, inspected, and torn down, so a user browsing three remote servers at once has three independent sessions the manager tracks — Cyberduck's browser-per-connection model rebuilt as engine sessions the shell renders as tabs. Each session carries a bounded log and a throughput sample (the `log` and `stats` modules, conceptually the same code as `ztunnel-core`'s), and the connection states are honest: `SessionStatus` runs `Disconnected`, `Connecting`, `Connected`, `Disconnecting`, `Error`, and a transport that is not wired reports `needs_transport` rather than faking a live link.

Two smaller niceties round out the port's fidelity to the whole workflow. Discovery is `discovery.scan`, behind the `discovery` feature over `mdns-sd`, finding local-network SFTP and WebDAV servers so a LAN user connects without typing a hostname. And credentials follow the fleet pattern exactly — `creds.set/clear/store/load/delete`, the safe default in memory and persistence to the platform keychain (behind the `keychain` feature over `keyring`, here pinned to v3 with the `apple-native`, `windows-native`, and `sync-secret-service` backends) — because a transfer client accumulates one credential per bookmark, potentially dozens, and getting their storage right is not optional. Browse, bookmark, discover, transfer, all behind one `zftp.invoke`.

Bookmarks, options, and reconnecting where you left off

A transfer-client refugee's first question is whether their bookmarks come with them, and `zftp-core`'s answer is a `Bookmark` model rich enough to reconnect exactly where the old client left off. `bookmark::from_url(url, name)` builds one from a URL, `import_path` sniffs a file and routes it to `parse_duck` (Cyberduck's plist, keyed on its `Protocol` value) or `parse_filezilla` (FileZilla's XML, importing every `<Server>`), and the resulting `Bookmark` carries the host, the `Protocol`, the path, and per-bookmark options set through `bookmark.set_options`. Because the options ride on the bookmark, an imported connection reconnects with the same charset, the same transfer mode, and the same starting directory the source client used — which is the

difference between “we can read your bookmarks” and “your bookmarks work.”

The `protocol` module is what turns a URL or an imported record into a connectable thing: `parse_url(url)` yields a `ParsedUrl`, and the `Protocol` enum’s sixteen variants each map, through `transport::for_protocol`, to the transport that speaks it. That indirection is why the bookmark layer is protocol-agnostic — a `Bookmark` for an S3 endpoint and one for an SFTP server are the same struct with a different `Protocol`, and the session manager opens either the same way. The whole import-bookmark-reconnect path is pure and testable: parse a `.duck` file, assert on the `Bookmark`, and never touch a network, because the parsing is separate from the connecting. Fidelity to the predecessor’s most-wanted file — the bookmark — is a property a test checks, not a claim the chapter makes.

Chapter 10 — zcontainer: Rebuilding Docker Desktop and Lens

The container and Kubernetes manager — and the marquee claim of the fleet: the first compiled-native desktop GUI for *both* Docker and Kubernetes, in a field where every competitor (Lens, FreeLens, Headlamp) is Electron or a TUI (k9s).

Containers, images, volumes, networks, Compose stacks

`zcontainer` is two products in one — a Docker Desktop and a Lens — and its Docker half talks to the Docker Engine’s HTTP API through `bollard`, the mature Rust Docker client. The container command surface is Docker Desktop’s whole vocabulary made explicit: `docker.containers.list`, and per-container `docker.container.start/stop/restart/remove`, `docker.container.inspect/exec/logs/logs.follow`, `docker.container.stats/commit/ diff/top/pause/rename`. Every button a user presses in Docker Desktop — start this container, tail its logs, exec a shell into it, inspect its config, see its stats — is one of those commands, and they all funnel through the one `zcontainer.invoke`.

Images, volumes, and networks round out the Docker object model, and Compose is the higher-level unit users actually organize around: `docker.compose.up/down/start/stop/restart/list` manages Compose stacks as first-class objects, so a user brings up a multi-container application with one command the way `docker compose up` does, but from the GUI with live status. The build surface is there too (`docker.builds.list/inspect/logs`), so watching an image build is a native panel rather than a scrolling terminal. This is Docker Desktop’s feature set, rebuilt against the Engine API, with `bollard` as the credited plumbing.

The claim that lifts `zcontainer` above “Docker Desktop in Rust” starts with what it manages *beneath* the API: its own `dockerd`. Docker Desktop’s actual hard problem on macOS and Windows is not the UI, it is running a Linux Docker daemon on a non-Linux host, and `zcontainer` owns that problem. The `daemon` module (with `daemon_linux` and `daemon_windows`) and the `vm` module manage a `dockerd` per OS: on macOS via an Apple Virtualization.framework guest, on Linux natively or rootless, on Windows via WSL2. The `docker.daemon.start/stop/status` and `docker.engine.pause` commands drive it. This is the part a naive port would shell out to Docker Desktop for; `zcontainer` does it itself, which is what makes it a *replacement* rather than a front end.

That daemon ownership is also where the `-core` pattern meets its most platform-bound requirement, and the split holds. Speaking the Engine API is pure-enough work that lives in `zcontainer-core`’s `docker` module over `bollard`; running a Virtualization.framework guest is deeply platform-specific and lives in the `vm` and `daemon` modules that the shell drives. The engine knows how to *talk* to a daemon; the shell knows how to *run* one. The boundary is the same one every chapter has drawn, placed at the hardest possible spot — between an API client and a hypervisor guest — and it holds because the API client is genuinely separable from the thing it talks to.

Multi-cluster Kubernetes: pods, workloads, services, CRDs, Helm

The Lens half talks to `kube-apiserver` through the `kube` crate and `k8s-openapi`, and its surface is Lens's whole scope. Contexts and namespaces (`k8s.contexts.list`, `k8s.namespaces.list`) give multi-cluster, multi-namespace navigation. Pods (`k8s.pods.list`, and per-pod `k8s.pod.logs/logs.follow/exec/portforward/debug`) are the center. The workload controllers are all there: `k8s.deployments.list`, `k8s.statefulsets/daemonsets/replicasets/jobs/cronjobs.list`. Services and nodes (`k8s.services.list`, `k8s.nodes.list`), metrics (`k8s.top.pods/nodes`), and the generic resource operations (`k8s.resource.get/apply/patch/delete/scale/restart/diff/events`) complete a cluster manager that does what `kubectl` does with a GUI over it.

Two features mark it as a *serious* Kubernetes tool rather than a pod list with buttons. First, custom resources: `k8s.crd.list` and the generic resource operations mean `zcontainer` handles CRDs, not just the built-in types, which is the difference between a tool that works on a toy cluster and one that works on a real cluster full of operators. Second, server-side apply: `k8s.apply.yaml` and the `k8s.resource.apply` path do server-side-apply, the correct modern way to reconcile YAML against a cluster, rather than a naive create-or-replace. These are the details a Kubernetes practitioner checks for, and their presence is why the “rebuilds Lens” claim is credible.

Helm is Kubernetes's package manager, and `zcontainer` manages it as a first-class subsystem: `k8s.helm.releases.list/install/upgrade/rollback/uninstall/history/values` and repository management (`k8s.helm.repo.add/repo.update/search`), with the release manifests decoded from Helm's base64-gzip storage format via `base64`, `flate2`, and `serde_yaml`. A user installs a chart, watches the release, rolls it back if it goes wrong — the whole Helm lifecycle — from the same GUI that manages the raw resources, which is exactly Lens's integration and exactly what a `kubectl-plus-helm` workflow wants unified.

The streaming operations — log follow, exec, port-forward — are why `zcontainer-core` has a subscription API where the other engines have only request/response. `subscribe`, `unsubscribe`, and `stream_send`, exposed as the Tauri commands `zcontainer_subscribe/unsubscribe/stream_send`, carry the bidirectional, long-lived channels that `k8s.pod.exec` and `docker.container.logs.follow` need — an exec session sends keystrokes and receives output, a log follow receives lines until cancelled. The engine's request/response `invoke` handles the CRUD; the subscription channel handles the streams; and the split is the `-core` pattern flexing to fit a requirement (Chapter 2) rather than breaking under it.

Live log streaming, in-pod exec, port-forwarding

These three are the features that separate a real container manager from a status dashboard, and they share a shape: a long-lived, often bidirectional channel between a UI panel and a remote process. A log follow streams lines from a container or pod until the user closes the panel. An exec session is a full terminal — the UI sends keystrokes, the remote process sends output — which is why `zcontainer`'s exec panels reuse the same embedded terminal (`terminal_spawn` and friends) the whole fleet ships. A port-forward holds a local socket open and tunnels it to a pod for the length of the session.

None of that fits `invoke(cmd, args) -> Value`, and that mismatch is the honest engineering pressure that produced the subscription API. Rather than distort `invoke` into something that could return a stream, or smuggle a raw socket into the GUI-free engine, `zcontainer-core` added a parallel channel: the engine's `subscribe` opens a stream and returns a handle, `stream_send` pushes input into it (the keystrokes of an exec, say), events flow back through the sink, and `unsubscribe` tears it down. The request/response door and the streaming door are separate but both lead into the same engine, which keeps the common case simple and the streaming case possible.

The `EngineEvent` enum and the sink carry the stream's output the same way every other engine's events

flow, which means the streaming case reuses the boundary machinery the whole fleet already has rather than inventing a new one. A log line, an exec output chunk, a port-forward status change — all are events emitted to the sink and delivered to the shell as WebView events, so the JavaScript listening for `zcontainer` events handles streamed output with the same listener it uses for a container-stopped notification. The subscription API added a way to *open* a stream; it did not need a new way to *deliver* one, because the event sink already existed.

This is the chapter's proof that the pattern scales to the hardest case in the fleet. `zcontainer` is the most demanding app — two full API clients, a hypervisor guest, and three kinds of live streaming — and the `-core` engine holds its shape: pure API clients over `bollard` and `kube`, request/response through `invoke`, streaming through the subscription channel, platform work (the daemon, the VM) in the shell, and the whole thing behind one `zcontainer_invoke` plus the streaming trio. If the engine-core, thin-shell thesis survives `zcontainer`, it survives anything the fleet is likely to build.

The “first compiled-native GUI for both” claim and its basis

The book makes the strong claim carefully, because a strong claim demands a stated basis. The competitive landscape for Kubernetes GUIs is Electron: Lens is Electron, its fork FreeLens is Electron, Headlamp is Electron; the fast alternative, k9s, is a terminal UI, not a GUI. The landscape for Docker GUIs is likewise Electron (Docker Desktop's own dashboard). `zcontainer` is a compiled-native Tauri app — a Rust backend and a WebView front end with no bundled Chromium runtime — that does *both* Docker and Kubernetes in one binary, and on the survey of shipping tools, no compiled-native desktop GUI covers both. That is the claim: first compiled-native desktop GUI for both Docker and Kubernetes.

The claim is narrow on purpose, and its narrowness is what makes it defensible. It is not “best” or “fastest” or “most complete” — the stack's own project bar kills those as goals — it is a *first*, at a specific, checkable intersection: compiled-native (excludes the Electron field), desktop GUI (excludes k9s the TUI), and both Docker and Kubernetes (excludes the single-purpose tools). Each qualifier excludes a named category of existing tool, and what is left is the intersection `zcontainer` claims to be first at. A claim you can check by naming the competitors and the qualifier that excludes each is a claim worth making; a vaguer superlative would not be.

The architectural reason `zcontainer` can be first there is the whole book. A compiled-native app doing both Docker and Kubernetes needs two mature API clients, and Rust has them (`bollard`, `kube`); it needs a way to run a Linux daemon on any host, and Tauri plus a platform VM layer provides it; it needs a serious GUI without Electron, and `zgui-core` provides it. The pieces that make the claim possible are exactly the fleet's shared substrate — the `-core` engine pattern, the Tauri shell, the shared toolkit — so `zcontainer` is not a special case that happened to be first; it is the fleet architecture pointed at the hardest domain and reaching a place the Electron field structurally cannot, because they carry a browser and it does not.

The book's final word on the claim is the book's word on every claim: it is credited honestly and cited to its basis. Docker Desktop and Lens are prior art for the workflow, credited by name; `bollard` and `kube` are prior art for the plumbing, credited as dependencies; the “first compiled-native GUI for both” is the original claim, stated with its qualifiers and its excluded competitors so a reader can check it. That is the same discipline the `zterminal` book applies to “the first terminal that live-edits a tmux server” and the `awks` book applies to “the first awk with a persistent bytecode cache” — locate precisely what is new against a named field, and claim only that.

`zcontainer-core`: the container/k8s engine

`zcontainer-core` (v0.2.0, ahead of most of the fleet) is the engine behind all of it, and its shape is the fleet shape with the streaming extension: `rlib/staticlib/cdylib`, a `struct Engine` with

`invoke` plus the `subscribe/unsubscribe/stream_send` trio, the `zcontainer_C` ABI in `src/ffi.rs` (`zcontainer_init`, `zcontainer_invoke`, `zcontainer_set_event_callback`, `zcontainer_string_free`, plus `zcontainer_subscribe`, `zcontainer_unsubscribe`, `zcontainer_stream_send`) with `include/zcontainer_core.h`, and the Tauri plugin registering all four commands. The `docker` and `kube` features gate their respective clients, so a host can embed just the Docker half or just the Kubernetes half.

The modules map cleanly to the two products plus the platform layer: `docker` (the `bollard` client), `kube` (the `kube/k8s-openapi` client), `daemon` with `daemon_linux/daemon_windows` and `vm` (the daemon and guest-VM management the shell drives), and `model`, `ffi`, `tauri_plugin`, and `error` (the fleet-standard scaffolding). The feature gates matter more here than anywhere else in the fleet, because the two halves have heavy, distinct dependency trees — `bollard` on one side, `kube` and `k8s-openapi` on the other — and a consumer that wants only one should not pay for both.

As an embeddable engine, `zcontainer-core` is the fleet's container/orchestration service. An app that wants to show the container a service runs in, or the pod behind a deployment, can embed the engine and query it through the same `invoke` surface — `docker.container.inspect`, `k8s.pods.list` — that `zcontainer` itself uses. The streaming API means an embedded consumer even gets live logs and exec, subject to the same `ZGui.embed.view` discipline as every other embed: mount the view scoped, feed the host shell, do not seize the global keyboard. The most complex engine in the fleet embeds by the same rules as the simplest.

And the version tells the same independent-versioning story one more time: `zcontainer-core` at `0.2.0`, the `zcontainer` app at `0.3.0`. The engine's API clients and daemon management have moved at their own pace while the app's packaging and UI moved at theirs, two artifacts shipping together, the shell one minor version ahead of the core. Across the fleet — `zgo` 0.2.0 over `zgo-core` 0.1.0, `zpdf` 0.2.0 over `zpdf-core` 0.1.0, `zcontainer` 0.3.0 over `zcontainer-core` 0.2.0 — the shells run ahead of their engines, which is the pattern's promise kept: chrome iterates fast, engines hold steady, and the version numbers are the receipt.

The command surface in full: two API clients' vocabularies

`zcontainer` is two products, and its command surface is the union of two full API vocabularies gated behind the `docker` and `kube` features so a host can embed one half without the other's heavy dependency tree. The Docker half is Docker Desktop made explicit: `docker.containers.list` and per-container `docker.container.start/stop/restart/remove/inspect/exec/logs/logs.follow/stats/commit/diff/top/pause/unpause/rename/kill/update/files`; images under `docker.images.list` and `docker.image.pull/run/remove/tag/history/inspect/prune/scan`; a registry browser under `docker.hub.search/repos/repo/tags`; networks and volumes (`docker.networks.list`, `docker.network.create/inspect/remove/prune`, and the parallel `docker.volume*`); Compose stacks (`docker.compose.list/up/down/start/stop/restart`); builds (`docker.builds.list/inspect/logs/remove`); and a system layer of `docker.system.info/df/prune`, `docker.ping`, `docker.stats`, `docker.events.follow`, `docker.metrics.history`, and `docker.topology`. Underneath the API sit the daemon commands `docker.daemon.status/start/stop` and `docker.engine.pause/resume`, which drive the guest `dockerd` the shell runs.

The Kubernetes half is Lens's scope, and its list surfaces are notably complete: `k8s.contexts.list`, `k8s.namespaces.list`, and the workload and resource lists `k8s.pods/deployments/statefulsets/daemonsets/replicasets/jobs/cronjobs/services/nodes/configmaps/secrets/ingresses/pvcs/pvs/events/crds.list`, plus a `k8s.generic.list` for arbitrary kinds. The `k8s.crds.list` and `k8s.generic.list` entries are the difference between a toy-cluster tool and one that works on a real cluster full of operators. Pods carry `k8s.pod.logs/logs.follow/exec/portforward/debug`; metrics are `k8s.top.pods/nodes`; the generic resource operations are `k8s.resource.get/apply/patch/delete/scale/restart/diff/events`; server-side apply is `k8s.apply.yaml`; and Helm is a first-class subsystem — `k8s.helm.releases.list`,

`k8s.helm.install/upgrade/rollback/uninstall/history/ values/search`, and the repositories `k8s.helm.repo.add/update/remove/list`, with release manifests decoded from Helm's base64-gzip storage via `base64`, `flate2`, and `serde_yaml`. On macOS the `vm` feature adds `vm.resources.get/vm.resources.set` for the `Virtualization.framework` guest. Two mature clients — `bollard` for Docker, `kube/k8s-openapi` for Kubernetes — behind one `zcontainer_invoke`, with the streaming trio of Chapter 4 carrying the follows, execs, and forwards that do not fit request/response.

Owning the daemon: the VM layer and the platform split

The claim that lifts `zcontainer` above “Docker Desktop in Rust” is that it manages its own `dockerd`, and the modules that do it are where the `-core` pattern meets its most platform-bound requirement. The engine's `daemon` module (with `daemon_linux` and `daemon_windows`) and its `vm` module manage a Docker daemon per OS: on Linux natively or rootless, on Windows via WSL2, and on macOS via an Apple `Virtualization.framework` guest. The macOS path is why the crate has a `vm` feature whose dependency list is a block of Apple bindings — `objc2`, `objc2-foundation`, `objc2-virtualization`, `block2`, `dispatch2`, and `libc` — pulled in only when `vm` is enabled, so a Linux or CI build never compiles the hypervisor bindings. The `docker.daemon.start/stop/status` and `docker.engine.pause/resume` commands drive it, and `vm.resources.get/.set` tune the guest's CPU and memory.

The split holds at the hardest possible boundary — between an API client and a hypervisor guest — because the two are genuinely separable. Speaking the Engine API is pure-enough work over `bollard` that lives in the `docker` module; running a `Virtualization.framework` guest is deeply platform-specific and lives in the `vm` and `daemon` modules the shell drives. The engine knows how to *talk* to a daemon; the shell (holding the entitlements a hypervisor needs) knows how to *run* one. A naive port would shell out to Docker Desktop for the guest; `zcontainer` runs it itself, which is precisely what makes it a replacement rather than a front end, and the fact that the boundary survives being placed between an HTTP client and a Linux VM on a non-Linux host is the strongest evidence in the book that the engine-core/thin-shell thesis scales to the fleet's hardest app.

Compose, builds, and the system layer

The parts of Docker Desktop that are not a single-container action are where a GUI earns its keep over the CLI, and `zcontainer` carries them as first-class objects. Compose stacks are managed through `docker.compose.list/up/down/start/stop/restart`, so a multi-container application comes up with one action and live per-service status rather than a scrolling `docker compose up` in a terminal. Image builds are `docker.builds.list/inspect/logs/remove`, so watching a build is a native panel with its log, not a firehose in a pane. And the system layer — `docker.system.info/df/prune`, `docker.ping`, `docker.stats`, `docker.events.follow`, `docker.metrics.history`, and `docker.topology` — gives the dashboard the fleet-wide readout a container operator actually watches: disk usage, a live event stream, historical metrics, the graph of how containers connect. The registry browser (`docker.hub.search/repos/repo/tags`) even puts image discovery in the same window.

Two of those deserve a note because they show the streaming door of Chapter 4 doing ordinary work. `docker.events.follow` and `docker.metrics.history` are the Docker analogue of the Kubernetes follows and execs: a long-lived subscription whose output flows back through the event sink as `Stream` events, so the dashboard's live activity feed uses the same machinery as a `kubectrl exec` session. The `docker` and `kube` features gate their heavy, distinct dependency trees — `bollard` on one side, `kube/k8s-openapi` on the other — so a host that wants only the Docker half, or only the Kubernetes half, pays for one client and not both. The two products share one engine, one `zcontainer_invoke`, and one streaming trio, and the feature gates keep that sharing from forcing a consumer to carry the half it does not use.

In-pod exec reuses the shared terminal

The streaming operations are not only an engine concern; on the front end they reuse a piece of family baseline, and the exec case is the clearest. A `k8s.pod.exec` or a `docker.container.exec` is a full terminal — the UI sends keystrokes, the remote process sends output — so `zcontainer`'s exec panels reuse the same embedded terminal (`terminal_spawn` and its siblings, from `zpw-embed-terminal`) that every app in the fleet ships for R3. The engine's `stream_send` carries the keystrokes into the subscription and the `Stream` events carry the output back; the terminal widget renders both. An exec session is thus a shared xterm.js surface wired to an engine stream rather than a bespoke terminal `zcontainer` had to build, which is the R3 baseline paying off in a place it was not designed for.

That reuse is the embed economy of Chapter 13 pointed inward at a single app. `zcontainer` did not build a terminal for exec, a log viewer for follow, and a socket manager for port-forward; it built one subscription channel in the engine (Chapter 4) and mounted the shared terminal, the shared log view, and the shared table on top of it. The three streaming features that separate a real container manager from a status dashboard — log follow, exec, port-forward — are the engine's subscription trio plus `zgui-core` widgets the fleet already had, which is why the hardest app is also, in front-end lines, one of the leaner ones. The engine grew the smallest thing streaming needed; the front end reused the components the family baseline already required; and the exec terminal is where those two economies meet in one panel.

Chapter 11 – The Document Suite: zoffice, zemail, zpdf, zphoto

Four apps that are also four engines the whole fleet embeds. `zoffice-core`, `zemail-core`, `zpdf-core`, and `zphoto-core` open a document, an attachment, a PDF, or an image *inside any other app*, which is why this chapter treats them together: they are the fleet's shared document plane.

zoffice: documents, spreadsheets, presentations (replacing MS Office)

`zoffice` rebuilds the office suite from scratch, and its engine covers the whole LibreOffice-shaped scope of Microsoft Office file compatibility. `zoffice-core`'s modules are the applications: `writer` (Word `.docx/.doc/.rtf` and ODF `.odt`), `calc` (Excel `.xlsx/.xls` and `.ods`), `impress` (PowerPoint `.pptx/.ppt` and `.odp`), plus `draw`, `math`, and `base` (Access `.odb`). The `App` enum names the six, and `Engine::app_for` routes a file to the right one. It reads and writes both the ODF and Office Open XML families, handles encrypted packages (`crypto`, `crypto_odf`) and digital signatures (`dsig`), and exports to PDF (`pdf`), with the OLE2 compound-document format (`ole2`) for the legacy `.doc/.xls/.ppt` files.

The dependency discipline is the durable-deps rule at office scale: `zip` and `quick-xml` for the OOXML and ODF containers (both are just zipped XML), RustCrypto for the encrypted packages, and *not* LibreOffice or UNO — the engine parses and writes the formats directly rather than embedding an office suite. That is a large undertaking, and the coverage is partial-by-design and honestly scoped, but the architecture is the same as every other core: a `struct Engine` with `invoke`, dispatching commands like `calc.open`, `calc.eval`, `calc.edit_cell`, `calc.pivot_tables`, `calc.charts`, `base.query`, `draw.connectors`, and the `writer.*`/`impress.*` families.

What makes `zoffice` matter to the *fleet* is not that it opens a spreadsheet standalone — it is that `zoffice-core` is on the embed matrix, consumed by every full GUI app. When `zemail` previews a `.docx` attachment, it is `zoffice-core` rendering it. When any app needs to show a document, `zoffice-core` is the engine. The office suite is not just an app; it is the fleet's document-rendering service, which is exactly why it was built GUI-free — so it can render a document into any host's window, not only its own.

`zoffice-core` is at v0.2.0 on the 2024 edition, with the full C ABI (`zoffice_invoke`, `zoffice_string_free` in `src/ffi.rs`, `include/zoffice_core.h`) and a Tauri surface in the app (`office_invoke`, `office_open`, `office_info`, `prefs_get`, `prefs_set`). It also carries a `diff` capability (`DiffOp`, the `diff` function) — document comparison as pure data, the same plan-as-data shape as `zftp`'s sync plan and `ztunnel`'s privilege plan — because comparing two versions of a document is a core office workflow and it belongs in the engine as a computed diff the shell renders.

zemail: a fast, owned desktop mail client

`zemail` is the fleet's mail client, a from-scratch Thunderbird-shaped desktop app, and `zemail-core` carries the whole mail stack: `net` for IMAP and SMTP (over the `imap` and `lettre` crates, with `mail-parser` for parsing), `store` for the local message database, `search` and `gloda` for full-text and global search, `filters` and `junk` for rules and spam classification, `addressbook` and

`carddav` for contacts, `calendar` for invites, and `openpgp/smime` for encryption. The command surface is a mail client's verbs: `account.add/autoconfig`, `folder.*`, `message.list/get/move/save_draft`, `imap.sync/search/idle`, `filter.add/run`, `junk.classify/train`, `contact.import_vcard`, `calendar.parse_invite/rsvp`.

The “owned” in “a fast, owned desktop mail client” is the pitch, and it is the same local-first, no-cloud-middleman argument `zreq` makes: your mail lives in a local store you own, spoken directly to your IMAP and SMTP servers, with no vendor's servers in the path. The junk classifier trains locally (`junk.train`, `junk.classify`), the search indexes locally (`gloda.search`, echoing Thunderbird's global-database search by name), the contacts sync over standard CardDAV rather than a proprietary API. It is a mail client as a piece of software you run, not an account you rent.

`zemail-core` (v0.1.0) has the full C ABI — prefix `zml_` (`zml_init`, `zml_invoke`, `zml_set_event_callback`, `zml_string_free`), headers `zemail_core.h` and `.hpp` — and the event sink matters here more than usual, because mail is asynchronous: `imap.idle` waits for the server to push new mail, and the arrival is an event, not a response. The engine narrates new mail, sync progress, and errors through the sink the same way `zreq` narrates request timing and `ztunnel` narrates connection status, so the shell's inbox updates live when mail lands rather than by polling.

As a fleet engine, `zemail-core` is on the embed matrix as the mail service — the “mail this” capability any app might want, the way `zftp-core` is the “transfer this” capability. An app that wants to send a document as an attachment can, in principle, embed `zemail-core` and drive a compose-and-send through its `invoke`. And `zemail` in turn embeds the *other* document cores: it previews `.docx` attachments through `zoffice-core`, PDFs through `zpdf-core`, and images through `zphoto-core`, which is the document plane composing — the mail client is both a provider of one service and a consumer of the other three.

zpdf: editing, forms, signatures, OCR, redaction (replacing Acrobat / Preview)

`zpdf` aims at the union of Adobe Acrobat Pro and macOS Preview, and `zpdf-core`'s surface is enormous because that union is enormous. It parses and writes the PDF object model directly over `lopdf`, and its modules span the whole Acrobat feature set: `page` (page management), `text` (text editing), `annot` (annotation and markup), `form` (AcroForms), `sign` and `security` (signatures and encryption, with a hand-rolled MD5 for the RC4 handler), `render`, `convert`, `ocr`, `linearize` (fast web view), `barcode`, `media` (rich media and 3D), and `js/actions` (PDF JavaScript and actions). The command dispatch is a long flat list — `add_note`, `add_ink`, `add_callout`, `apply_redactions`, `certify`, `accessibility_check`, `add_qr_code`, `bates_number`, `flatten_annotations`, `downsample_images` — one per Acrobat operation.

The breadth is why `zpdf-core` is the one engine that ships *no* C ABI, and the chapter is candid about it. There is no `include/` directory and no `extern "C"` in the crate; it exposes the native Rust API and the Tauri plugin (`zpdf_invoke`) only. The reason is that every consumer of PDF in the fleet is a Rust/Tauri app that links the crate natively — the app itself, and the sibling apps that embed it for preview — so the C door has no caller yet, and the durable-deps philosophy of “don't carry what nothing uses” applies to ABIs as well as dependencies. When a non-Rust host wants PDF preview, the four-function skin gets added; until then the crate does not carry it. This is the fleet's uniform pattern with its one honest exception, flagged rather than smoothed over.

`zpdf-core` is, by the embed matrix, the single most-embedded engine in the fleet — the `pdf-core` column is consumed by `zcite`, `zreq`, `zoffice`, `zemail`, `zphoto`, `ztunnel`, and every other full GUI app. The reason is obvious once stated: PDF is the universal document format, and every app eventually needs to show one — a cited paper in `zcite`, an API doc in `zreq`, an exported document in `zoffice`, an attachment in `zemail`. Building the PDF engine once, GUI-free, and embedding it everywhere is the embed matrix's clearest win, and it is why `zpdf-core` being embeddable was worth the enormous effort of parsing the PDF object model directly.

The Acrobat-plus-Preview scope also means `zpdf` carries the operations that separate a real PDF editor from a viewer: redaction that actually removes content (`apply_redactions`), signatures that certify a document

(`certify`, `add_typed_signature`), OCR that makes a scan searchable (`ocr`), accessibility tagging (`auto_tag`, `accessibility_check`), and optimization (`downsample_images`, `linearize`). These are the features Acrobat charges for and Preview lacks, rebuilt over the raw object model, and their presence is why the “replacing Acrobat” claim is more than “we can display a PDF.” A viewer shows the page; an editor changes the file, and `zpdf` changes the file.

zphoto: layers, selections, filters, non-destructive adjustments (replacing GIMP / Photoshop)

`zphoto` is the raster editor, a from-scratch GIMP/Photoshop port, with a vector mode (an Illustrator port) layered on top that renders into the same raster stack. `zphoto-core` (v0.2.0) is built around a layer model (`model`), the pixel operations (`ops`, `paint`, `fill`, `filter`, `text`), a vector subsystem (`vector`, with `vec.*` commands), and image codecs (`codec`, `b64`). Its command surface is a photo editor’s: `edit.undo/redo/copy/cut/paste/copy_merged/fade/history`, `fill.bucket/gradient/pattern/clouds/plasma/fibers`, a long `filter.*` family (`accented_edges`, `angled_strokes`, and the rest of the GIMP filter menu), `brush.define/stamp`, `action.record/play/stop` (macros), and `comp.capture/apply` (layer comps).

The non-destructive story is the part that makes it a *modern* editor rather than a paint program: adjustments, layer comps, recorded actions, and a real undo/redo history (`edit.history`) mean edits compose and reverse rather than baking into pixels. The `action.record/play/stop` macro system is Photoshop’s actions rebuilt — record a sequence of operations, replay it on other images — which is the kind of professional workflow feature that signals the port took the predecessor seriously. The vector mode sharing the raster stack is `zphoto`’s own integration: an Illustrator-style vector layer that rasterizes into the same document, with SVG interchange, and AI/SVG import routed through `zpdf-core` (because an `.ai` file is a PDF underneath) — a concrete cross-engine embed inside the document suite itself.

`zphoto-core` carries the full C ABI (`zphoto_init`, `zphoto_invoke`, `zphoto_string_free` in `src/ffi.rs`, `include/zphoto_core.h`) and the Tauri `zphoto_invoke`, and like the others it is on the embed matrix as the image service. Any app that shows an image — `zemail` previewing an inline image, `zcite` showing a figure, `zoffice` embedding a picture — can render it through `zphoto-core`. The raster engine is the fleet’s imaging plane, built GUI-free so it can composite into any host’s window, which is the same argument as `zpdf-core` for documents and `zoffice-core` for office files.

The four document cores together are what the chapter’s title calls the shared document plane, and the reason to treat them as one chapter is that they are one capability with four faces. Between them they can open, render, and edit essentially any document a desktop user encounters — an office file, an email, a PDF, an image — and because all four are GUI-free engines with the fleet’s uniform `invoke` surface, any app can embed any of them and show any of those formats inside its own window. The document plane is the payoff of the whole `-core` pattern at its most useful: not ten apps that each reinvent document preview, but four shared engines every app composes.

The document plane: previewing any format inside any app

Step back from the four apps and a single capability comes into focus: given a file of essentially any common type, some engine in the fleet can render and edit it, and because that engine is GUI-free with a uniform `invoke` surface, any app can host that rendering inside its own window. That is the document plane — the union of `zoffice-core`, `zemail-core`, `zpdf-core`, and `zphoto-core` — and it is the single most valuable thing the embed matrix produces, because “show me this file” is a need every desktop app has and the plane solves it once for all of them.

The mechanism is the whole book stacked up. The engines are GUI-free (Chapter 1) so they can render into a host's surface. They expose a uniform `invoke` (Chapter 2) so a host drives any of them the same way. They mount through `ZGui.embed.view` (Chapter 3) so the embedded viewer feeds the host shell rather than fighting it. And they are versioned independently (Chapter 2) so a host pins the engine revision it tested against. A `zcite` showing the PDF of a cited paper is `zpdf-core` mounted through `ZGui.embed.view`, driven by `zct`'s shell through `zpdf`'s `invoke`, pinned to a tested `zpdf-core` revision — every layer of the architecture participating in one live preview.

The document plane is also where the fleet's "faithful port" and "novel architecture" halves meet. Each engine is a faithful rebuild of a predecessor — `zoffice-core` of an office suite, `zpdf-core` of Acrobat, `zphoto-core` of GIMP — and none of those predecessors can be embedded inside a mail client the way `zoffice-core` is embedded inside `zemail`. The predecessors are the prior art for *what a document is*; the composable, embeddable engine is the original architecture for *how a fleet shares document handling*. Neither half is diminished by crediting the other: Acrobat defined PDF editing, and no Acrobat is a library `zcite` links to preview a paper.

Chapter 13 makes the plane concrete by reading the embed matrix that records it — which app consumes which core — but the plane is the reason the matrix is worth reading. Without it, the matrix would be a bookkeeping table; with it, the matrix is the map of a genuine capability, the record of how four document engines reach every app in the fleet so that no app has to know how to render a spreadsheet, a message, a PDF, or an image. It only has to know how to embed the engine that does.

Four engines, four different ABIs, one document plane

The document suite is the sharpest illustration of the C-ABI spectrum from Chapter 2, because its four engines sit at four different points on it and the reason each sits where it does is legible from its domain. `zemail-core` carries the full canonical door: `zml_set_event_callback`, `zml_init`, `zml_invoke`, `zml_string_free`, plus a header and a C++ RAI facade — the full four because mail is asynchronous (`imap.idle` waits for the server to push, and the arrival is an event through the sink). `zphoto-core` carries three — `zphoto_init`, `zphoto_invoke`, `zphoto_string_free` — with no event callback, because an image editor computes on request. `zoffice-core` carries two — `zoffice_invoke`, `zoffice_string_free` — with no `init` (its `Engine` is a stateless unit struct) and no events, and is written on the 2024 edition with `#[unsafe(no_mangle)]`. And `zpdf-core` carries none at all, `rlib`-only, the documented exception. Four engines that share one embed matrix and present four different C surfaces, each fitted to whether its domain has state and whether it has anything to narrate.

The internal shapes differ as legibly as the ABIs. `zoffice-core` routes files to applications through an `App` enum — `Writer`, `Calc`, `Impress`, `Draw`, `Math`, `Base` — and `app_for(path)` maps extensions (`odt/docx/doc/rtf` to `Writer`, `ods/xlsx/xls/csv` to `Calc`, `odp/pptx/ppt` to `Impress`, and so on); its document-compare capability is a `DiffOp` struct and a `diff(a, b) -> Vec<DiffOp>` defined inline in `lib.rs`, plan-as-data like `zftp`'s sync and `ztunnel`'s privilege plan. Its command families are wide and shape-preserving — `writer.*` with `render/pdf/html/markdown/structure/revision_authors`, `calc.*` with `eval/edit_cell/pivot_tables/charts/conditional_formats`, `impress.*`, `draw.*`, `math.*`, `base.open/tables/query` — parsing OOXML and ODF directly over `zip` and `quick-xml` with `RustCrypto` for the encrypted packages, and never embedding an office suite.

`zpdf-core` is the far pole: the one engine with no C ABI and the most-embedded engine in the fleet, and its Rust surface is enormous — roughly three hundred operations in the `zpdf_invoke` dispatch, a flat list rather than a dotted namespace. A representative slice shows the Acrobat-plus-Preview union it targets: `add_note`, `add_ink`, `add_callout`, `apply_redactions`, `redact_search`, `certify`, `add_typed_signature`, `verify_signatures`, `accessibility_check`, `auto_tag`, `reading_order`, `add_qr_code`, `add_data_matrix`, `bates_number`, `flatten_annotations`, `flatten_form`, `downsample_images`, `linearize`, `to_pdf_a`, `to_pdf_x`, `ocr_page`, `make_searchable`, `export_docx`, `export_pptx`, `export_xlsx`, `create_from_images`, `split_by_bookmarks`, `encrypt_aes256_with_permissions`, and `run_javascript` (its own `boa_engine` JavaScript, for PDF actions). That surface is why the crate links natively into every consumer

and why the C door has no caller yet: three hundred operations over `lopdf`'s object model, every one of them the kind of thing Acrobat charges for or Preview lacks, all reachable through the native Rust API that seven sibling apps already speak.

zemail: the full mail stack behind `zml_invoke`

`zemail-core` carries more of a real mail client than its one-line pitch suggests, and the command surface shows it. The account and folder verbs are ordinary (`account.add/remove/list/autoconfig`, `folder.list/create/rename/delete`), but the message surface is a modern mail client's whole workflow: alongside `message.list/get/move/delete/set_flags/threads/save_draft` sit `add_label`, `remove_label`, `categorize`, `junk`, `pin`, `snooze/unsnooze`, `followup`, `set_aside`, `expire`, and `unsubscribe`, plus whole families for the features a power user reaches for — `screeener.*` (approve/pending), `vacation.*` (auto-reply), `vip.*`, `list.*` (mailing lists), `template.*`, `outbox.*` (queue/schedule/due), and `thread.*` (mute/unmute). The transports are `imap.sync/search/idle/folders/store_flags` and `smtp.send` (with `pop3.fetch`), over the `imap` and `lettre` crates behind the `net` feature, with `mail-parser` doing the parsing non-optionally.

The “owned, local-first” pitch is legible in which subsystems are local and which speak standards. Junk is a locally trained classifier (`junk.classify/train/run`); search is a local index with `search.run/save/saved` and `gloda.search`, the last echoing Thunderbird's global-database search by name; contacts sync over standard CardDAV (`contact.import_vcard/export_vcard`, `carddav.fetch/put`) rather than a proprietary API; and calendar invites are `calendar.parse_invite/rsvp`. Encryption is two feature-gated stacks — `openpgp.*` (gen_key/encrypt/decrypt/sign/verify, over the `pgp` crate) and `smime.*` (the same verbs over a pinned pre-release RustCrypto CMS stack) — so a build takes exactly the crypto it wants. The event sink matters more here than anywhere else because mail is asynchronous: `imap.idle` waits for the server to push, and the arrival is an `EngineEvent::SyncFinished { fetched }`, not a response, so the inbox updates live. And `zemail` is simultaneously a provider and a consumer of the document plane — it emits the “mail this” capability through `zml_invoke` while previewing `.docx` attachments through `zoffice-core`, PDFs through `zpdf-core`, and images through `zphoto-core`.

zphoto: layers, non-destructive ops, and the vector stack

`zphoto-core` is a from-scratch GIMP/Photoshop port with an Illustrator-style vector mode layered on top, and its module list is the editor decomposed: `model` (the layer model), `ops`, `paint`, `fill`, `filter`, and `text` (the pixel operations), `vector` (the vector subsystem), and `codec/b54` (image I/O). The command surface is far wider than a paint program's. The non-destructive core is `edit.undo/redo/copy/cut/paste/copy_merged/fade/history` and the adjustment family `op.*` — `levels`, `curves`, `hue_saturation`, `color_balance`, `brightness_contrast`, `exposure`, `black_white`, `gradient_map`, `threshold` — which compose and reverse rather than baking into pixels. The painting family `paint.*` is a professional toolset — `stroke`, `clone`, `heal`, `smudge`, `dodge`, `burn`, `airbrush`, `mixer` — and `select.*` carries real selection tools (`wand`, `color_range`, `feather`, `grow`, `shrink`, `focus_area`, `skin_tones`).

Two subsystems mark it as a modern editor rather than a viewer with a brush. The macro system `action.record/stop/play/load` is Photoshop's actions rebuilt — record a sequence, replay it on other images — and layer comps (`comp.capture/apply`) plus a real undo/redo history mean edits are reversible and composable. The vector mode shares the raster stack: `vec.*` is a large family (`new`, `rect`, `ellipse`, `polygon`, `star`, `pen/path.add`, `pathfinder`, `blend`, `warp`, `gradient_mesh`, `symbol.*`, `artboard.add`) that rasterizes into the same document with SVG interchange, and AI/SVG import routes through `zpdf-core` because an `.ai` file is a PDF underneath — a concrete cross-engine embed inside the document suite itself. Codecs are behind the `codecs` feature (the `image` crate) and text behind `text` (`ab_glyph`), so a headless build that only manipulates layers takes neither. The whole thing is `zphoto_init`, `zphoto_invoke`, `zphoto_`

`string_free` — three C functions and no events — because an image editor computes on request and has nothing to narrate.

zoffice: parsing the containers, not embedding a suite

`zoffice-core`'s durable-deps decision is the most ambitious in the fleet, because it declines to embed LibreOffice or UNO and instead parses and writes the office formats directly. The trick that makes this tractable is that the modern formats are zipped XML: an `.xlsx` or `.docx` is a `zip` archive of `quick-xml`-parseable parts, so the crate's mandatory dependencies are `zip` (deflate only) and `quick-xml`, and the `writer`, `calc`, and `impress` modules read and write those parts directly. The legacy binary formats — `.doc`, `.xls`, `.ppt` — are OLE2 compound documents, handled by the `ole2` module; encrypted packages go through `crypto` and `crypto_odf` over a RustCrypto stack (`aes`, `cbc`, `sha1`, `sha2`, `hmac`, `rsa`, `x509-cert`); and digital signatures live in `dsig`. There is no office suite inside `zoffice`; there is a set of format parsers, which is exactly what “rebuilt from scratch” has to mean for file compatibility.

The command families are wide and deliberately shape-preserving, because the value of a document engine is round-tripping structure, not just extracting text. `writer.*` carries not only `render/pdf/html/markdown/text` but `structure`, `runs`, `tables`, `images`, `links`, `notes`, `footnotes`, `fields`, `comments`, `sections`, `style_definitions`, and `revision_authors` — the document's real anatomy. `calc.*` carries `eval`, `edit_cell`, `formulas`, `sort`, `charts`, `named_ranges`, `validations`, `conditional_formats`, `pivot_tables`, and `print_setups` alongside the render targets. `impress.*`, `draw.*`, `math.*` (with `starmath`), and `base.*` (with `query`) round out the six applications the `App` enum names, routed by `app_for`. Two facts keep the chapter honest: the coverage is partial-by-design and scoped as such, and the ABI is the smallest in the fleet — two functions, `zoffice_invoke` and `zoffice_string_free`, no `init` and no events, because the `Engine` is a stateless unit struct that computes each call from the bytes it is given. Its `diff(a, b) -> Vec<DiffOp>` for document comparison is plan-as-data like `zftp`'s `sync`, defined inline in `lib.rs` rather than in a module — a small honest detail the source shows.

Why four engines and not one document engine

The document plane is four crates, and a reasonable reader will ask why it is not one. The formats overlap — a PDF export path exists in the office engine, an image codec is needed by three of them, text layout is a problem all four have — and a single `zdocument-core` would deduplicate that overlap and present one embedding target instead of four. The reason the fleet did not do that is worth stating, because it is the clearest case in the book of the factoring rule doing something non-obvious.

The rule that governs every split in this stack is *change rate and consumer set*, not subject-matter similarity. The four document domains look adjacent from outside and have almost nothing in common on the inside: a PDF engine is an object-model manipulator over a mature, frozen file format; an office engine is a family of XML and ZIP container parsers tracking two evolving specifications; an image editor is a pixel pipeline with an undo/redo history and a codec layer; a mail engine is a protocol client with a message store and a sync loop. They change for entirely unrelated reasons, they are used by different code paths in every consuming app, and their dependency trees have essentially no intersection — `lopdf` and `boa_engine` on one side, `zip` and `quick-xml` and the RustCrypto stack on another, `image` and `ab_glyph` on a third, `imap` and `lettre` and `mail-parser` on a fourth.

A merged crate would make every consumer of any one of those carry all four dependency trees, which is the exact cost the feature gates of Appendix J exist to avoid — and the gates would have to be reintroduced *inside* the merged crate to fix the problem the merge created, at which point the merge has bought nothing but a shared `lib.rs`. It would also merge four command surfaces into one namespace, so a change to a `writer.*` command name becomes a version bump every PDF consumer has to evaluate. The overlap that motivated the merge, meanwhile, is smaller than it looks: “the office engine exports PDF” is a writer that emits PDF syntax, not a dependency on the full PDF object model and its three hundred operations, and the fleet's

answer to genuine overlap is the same as everywhere else — one engine embeds another, exactly as any app does.

That is the more general point the document plane makes, and it is the reason this chapter sits where it does in the book. The plane is a *capability* — open, render, and edit essentially any file a desktop user meets — and the fleet delivers capabilities by composing separately versioned engines rather than by growing one engine large enough to cover a category. Four engines that compose can be adopted one at a time, gated independently, versioned independently, and embedded by hosts that want exactly one of them; a single document engine could be adopted only whole. For a fleet whose defining property is that any app can mount any engine, the granularity of the engines *is* the granularity of what an app can choose to become, which is an argument for keeping them small and separate even where their subject matter appears to want merging.

Chapter 12 — zgui-core as a Design System

Chapter 3 introduced `zgui-core` as the shared shell; this chapter reads it as a design system. The distinction matters because a shell is a component you mount and a design system is a vocabulary you speak — tokens, a type scale, a component matrix, a conformance spec — and `zgui-core` is the second thing. It is the single source of truth for every control, dialog, table, toast, modal, palette, and chrome element in the fleet, and the whole of the family's visual and interactive coherence is a property of this one submodule. This chapter walks its four pillars: the component surface, the app shell, the token system, and the R1–R10 conformance spec that makes consuming all of it a MUST rather than a suggestion.

The component surface: one global, hundreds of widgets

`zgui-core`'s `webui/` directory is roughly two hundred component files, each an IIFE that hangs one namespace off the single global `window.ZGui`. There is no build step to import a widget and no per-app copy to drift: an app loads the submodule's files and calls `ZGui.<name>`. The surface is large but not sprawling, because it is the accumulated set of “the generic kernel of something one or more apps had already built,” extracted so the N apps stop maintaining N divergent tables — and the README's `webui/` table is the human-maintained roster of what exists.

The everyday subset is small. `ZGui.appShell` is the shell. `ZGui.palette` is the command palette. `ZGui.fzf` is the fuzzy matcher. `ZGui.tabs`, `ZGui.toast`, `ZGui.modal` (with `open`, `confirm`, and `prompt`), `ZGui.settings`, and `ZGui.shortcuts` are the daily chrome. The two table components — `ZGui.table` (a `{ columns, sort, batch, multiFilter }` toolkit of resize/sort/batch engines) and `ZGui.dataTable` (a self-drawing table built on top of `ZGui.table.sort` and `ZGui.table.columns`) — cover R8. Beyond them the surface runs deep: form controls (`textField`, `checkbox`, `toggle`, `range`, `select`, `radio`, `searchBox`, `searchSelect`, `pinInput`, `datePicker`, `colorPicker`), layout (`splitPane`, `dockPanel`, `resizable`, `drawer`, `accordion`, `scrollArea`), HUD (`statStrip`, `headlineStats`, `playingPill`, `heatColor`), data views (`tree`, `jsonView`, `xmlView`, `csvView`, `tomlView`, `logView`, `markdown`, `codeEditor`), the node-graph editor (`nodeGraph`), the tile dashboard (`tiles`, `tileGrid`), the preferences shell (`prefsShell`), the launcher chrome (`launcher`), and the effects layer (`splash`, `crt`, `neonFrame`, `glitchText`). The naming has a few deliberate edges the book uses throughout: the palette is `ZGui.palette`, not `commandPalette`; the HUD helpers are top-level `ZGui.statStrip/headlineStats`, not grouped under `ZGui.hud`; and `ZGui.table` and `ZGui.dataTable` are different things.

The app shell in one call

`ZGui.appShell(root, opts)` is the keystone that delivers most of the conformance spec in a single mount. It reads an options object and returns a handle:

```
ZGui.appShell(root, {
  brand: { glyph, title, subtitle }, // R6: logo + title, top-left
  filterPlaceholder, onFilter(q), // R7: fuzzy filter, highlighted
  palette: [{ label, hint, run }], // R1: the Cmd/Ctrl+K command palette
  settings, settingsExtra(bodyEl), // one settings panel; apps inject their rows
  onColorscheme(id), // R4: theme switch over the token set
  onOpenLog(), onOpenLogDir(), onOpenDataDir(), // diagnostics buttons
  actions: [Node], version, hotkeys, contextMenu, splash,
}) // -> { el, body, filterInput, openSettings, openShortcuts, setPaletteItems }
```

The handle's `body` is where the app puts its content; `setPaletteItems(items)` feeds the one palette (it does `palette.clear()` then `palette.register(items)`); and `openSettings/openShortcuts` open the shared panels. The shell is a *single-instance* component, guarded by `document.documentElement.dataset.zguiShellMounted` (`data-zgui-shell-mounted`), and mounting it twice throws with a message that is also the architecture in one sentence: “appShell already mounted — only the top-level app mounts the shell; an embedded core must use buttons + scoped APIs.” On mount it stamps a *baseline* set — `colorscheme`, `customScheme`, `filter`, `crt`, `neon`, `logAccess`, `palette`, `shortcuts` — recording which family-baseline features the app has wired, and it toggles effects with body classes (`no-scanlines`, `no-vignette`) driven by `window.prefs`. An app's `main.js` is, in large part, one `appShell` call plus the wiring of its engine's JSON into that handle.

The token system: colours, glass, and two bundled fonts

The look is a token system, not a skin, and the tokens are exact. `cyberpunk.css` defines the palette as CSS custom properties on `:root`:

```
:root {
  --bg-primary:#05050a; --bg-secondary:#0a0a14; --bg-card:#0d0d1a; --bg-hover:#12122a;
  --accent:#ff2a6d; --accent-light:#ff6b9d; --accent-glow:rgba(255,42,109,0.4);
  --cyan:#05d9e8; --magenta:#d300c5; --green:#39ff14; --red:#ff073a;
  --text:#e0f0ff; --text-dim:#7a8ba8; --text-muted:#3d4f6a; --border:#1a1a3e;
  --glass-blur: blur(16px) saturate(165%);
  --glass-fill: linear-gradient(145deg, rgba(90,140,180,0.16), rgba(10,12,26,0.40));
  --glass-sheen: inset 0 1px 0 rgba(255,255,255,0.22);
}
```

The hot magenta-pink `--accent: #ff2a6d`, the cyan `--cyan: #05d9e8`, the near-black `--bg-primary: #05050a`, and the glassmorphism group (a frosted `--glass-fill`, the `blur(16px) saturate(165%)` blur, and the `--glass-sheen` inner highlight) are the whole visual vocabulary, and R4 requires new surfaces to read the variables rather than hardcode colours so an element moved between apps looks the same untouched. The type is part of the system and part of the reason it survives a clean machine: two fonts, **Orbitron** (variable weight 400–900, for headings and HUD numbers) and **Share Tech Mono** (for terminal and body), are bundled as `@font-face` from vendored TTFs (`Orbitron-VF.ttf`, `ShareTechMono-Regular.ttf`) with `font-display: block`, not resolved against a system install — because the requirements spec's first pitfall is that an unbundled font renders tofu on a machine that lacks it.

Eight schemes over twenty-four keys

Because it is a token system, theming is a runtime operation, not a rebuild. `ZGui.colorscheme` ships eight built-in schemes — `cyberpunk`, `midnight`, `matrix`, `ember`, `arctic`, `crimson`, `toxic`, `vapor` — each with a dark set and a `lightVars` variant, and applying one is writing a fixed set of custom properties onto `:root`. That set is `SCHEME_VAR_KEYS`, twenty-four keys: `--accent`, `--accent-light`, `--accent-glow`, `--cyan`, `--cyan-glow`, `--cyan-dim`, `--magenta`, `--magenta-glow`, `--green`, `--green-bg`, `--yellow`, `--yellow-glow`, `--orange`, `--orange-bg`, `--red`, `--text`, `--text-dim`, `--text-muted`, `--bg-primary`, `--bg-secondary`, `--bg-card`, `--bg-hover`, `--border`, and `--border-glow`. Every component draws from those

twenty-four, so picking a scheme recolours the whole app at once, and the API is correspondingly small: `apply`, `applyVars`, `setLight`, `load`, `buildSwitcher`, a custom-scheme editor (`buildEditor`, `applyCustom`), pre-set save/load (`savePreset/loadPreset`), and the pane-scoped `scope(el, schemeOrVars)` that Chapter 14 needs for embeds. The effects layer — CRT scanlines (`crt`), neon glow (`neonGlow`), the splash overlay (`splash`) — is likewise centralized and toggleable, disabled per body with `no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim` for accessibility. The visual language is coherent because it is one token file, one font pair, one scheme engine, and one effects layer, consumed by every app.

R1 through R10: the conformance spec as a live checklist

`GUI_APP_REQUIREMENTS.md` turns all of the above from “available” into “required,” and its opening line is the design philosophy in three words: “**Divergence is a bug.**” Ten requirements are a MUST for every app, each naming the canonical shared source an app must consume rather than reimplement:

- **R1 — Command palette (Cmd/Ctrl+K):** a fuzzy palette listing every actionable command, backed by `command-palette.js`, matched by the shared `fzf` (R7); every button and shortcut must also be palette-reachable.
- **R2 — Stryke Hooks editor:** every app embeds the shared `zpwr-hooks-editor` (Monaco), built once as an esbuild bundle and vendored — apps never fork it.
- **R3 — Embedded terminal:** a real PTY-backed terminal (`zpwr-embed-terminal`, `xterm.js`) on Ctrl+back-tick and Cmd+T with a palette entry, dock-to-corner drag, a “Kill & close” (X) that kills the PTY and a hide (-) that does not, rendering a *bundled* Nerd Font, never a system one.
- **R4 — Shared styles:** one visual language from `cyberpunk.css` — the tokens, the glass surfaces, Orbitron and Share Tech Mono — with new surfaces using the token variables so an element moved between apps looks identical.
- **R5 — Tile dashboard + tab bar:** a tab bar switching top-level views (active tab unambiguous) and a tile-grid landing surface, both from the shared components.
- **R6 — Logo, top-left:** the MenkeTechnologies logo top-left in the shared header (logo and title left, status and chips right).
- **R7 — Filters: fzf with highlighted matched characters:** every filter is fuzzy via the shared matcher and highlights matched chars — no `includes()` substring filter, no second fuzzy matcher. “One matcher, one highlight style, everywhere.”
- **R8 — Tables: sortable + resizable columns:** click-to-sort with an active indicator and drag-to-resize with widths persisted per table, from the one shared table component, never hand-rolled.
- **R9 — DAW arrangement grid:** every app embeds the shared `zpwr-clip-engine` grid via `createGrid({ canvas, domain, store, storageKey, onChange, popover })` — “one renderer, one interaction model, N domains, host-injected transport.”
- **R10 — File browser:** the multi-pane browser ported from Audio-Haxor and promoted to the shared `zpwr-file-browser` submodule, with resizable panes, a hidden-file toggle, rename/copy/move, keyboard nav, per-pane path persistence, and fs access behind a host-provided backend.

The spec is candid about where conformance is still being reached, and the book follows it: the command palette is not yet single-source across the older audio apps, the two substrates still split their stylesheet source, and R9’s arrangement grid is embedded only in the DAW so far. What is fully converged is the newer Tauri roster this book is about — `zreq` and `zcite` are called out by name as routing every table, filter, and tab through `ZGui.table`, `ZGui.fzf`, and `ZGui.tabs`. R1 through R10 is a live checklist, green where it matters most and closing the rest, and the fuzzy matcher underneath R1 and R7 emits its matched characters wrapped in `<mark class="fzf-hl">` so that “one highlight style, everywhere” is a literal fact about the markup, not an aspiration.

Extracted, not designed: the lineage of the toolkit

A design system this large from one author is only plausible because it was not designed top-down; it was *extracted*. `zgui-core`'s README is explicit about the lineage: the toolkit was pulled out of Audio-Haxor, `zreq`, and `zpw-r-patch-core` — the apps that had already, independently, built the table, the palette, the fuzzy filter, and the HUD strip that the fleet now shares. That provenance is why so many components carry the exact markup those apps had settled on: `ZGui.fzf` is Audio-Haxor's matcher extracted verbatim so every filter highlights identically, and `ZGui.prefsShell` is the preferences window distilled from `zgo`'s Alfred-style prefs and promoted into the toolkit, which `zgo` then consumes back. The reuse loop runs in both directions: an app builds the generic kernel of something, the kernel is lifted into `zgui-core`, and every app — including the one that built it — consumes the shared version.

Extraction rather than design is also why the surface is large without being incoherent. Each component earned its place by being the deduplicated form of something N apps had already written N times, so the toolkit is the accumulated answer to real divergence rather than a speculative library of widgets nobody asked for. It is also why the toolkit evolves the same way the engines do: when the design changes, it changes in `zgui-core` and reaches every app on the next submodule bump — the same reuse story as the engines, applied to the front end. “Ten apps cannot look like ten apps” is answered not by a shared aesthetic each app re-implements but by a shared dependency each app pins, grown from the code those apps had already written.

Effects, splash, and accessibility toggles

The visual identity is not only the colours and the type; it is also the effects layer, and `zgui-core` treats effects as a centralized, toggleable system rather than a scatter of per-app CSS. The CRT scanlines (`ZGui.crt`), the neon glow (`ZGui.neonGlow`), the bezel vignette, and the boot splash (`ZGui.splash`) are all components with a single implementation, applied through the shell, so the whole fleet shares one look down to the animation. Because they are centralized they are also controllable: the shell toggles them per body with classes — `no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim` — driven by `window.prefs`, so a user who finds the scanlines distracting or needs reduced motion turns them off once and every component respects it.

That accessibility surface is a design decision, not an afterthought, and it interacts directly with the embed contract. The effects are full-document overlays — a CRT layer or a splash spans the whole window — which is exactly why `ZGui.crt`, the `splash` methods, and `neonGlow.set/toggle` are on the `FORBIDDEN` list an embedded core may not call (Chapter 14): a guest that painted a CRT layer would paint over its host. So the effects belong to the top-level app, toggled through the shell's settings panel, and an embedded core inherits whatever the host has set rather than imposing its own. One effects system, one set of accessibility toggles, owned by the shell and off-limits to guests — the same centralization that makes the look coherent is what makes it safe to share a window.

table and dataTable: two components, one job

The two table components are a good case study in how the toolkit factors a capability, because they are different things that satisfy R8 at different levels. `ZGui.table` is the low-level toolkit — an object of engines, `{ columns, sort, batch, multiFilter }` — where `columns` handles drag-to-resize with persisted widths, `sort` handles click-to-sort with a persisted active state, `batch` handles checkbox multi-select, and `multiFilter` handles column filtering. An app with an existing table markup reaches for these engines to make its table sortable and resizable without re-rendering it. `ZGui.dataTable` is the higher-level convenience built on top of `ZGui.table.sort` and `ZGui.table.columns`: give it `{ columns, rows, rowKey, onRowClick, sortScope, resizable }` and it draws the table itself, returning `{ el, setRows, sort, getSort }`.

The reason both exist rather than one is that they serve two consumption styles without divergence. An app that

wants a table drawn for it uses `dataTable` and gets sorting and resizing for free; an app that already renders its own rows (because the cells are custom) uses the `table` engines to add sorting and resizing to markup its controls. Crucially, `dataTable` is not a fork of the sorting logic — it calls `table.sort` and `table.columns` — so there is still one sort implementation and one resize implementation in the fleet, satisfying R8’s “from one shared table component, never hand-rolled” even though two names appear on `window.ZGui`. It is the design system’s general move in miniature: factor the capability into a reusable kernel, then offer a convenience layer over it, and make sure the convenience layer consumes the kernel rather than duplicating it.

One global, no build step: what that choice gives and gives up

The toolkit’s delivery mechanism is worth examining on its own, because it is unusual enough in a modern front-end context to look like an oversight rather than a decision. `zgui-core` is roughly two hundred files, each an IIFE that hangs one namespace off `window.ZGui`, loaded as plain scripts. There is no module graph, no bundler, no tree shaking, no import statement anywhere in a consuming app. An app loads the submodule’s files and calls `ZGui.<name>`, and that is the entire integration story.

What that buys is proportional to how much of this book is about a solo author. It means the toolkit has no build step to maintain, which means it has no build-tool dependency to keep working across a decade, which means the toolkit cannot be broken by an upgrade to something that is not the toolkit. It means a component is editable and immediately testable by reloading a page, with no compile between the edit and the result. It means the vendoring step of Appendix D is a directory copy rather than a package resolution, which is why `copy-webui.mjs` can be a `cpSync` and why the same files run unchanged on the JUCE substrate — there is nothing substrate-specific about a script tag. And it means the embed contract of Chapter 14 is even implementable: `ZGui.embed.view` swaps a fixed set of paths on a live object for the duration of a call, which is a trivial operation on a namespace object and an awkward one against a module graph whose bindings are resolved at load.

What it gives up is equally concrete and the chapter should say so. There is no per-app tree shaking, so an app that uses forty components ships the same toolkit as one that uses two hundred; for a desktop app loading from local disk that cost is paid once at startup rather than over a network, which is why the fleet can accept it and a web product could not. There is no static analysis of the surface, so a call to a component that does not exist is a runtime `undefined`, not a compile error — the same trade the command bridge makes at the Rust boundary, arriving at the front end for the same reason and mitigated the same way, by the surface being enumerated in one place (Appendix C) and exercised by the apps that consume it. And load order matters: a script that calls `ZGui.appShell` before the shell’s file has executed gets nothing, which is a class of bug a module system would have made impossible.

The consistent thread is that the toolkit optimizes for durability and directness over the guarantees a build pipeline provides, which is the same instinct as the durable-deps rule one layer up. A `-core` crate refuses a dependency it could hand-roll in a few hundred lines; the toolkit refuses a build system it can do without. In both cases the thing being avoided is not complexity for its own sake but a *moving* dependency — a crate that will churn, a bundler that will need migrating — in a stack whose author intends to maintain it for a long time from one desk. Read that way, “two hundred IIFEs on one global” is not a legacy pattern the toolkit has not modernized past; it is the same endgame reasoning the engines apply, expressed in JavaScript.

The one place the choice creates a genuine constraint rather than a trade is naming. Because everything lives on one object, every component’s name is a global name, and the surface has to stay legible without namespacing to disambiguate. That is why Chapter 12 had to spend a paragraph on the naming edges — the palette is `ZGui.palette` and not `commandPalette`, the HUD helpers are top-level rather than grouped under `ZGui.hud`, `table` and `dataTable` are different things — and it is why those edges are worth documenting rather than smoothing over. In a module system those would be file paths and nobody would need to remember them; on one global they are vocabulary, and vocabulary has to be learned once and then used consistently by every app. The flat namespace is a readable API exactly as long as its names are chosen as carefully as an engine’s command strings are, and for the same reason: in both cases the name *is* the interface.

Chapter 13 – The Embed Matrix

The payoff of the whole design, made concrete. This chapter reads the `COMPONENTS.md` consumption matrix column by column: which engine each app embeds, why the JUCE audio plugins embed only the clip/patch engines, and how a change in one `-core` propagates to every shell that vendors it.

Reading the consumption matrix

`COMPONENTS.md` is the fleet's ledger, and its consumption matrix is a table with apps down the side and shared components across the top, a checkmark where an app consumes a component. The source of truth is each app's `.gitmodules`; the table is the human-readable projection of which submodules each app actually pins. Reading it is reading the architecture's realized shape rather than its intended one — not “which apps *could* share this engine” but “which apps *do*,” recorded in the git history as submodule pointers.

The columns divide into two kinds, and the distinction is the chapter's organizing idea. Some components are consumed by copy at build time and sit outside the per-component matrix — `zgui-core` (the `window.ZGui` toolkit) and `zpwr-i18n` (localization) are vendored into each app's `frontend/` by a copy step, the way `GUI_APP_REQUIREMENTS.md` describes. Others are the embeddable engines and shared web modules the matrix tracks by column: `zpwr-embed-terminal`, `zpwr-hooks-editor`, `zpwr-file-browser`, and the document cores `office-core`, `mail-core`, and `pdf-core`. The matrix's own note is explicit that “any `-core` engine can be embedded in any GUI app, and every GUI app embeds the full shared component set it needs.”

The document-core columns are the ones this book cares about most, and the matrix shows them consumed nearly universally: `office-core`, `mail-core`, and `pdf-core` carry checkmarks across essentially every full GUI app — `zcite`, `zreq`, `zphoto`, `zoffice`, `zemail`, `zpdf`, `zterminal`, and the audio apps — with `(source)` marking the app that owns each engine (`zoffice` owns `office-core`, `zemail` owns `mail-core`, `zpdf` owns `pdf-core`). That near-solid block of checkmarks is the document plane of Chapter 11, drawn as a table: every app previews office files, mail, and PDFs because every app embeds the three engines that handle them.

Some engines have no column because they are app-specific, and the matrix says so: `zphoto-core`, `ztmux-core`, and by the same logic the newer `zreq-core`, `zcite-core`, `ztunnel-core`, `zgo-core`, `zftp-core`, and `zcontainer-core` are engines owned by one app that *can* be embedded but are tracked by their own `.gitmodules` rather than given a standing matrix column. The matrix is a living document — “last reconciled” dated, with a notes section tracking splits and extractions in progress — so its columns are the engines common enough to be worth a standing column, and the absence of a column means “app-specific, embed it if you need it,” not “not embeddable.”

Shared components vs. per-app engines

The matrix draws a line between two things that both get shared but get shared differently. The shared *component set* — the embedded terminal, the hooks editor, the file browser, the command palette, the fzf matcher, the arrangement grid — is the family-baseline chrome from `GUI_APP_REQUIREMENTS.md`, the R1-through-R10 machinery every app must consume so that “open any app and the palette, terminal, tables, and file browser behave identically.” These are consumed by nearly every app almost by definition, because conformance requires them. An app that did not embed the shared terminal would fail R3.

The per-app *engines* are the other kind of sharing, and they are consumed by need rather than by mandate. No requirement says `zreq` must embed a citation engine, but if it wanted “cite this endpoint’s docs” it could embed `zcite-core`; no rule says `zoffice` must embed a transfer engine, but if it wanted “save to S3” it could embed `zftp-core`. The document cores sit in between — so universally useful that they are consumed almost everywhere, but by the pull of “every app shows documents” rather than by a conformance checkbox. The matrix’s checkmarks for `office-core/mail-core/pdf-core` are need made near-universal, not mandate.

The JUCE audio plugins are the instructive exception, and the matrix explains them in a note. `zpwr-synth`, `zpwr-fx`, and `zpwr-midi-fx` consume *only* the clip and patch engines (`zpwr-clip-engine`, `zpwr-patch-core`) and none of the document cores or shared terminal — because a synthesizer plugin loaded inside a DAW is not a desktop app that shows PDFs, it is a real-time audio processor with a tiny UI. The matrix’s dashes across those rows are not gaps to close; they are “not applicable to that app,” a plugin correctly declining the desktop-app baseline it does not need. The consumption matrix is honest about non-consumption, which is what makes the checkmarks meaningful.

That distinction — baseline chrome every app must consume, engines each app consumes by need, and plugins that consume only what a plugin needs — is the whole embed economy in one paragraph. The chrome makes the apps a recognizable family; the engines make them compose; the plugins opt out of what does not apply. And all three are recorded in one table backed by real `.gitmodules`, so the architecture’s sharing is not a diagram of intent but a checkable fact about what each app actually pins.

How an engine bump reaches ten apps

When a `-core` engine gets a fix, the propagation is a submodule-pointer cascade, and the mechanics are worth spelling out because they are the daily cost of the whole design. `zpdf-core` is a git repository; each app that embeds it pins a specific commit of it as a submodule. A fix to `zpdf-core` lands in the engine’s repository as a commit; then each consuming app must bump its submodule pointer to that commit, test, and commit the bump. Ten consumers means ten pointer bumps to fully propagate a fix, each one a recorded commit saying “this app now ships `zpdf-core` at revision X.”

This is slower than a monorepo where one edit changes every consumer at once, and the book does not pretend otherwise. The upside is precision and isolation: each app upgrades on its own schedule, pins exactly the revision it tested against, and cannot be broken by an engine change it has not yet adopted. A regression in `zpdf-core` reaches only the apps that have bumped to it; the rest are still on the old, working pointer. The cascade trades propagation speed for the guarantee that no app ships an engine revision it did not choose, which for a fleet of independently released products is usually the right trade.

The matrix is what makes the cascade tractable, because it tells you the blast radius before you start. Bumping `zpdf-core`? The `pdf-core` column tells you exactly which apps consume it and therefore which pointers need bumping. Bumping something in the baseline chrome? The requirements roster tells you it is everyone. The propagation is still manual work, but it is *bounded, known* work — you are never guessing which apps a change touches, because the matrix records it. A fleet without that ledger would face the cascade blind; with it, every bump has a checklist.

The stack’s own tooling reinforces the cascade at the top: the superproject that tracks all the apps as submodules gets a pointer bump when any app moves, so the meta-repository is a cascade of cascades — engine bumps into apps, app bumps into the meta. It is submodules all the way up, and each level is a commit recording exactly which revision of everything below it shipped together. That is heavy machinery for a solo author, which is why Chapter 15 is about whether it is worth it; the answer the matrix suggests is that the ledger is what keeps the machinery from becoming chaos.

The cost model: what sharing buys and what it constrains

Sharing buys reuse, and the reuse compounds. Build `zpdf-core` once and every app previews PDFs; build `zgui-core`'s table once and every app has sortable, resizable columns; build the `invoke`/C-ABI/Tauri-plugin pattern once and every new engine gets three front doors for free. The tenth app is dramatically cheaper than the first, because the tenth app is a thin shell over an engine, mounting a shell it did not write, embedding engines it did not build, speaking a boundary pattern it did not invent. That compounding is the entire economic case for the architecture, and the embed matrix is its evidence — every checkmark is a capability an app got without building it.

Sharing constrains, too, and the constraints are real. A shared component is a shared *contract*: `zgui-core` cannot change `ZGui.appShell`'s signature without touching every app that mounts it, so the toolkit's API is harder to evolve than a per-app widget would be. A shared engine is a shared *dependency surface*: a breaking change to `zpdf-core`'s `invoke` commands breaks every consumer, so the engine's command names are effectively public API. The freedom to change any one piece is traded for the leverage of changing it everywhere at once, and past a certain fleet size the shared pieces calcify into interfaces you evolve carefully rather than freely.

The `ZGui.embed.view` mechanism from Chapter 3 is the clearest case of a constraint that is worth its cost. Sharing one document and one keyboard between a host and an embedded core *requires* the discipline that the core not seize globals — and rather than trust every core author to remember, the toolkit enforces it by turning “seize a global” into an exception. That enforcement is a constraint on how a core may behave when embedded, and it is exactly the constraint that makes live embeds safe. The cost model here is favorable: a small, mechanically enforced constraint (feed the host, don't grab globals) buys a large capability (any engine live inside any app).

The honest bottom line is the one the whole book has been building toward: the shared architecture makes a ten-app fleet *cheaper to build* and *harder to change*, and for a solo author shipping faithful ports of well-understood products, that is the right side of the trade. The products are known — Postman, Zotero, Cyberduck — so their engines' interfaces are stable, so calcifying those interfaces costs little. The leverage of one toolkit, one shell, one boundary pattern, and a document plane every app embeds is enormous. Sharing constrains change and buys reach, and a fleet of stable-target ports wants reach more than it wants freedom to churn its interfaces. Chapter 15 is what that trade looks like from the one desk that made it.

The consumption matrix, reproduced

The meta-repository's `COMPONENTS.md` is the ledger, and it is worth reading in the concrete rather than the abstract. Its consumption matrix has apps down the side and shared submodules across the top, a check where an app consumes a component, a dash where it does not apply, and `(source)` where an app owns the engine that column tracks. The columns are the family-baseline components and the document cores; a condensed reading of the real table looks like this:

app	embed- terminal	hooks- editor	file- browser	i18n	office- core	mail- core	pdf- core
zoffice	✓	✓	✓	✓	(source)	✓	✓
zemail	✓	✓	✓	✓	✓	(source)	✓
zpdf	✓	✓	✓	✓	✓	✓	(source)
zcite	✓	✓	✓	✓	✓	✓	✓
zreq	✓	✓	✓	✓	✓	✓	✓
zphoto	✓	✓	✓	✓	✓	✓	✓
zterminal	-	✓	✓	✓	✓	✓	✓
zpwir-synth	-	-	-	-	-	-	-
zpwir-fx	-	-	-	-	-	-	-

Three things read straight off that block. First, the document-core columns — `office-core`, `mail-core`, `pdf-core` — are a near-solid wall of checks across every full GUI app, which is the document plane of Chapter 11 drawn as a table: every app previews office files, mail, and PDFs because every app embeds the three engines that handle them, with `(source)` marking the one app that owns each. Second, `zgui-core` and `zpwir-i18n` are deliberately *not* columns — the note is explicit that they are consumed by copy at build time (“copy `webui/*` into each app’s `frontend/`”), so they sit outside the per-component matrix. Third, the JUCE plugins (`zpwir-synth`, `zpwir-fx`, `zpwir-midi-fx`) are a row of dashes across the document cores and the terminal, because a synth loaded inside a DAW is a real-time audio processor with a tiny UI, not a desktop app that shows PDFs — the dashes are “not applicable,” a plugin correctly declining a baseline it does not need. The matrix is honest about non-consumption, which is what makes the checks mean something.

Submodules all the way up: the superproject cascade

The matrix records who embeds what; the git structure records it as pinned commits, and the shape is submodules all the way up. Each `-core` engine is its own git repository; each app pins the engines and shared crates it consumes as submodules (the Rust pieces under `crates/`, the JavaScript toolkits under `frontend/lib/`); and the meta-repository pins all the apps. It is a tree of pinned commits whose leaves are engines and toolkits and whose trunk is the fleet frozen at one consistent revision set. A fix to `zpdf-core` is a commit in the engine; then a submodule-pointer bump in each app the `pdf-core` column shows consuming it; then a pointer bump in the meta. Ten consumers is ten recorded bumps, each a commit that says “this app now ships `zpdf-core` at revision X.”

That is slower than a monorepo where one edit changes every consumer at once, and the book does not pretend otherwise — but the matrix is what makes the cascade tractable rather than blind. Bumping `zpdf-core`, you read the `pdf-core` column and know exactly which pointers need bumping; bumping something in the baseline chrome, the requirements roster tells you it is everyone. The propagation is manual but *bounded*, *known* work, never a guess about blast radius. And the upside the slowness buys is isolation: a regression in an engine reaches only the apps that have bumped to it, and the rest sit on their old, working pointers until they choose to move. The cascade trades propagation speed for the guarantee that no app ever ships an engine revision it did not deliberately adopt — which, for a fleet of independently released products, is usually the right trade, and the ledger is what keeps the machinery from becoming chaos.

Reading the matrix backwards

The matrix is normally read forwards — pick an app, see what it consumes — and reading it in the other direction is a more useful exercise, because it converts a table of checkmarks into an estimate of work not done. Take a row and ask what that app would have had to build if every check in it were a dash.

`zcite` is the instructive row because its checks are unremarkable. It consumes the embedded terminal, the hooks editor, the file browser, the `i18n` catalogue, and all three document cores. Strike them out and the reference manager’s scope changes shape entirely. Showing the PDF of a cited paper — the single most characteristic thing a reference manager does — becomes building a PDF renderer, which is not a feature of a

citation manager but a project comparable in size to the citation manager itself. Previewing an attached `.docx` becomes an OOXML parser. Previewing an attached image becomes a codec layer. The terminal and file browser become a PTY driver and a file manager's worth of filesystem operations. What remains after those subtractions — the library model, the saved searches, the CSL processor, the identifier lookups, the duplicate detection — is `zcite`'s actual domain, and it is a fraction of what the shipped application does.

That subtraction is the compounding-reuse argument stated as a quantity rather than a principle, and it explains a fact about the fleet that is otherwise puzzling: how the apps can be *feature-comparable* to their predecessors while being individually small. They are not small. They are thin at the top of a stack that is large, and the matrix is the record of how much of each app is stack rather than app. An app's own repository contains its domain engine and a shell; everything the matrix checks arrives already built, already tested, already conformant, and already fixed for whatever bugs the other consumers have found in it.

The reading also makes the constraint side of Chapter 13's cost model concrete in the same motion. Every check is a capability an app got for free *and* an interface the app is now pinned to. `zcite` did not write a PDF viewer, and `zcite` also cannot change how the PDF viewer behaves without changing it for `zreq`, `zoffice`, `zemail`, `zphoto`, and every other row with a `pdf-core` check. The checkmark is simultaneously the leverage and the lock-in, which is why the same table serves as both the reuse ledger and the blast-radius map for a change. A row read forwards tells you what an app inherits; a column read downwards tells you who you will break.

There is a third reading, and it is the one that says the most about whether the architecture is working. Look for rows where an app consumes an engine it has no obvious domain relationship to. A mail client consuming an office engine is that: nothing about IMAP requires reading spreadsheets, and `zemail` consumes `office-core` because users attach documents to mail, which is a fact about users rather than about mail protocols. Those cross-domain checks are the evidence that the embed matrix is describing genuine composition rather than a shared-utilities table dressed up as architecture. A fleet whose sharing was only chrome and only utilities would have a matrix where every check was predictable from the app's category. This one has checks that are predictable only from what users actually do with files, which is the property Chapter 11's document plane claims and the matrix is the receipt for.

Chapter 14 – The Embed Contract and Cross-App Composition

The embed matrix of Chapter 13 records *which* app consumes which engine; this chapter is *how* a consumed engine mounts live inside a host without breaking it. The requirement, stated in Chapter 3 and cashed here, is that a cross-app embed be real — a working sibling engine drawing into the host's window, not a screenshot or a “coming soon” panel. That is only possible because the engines are GUI-free (Chapter 1) and the chrome is shared (Chapter 12), but it introduces one genuine hazard, and `zgui-core` answers the hazard mechanically rather than by asking authors to be careful.

The hazard: two components, one document, one keyboard

The failure is concrete and it is about globals. A host app binds Cmd+K for its command palette; if an embedded core also binds Cmd+K for *its* palette, the two bindings fight on one `document` and the app reads as broken. The same collision lurks for the settings panel, the global shortcut map, the `:root` colorscheme, and the full-document CRT and splash overlays — every place a component reasonably assumes “I am the whole app.” An embedded core that innocently calls `ZGui.appShell`, or `ZGui.palette.bindHotkey()`, or `ZGui.colorscheme.apply()` assumes exactly that, and inside another app the assumption corrupts the host: a second shell mount, a stolen hotkey, a `:root` theme written out from under the host's own.

The requirements spec names this in its pitfalls — “embedded cores feed the host's keyboard; they never bind their own” — and points at the two-⌘K collision as the canonical break. The reason the rule cannot be left to discipline is that the innocent call and the corrupting call look identical: `ZGui.colorscheme.apply("matrix")` is correct in a top-level app and wrong in an embedded pane, and nothing about the call site tells the author which world the code is running in. So the toolkit has to draw the line at runtime, where it knows.

`ZGui.embed.view(fn)`: forbidden globals as throwing stubs

`webui/embed.js` enforces the boundary by swapping a hard-coded set of global-seizing `ZGui` paths for throwing stubs, for the duration of the embedded view's init. The `FORBIDDEN` list is exact:

```
// ZGui.embed – the paths an embedded core may NOT call (swapped for throwing stubs)
["appShell"],
["palette", "bindHotkey"],           // seizes the global ⌘K
["shortcuts", "init"],              // installs the global keymap
["colorscheme", "apply"], ["colorscheme", "applyVars"], ["colorscheme", "setLight"],
["colorscheme", "load"], ["colorscheme", "buildSwitcher"], ["colorscheme", "buildEditor"],
["splash", "show"], ["splash", "hide"], ["splash", "setVersion"],
["crt"], ["neonGlow", "set"], ["neonGlow", "toggle"], // full-document overlays
["baseline", "markAll"], ["baseline", "mark"], ["baseline", "require"],
```

An embedded view runs its setup inside `ZGui.embed.view(fn)`; for the length of `fn` each of those paths is a stub that throws, and the originals are restored in a `finally`. The thrown message is also the way out: `"[zgui]`

`ZGui.<path>()` is an app-level global call – forbidden in an embedded core view. Open it from a button with the scoped API (`buttonBar / palette.create({scope}) / colorscheme.scope`).² An embedded core that reaches for a document-global singleton fails loudly at init, naming both the forbidden call and the scoped API to use instead — a corrupted host is turned into an immediate, self-explaining exception. Note that `baseline.markAll` is on the list: an embedded core may not stamp the family-baseline flags, because that is the top-level app’s claim to make, not a guest’s.

The scoped APIs: contribute through the host, don’t seize globals

The APIs that remain callable inside `embed.view` are the whole trick, because they let a core *add* to the host instead of *replacing* it. A core opens its own palette from a button with `ZGui.palette.create({ items, scope: paneEl })` — a scoped overlay rendered inside its pane, not a grab of the global Cmd+K. It themes only its pane with `ZGui.colorscheme.scope(paneEl, "matrix")` — writing the twenty-four scheme variables onto that element, not onto `:root`. It builds its local controls with `ZGui.buttonBar` rather than mounting a second `appShell`. And it feeds the host’s one palette and one settings panel through the shell handle it was given — `setPaletteItems(...)` and the `settingsExtra(bodyEl)` callback — so the host shell still owns the single palette, the single settings panel, and the global keyboard, while the guest contributes entries to them. The embed reads like this:

```
ZGui.embed.view(() => {
  // ZGui.palette.bindHotkey() here would THROW – the FORBIDDEN stub fires.
  const pal = ZGui.palette.create({ items: coreCommands, scope: paneEl });
  coreButton.onclick = pal.open; // button-opened, no global ⌘K
  ZGui.colorscheme.scope(paneEl, "matrix"); // theme this pane, not :root
  hostShell.setPaletteItems(coreCommands); // contribute to the host palette
});
```

A live embed, walked: `zcite` shows a cited paper’s PDF

Put the pieces together and a real cross-app embed is legible end to end. A user in `zcite` clicks a cited paper’s attachment; `zcite` wants to show the PDF inline. The PDF engine is `zpdf-core`, which `zcite` already vendors as a submodule (the `pdf-core` check in the matrix) and links natively — no C ABI needed, because both are Rust/Tauri. `zcite`’s shell mounts `zpdf`’s mountable view into a pane inside `ZGui.embed.view`, so the PDF viewer’s setup runs under the forbidden-globals guard; the viewer drives `zpdf-core` through the same `zpdf_invoke` command surface the standalone `zpdf` app uses (`render_page`, `list_annotations`, `search`), pinned to the `zpdf-core` revision `zcite` tested against; and the viewer opens its own tools from a scoped palette and a button bar, contributing a “cite this page” entry to `zcite`’s host palette rather than seizing Cmd+K. Every layer of the architecture participates in one preview: GUI-free engine, uniform `invoke`, scoped embed, independent version pin.

The same walk describes `zemail` previewing a `.docx` attachment through `zoffice-core`, an inline image through `zphoto-core`, or an attached PDF through `zpdf-core` — the mail client is simultaneously a provider of one service (`zemail-core`, the “mail this” capability) and a consumer of the other three. That composition is the payoff the whole book has been building toward, and it is the document plane of Chapter 11 made interactive: not ten apps that each reinvent document preview, but four shared engines every app mounts live, each behind a boundary that turns “seize a global” into an exception so the mount is safe. The embed contract is small — a list of forbidden paths, a handful of scoped APIs, one throwing stub each — and it is exactly that smallness, mechanically enforced, that makes the fleet’s cross-app composition a set of working features rather than a wall of placeholders.

Why enforcement beats convention

It is worth dwelling on why the embed contract is a mechanism and not a style guide, because the choice is the same one the whole stack keeps making. The corrupting call and the correct call are textually identical — `ZGui.colorscheme.apply("matrix")` is right in an app and wrong in an embedded pane — so a convention (“don’t seize globals when embedded”) asks an author to remember, at every call site, which world the code is running in. Authors forget, and the failure is silent: a stolen Cmd+K or a `:root` theme written from under the host does not throw, it just makes the app feel broken in a way that is hard to trace back to the guest. A convention that fails silently is the worst kind of contract, because the cost of a lapse is paid far from its cause.

Turning the lapse into an immediate, self-naming exception moves the cost back to the cause. Because `ZGui.embed.view` swaps the forbidden paths for throwing stubs during init, a guest that reaches for a document-global fails at mount, with an error that names both the forbidden call and the scoped API to use instead. The author learns of the mistake at the first run, at the exact line, rather than from a bug report about a dead keyboard weeks later. That is the same reasoning behind `ztunnel`’s `InvalidState` errors on illegal connection transitions, `zreq`’s loud unknown-command string, and the type-level honesty of `Backend::launch_kind` — the stack prefers a mechanism that makes the wrong thing impossible or immediately loud over a convention that makes it merely discouraged. A small, mechanically enforced constraint (feed the host, don’t grab globals) buys a large capability (any engine live inside any app), and the enforcement is what makes the capability safe enough to use everywhere.

The scoped APIs, mechanically

The APIs that survive inside `embed.view` are worth a closer look, because their mechanics are what make “contribute, don’t seize” concrete. `ZGui.palette.create({ items, scope })` renders its overlay *inside* the scope element with a `palette-overlay--scoped` class instead of full-screen, and returns an instance with `open`, `close`, `isOpen`, and `setItems` — so a core’s commands are reachable from a button it owns, filtered by the same `ZGui.fzf` matcher the global palette uses, without ever touching the host’s Cmd+K binding. The host palette remains the singleton with `register/open/bindHotkey`; the guest palette is an instance scoped to a pane, and the two never collide because only one of them binds a global key.

`ZGui.colorscheme.scope(el, schemeOrVars)` is the theming analogue: it writes the twenty-four `SCHEME_VAR_KEYS` onto the given element as inline custom properties rather than onto `:root`, so a pane can be `matrix` while the host stays `cyberpunk` and every component inside the pane recolours from the scoped variables it inherits. That is why `colorscheme.apply` and `applyVars` (which write `:root`) are forbidden in an embed while `scope` is not — the difference is precisely whether the write lands on the shared document root or on the guest’s own element. Together, scoped palette and scoped colorscheme let an embedded core have its own commands and its own theme *within its pane* while the host keeps ownership of the one keyboard, the one settings panel, and the document root. The embed contract is not “guests get nothing”; it is “guests get a scope,” and the scoped APIs are the exact surface that scope is expressed through.

Composition is architectural, not brute-forced

The last thing to say about the embed contract is why it is the fleet’s real differentiator rather than a nice-to-have. A single author, given unlimited time, could build ten desktop apps ten separate ways — but they still would not have a mail client that previews office documents, a reference manager that shows the PDF of a cited paper, or a photo editor that imports Illustrator files through the PDF engine, because those capabilities are not features you add to an app; they are properties of how the apps are built. Composition requires the engines to be GUI-free and embeddable and the chrome to be shared and boundary-safe, and none of that can be brute-forced per app after the fact. It is decided at the architecture, in the zero-GUI rule, the uniform `invoke`, and the `embed.view` contract, or it is not available at all.

That is why the document plane is the payoff the whole book builds toward and the embed contract is the

chapter that makes it real. The plane is four engines — `zoffice-core`, `zemail-core`, `zpdf-core`, `zphoto-core` — that between them open, render, and edit essentially any file a desktop user meets, and the contract is what lets any app mount any of them live inside its own window without breaking. Every app previews office files, mail, PDFs, and images not because ten apps each solved document preview but because four shared engines solved it once and the embed contract made them safe to compose. A faithful port credits its predecessor; a composable fleet is the original architecture, and the two halves meet exactly here — Acrobat defined PDF editing, and no Acrobat is a library a reference manager links to show a paper. The predecessors are the prior art for what a document is; the embeddable engine behind a mechanically enforced boundary is the new thing, and it is new because it is architectural.

Chapter 15 — Doing It Solo

The book closes where it began: this is one person's stack. This chapter is the method — how the `-core` pattern, the single component library, and one monorepo of submodules make a ten-app fleet tractable for a solo author, and where the seams are.

The monorepo of submodules and pointer cascades

The whole fleet lives as submodules under one meta-repository, and that structure is the solo author's substitute for a team's shared codebase. Each app is a submodule; each app's engine is a submodule inside it; each shared component — `zgui-core`, `zpwr-il8n`, `zpwr-embed-terminal`, `zpwr-file-browser`, `zpwr-hooks-editor` — is a submodule pinned into every app that uses it. The meta-repository pins all of them. It is a tree of pinned commits, and its leaves are the engines and toolkits, and its trunk is the fleet as a whole frozen at one consistent set of revisions.

Working in that tree means working at whatever level a change lives and then cascading the pointer bumps upward. A fix to `zpdf-core` is a commit in the engine, then a pointer bump in each app that embeds it, then a pointer bump in the meta. The shorthand the author uses — `cp` for “commit and push, cascading the pointer bumps up each level” — exists precisely because this cascade is the daily rhythm. It is more ceremony than a monorepo edit, but it buys the isolation the last chapter described: every level records exactly what it shipped, and no change propagates until its pointer is deliberately bumped.

The cascade has real hazards a solo author learns to respect, and the stack's own memory of them is instructive. An IDE that polls git can hold the index lock at the wrong moment; a submodule can be hard-reset to its pinned pointer and wipe uncommitted work; a concurrent edit in one session can clobber a sibling's staged changes if pointers are bumped through the index carelessly. These are not hypotheticals — they are logged lessons — and the discipline they teach is “commit inside submodules early and often, bump pointers deliberately, fetch before judging code missing.” The tree is powerful and unforgiving in equal measure.

The payoff is that one person can hold the whole fleet in one checkout, at one consistent set of versions, and move a change through it with full record of what shipped where. That is what a team gets from a shared monorepo and a CI system, approximated by a solo author with submodules and a pointer discipline. It is heavier than a monorepo and lighter than ten independent release processes, and for a fleet of this size in one person's hands, it is the structure that makes the whole thing hold together rather than scatter.

One toolkit, one shell, one build

The single most important solo-scaling decision is the one the whole book is about: one toolkit (`zgui-core`), one shell (`ZGui.appShell` over Tauri v2), one engine pattern (`invoke` plus C ABI plus Tauri plugin). A solo author cannot maintain ten bespoke UI toolkits, ten shell implementations, ten boundary designs — the surface area would swamp one person. By collapsing all of it to one of each, the per-app work drops to the genuinely app-specific: the engine's domain logic and the thin glue between the engine's JSON and the shared widgets. Everything else is inherited from the shared pieces.

You can see the collapse in how small a new app's shell is. A new app's `main.js` mounts `ZGui.appShell`, relocates its engine's mountable view into `shell.body`, and wires the engine's commands into the shell's palette and settings. Its `src-auri/main.rs` registers the engine's one `invoke` command, the four terminal commands, the file-browser commands, and a handful of platform commands. That is nearly the whole shell,

and it is nearly identical across apps, because the shell is a configuration of shared pieces rather than an implementation. The tenth shell is boilerplate; the interesting code is the tenth engine.

“One build” is the same idea applied to the toolchain. Every app is a Tauri v2 app with a static or workspace `frontend/`, a `beforeBuildCommand` copy step that vendors the shared web modules, and a `pnpm/cargo` build. The requirements spec even documents the two substrates — Tauri and JUCE — behind one frontend contract, so the shared web modules run unchanged in both via a transport shim. A solo author who has built one app’s build has built them all, because the build is the same build with a different engine crate and a different brand string. The uniformity is not aesthetic; it is what lets one person’s muscle memory carry across ten apps.

The deepest form of this reuse is the pattern itself as a reusable asset. Adding an eleventh app is not designing an app; it is instantiating a known template — pure engine with `invoke`, C ABI in `src/ffi.rs`, Tauri plugin in `src/tauri_plugin.rs`, mountable webui in `webui/`, shell that mounts `ZGui.appShell`. The design decisions are all already made; the new work is the domain. That is what “one toolkit, one shell, one build” buys a solo author: not just reused code, but a reused *design*, so the marginal app is domain logic wrapped in a template rather than a product built from scratch.

What the shared architecture makes cheap – and what it makes expensive

The shared architecture makes breadth cheap. Because the tenth app is a shell over an engine over a shared toolkit, a solo author can ship a fleet — Postman, Zotero, Tunnelblick, Alfred, Cyberduck, Docker Desktop, an office suite, a mail client, Acrobat, GIMP — that would be unthinkable to build ten separate ways. Breadth is the thing solo authors normally cannot afford, and the architecture is a machine for affording it: pay the fixed cost of the toolkit and the pattern once, and each additional app is cheap. The fleet exists because the shared architecture made its breadth economically possible for one person.

It also makes composition cheap, which is the part no amount of solo effort buys without the architecture. A single author building ten apps ten separate ways would still not have a mail client that previews office documents, a reference manager that shows PDFs, a photo editor that imports Illustrator files through the PDF engine — because composition requires the engines to be embeddable, and embeddability is an architectural property, not an effort you can brute-force per app. The document plane, the citation service, the transfer engine as shared capabilities are things the architecture makes cheap and that no amount of per-app work would produce.

What it makes expensive is change to the shared surfaces. Evolving `ZGui.appShell`, or renaming a document core’s `invoke` commands, or reworking the embed contract, touches every consumer, and a solo author feels that cost directly — there is no team to parallelize the cascade of fixes across. The shared pieces calcify into interfaces the author evolves carefully, with the matrix as the map of what each change will touch. The freedom to churn any one app’s UI or any one engine’s API is the price paid for the leverage of building them once, and the solo author pays it in careful, matrix-guided, cascading edits rather than free local changes.

The other expensive thing is the cascade itself, the pointer-bump ceremony and its hazards from two sections ago. A team amortizes that overhead across people; a solo author pays all of it, and the memory of index locks and hard-reset submodules is the scar tissue of paying it. The honest accounting is that the shared architecture moves the cost from “building ten apps” to “maintaining one shared substrate and cascading its changes” — and for a fleet of faithful ports of stable products, where the substrate is more valuable than the freedom to churn it, that is a cost worth moving. The architecture is not free; it is a different, cheaper-for-this-shape bill.

Where the fleet goes next

The fleet’s next moves are legible in its own honesty notes, because the stack marks what is provisional rather than rounding it up. The conformance gaps in `GUI_APP_REQUIREMENTS.md` are a roadmap: converge the

older apps' command palettes on the single shared module, unify the two substrates' stylesheet sources, embed the arrangement grid in the apps that still lack it, finish routing every table and filter through the shared widgets. Closing those gaps is not new architecture; it is the existing architecture reaching full conformance across the whole roster, and the checklist says exactly what "done" means.

The engines have their own stated frontiers. `zreq`'s two missing Postman features — a full JS test sandbox and per-host client TLS certs — are named in its feature matrix. `ztunnel`'s network backends and privileged helper are marked as phased. `zftp`'s cloud transports report `needs_transport` until wired. `zcontainer`'s daemon management spans three OSes at different maturities. None of these are hidden; each is a marked boundary in the code or the docs, which is how a project that credits its predecessors honestly also tracks its own unfinished edges. The fleet advances by closing named gaps, not by discovering it overclaimed.

The C ABI is a frontier of its own. Nine of ten engines carry the four-function C door; `zpdf-core` does not, because nothing non-Rust needs it yet. The moment a non-Rust host wants to embed PDF preview, that door gets added, and the same is true for any engine as its embed demand grows beyond the Rust/Tauri fleet. The uniform pattern means adding a C ABI to an engine is a known, small piece of work — the four functions over the existing `invoke` — so the fleet's reach into non-Rust hosts is bounded only by demand, not by design work. The doors are cheap to add because the pattern already exists.

The book ends where it began, on the thesis, now earned across ten apps: every app is a thin Tauri v2 shell over a pure-Rust `*-core` engine with two front doors, all sharing one toolkit and one HUD, all embedding one another through a live embed contract, all versioned independently and tracked in one submodule tree with one consumption matrix. The products are faithful ports of tools people already know, credited by name; the plumbing is credited to its dependencies; the architecture — engines that compose, a document plane every app embeds, a fleet one person can hold in one checkout — is the original work. Ten shells, a fleet of engines, one author, and the one sentence that ties them together, demonstrated rather than asserted. That is the desktop, rewritten in Rust.

The version skew, read honestly across the fleet

Independent versioning is the `-core` pattern's promise, and the fairest way to assess it is to read the actual numbers rather than assert a tidy story. The skew is real but partial. Four apps run visibly ahead of their engines: `zgo` at `0.2.0` over `zgo-core 0.1.0`, `zpdf` at `0.2.0` over `zpdf-core 0.1.0`, `zcontainer` at `0.3.0` over `zcontainer-core 0.2.0`, and `zterminal` at `0.29.7` over its `ztermux-core 0.1.0` — the shell iterating on packaging, window behaviour, and integration while the engine held steady. But many apps ship in lockstep: `zreq`, `zcite`, `ztunnel`, `zftp`, and `zemail` sit at `0.1.0` with their cores at `0.1.0`, and `zoffice` and `zphoto` at `0.2.0` with their cores at `0.2.0`. The honest reading is not "shells always run ahead of engines" but "shells *can* run ahead, and several do, because the versions are genuinely independent — and where they happen to match, it is because the two artifacts moved together this cycle, not because they are welded."

That partial skew is the pattern working exactly as designed. The point of independent versioning was never that the numbers must diverge; it was that they *may*, so that a shell can iterate on chrome without dragging its engine's version and an engine can hold at a stable revision while three consumers pin it. `zgo`'s two minor versions of menu-bar and hotkey work over one stable engine version is the skew made visible; `zoffice` and its core moving together at `0.2.0` is the same freedom producing a matched pair this cycle. Both are the receipt of the same mechanism — each artifact carries its own version and moves on its own schedule — and a fleet where every shell was forced to match its core would have neither the skew nor the lockstep, only a release train.

The cascade and its logged hazards

The pointer cascade is the daily rhythm, and the shorthand the author uses for it — `cp`, "commit and push, cascading the pointer bumps up each level" — exists precisely because a change that must reach the meta is a fix in the engine, then a bump in each consuming app, then a bump in the meta, three levels of commit

and push. It is more ceremony than a monorepo edit, and it has real hazards a solo author learns to respect, logged as scar tissue rather than hypotheticals. An IDE that polls git can hold the index lock at the wrong moment, so a commit that stages by path in one shot beats one that races the poller. A submodule can be hard-reset to its pinned pointer by the IDE and wipe uncommitted work, so the discipline is to commit inside submodules early and often. And the premiere-port style of running the same task in several sessions at once means a concurrent edit can clobber a sibling's staged changes if pointers are bumped through the index carelessly, so the rule is to fetch before judging code missing and to bump gitlinks through the index rather than by re-adding a whole subtree.

Those lessons are not incidental; they are the cost of the tree structure paid in full by one person. A team amortizes the cascade overhead across people and a CI system; a solo author pays all of it, and the memory of index locks and hard-reset submodules is the receipt. The honest accounting from Chapter 13's cost model applies here at the level of daily work: the shared architecture moves the cost from "building ten apps" to "maintaining one shared substrate and cascading its changes," and the cascade's hazards are the concrete form that moved cost takes. For a fleet of faithful ports of stable products — where the substrate is more valuable than the freedom to churn it — the trade is worth making, but it is not free, and the logged hazards are what "not free" looks like at the keyboard.

The eleventh app is a template, not a design

The deepest form of the reuse is that the pattern itself is a reusable asset, so adding an eleventh app is not designing an app — it is instantiating a known template. The template is fixed: a pure engine crate with `crate-type = ["rlib", "staticlib", "cdylib"]` and one `Engine::invoke`; a C ABI in `src/ffi.rs` fitted to the domain (the events callback if it has events, `init` if it holds state); a Tauri plugin in `src/tauri_plugin.rs` behind the `tauri` feature; a mountable webui in `webui/` that auto-detects its transport; and a shell whose `main.js` mounts `ZGui.appShell` and whose `src-tauri/main.rs` registers the one engine command, the four `terminal_*` commands, the `fs_*` family, and a handful of platform commands. Every one of those decisions is already made; the only genuinely new work is the domain — the parser, the model, the dispatch that is *this* product and no other.

That is what "one toolkit, one shell, one build" buys a solo author: not just reused code but a reused design, so the marginal app is domain logic wrapped in a template rather than a product built from scratch. The tenth shell is boilerplate and the tenth `main.js` is nearly identical to the ninth; the interesting code is the tenth engine. It is also why breadth — the thing solo authors normally cannot afford — became affordable here: the fixed cost of the toolkit, the shell, and the boundary pattern was paid once, and each additional app draws it down. A single author building ten apps ten separate ways could not ship this fleet, and could never have the composition — a mail client that previews office documents, a reference manager that shows PDFs, a photo editor that imports Illustrator files through the PDF engine — because composition is an architectural property, not an effort you can brute-force per app. The template makes the eleventh app cheap; the architecture makes the eleventh app compose; and both are what let one desk hold the whole fleet.

Chapter 16 — The Failure Contract: How Ten Engines Fail

The chapters so far described what the engines *do*; this one is about what they do when they cannot. Failure is not a footnote to a boundary design — it is half of it, because a contract that says only what a call returns on success, and goes silent on what it does on error, is a contract a host cannot actually program against. The fleet has one failure contract, uniform across all ten engines and all three doors, and it is as deliberately boring as the memory rule: an operation either produces a JSON result or produces a typed error that crosses the boundary as JSON too, and nothing — no error, no missing capability, no timeout — is allowed to unwind past the FFI seam or poison the shared engine.

thiserror in every crate: errors as a typed enum

Every `-core` in the fleet carries an `error` module, and Appendix F lists it as one of the three scaffolding modules — `ffi`, `error`, `tauri_plugin` — that every crate has regardless of domain. The module is the same shape everywhere: a `thiserror`-derived enum, one variant per failure kind, with `#[error("...")]` messages that read as sentences and `#[from]` conversions that let the `?` operator lift a lower-level error (a `serde_json::Error`, an `io::Error`, a `reqwest::Error` behind its feature) into the engine's own error type without a hand-written `map_err` at every call site. The engine's public functions return `Result<Value, <Name>Error>`, and the domain code reads in the ordinary Rust idiom — fallible calls threaded with `?`, the error type widening as it climbs — right up until the boundary, where the error has to become data.

The reason the error is a *typed enum* rather than a stringly `anyhow::Error` is the same reason the commands are enumerated: the failure surface must be auditable and stable, because hosts branch on it. A host that gets back “this bookmark needs a transport backend that is not wired” must be able to tell that apart from “this bookmark’s credentials were rejected,” and it can only do so if the two are distinct variants with distinct tags rather than two English strings that a future edit might reword. `thiserror` gives the engine the ergonomics of `anyhow` inside the crate — cheap `?`, cheap `#[from]` — while keeping a closed, named set of failures at the surface, which is exactly the split the boundary wants: convenient in the middle, precise at the edge.

One Result, three ways to surface it

The native door surfaces failure the Rust way: `Engine::invoke` returns `Result<Value, Error>`, and a Rust host matches on it or `?`-propagates it like any other result. The other two doors cannot return a Rust `Result` — a C caller has no `Result`, a WebView has no Rust type — so the boundary makes the same decision it made for commands and arguments: the error becomes JSON. The C door’s `<prefix>_invoke` never returns a null or a negative error code to signal failure; it returns a heap JSON string exactly as it does on success, and the failure is *in* the JSON — an object carrying an error shape the host parses, with the variant’s tag and its message. The Tauri door does the same: the `#[tauri::command] <prefix>_invoke` resolves to a JSON value whether the operation succeeded or failed, and the front end reads the same error shape a C host reads.

This is a deliberate choice with a cost worth naming. Encoding errors in the return JSON rather than in a separate channel (a C error code, a rejected JavaScript Promise) means the host must inspect the payload to know whether a call succeeded — there is no out-of-band signal to branch on first. The benefit is that the

failure contract is identical across all three doors: a bug in error handling is a bug in one place, the error shape is testable by the same `cargo test` that tests success (Chapter 19), and a host written against one door ports to another door unchanged because the error surface did not move. The fleet chose one uniform in-band error shape over three idiomatic per-door error channels, for the same reason it chose one command dispatch over a hundred typed commands: uniformity across hosts is the property that a three-host engine most needs to preserve.

The `Error` event: failures that arrive late

Not every failure is the return value of a call, because not every operation is synchronous. A request that is in flight, a sync that is running, a script filter that is computing — these were dispatched, returned an acknowledgement, and are now doing their work on another task, so when they fail the call that started them is long since returned. For those, the failure contract has a second surface: the `Error { tag, message }` variant that Appendix H shows in six of the seven event-bearing engines' `EngineEvent` enums. A late failure is narrated through the same event sink that narrates a late success — `RequestFinished` and `Error` are siblings in `zreq-core`'s event enum, `SyncFinished` and `Error` in `zemail-core`'s — so the host that is already listening for progress is, by the same listener, listening for failure.

The symmetry is the point. There are exactly two ways an operation can end — it returns or it narrates — and each way has its success form and its failure form: a synchronous call ends in a result JSON that is either data or an error object; an asynchronous operation ends in an event that is either a completion variant or an `Error` variant. A host never has to wonder which channel a failure will come down, because a failure always comes down the *same* channel its success would have. This is why `zcontainer-core`'s streaming, the one genuinely different-shaped subsystem, still fits: a stream that dies emits `StreamError { id, tag, message }` on the sink, keyed by the same subscription id as its `Stream` chunks, so even the hardest case reuses the “failure rides the success channel” rule rather than inventing a third.

`needs_transport` and `needs_privilege`: the non-error, honestly

The most interesting part of the failure contract is a category that is not a failure at all: the operation the engine *could* describe but *cannot* perform, because the performing belongs to the shell's half of the bargain. `zftp-core` can resolve a bookmark, plan a transfer, and name the protocol, but if the transport backend for that protocol is not wired it does not raise an error — it returns a result whose status is `needs_transport`. `ztunnel-core` can build the OpenVPN command line and the firewall plan, but if root is not held it returns `needs_privilege` rather than pretending to have connected. These are honest data, not exceptions: the call succeeded — the engine did exactly the pure computation it is responsible for — and the result truthfully reports that the impure remainder is the shell's to complete or is not yet available.

Modelling a missing capability as a *value* rather than an *error* is a small decision with a large honesty payoff, and it is why Appendix K can list these without embarrassment. An error says “something went wrong”; `needs_transport` says “nothing went wrong, and here is precisely the boundary between what I did and what remains.” A host can render that boundary in the UI — a bookmark that is reachable-in-principle but not-yet-wired, a tunnel whose plan is ready but whose elevation is pending — instead of showing a red failure for a feature that is simply phased. The fleet's honesty compact (the front matter's promise not to fake a live connection) is not enforced by good intentions; it is enforced by the type, because an engine that returns `needs_privilege` as a first-class status literally cannot report a connection it did not make. The non-error is the honesty compact compiled into the result shape.

Poison, panics, and the mutex that must not die

The one failure the contract absolutely forbids is an unwinding panic crossing the FFI seam, because a `panic!` that unwinds past an `extern "C"` boundary is undefined behaviour, and undefined behaviour in an engine that ten apps embed is a fleet-wide liability. So the boundary functions are the one place the durable-deps austerity bends toward defensiveness: the C door parses its inputs fallibly (a non-UTF-8 command string or malformed args JSON becomes an error result, never a panic), and the dispatch is written so that a domain error is a `Result::Err` — data — rather than an `unwrap` that could abort. The engine treats its own bugs as the exception to catch, not as a reason to take the host's process down.

The process-global `Mutex<Engine>` adds a second hazard the contract has to answer: lock poisoning. If a thread panicked while holding the engine lock, the standard library marks the mutex poisoned and every subsequent `lock()` returns an error, which would turn one transient bug into a permanently dead engine — every future `invoke` failing on a poison the original panic left behind. The fleet's answer is the same discipline that avoids the FFI unwind: keep the code under the lock panic-free by construction (fallible parsing, `Result` propagation, no `unwrap` on host-supplied data), so the lock is never poisoned in the first place, and recover the guard rather than propagate the poison if it ever is. The single-global-engine simplicity of Chapter 4 buys first-try correctness for hosts; the price is that the one shared lock must be treated as load-bearing, and the failure contract's last clause is the rule that keeps it alive: nothing under the engine lock is allowed to panic, so nothing under it is allowed to poison.

The taxonomy a host actually branches on

A failure contract is only as useful as the decisions it lets a caller make, so it is worth asking what a host does with an error rather than only how the error travels. In practice a front end branches four ways, and the fleet's error surface maps onto those four with reasonable precision — which is the real argument for a typed enum over a string, more so than the auditability argument Chapter 16 leads with.

The first branch is *retry*. A transient failure — a network hiccup, a server that was briefly unavailable — is one where the same call with the same arguments may succeed later, and the correct UI is a retry affordance rather than an explanation. The second is *fix the input*. A malformed URL, an unparseable import file, a command whose arguments do not deserialize: the call will never succeed as issued, and the correct UI points at the field. The third is *fix the environment*. Credentials rejected, a daemon not running, a path not readable: nothing about the request is wrong, and the user has to change something outside the app. The fourth is *this is not a failure at all*, which is where `needs_transport` and `needs_privilege` live — a successful call whose result says the impure remainder is pending.

Reading the fleet's error enums against those four branches shows why the variants are shaped the way they are. `Error::UnknownCommand(cmd)` is unambiguously branch two, and it carries the command name because the only useful thing to do with it is show a developer what was not recognized. The `#[from]` lifts are where the boundary between two and three gets interesting: a `serde_json::Error` lifted into the engine's enum is almost always a bad input, while an `io::Error` or a lifted `request::Error` is almost always an environment problem, and the fact that they arrive as distinct variants rather than as one stringly “something failed” is what lets a front end render them differently without parsing English. A single *anyhow*-style error would collapse branches two and three into one message and force the UI to guess.

The `Error { tag, message }` event variant is the same taxonomy arriving on the asynchronous rail, and the two-field shape is doing deliberate work. The `tag` is what a host branches on; the `message` is what a host displays. Splitting them means the displayed text can be reworded — improved, localized through the catalogue of Chapter 20 — without breaking a front end that keyed off the wording, which is precisely the failure mode that makes stringly errors expensive over time. The same discipline appears in `StreamError { id, tag, message }`, where the `id` adds the third thing a streaming host needs: not just what went wrong and what to show, but which of several open panels to show it in.

Where the taxonomy is weakest is *retry*, and honesty requires saying so. Nothing in the fleet's error surface

marks a failure as transient; a host distinguishing “try again” from “this will never work” is inferring it from the variant rather than reading it from a flag. For the common cases the inference is easy — a lifted network error is retryable, an unknown command is not — and for the uncommon ones it is a judgement the front end makes with incomplete information. A richer contract would carry retryability as data the way `needs_transport` carries phasing as data, and the reason the fleet has not needed it is that the apps’ long operations are the ones the domains modelled as entities with status (Chapter 22), where retry is a domain verb — resend from history, reconnect a session, re-run a transfer — rather than a generic property of an error. That is a defensible place to have stopped, and it is a place, not a completed design.

Chapter 17 — Persistence: The `store` Module and the Data Directory

An engine that forgot everything between launches would be a calculator, not an application, and yet persistence is exactly the kind of impure, platform-shaped concern the zero-GUI rule seems to push out of the engine. This chapter is about how the fleet resolves that tension: the engine owns its *state* and its *serialization* — what to save and in what shape — while the platform-specific question of *where* the bytes land is answered by one convention shared across every app. The result is that persistence, like failure, is uniform across ten engines despite ten different things being saved.

Every engine owns a `store`

Appendix F's module lists name a `store` module in six of the ten cores — `zreq-core`, `zcite-core`, `ztunnel-core`, `zgo-core`, `zftp-core`, and `zemail-core` — and the four that do not are a group with reasons of their own, taken up later in this chapter. The six are the engines that hold a durable model of their own, and the `store` module is where the engine's durable model lives: `zreq`'s collections, environments, and history; `zcite`'s libraries and items; `ztunnel`'s configs and server lists; `zgo`'s workflows, snippets, clipboard history, and frequency table; `zftp`'s bookmarks. It owns the load path (read the file at startup, deserialize into the in-memory model, tolerate a missing or older file) and the save path (serialize the model, write it back), and it is pure in the sense that matters: it depends on `serde` and `dirs`, not on a GUI, so a test can point it at a temp directory and exercise the whole round-trip with no window.

Keeping the store *inside* the engine rather than in the shell is what makes persistence compose the way everything else composes. When `zcite`'s citation engine is embedded in `zemail` to format a reference, or `zgo`'s launcher logic is embedded elsewhere, the embedded engine reads and writes its own store through its own `store` module — the host app does not have to know how a Zotero library is laid out on disk in order to embed a Zotero engine. The store travels with the engine because it *is* part of the engine, and that is only possible because the store depends on nothing graphical. The same rule that made the engine testable and embeddable makes its persistence embeddable too.

The data directory, by bundle id

The one thing the engine does *not* decide is where its store file lives, because that is a platform question with a platform-specific answer, and the fleet answers it with a single convention: every app — including every embedded core — persists under `~/Library/Application Support/com.menketechologies.<app>` on macOS, resolved through Tauri's `app_data_dir()` keyed by the app's bundle identifier, not through a raw `dirs::data_dir()` join with a product name. The distinction is not pedantic. Resolving by bundle id means an app and every core it embeds land in one predictable directory that matches the app's identity, so the data a user sees under an app's name is exactly the data that app and its embeds wrote — no stray directory named after an engine, no collision between an app and a core that happens to share a product string.

This convention is the shell's contribution to persistence, and it slots cleanly into the shell's-half-of-the-bargain split from Chapter 1. The engine's `store` module knows *what* to write and *how* to serialize it; the shell hands the engine the resolved data directory at startup, and the engine writes there. An engine driven by a plain `cargo test` gets a temp directory instead and never touches the user's real data; an engine embedded in a

sibling app gets the host's resolved directory and cooperates with the host's identity. The “where” is a single parameter the shell supplies, which is why the convention could be stated once, in the fleet's own memory notes, and then hold across every app rather than drifting per-crate.

JSON on disk, not a database

The fleet's default persistence substrate is JSON files, not an embedded database, and the choice follows directly from the durable-deps rule: `serde_json` is already a mandatory dependency in every core (it is how the boundary speaks), so persisting the model as JSON adds no new dependency, no schema migration engine, no query layer — the same `serde` derives that marshal a result across the boundary serialize the model to disk. For the shapes most of these engines hold — a few hundred requests, a library of references, a list of bookmarks and servers, a workflow graph — a JSON document read into memory at startup and written back on change is exactly the right weight: simple, inspectable, diffable, and trivially correct.

The book is honest about where that choice reaches its edge, because it does. A JSON document that is loaded and saved whole does not scale to the shapes a few engines approach: `zemail`'s message store and `zcite`'s attachment index are the kind of growing, queryable data that a whole-file JSON model serves poorly, and those engines lean on more structured storage and indexing (`zcite`'s `search/identifier` machinery, `zemail`'s `gloda` search) rather than pretending a single JSON blob is a mail store. The honest framing is that JSON-on-disk is the fleet's *default*, chosen because it is dependency-free and correct for the common case, and the engines whose data outgrows it say so in their module structure rather than forcing every engine into a database it does not need. Persistence, like the C-ABI's function count, is a spectrum fitted to the domain, not a uniform stamped everywhere.

Credentials do not live in the store

There is one class of state the `store` module deliberately refuses to hold: credentials. A VPN's private key, an FTP account's password, an OAuth refresh token — a JSON file in the data directory is exactly the wrong home for these, and the fleet keeps them out of the store on purpose. `ztunnel-core`'s `creds` module and `zftp-core`'s `creds` module, gated behind a `keychain` feature that pulls in the `keyring` crate, route secrets to the OS keychain, and the store persists only a *reference* — which bookmark or config a secret belongs to — not the secret itself. Delete the JSON store and you lose your bookmarks; you do not leak your passwords, because your passwords were never in it.

This split is another instance of the plan-versus-effect discipline that runs through the fleet. The store holds the inert, shareable, diffable part of the state — the configuration a user could reasonably email themselves — and the keychain holds the part that must be protected by the platform's own secret store. The engine's `creds` module is the seam between them, and it is feature-gated precisely so that a `--no-default-features` build, or a test, or an embed that has no business touching the user's keychain, compiles without the secret path at all. Secrets are the one kind of persistence the engine hands back to the platform, for the same reason it hands back privilege and pixels: the platform's secret store is a thing no pure engine should reimplement.

Migrations without a migration framework

A store that outlives its schema needs a migration story, and the fleet's is deliberately lightweight, because a heavy one would be a dependency the durable-deps rule would reject. The load path is written to *tolerate* an older or partial document rather than to run a versioned migration pipeline: `serde`'s `#[serde(default)]` fills fields a newer model added but an older file lacks, `Option<T>` and defaulted collections absorb shapes that changed, and unknown fields an older reader does not understand are handled rather than fatal. A file written by last month's build loads in this month's build because the model was designed to deserialize forgivingly, not because a migration script ran.

Where a change is too large for `serde` defaults to absorb — a field that moved, a shape that split — the engine reads the old shape explicitly and rewrites it to the new one on the next save, a migration expressed as ordinary Rust in the `store` module rather than as an entry in a framework's migration table. The trade is the familiar durable-deps one: this approach does not give you a database's rigorous, ordered, reversible migrations, and for the whole-file JSON stores the fleet uses it does not need to, because the whole document is in memory and a migration is just a transformation of a value. The engines whose data is genuinely database-shaped carry heavier machinery; the many whose data is a few hundred `serde`-derived structs migrate by deserializing forgivingly and reserializing current, which is the least machinery that keeps a user's data loading across versions.

The four cores that own no `store`

The six cores named above are exactly six of ten, and the four that name no `store` module are worth reading as a group, because they are not four engines that forgot to persist. `zcontainer-core`, `zoffice-core`, `zpdf-core`, and `zphoto-core` each have a reason, and the reasons are different enough that the absence is not one pattern but three.

`zcontainer-core` holds no durable model because the durable model is not its to hold. The containers belong to a daemon, the clusters are described by a kubeconfig the operating system already owns, and the workloads are described by YAML in the user's repositories. An engine that cached any of that would be maintaining a second, staler copy of somebody else's source of truth, and every disagreement between the two copies would be a bug the user experiences as the app lying about their cluster. The correct design for a client of an authoritative remote is to hold nothing across launches, and `zcontainer-core`'s module list — `model`, `daemon`, `daemon_linux`, `daemon_windows`, `docker`, `kube`, `vm` — is that decision made visible.

`zoffice-core`, `zpdf-core`, and `zphoto-core` are the other case: their durable state is the *user's file*. A document engine's persistence is `save_pdf`, or a writer export, or an image encode — the model round-trips through a format the user chose, at a path the user chose, and there is nothing left over to keep in an application data directory. This is why they are also the engines with no importers (Chapter 27) and, for two of the three, no events (Appendix H): a request-response transformation engine with no session has nothing to narrate and nothing to remember. The absence of a `store` in a document engine is the same fact as the absence of an `EngineEvent`, seen from the persistence side.

The split has a consequence for the whole architecture that is easy to miss. The six engines with a `store` are the six with a *user-facing model of their own* — a workspace, a library, a config set, a workflow set, a bookmark set, an account set — and those are exactly the engines whose embedding is complicated by the next section's problem. The four without a store embed cleanly into anything, because a guest that persists nothing cannot disagree with its host about where its bytes go. `zpdf-core` being both the fleet's most-embedded engine and one of the four storeless ones is not a coincidence; a stateless guest is a cheap guest.

One directory, many guests

The bundle-id convention resolves the question of *where* cleanly, and it produces one consequence that deserves to be stated plainly rather than left for a reader to discover: an engine embedded in a host does not see the data it holds in its own app.

Follow the rule through. `zcite` resolves its data directory from its own bundle id and its `zcite-core` reads a library there. When `zemail` embeds `zcite-core` to format a citation, that guest engine is handed `zemail`'s resolved directory, because that is what the convention says — an app and every core it embeds land in one directory keyed to the app's identity. So the embedded `zcite-core` in `zemail` starts empty. It has all of the engine's logic — the schema, the CSL processor, the seven importers — and none of the user's references.

That is the correct behaviour for the reasons Chapter 21 gives about the trust boundary, and it is a real limitation of the composition story, so both halves belong here. It is correct because the alternative is worse: an app that reached into a sibling app's data directory would be an app that could read a user's mail store because it

happened to embed a mail engine, and the bundle-id keying is exactly what makes that impossible without the user's involvement. Embedding an engine grants you the engine, not the neighbour's data.

It is a limitation because it silently divides the embedded engines into two classes. An engine embedded for *computation* — `zpdf-core` rendering a page, `zcite-core` formatting a supplied reference into a bibliography, `zoffice-core` previewing an attachment — needs nothing from the user's store and works perfectly as a guest. An engine embedded for its *data* would need the host to be handed the other app's directory, and nothing in the convention does that. The embed matrix of Chapter 13 records which app consumes which engine; it does not distinguish these two kinds of consumption, and the distinction is the one that determines whether an embed needs anything beyond mounting.

The honest framing is that the fleet's composition story is a story about sharing *capabilities*, not about sharing *state*. Every live embed the book describes is a computation embed, which is why the limitation has not bitten: the document plane is capability sharing by nature. A state-sharing embed — `zemail` showing your actual `zcite` library rather than formatting a reference you handed it — would need a deliberate mechanism the fleet has not built, and building it would mean deciding how one app asks a user for permission to read another's directory. That is a design problem, not an oversight, and naming it is better than implying the composition story covers a case it does not.

Whole-file saves and the writers you do not have

One more property of the JSON-on-disk choice is worth making explicit, because it is the kind of thing that is fine until precisely the moment it is not. A store that loads whole and saves whole is a last-writer-wins store: the save path serializes the in-memory model and replaces the file, so any change made by a different writer since the load is gone. The question that matters is therefore how many writers a given store can have.

Within a process the answer is one, and it is enforced rather than assumed. The process-global `OnceLock<Mutex<Engine>>` of Appendix O means every command against an engine is serialized, so a save cannot interleave with another command's mutation of the same model. Chapter 22's critical-section discussion is usually read as a constraint on long operations; from the persistence side it is also the property that makes whole-file saves safe, because there is no second thread inside the engine holding a stale model.

Across processes the answer is that the bundle-id convention keeps the writers apart. Two different apps never write the same store, because their directories are keyed to different identities — and the previous section's limitation is exactly the same fact seen from the other side. What the convention does *not* cover is two instances of the *same* app, where two processes would resolve the same directory and each hold a whole model. Nothing in the persistence design prevents that; what prevents it is a shell-level concern about how many instances of an app may run, which lives on the shell side of Chapter 1's split along with every other window-and-process question.

The reason this is acceptable rather than alarming is the shape of the data. These stores are configuration and collections — a workspace, a bookmark set, a workflow set — mutated by one person at desktop speed, not shared documents under concurrent edit. Whole-file JSON is correct for that shape and it stops being correct exactly when the shape changes, which is the same edge the chapter already named for `zemail`'s message store and `zcite`'s indexes. The engines whose data grows past a single in-memory document are the engines whose concurrency assumptions need revisiting too, and they are the same engines. That the two limits arrive together is not a coincidence: they are both consequences of holding the whole model in memory, which is the one assumption the default persistence substrate makes.

Chapter 18 – The Shared Substrate: Terminal, File Browser, and the Commands Every App Registers

Not everything shared across the fleet is an engine or a widget. Beneath the app-specific engine and above the raw platform sits a thin layer of *shared capability* — an embedded terminal, a file browser — that almost every app wants and no app should build twice. These are not part of any one product’s domain; a terminal is not “the Postman part of `zreq`” and a file browser is not “the Zotero part of `zcite`.” They are amenities the shell offers, and the fleet factors them into shared Rust crates that every app’s `invoke_handler` registers the same way, so that the twentieth app inherits a working terminal and a working file browser as a block of boilerplate rather than a feature to implement.

`zpwr-embed-terminal`: four commands, one PTY

The embedded terminal is a shared crate, `zpwr-embed-terminal`, pinned under `crates/` in every app that mounts one, and it registers exactly four Tauri commands: `terminal_spawn`, `terminal_write`, `terminal_resize`, and `terminal_kill`. The shape is the pseudo-terminal lifecycle reduced to its four verbs — spawn a PTY running the user’s shell, write bytes into it (the keystrokes the front-end terminal widget captures), resize it when the pane resizes, and kill it when the pane closes — with the PTY’s output streamed back to the WebView as events the terminal widget renders. It is, in miniature, the same two-door shape as an engine: a small command surface in, an event stream out, and the interesting long-lived resource (the PTY) owned on the Rust side.

The terminal is a clean illustration of why some things are the *shell*’s and not the *engine*’s, from Chapter 1’s split. A PTY is a long-lived, platform-specific, streaming resource — precisely the kind of thing the zero-GUI engine rule pushes out of a core and into the shell’s async world. So the terminal does not live in any `-core`; it lives in a shared shell-side crate, and an app gets it by pinning the crate and registering its four commands. The frontend side is equally shared: the terminal *widget* is a `zgui-core` component (the toolkit’s terminal view), so an app inherits both halves — the Rust PTY driver and the JavaScript renderer — from shared submodules, and writes none of it. One terminal, built once, mounted everywhere.

`zpwr-file-browser`: the `fs_*` family

The file browser is the larger of the two shared substrates, a crate `zpwr-file-browser` that registers the `fs_*` command family Appendix D enumerates: `fs_list_dir`, `fs_get_info`, `fs_grep`, `fs_hash`, `fs_compress`, `fs_extract`, `fs_read_file_base64`, `fs_git_status`, the `fb_watcher_set` live-watch command, and roughly thirty more. This is a genuine file manager’s worth of operations — directory listing with metadata, recursive grep, content hashing, archive compress and extract, base64 file reads for previewing, per-file git status, and a filesystem watcher that pushes change events — and it is shared because every app that shows the user a file tree wants the same operations, and because filesystem access is exactly the platform-touching work an engine must not do.

The `fs_*` family is where the “thin shell” accounting from Chapter 4 gets its most striking number. When that chapter says an app’s backend is “one engine command, a handful of platform commands, and a block of

shared registrations,” this crate is most of the block: an app registers thirty-odd `fs_*` commands with one line each, and inherits a complete file browser. The commands are deliberately the un-shareable-by-the-engine, eminently-shareable-across-apps operations — no single app’s domain, every app’s need — which is exactly the profile that justifies a shared crate over per-app code. A file browser in `zftp` and a file browser in `zcontainer` are the same file browser, because browsing files is not an FTP concern or a container concern; it is a desktop concern the fleet solved once.

Why these are crates, not per-app code

It is worth being precise about *why* the terminal and the file browser are factored out when so much else in an app is app-specific, because the criterion is the same one that governs the whole architecture: factor out what changes at the same rate and is reused by many. A terminal’s behaviour changes when terminals change — rarely — and every app that has one wants the identical behaviour, so per-app terminals would be N copies of one thing drifting apart, which is exactly the failure mode shared crates exist to prevent. The same is true of the file browser: there is no app-specific way to list a directory or hash a file, so N implementations would be N chances to get `fs_grep` subtly different, N places to fix a symlink bug, N terminals that render escape sequences slightly differently.

This is the engine-core pattern applied one layer up. The `-core` crates factor out an app’s *domain* engine so it can be tested and embedded; the shared substrate crates factor out the app’s *amenities* — terminal, file browser, and the toolkit’s widgets — for the same reason at the shell layer. The result is that an app is a thin composition at *two* levels: a thin shell over a shared engine (the book’s thesis) and a thin registration over a shared substrate (this chapter’s corollary). What remains genuinely per-app, after both factorings, is small — the engine’s domain logic and the few platform commands only that app needs — which is the whole reason one author can hold ten apps.

The registration boilerplate as a contract

The block of registrations in an app’s `src-tauri invoke_handler` is boilerplate, but it is boilerplate that functions as a contract, and reading it tells you almost everything about what an app is. Appendix D lays out the standard block: the engine’s one `<prefix>_invoke`; the four `terminal_*`; the `fs_*` family; and a few app-specific platform commands. An app’s identity is legible in the deltas from that standard block — `zgo` adds `zgo_hide/zgo_reveal/zgo_trash` because it is a launcher; `zcontainer` adds the `subscribe/unsubscribe/stream_send` trio because it streams; the diagnostics-panel apps add `open_log_file/open_log_dir/open_data_dir`. Everything else is the same in every app, which is what “boilerplate” means and why it is a feature, not a smell.

That uniformity is what makes the registration block a *contract* rather than merely repeated code. Because every app registers the same terminal and file-browser commands under the same names, the shared frontend widgets that call those commands work in every app without configuration — the `zgui-core` terminal view calls `terminal_spawn` and it is there; the file tree calls `fs_list_dir` and it is there. The registration block is the promise each app makes to the shared frontend: *these commands exist, by these names, with these shapes*. An app that honours the block gets the whole shared substrate for free; an app that deviated would break the shared widgets that assume it. The boilerplate is the interface, and copying it faithfully is how an app joins the fleet rather than forking from it.

The substrate has a cascade of its own

The engines get the attention in Chapter 13’s propagation story, but the shared substrate cascades too, and its cascade has a different risk profile that is worth separating. A `zpdf-core` bump reaches the apps the `pdf-core` column lists — a bounded, matrix-visible set. A `zpwr-file-browser` or `zpwr-embed-terminal`

bump reaches *everyone*, because the substrate is baseline rather than optional: R3 requires the terminal and R10 requires the browser, so there is no app in the conforming roster that does not consume both.

That difference matters in both directions. On the upside it is the mechanism Chapter 21 leans on — a hardening fix in the file browser is a fleet-wide fix, because there is no app holding a divergent copy to miss it. On the downside it means the substrate crates have the largest blast radius in the tree and the least visibility in the ledger, since they are precisely the components the matrix records as universal and therefore stops being informative about. A column of all checks tells you nothing you did not know; the apps that would notice a regression in `terminal_write` are all of them.

The registration block is what makes this manageable, and reading it as a versioned interface rather than as boilerplate is the useful reframing. Because every app registers the same command names with the same shapes, the shared frontend widgets call them without configuration — the toolkit's terminal view calls `terminal_spawn` and it is there. That works only as long as the names and shapes are stable across every pointer an app might be sitting on, which turns the substrate's command surface into a compatibility contract with a wider consumer set than any engine's. Renaming an engine command breaks the apps that embed that engine; renaming `fs_list_dir` breaks the shared widget that every app mounts, in every app, at whichever pointer each of them happens to be pinned to.

The practical discipline that follows is that the substrate's surface should be the slowest-moving thing in the tree, additive where it changes at all — a new `fs_*` command is safe because old apps simply do not call it, while a changed shape on an existing one is not. This is the same calcification Chapter 13 describes for shared engine interfaces, arriving one layer down and biting harder because the consumer set is universal rather than matrix-bounded. It is also the strongest argument for keeping the substrate's scope narrow: every capability added to `zpwr-file-browser` is a capability every app in the fleet now registers, carries, and is exposed to, whether or not it wanted it. A shared crate at the bottom of a fleet earns its place by being complete and then holding still.

Chapter 19 — Testing a Fleet Without a Window

The zero-GUI rule was introduced in Chapter 1 as the precondition for embedding and the C ABI; this chapter is about its third and least glamorous payout, which is in some ways the one that keeps the fleet alive: testability. An engine with no GUI dependencies is an engine a plain `cargo test` can exercise completely, with no display server, no WebView, no event loop — and for a solo author maintaining ten products, a test suite that runs in milliseconds on every save is not a nicety but the substitute for the QA team a lone developer does not have. This chapter is how the fleet tests itself, and, honestly, what those tests cannot reach.

The zero-GUI rule pays out in tests

The mechanics are as simple as the rule makes them. Because a `-core` is a plain library whose public surface is `Engine::invoke(cmd, args) -> Result<Value>`, a test constructs an `Engine`, calls `invoke("collection.add", args)`, and asserts on the returned JSON — no window is opened because there is no window to open, and the thing under test is a data structure and a function, which is exactly what a unit test wants. Chapter 1 cited the concrete numbers: `zcite-core` claims 57 unit plus 13 integration tests; `zreq-core` scores a matrix of 83 enumerated Postman behaviours; and none of those tests touch a display, because the engine under test cannot draw and does not try.

The subtle, load-bearing consequence is that these tests exercise *the same code all three hosts run*. `invoke` is the one dispatch; the C door and the Tauri door are translators over it (Chapter 1’s “one body, three hallways”); so a test that drives `invoke` directly is testing what the C host and the JavaScript host will see, minus only the thin type-marshalling of the doors themselves. Coverage measured against the native `invoke` is coverage of the whole product’s behaviour, which is why the fleet can make coverage claims at all — the number means something because the thing measured is the thing shipped. An architecture where each host had its own logic would need three test suites and could trust none of them; the fleet needs one, because there is one body to test.

Feature matrices as executable specifications

The fleet’s most distinctive testing artifact is the *feature matrix*: an enumerated list of a predecessor’s behaviours, each scored full, partial, or missing, and each backed by a test that establishes the score. `zreq-core`’s 83 Postman behaviours and `zcite-core`’s 60 Zotero features are not marketing tallies rounded up for a README; they are the honesty compact made executable, a specification written as a checklist against the tool being ported and then held to by tests. The matrix answers the question a faithful port must answer — “how much of the original do you actually do?” — with a number whose denominator is named and whose numerator is defended by code.

The matrix does something a plain test suite does not: it makes *incompleteness* a first-class, tracked value rather than a silent gap. When `zreq`’s matrix records that a full JavaScript test sandbox and per-host client TLS certificates are the two missing behaviours (Chapter 15, Appendix K), that is the matrix functioning as a roadmap and as an honesty ledger at once — the port knows exactly what it does not do, names it, and does not round the 92.2% up to a hundred. This is the opposite of the “X but faster” projects the fleet’s own bar rejects: a project that measures itself against its predecessor behaviour-by-behaviour, publishes the

denominator, and marks its own gaps is a project whose completeness claims can be checked, which is the whole point of Appendix K. The feature matrix is a test suite that also tells the truth about coverage.

Coverage numbers with denominators

The discipline that separates the fleet's coverage claims from vapor is that every percentage carries its denominator and its breakdown. `zreq` is not “basically complete”; it is 92.2% over 83 behaviours — 72 full, 9 partial, 2 missing — and the two missing are named. `zcite` is 95.0% over 60 features — 54 full, 6 partial, 0 missing. A number with a denominator is falsifiable: a reader can, in principle, walk the 83 behaviours and check each score against the code, which is exactly the checkability the front matter promises. A number without a denominator — “90% of Postman” — is a mood, and the fleet does not ship moods as metrics.

Naming the *partial* count matters as much as the missing count, because partial is where dishonesty usually hides. A feature scored “full” that is really “mostly” is a claim a test can catch; a feature quietly scored “full” when it is partial is the rounding-up the honesty compact forbids. By carrying 9 partials in `zreq` and 6 in `zcite` as their own category rather than folding them into full or missing, the matrices keep the middle honest — a partial is a behaviour the port does incompletely and says so — and the percentage is computed with partials weighted as partials, not as wins. The number is conservative by construction, which is why the book can quote it without hedging: it was already hedged, in the direction of underclaiming, before it was published.

What a headless test cannot catch

The chapter would be dishonest if it stopped at the payout, because the zero-GUI test strategy has a real blind spot, and the fleet's own honesty compact demands naming it. A `cargo test` that drives `invoke` verifies the engine's *logic* and never sees the *shell*: it does not open the `WebView`, does not render a `zgui-core` widget, does not mount `ZGui.appShell`, does not exercise the Tauri command marshalling or the event emit, does not click anything. The layer the tests cover completely is the layer with no GUI; the layer they cannot cover is precisely the layer that *is* GUI — the chrome, the mount, the front-end glue between the engine's JSON and the widgets.

This is why the fleet's quality story is not “the tests are green” but “the tested layer is the layer that carries the domain risk, and the untested layer is thin by design.” The argument only holds because the shell is genuinely thin: if domain logic leaked up into the front-end glue, the headless tests would be testing the wrong half and the coverage numbers would be lies of omission. The thin-shell thesis and the headless-test strategy are the same bet made twice — that the risky code is the engine and the engine has no GUI — and they stand or fall together. Where the shell is thicker than it should be (an older app whose front end still carries logic the engine should own), the tests reach less far, and the `GUI_APP_REQUIREMENTS.md` conformance gaps that Chapter 15 treats as a roadmap are, read through this chapter, also a map of where the headless tests stop covering. The fleet tests what it can test completely and is honest that the seam it cannot reach is the seam it works to keep thin.

Testing across the feature gates

There is a last subtlety a fleet of feature-gated crates must handle: an engine has many builds, not one, and a test suite that only exercises the default build tests only one of them. Appendix J shows every core as a set of Cargo features — `net`, `tauri`, `wireguard`, `docker`, `kube`, and the rest — and the important property the gates promise is that `--no-default-features` yields a pure, network-free, GUI-free library. That promise is only real if it is tested: a core whose non-network build silently failed to compile would have a broken embedding story the moment a host wanted the pure library, and no default-build test would ever notice.

So the testing discipline extends to the gates. The pure core — the part that must compile and pass with no features — is where the bulk of the unit tests live, because that is the part every consumer gets and the part the C ABI and the embeds depend on. The gated parts are tested with their gate on, but the invariant the fleet

most needs to hold is that the ungated core is complete enough to be worth testing on its own: a `zreq-core` that could resolve variables, sign auth, and generate code without the `net` feature is a `zreq-core` whose logic is genuinely separable from its wire, and the way you *know* it is separable is that it builds and tests without `net`. The feature gates are the embedding story's fine print, and testing the ungated build is how the fleet keeps that fine print honest — the same way the coverage denominators keep the completeness claims honest and the `needs_transport` status keeps the connection claims honest. Testability, in the end, is not a separate virtue bolted onto the architecture; it is the zero-GUI rule and the feature gates seen from the test runner's side.

What a matrix row looks like as a test

A feature matrix is only an honesty instrument if a score is defended by something executable, so it is worth walking one row from behaviour to assertion. Take Postman's variable precedence: a collection-level variable shadows an environment variable of the same name, which shadows a global. As a matrix row it is one behaviour with one score. As a test it is a `Scope::build(globals, env, collection)` over three inputs that deliberately collide, a `Scope::render` of a string containing the contested token, and an assertion that the collection's value is the one that came out. No network, no window, no fixture file — three maps and a string, because the behaviour under test is a pure function and the port built it as one.

That shape is what makes the scores cheap enough to maintain across two matrices and several hundred rows. The rows that are expensive to test are the rows whose behaviour is entangled with an effect, and the fleet's architecture systematically converts those into rows that are not. Auth signing would be an expensive row if a signature could only be checked by sending a request; it is a cheap row because `http::prepare` produces a `PreparedRequest` and `auth::sign` operates on it, so the test signs a known request with a known key at a fixed timestamp and asserts on the header. Code generation would be expensive if a snippet were derived independently; it is cheap because `codegen::generate` consumes the same `PreparedRequest`, so the assertion is a string comparison. The pure half of the pure/impure seam is not only where the interesting logic lives — it is where the *checkable* logic lives, and the matrix scores what is checkable.

The rows that resist this are the ones the scoring vocabulary handles by refusing to round. A behaviour that can only be confirmed against a live third-party service — an identifier lookup resolving a real DOI, a sync against a real IMAP server — cannot be pinned by a test that runs on every save without inventing a mock whose fidelity is itself unverified. The honest handling is to score what the engine's own half guarantees, mark the remainder partial, and say which half is which, rather than to write a mock that always agrees and call the row full. That is the same instinct as `needs_transport`: when the boundary between what was done and what remains is real, make the boundary visible instead of papering it.

There is one more thing a matrix row does that a test alone does not, and it is why both artifacts exist. A test asserts that the behaviour the port implemented is correct; the row asserts that the behaviour is the *predecessor's*. A test suite with perfect coverage of a wrong model is green, and the only thing that catches it is a row that names the original behaviour it is scoring against. The matrix is the link between the code and the program being ported, and the test is the link between the code and its own intent — which is why Chapter 25 can call the format round-trips the objective instrument, since a round-trip is the one check that tests the model against the predecessor's own artifacts rather than against either the author's tests or the author's reading of the original.

Chapter 20 – Speaking Every Language: The Shared `i18n` Catalogue

A fleet built from one toolkit should speak one language *system*, and it does. Localization is a cross-cutting concern of exactly the kind the fleet handles by sharing rather than duplicating: not ten translation files, one per app, drifting apart as strings are added, but one shared catalogue every app draws from, so that a term translated once for one app is translated for all of them. This chapter is how the fleet says the same thing in every language without saying it ten times.

One catalogue, namespaced keys

The translations live in a shared submodule, `zpwr-i18n`, pinned into every app that shows localized text, with the strings organized as per-locale JSON files keyed by namespaced, dotted keys. The namespacing is what lets one catalogue serve ten apps without collision: a shell-chrome string that every app shares (a settings-panel label, a palette hint, a common toast) lives under a shared namespace and is translated once for the whole fleet, while an app-specific string lives under that app's namespace. The shared toolkit's chrome — the `zgui-core` shell, settings, shortcuts overlay — is localized through the shared keys, so an app inherits a translated shell the same way it inherits a styled one: by consuming the submodule, not by re-translating the buttons.

Storing the catalogue as JSON keyed by dotted names is the same durable, dependency-free choice the stores make: no localization framework, no compiled message catalogues, just `serde`-parsed JSON the toolkit looks strings up in at runtime. It also means the catalogue is diffable and mergeable as ordinary text, which matters for the concurrency story below. The key namespace is, in effect, a second public surface the fleet maintains alongside the command namespace of Appendix A — one names an engine's operations, the other names the fleet's user-visible strings — and both are kept auditable in one place for the same reason: a shared surface that many consumers depend on has to be legible in a single file.

`ZGui.i18n` and the runtime merge

The consumption side is a `zgui-core` facility, `ZGui.i18n`, listed among the toolkit's infra surface in Appendix C. It is the front end's single lookup point: a widget or an app asks `ZGui.i18n` for a key in the active locale and gets the translated string, falling back to a base locale when a key is missing so a partially-translated locale renders in its own language where it can and the base language where it cannot, rather than rendering a raw key. The active locale is a shell-level setting, so changing it recolours the app's language the way changing a colorscheme recolours its palette — one setting, applied through the shared toolkit, reflected everywhere the toolkit drew.

The merge is where the shared-plus-specific split becomes runtime behaviour. An app's effective catalogue is the shared `zpwr-i18n` strings layered with any app-specific strings, resolved so that the shared chrome always finds its keys and the app's own strings sit atop them. One case in the fleet's memory makes the layering vivid: an app whose UI strings came from a SQLite store had to have the shared `zpwr-i18n` JSON merged in *underneath* the store at runtime, so that shared keys always resolved even where the app's own store was silent. That is the merge doing its job — the shared catalogue as the floor every app stands on, the app's own strings as whatever it adds above — and it is why a new shared string appears in every app at once: it was added to the floor.

Union-merge and concurrent translation

A single shared catalogue that ten apps' development touches is a concurrency hazard the same way the pointer cascade is, and the fleet handles it with the same deliberate care. When two lines of work each add strings to the same locale file — a new key for one app, a new key for another — a naive merge of the JSON would have them collide, and the fleet's answer is a union-merge discipline that combines both additions rather than forcing one to lose. The keys are namespaced precisely so that additions rarely conflict in *content* — two apps adding their own namespaced keys are adding to different regions of the same file — and the merge strategy resolves the structural overlap so both survive.

This is the i18n instance of a pattern the whole book documents: a shared surface is leverage, and leverage has a coordination cost the solo author pays in discipline. The shared catalogue means a term is translated once and every app benefits, which is the leverage; the price is that concurrent edits to the shared file must be merged as a union rather than a last-writer-wins overwrite, lest one session's new strings silently drop another's. It is the same trade as the shared `zgui-core` and the shared engines — cheap to consume, careful to change — and the union-merge on the locale files is the concrete form the care takes for translations, exactly as the matrix-guided cascade is the form it takes for pointers.

What is translated, and what is not

Honesty about scope applies to localization as much as to coverage. What the catalogue translates is the *chrome and the app-specific UI strings* — the labels, hints, menu items, settings, toasts, and overlays a user reads. What it does not translate, and should not, is the *data*: a PDF's contents, a citation formatted in a style's own language, a container's log lines, an email's body, a filename on disk. The engine returns data in whatever language that data is in; the toolkit localizes the frame around the data. Conflating the two would be a category error — asking the i18n catalogue to translate a user's document is asking the frame to rewrite the picture — and the fleet keeps the line where it belongs.

There is also an honest bound on completeness, the same kind Appendix K keeps for coverage: a shared catalogue is only as localized as its locale files are filled, and a locale that is partially translated renders in the base language where its keys are missing. That is the fallback working as designed, not a bug, but it is a bound worth stating plainly rather than implying every app is fully localized in every locale. The architecture guarantees that a string translated once is available to every app; it does not, by itself, guarantee that every string is translated in every language. The catalogue is the mechanism that makes fleet-wide localization *possible* and cheap; filling it is ongoing work, and the fleet marks the difference between the mechanism and its completeness the way it marks every other frontier in its own honesty notes.

A catalogue is not a framework, and what that costs

The i18n choice is the durable-deps rule applied to localization: per-locale JSON parsed by `serde` at runtime, looked up by dotted key through `ZGui.i18n`, with no localization framework and no compiled message catalogues. The benefits were stated — no dependency, diffable and mergeable as ordinary text, the same shape as the stores — and the costs deserve the same treatment, because a key-value catalogue is a strictly weaker tool than a message-formatting library and the difference shows up in specific places.

The first is plurals. “1 item” and “2 items” are one message in a framework that understands plural categories and two keys in a catalogue that does not, and languages with more than two plural forms need more keys still. A JSON catalogue can express this — a key per form, selected by the caller — but the selection logic is then the caller's, and a caller that gets it right for English and wrong for a language with four forms has produced a bug that only speakers of that language will ever see. The second is interpolation and ordering. A message with two substituted values has a fixed order in the base language and may need a different order in a translation, which a positional scheme handles and a naive concatenation does not; a catalogue does not enforce which one a caller used. The third is anything genuinely locale-sensitive that is not a string at

all — number grouping, date rendering, sort collation — which a catalogue has no vocabulary for and which therefore lands wherever the component that needed it put it.

None of these make the choice wrong, and the fleet's position on them is consistent with every other durable-deps decision in the book: the mechanism is deliberately small, the gap is real, and the gap is named rather than implied away. A framework would solve plurals and ordering and would add a dependency, a compile step, and a message syntax to every string in every app — for a fleet whose UI strings are overwhelmingly short labels, buttons, and toasts rather than prose with embedded counts. That is the calculation, and it is the same one `zreq-core` made about a crypto crate: the small thing, owned, until the domain demands the large one.

The honest bound, then, is narrower than “the fleet is localized.” The catalogue guarantees that a string translated once is available to every app, that the shared chrome arrives translated with the toolkit, and that a missing key renders in the base language rather than as a raw key. It does not guarantee that a translation is grammatically correct in a language whose plural or ordering rules the calling code did not consider, and no runtime lookup could. That is a bound on the mechanism, not a defect in the catalogue, and stating it is the same discipline Appendix K applies to coverage percentages — the mechanism is real, the completeness is ongoing, and the book marks which is which.

Chapter 21 – The Security Boundary: What the WebView May Ask For

Every chapter so far has treated the command bridge as an engineering seam — a place where types are marshalled and ownership changes hands. It is also a trust boundary, and the two facts are in tension. A `WebView` is a browser engine rendering HTML and running JavaScript; a `#[tauri::command]` is a Rust function with the full authority of the user's account. Whatever the front end can call, the front end can call with that authority, and “the front end” is a category that includes the app's own `main.js`, the `zgui-core` components it mounts, the mountable webui a `-core` ships, and anything that ever manages to execute in that document. This chapter is about where the fleet's architecture helps that situation, where it does not, and what the honest boundary actually is.

The bridge is an authority surface, not just an API

The single-command bridge of Chapter 4 was argued there on maintenance grounds: one `zrq_invoke` instead of a hundred named commands means the IPC surface stops growing with the feature set. Read as a trust boundary, the same shape says something sharper and less comfortable. Collapsing a hundred operations into one command does not reduce what the front end can do; it reduces what the *framework* can see it doing. A per-feature IPC surface would let a host-level permission system distinguish “may list collections” from “may send a request to an arbitrary URL,” because those would be two registered commands. The fleet's bridge presents them as one command and a string, so any permission decision a framework could make about `zrq_invoke` is a decision about the whole engine at once.

That is a real trade, and the book should name it rather than let the maintenance argument stand alone. The fleet takes the trade for a specific reason: the `WebView` in these apps is not a container for untrusted third-party content. It renders the app's own vendored frontend, copied at build time from sources the same author controls (Appendix D's `copy-webui.mjs` step), with no remote origin loaded into the window and no user-supplied HTML executing as app chrome. Under that assumption, a fine-grained per-command permission model would be adding locks between two rooms the same person owns. The assumption is what makes the trade defensible, which means the assumption is the thing to defend — and the moment an app in the fleet renders content it did not author *as chrome*, rather than as data inside a viewer widget, the calculation changes and the coarse bridge stops being free.

The engines make that assumption easier to hold than it would otherwise be, because of where their impurity lives. An engine that is pure computation over data has an authority footprint bounded by what it can compute: `zreq-core`'s `prepare` cannot touch anything, and it is `http::send` behind the `net` feature that reaches the network. `zcite-core`'s formatter cannot touch anything at all. The dangerous verbs in the fleet are concentrated, not spread — network sends, filesystem operations, credential reads, process spawns — and each of them is either feature-gated in an engine or living in the shell where Chapter 1's split put it. A coarse bridge over engines whose default build is inert is a materially better position than a coarse bridge over engines that can each do anything.

The `fs_*` family is the real amplifier

If you were going to point at one part of an app's registered surface and call it the security-relevant part, it would not be the engine command. It would be the shared substrate of Chapter 18. The `fs_*` family from

`zpwr-file-browser` is a file manager's worth of authority registered into every app that mounts a browser: directory listing with metadata, recursive grep, content hashing, archive extract, base64 file reads, per-file git status, a filesystem watcher. `terminal_spawn` is stronger still — it is a PTY running the user's shell, which is to say arbitrary process execution, offered as a family-baseline amenity in every conforming app.

This is not a criticism of the design; it is a description of where the authority actually sits, and it inverts a natural intuition. The instinct is to worry about the engine, because the engine is the app's substance. The engine is the *least* privileged part of the app. `zcite-core` formatting an APA citation has no more authority than a pure function; the terminal command sitting next to it in the same `invoke_handler` can run anything the user can run. An app's threat surface is not proportional to its domain's complexity — `zpdf-core`'s three hundred operations are three hundred transformations of a document — it is proportional to the platform-touching commands the shell registers, which is a much shorter list and an almost identical one across the fleet.

The uniformity is the useful consequence. Because every app registers the same terminal and file-browser commands under the same names (Chapter 18's registration block as a contract), the fleet's most privileged surface is *one* surface, implemented once, in two shared crates. A hardening decision made in `zpwr-file-browser` — a path normalization, a symlink policy, a refusal to follow a traversal out of a requested root — applies to every app the moment its pointer is bumped. The pointer cascade of Chapter 13 is usually discussed as a cost; for the privileged substrate it is the mechanism that lets one fix reach the whole fleet. Ten hand-rolled file browsers would mean ten independently wrong path handlers and ten separate fixes, which is exactly the divergence-is-a-bug argument arriving at a place where the stakes are higher than whether two tables sort the same way.

What the embed contract is, and what it is not

Chapter 14 described `ZGui.embed.view(fn)` as a mechanism that turns “seize a global” into an immediate exception, and it is worth stating plainly that this is a *correctness* mechanism and not a *security* mechanism. The distinction matters because the two look similar from a distance: both are enforcement, both replace convention with machinery, both fail loudly. But the `FORBIDDEN` list swaps a fixed set of `ZGui` paths for throwing stubs during an embedded view's init, and that is all it does. It does not sandbox the guest, it does not restrict what the guest may call on the Tauri bridge, and it does not survive an author who saves a reference to `ZGui.appShell` before entering the view or who defers a forbidden call past the `finally` that restores the originals.

None of that is a flaw, because the hazard the contract exists to answer is not an adversary. It is a mistake — an author calling a top-level API from inside an embedded pane, where the call is textually identical to the correct one and the failure is silent and remote from its cause. Against that hazard the mechanism is well matched: it fires at init, at the exact line, with a message naming the scoped alternative. Against an adversarial guest it would be trivially bypassable, and the fleet does not pretend otherwise, because the guests are sibling engines from the same monorepo. The embed contract's peer in the security world is a lint, not a jail, and the book gains nothing by blurring that.

The honest formulation is that the fleet has one trust boundary that matters and it is the process boundary. Everything inside an app's process — the shell, the engine, every embedded core, the whole `window.ZGui` surface — is one trust domain, and the mechanisms described in this book are there to keep that domain *correct*, not to partition it. The architecture's genuine security contributions are elsewhere and are structural: the engines' default builds cannot reach the network, credentials never enter the store (Chapter 17), the privileged work is refused by the engine and handed to a helper the shell owns (`needs_privilege`), and the most powerful commands live in shared crates where they are audited and fixed once. Those are real. A throwing stub on `colorscheme.apply` is a good idea for a different reason.

Secrets, and the discipline of not holding them

The credential story from Chapter 17 reads differently through this chapter's lens. `ztunnel-core` and `zftp-core` route secrets to the OS keychain behind the `keychain` feature and persist only a reference in the store, which was framed there as a persistence decision. It is also the strongest security posture in the fleet, and it is strong for a structural reason rather than a cryptographic one: the engine that never holds a secret cannot leak one. An engine whose store contains a bookmark id and whose `creds` module is compiled out under `--no-default-features` has nothing to disclose if its JSON is copied, diffed, emailed, or synced into a backup the user forgot about.

This is the plan-versus-effect discipline arriving at secrets. The engine computes with a *reference* to a credential and hands the impure act of resolving it to a gated module that talks to the platform's own secret store, the same way it computes a `PrivilegePlan` and hands the elevation to a helper. In both cases the engine's purity is not an aesthetic — it is the property that bounds the damage, because a component that cannot perform an effect cannot perform it wrongly. A fleet built the other way, with credentials in the engine's model because that is where the connection code lives, would have ten crates each holding secrets in memory and each a candidate for putting one somewhere it does not belong. The zero-GUI rule and the keychain split are the same instinct applied to two different kinds of impurity, and the security benefit of the second is a direct consequence of taking the first seriously.

The bound worth stating is that the protection is only as good as the gate. A build with `keychain` enabled does hold secrets in process memory for the duration of a connection, because it must; the claim is not that the engine never touches a credential but that it never *persistent*s one and never needs one in the paths that are tested, embedded, and built by default. That is a smaller claim than “secure by design,” and it is the one the code supports, which is the standard Appendix K holds every other claim in this book to.

Chapter 22 – Concurrency: One Lock, Three Hosts, and the Long Operation

Chapter 4 introduced the process-global `OnceLock<Mutex<Engine>>` as a simplicity decision — no handles for a C host to juggle, no way to end up with two divergent engines — and Chapter 16 returned to it as a poisoning hazard. Neither chapter asked the question a systems engineer asks immediately on seeing a single global mutex behind a GUI: what happens when an operation under that lock takes a long time? This chapter is that question, and the answer is a set of design constraints that explain several decisions the book has so far presented as independent.

Every invoke is a critical section

Start from the mechanics. `<prefix>_invoke` locks the engine, dispatches the command, and returns; the lock is held for the entire duration of the dispatch. That is true on all three doors, because all three doors are translators over the same body: a Tauri command that locks and calls `invoke`, a C function that locks and calls `invoke`, a Rust caller holding the same `Mutex`. There is exactly one engine and exactly one lock, so every operation an engine performs is serialized against every other operation on that engine, for the whole process.

For the overwhelming majority of the fleet's commands this is invisible and correct. `vars.resolve` renders a string. `bib.citation` formats a reference. `item.get` reads a map. `codegen.generate` walks a `PreparedRequest` and emits a snippet. These are pure functions over in-memory data, and serializing them costs nothing a user could perceive while buying the single most valuable property a concurrent data structure can have, which is that there is no concurrency to reason about. The engine's domain code is written as ordinary single-threaded Rust — `&mut self`, no interior mutability, no `Arc` graph, no lock ordering to get wrong — and that is a substantial simplification bought with a lock that is essentially never contended in the common case.

The interesting cases are the ones that are not fast. `http::send` waits on a remote server. `imap.sync` fetches a mailbox. `k8s.pods.list` waits on an API server across a network. `identifier.lookup` queries an external service. `ocr_page` grinds through pixels. Each of these is an `invoke` that holds the one lock for as long as it takes, and while it does, every other command on that engine blocks — including the cheap ones the UI wants to run to stay responsive, and including whatever command the user pressed a button to cancel with. This is the sharp edge of the single-global-engine decision, and the fleet's architecture answers it in three ways that were introduced separately and are really one answer.

The three escapes, seen together

The first escape is the event sink. An operation that narrates rather than returns does not need to hold the lock while it waits, because the waiting part is not the part that touches the engine's state. `zreq-core` emits `RequestStarted { method, url }` and later `RequestFinished { status, time_ms }`; `zemail-core` emits `SyncStarted` and `SyncFinished { fetched }`. Read as a concurrency mechanism rather than a UI convenience, the sink is what lets the engine tell the host “this is under way” without keeping the host's next call queued behind it. The events exist because polling is wasteful; they *matter* because narration is how a serialized engine reports progress on something that cannot be a single fast critical section.

The second escape is the streaming door. `zcontainer-core`'s `subscribe/unsubscribe/stream_send`

trio was justified in Chapters 4 and 10 on shape grounds — a log follow does not fit `invoke(cmd, args) -> Value`. The concurrency reading is at least as strong: a `k8s.pod.logs.follow` that ran as an ordinary `invoke` would hold the engine lock for the lifetime of the follow, which is to say until the user closed the panel, during which the app could not list a pod, inspect a container, or do anything else at all. The subscription trio's real work is that `subscribe` returns *immediately* with a numeric handle and the stream's output arrives through the sink, so the long-lived operation never occupies the critical section. The trio is not only a shape accommodation; it is the mechanism that keeps the fleet's most stream-heavy app from stalling its own UI on its own mutex.

The third escape is the feature gate, and it is the quietest. Because the network is behind `net` in `zreq-core`, `zcite-core`, `zftp-core`, and `zemail-core`, the `--no-default-features` build of those engines has no long operations at all — every command is pure computation over memory, and the single lock is uncontended by construction. That is why the same crate can be a well-behaved embedded guest inside a host that wants only the pure half (Chapter 9's argument for embedding `zftp-core` without `opendal`), and it is why the tested build and the embedded build are the ones where the lock story is trivially correct. The gate that Chapter 19 discussed as a testing discipline is also the gate that determines whether an embedded engine can stall its host.

Reentrancy: the hazard the sink introduces

There is a hazard the sink creates that is worth naming precisely, because it is the kind of thing that is obvious once stated and invisible until then. The `EventSink` is a `Box<dyn Fn(EngineEvent) + Send + Sync>` that the *host* installs and the *engine* calls. If the engine calls the sink while holding its own lock, and the host's sink implementation synchronously calls back into `invoke`, the second `invoke` blocks forever on a lock the first call still holds. The host wrote nothing exotic — it received an event and reacted to it, which is what event handlers do — and the engine did nothing exotic either. The deadlock lives entirely in the interaction.

The fleet's structure keeps this from biting in the shapes it actually uses, and the reason is instructive. Under Tauri, the sink is wired to `app_handle.emit(...)`, which hands the event to the WebView; the JavaScript that reacts to `"zrq-event"` runs on the front end and reaches the engine only by issuing a fresh `invoke` through the bridge — an asynchronous round trip, not a synchronous call from inside the sink. The event handler's reaction is queued as a new call rather than nested inside the emitting one, so the recursion never forms. Under the C ABI the same discipline is what a host must observe, and it is the one genuinely non-obvious obligation the otherwise boring FFI contract imposes: the registered `EventCb` should record, enqueue, or signal, and should not turn around and call `<prefix>_invoke` from inside the callback.

That obligation deserves to be stated as part of the contract rather than discovered. Appendix B's three rules — caller-owned strings, one process-global instance, JSON in and JSON out — are the memory-and-threading contract; this is its fourth clause, and it follows from the second. A single global engine behind a mutex is a wonderfully simple thing for a host to hold correctly on the first try, and the price of that simplicity is that the host may not reenter it from a callback the engine invoked. The C++ RAII facades that ship alongside the headers are the natural place for that discipline to be encoded once, for the same reason they are the natural place to keep the command string literals in a small reviewed set.

Cancellation is a domain verb, not a runtime primitive

The last consequence of the design is that there is no generic way to cancel an `invoke`. The signature has no cancellation token, the C door has no abort function, and a caller blocked on a lock held by a slow operation has no lever to pull. This is not an oversight; it is what a synchronous, string-dispatched, one-lock boundary implies. Adding a generic cancel would mean threading a token through every dispatch arm, giving the C door a handle to reference an in-flight call, and abandoning the property that a host tracks no state beyond the strings it must free — the whole simplicity argument of Chapter 4 in reverse.

What the fleet does instead is model cancellation where it is meaningful, as a domain command over domain

state. `zftp-core` has `transfer.cancel`, because a queued transfer is an object with a `TransferStatus` the engine advances and cancelling it is a state transition — `Queued` or `Running` becomes `Cancelled`, and the event sink narrates the change. `zcontainer-core` has `unsubscribe(id)`, because a subscription is an object with a handle and tearing it down is an operation on that handle. `ztunnel-core` has `vpn.disconnect`. In every case the thing being cancelled is a long-lived entity the engine models, the cancel is a fast command that mutates that entity's state, and the slow work observes the state and unwinds. Cancellation rides the same request/response rail as everything else, because it is not a runtime facility retrofitted onto the boundary — it is a verb in the domain.

The honest limit is that this covers exactly the operations the domain modelled as cancellable and nothing else. A single `request.send` against an unresponsive host, a `k8s.pods.list` against a wedged API server, an `ocr_page` on a pathological scan — these run to completion or to their own timeout, and the user waits. The mitigation is the same one that makes the whole design work: the operations that are genuinely long-lived are the ones the engines modelled as entities with status, and the operations that are not modelled that way are the ones that are short. Where that stops being true, the pattern the fleet already has is the one to reach for — give the slow thing an identity, a status, and a verb — rather than a general cancellation mechanism at the boundary, which would cost the boundary the property that makes it usable correctly on the first try.

Chapter 23 — The Cost of JSON: Where the Uniform Boundary Gets Expensive

The boundary's data contract is `serde_json` in both directions and a dotted string in the middle, and the book has praised that choice repeatedly: it is the one language all three hosts speak, it makes arguments forward-compatible, it makes errors uniform, it makes the C door a translator rather than a second implementation. Every one of those arguments is about *correctness and reach*. None of them is about cost, and a uniform boundary that is cheap for a command like `tag.set_color` is not automatically cheap for a command that produces a rendered page. This chapter is the cost side of the ledger: where JSON-in/JSON-out is close to free, where it is wasteful, and what the fleet does at the places where the uniform answer stops being the right one.

Where the cost is nothing

The great majority of the fleet's commands move small structured values, and for those the serialization cost is irrelevant next to the work being described. A `request.get` returns a request; a `search.quick` returns a list of item summaries; a `config.list` returns a handful of VPN configurations; `platform.info` returns a small object. These are modest structured payloads produced by `serde` derives that were going to exist anyway, because the same derives are what persist the model to disk (Chapter 17) and what the engine's own tests assert against. The marginal cost of the boundary for these commands is a `to_string` and a parse over data that is small by the nature of what it describes.

There is also a real benefit hiding in the sameness that a performance-minded reader might not weight properly. Because the wire format is the same as the disk format and the same as the test-assertion format, there is exactly one representation of a model to get right, and no impedance layer between the engine's internal types and what a host sees. A design that optimized the boundary with a binary framing would have introduced a second representation, a second set of derives, a versioning question about the binary layout, and a place for the two to disagree — for a saving that, on a `collection.list`, nobody could perceive. The fleet's boundary is not fast because JSON is fast; it is cheap because the payloads are small and the alternative would have cost more in complexity than it saved in bytes.

Where the cost is real: bulk bytes

The cost becomes real exactly when a command's result is not a small structure but a large opaque blob. Three shapes in the fleet have this property. The first is rendered imagery: `zpdf-core`'s page rasterization, `zphoto-core`'s image operations, `zoffice-core`'s `render` family. The second is file contents: reading a document, extracting an attachment, pulling an image out of a PDF. The third is high-rate streams: a container's log follow at volume, an exec session's output.

For bulk bytes, JSON is a bad container in three separate ways that compound. It cannot represent binary at all, so bytes must be base64-encoded, which inflates them before anything else happens. The encoding and the decoding are both real work proportional to the payload. And the result is a single large string that gets allocated, copied across the boundary, and — on the C door — heap-allocated for the caller to free, which

means a rendered page becomes a large `char*` whose lifetime the host now manages. Chapter 4's memory rule is admirably simple; it is simple partly because the strings it was designed around are small.

The book has already shown the fleet's two answers without labelling them as such, and they are worth putting side by side. The first answer is that the native door does not have to be JSON at all. Chapter 1's very first code sketch is `render_page(doc: &Document, page: usize, scale: f32) -> RgbaImage` — a Rust function returning a Rust image type, not a command returning a `Value`. That is one reason `zpdf-core` can be the fleet's most-embedded engine while shipping no C ABI: every consumer of it is a Rust host that links it natively and takes the bitmap as a value, so the highest-bandwidth engine in the fleet is precisely the one whose consumers never cross a string boundary to use it. The absence of `zpdf-core`'s C door, which Appendix K records as an honesty note about uniformity, is also a fact about bandwidth: nothing needed the door, and the door would have been the expensive way in.

The second answer is base64 where a byte payload genuinely has to cross as JSON, and the fleet is explicit about it rather than sneaking it in. `fs_read_file_base64` has the encoding in the command's name — the shared file browser's way of saying “this one hands you bytes, and here is the tax.” Naming it is better than hiding it because it tells a front-end author exactly which call is the expensive one in a directory-browsing session where every other `fs_*` call is metadata. The same is true of the engines that return encoded imagery for preview: the boundary is uniform, the cost is on the specific calls that carry pixels, and those calls are identifiable by what they return.

Streams and the per-event tax

Streaming has a different cost profile and it is worth separating. A subscription's output arrives as `EngineEvent::Stream { id, data }` — one serialized event per chunk, each one a JSON object with a tag, an id, and a payload, emitted to the sink and delivered as a WebView event or a C callback. The per-chunk overhead is small in absolute terms and irrelevant for a log line every few seconds. It is not irrelevant for a high-rate follow across a busy deployment or an exec session echoing a fast-scrolling build, where the cost is not the bytes but the *count* — many small serializations, many event dispatches, many front-end handler invocations, each doing a `type` switch and a route to the panel that owns the id.

The mitigation the architecture already provides is chunking, and it is a property of where the boundary sits rather than a feature anyone added. The engine reads from the underlying stream and emits what it read; a read that returns a kilobyte becomes one event, not a thousand. The granularity of `Stream` events is the granularity of the engine's reads, which means the tax is paid per read rather than per line, and a well-behaved reader amortizes it naturally. This is the same reason the exec panel can reuse the shared terminal widget (Chapter 10) without a bespoke fast path: `xterm.js` is built to consume chunks, and the boundary happens to deliver them.

The honest note is that the fleet has not needed to find the limit of this, and the book will not pretend to know where it is. The claim supportable from the architecture is narrow: the per-event cost is bounded by chunk size rather than line count, the same sink carries both a rare status change and a dense output stream, and the design did not introduce a second delivery path for the fast case. Whether that holds at a rate nobody has yet driven it to is exactly the sort of thing Appendix K exists to keep the book from claiming.

What the uniformity buys back

It would be an incomplete accounting to list the costs without weighing what they purchase, because the fleet's boundary was not chosen in ignorance of the alternatives. A hand-optimized boundary — binary framing for bulk results, shared memory for bitmaps, a separate high-rate channel for streams — would be faster on the three shapes above and would cost the property that every other chapter in this book depends on. It would mean the C door and the Tauri door no longer see the same result shape, so a host written against one no longer ports to the other unchanged (Chapter 16). It would mean the tests that drive native `invoke` no longer exercise what the other hosts receive, so the coverage numbers of Chapter 19 would stop meaning what

they claim. It would mean the error contract fragments, because an out-of-band binary channel needs its own failure signalling.

So the fleet's position is a deliberate one: pay JSON's cost everywhere, take the native Rust API where the cost would actually hurt, and name the base64 calls so nobody is surprised. That is a defensible engineering trade for a fleet whose bandwidth-heavy consumers are all Rust hosts linking natively and whose cross-language consumers are moving structured data. It would be the wrong trade for a stack whose primary consumer was a C++ host demanding rasterized pages at frame rate, and the reason the fleet can take it is a fact about its own consumers, not a general claim about boundaries. The architecture's uniformity is worth its cost here because of who is on the other side of it — which is the same reasoning, applied to a different question, as Chapter 21's argument about why the coarse command bridge is acceptable in an app that renders only its own frontend.

Chapter 24 – The Mountable Webui and the Two Substrates

The front-end half of the `-core` pattern has appeared in fragments across the book: a `webui/` directory inside `zreq-core` carrying `zreq.{html,css,js}`; a build step that copies it into the app's served `frontend/`; a transport auto-detection that reaches Tauri when Tauri is present and falls back to a mock in a plain browser; a shim that lets the same modules run on JUCE. Those fragments are one design, and it is a consequential one — it is what makes an engine embeddable *with its own view* rather than as a headless library a host must build a UI for. This chapter assembles it.

The engine ships its own view

The unusual move is that the view lives in the engine's repository, not the app's. `zreq-core` contains `webui/zreq.html`, `webui/zreq.css`, and `webui/zreq.js`, and the `zreq` app's build copies them into the directory Tauri serves. Chapter 15 described the consequence — a new app's `main.js` “relocates its engine's mountable view into `shell.body`” — without dwelling on why the view is on that side of the boundary, and the reason is the embed matrix. If the view belonged to the app, then `zcite` showing a PDF would have to either build its own PDF viewer or vendor a piece of the `zpdf application`, which is a dependency on a shell rather than on an engine. Because the view belongs to the engine, embedding `zpdf-core` brings the viewer along, and the host mounts it (Chapter 14's walk). The mountable webui is what makes “embed an engine” mean “embed a capability the user can see and use” instead of “embed an API you must now build a UI for.”

That framing also explains the sharpest constraint on such a view, which Chapter 14 enforces mechanically: it must be mountable into a *pane*, not into a *window*. A view that assumes it owns the document — that mounts `appShell`, binds the global palette hotkey, writes `:root` — is a view that works exactly once per process, in the app that owns it, which would defeat the entire purpose of shipping it with the engine. The `FORBIDDEN` list is, read this way, the specification of what a mountable view may assume about its surroundings: it may assume `window.Zgui` exists and it may assume it has an element to draw in, and it may assume nothing else. Everything it wants that is document-global — a palette, a theme, a settings row — it obtains through the scoped APIs or by contributing to the host shell it was handed.

The standalone app, then, is a special case of the embed rather than the other way round. `zreq` the application is the `zreq-core` view mounted into a shell whose body is the whole window; `zcite` embedding a PDF viewer is the `zpdf-core` view mounted into a pane. Same view, same command surface, different container. That inversion — the app as the trivial host — is why the tenth shell is boilerplate (Chapter 15) and why a view written for a standalone app is embeddable without a rewrite. The mountable view is the front-end analogue of the zero-GUI rule: keep the component free of assumptions about its surroundings, and it composes.

Transport auto-detection and the mock

The view drives its engine through a transport it discovers at load time rather than one it is configured with. Under Tauri it calls `invoke("zrq_invoke", { cmd, args })` and listens for `"zrq-event"`. In a plain browser, with no `window.__TAURI__` present, it falls back to an in-memory mock so the view renders in preview mode with no backend at all. The shape is small:

```
// the mountable view picks its transport at load, not at build
const T = window.__TAURI__
  ? { invoke: (cmd, args) => tauriInvoke("zrq_invoke", { cmd, args }),
      listen: (h) => tauriListen("zrq-event", h) }
  : mockTransport(); // in-memory, so the view renders with no backend
```

Two things follow from that, and the second is the interesting one. The obvious payoff is development: a view that renders in a browser tab is a view you can iterate on without building and launching a desktop app, with the whole browser toolchain available. The less obvious payoff is that the mock is a *design constraint* with teeth. A view that can render against an in-memory mock is a view that does not assume the engine is present, does not assume a command is fast, and does not hide domain logic in its own JavaScript — because whatever logic lived in the view would have to be duplicated in the mock to make the preview coherent. The fallback quietly enforces the thin-front-end discipline that Chapter 19 identified as the load-bearing assumption behind the headless-test strategy: the tests only cover the risky layer if the risky layer really is the engine, and a view that must render against a mock has a hard time accumulating risk.

It is worth being precise about what the mock is not. It is not a test double the test suite drives, and it does not make the front end tested — Chapter 19’s blind spot is still the blind spot. It is a rendering fallback that makes the view previewable and, as a side effect, makes it obvious when the view has started depending on the engine for something other than data. Those are useful properties and they are smaller than “the front end is covered,” which is the sort of overstatement the honesty ledger exists to prevent.

Two substrates, one frontend contract

The transport indirection pays a second dividend that a Tauri-only fleet would not need. The requirements spec documents two native substrates behind one frontend contract: Tauri, where the backend is reached through `window.__TAURI__`, and JUCE, where it is reached through `window.Juce` and `window.__JUCE__.backend`. The shared web modules — the whole `zgui-core` toolkit and the mountable views — run unchanged on both, because the only substrate-specific thing they do is the transport lookup they were already doing to support the browser mock. The shim is the third branch of a decision the view already had to make.

That is the payoff of having put the substrate dependency in exactly one place. A toolkit that had scattered `window.__TAURI__` checks through two hundred component files would face a two-substrate requirement as a two-hundred-file problem; a toolkit whose components never touch a transport at all, and whose views resolve one at load, faces it as a shim. The spec’s own phrasing — “divergence across substrates is a bug too” — extends the design system’s founding principle from “ten apps must look alike” to “two runtimes must behave alike,” and the mechanism that delivers it is the same in both cases: one implementation, consumed rather than reimplemented, with the variation isolated to a single seam.

The JUCE substrate also clarifies what the audio plugins’ row of dashes in the Chapter 13 matrix means. A `zpw-synth` consuming only the clip and patch engines, with no document cores and no shared terminal, is not an app that failed to adopt the baseline; it is a plugin on the other substrate consuming the parts of the shared frontend that apply to it and declining the parts that assume a desktop application. The frontend contract is what makes that a coherent position rather than a fork: the plugin runs the same widgets from the same toolkit through a different transport, so a fix to a component reaches the plugin and the desktop app alike, even though the plugin will never register an `fs_*` command in its life.

The build step, and why the served copy is disposable

The mechanics that hold this together are the vendoring step from Appendix D: a `beforeBuildCommand` (and its `beforeDev` twin) that copies the shared web modules and the engine’s `webui/` into the directory the app serves, recursively, dereferencing symlinks, filtering `.git`. The copies are gitignored and regenerated on every build, and the served frontend is never hand-edited — the rule is to edit the source under the engine’s or the toolkit’s own tree.

That rule is worth stating as a rule because violating it is easy and the consequence is silent. A served copy is exactly where a developer debugging a layout problem in a running app naturally reaches, and an edit made there works, survives testing, and vanishes on the next build with no error and no diff to explain the regression. Treating the served directory as build output — regenerated, gitignored, never authoritative — is the same discipline as never editing a `target/` directory, applied to a place where the files look like source because they are HTML and CSS. The copy step is what makes the engine's view the single source of truth in practice and not merely in intent, and the gitignore is what keeps a stale copy from becoming a second source that drifts.

The copy also explains a detail of the Chapter 13 matrix that is otherwise puzzling. `zgui-core` and `zpwr-i18n` are deliberately *not* columns in the consumption table, because they are consumed by copy at build time rather than pinned per-component the way the engines are. The matrix tracks what an app pins as a submodule and links as a dependency; the toolkit and the catalogue are vendored into the served frontend by a build script. Both are shared, both are single-source, and the difference in how they arrive is exactly the difference between a Rust crate the app links and a directory of web modules the app serves.

Chapter 25 – Reading a Predecessor as a Specification

Ten of these apps are ports, and a port has a problem no greenfield product has: the specification already exists, but it is not written down anywhere. It exists as a shipping program with decades of accumulated behaviour, as the muscle memory of its users, and as a set of file formats other tools read. This chapter is about the method the fleet uses to turn that into something a Rust crate can be built against and measured by — the feature matrix — and about what the method can and cannot capture. It is the missing methodology behind every “the port is 92.2% of Postman” claim the book has made.

Enumerate before you build

The first move is to write the predecessor down as a list of discrete behaviours before writing the engine that implements them. `zreq-core` carries `scripts/port_features.tsv` with 83 enumerated Postman behaviours; `zcite-core` scores 60 Zotero features. The artifact is deliberately dull: a row per behaviour, a score, and enough identification to find the behaviour in the original and the code in the port. It is not a design document and it is not a roadmap in the aspirational sense; it is an inventory taken of something that already exists.

Doing this first, rather than as a retroactive audit, changes what gets built. An enumerated predecessor forces the question “what *is* a behaviour here” at a point where the answer can still shape the model — and the answers are what produced several of the design decisions earlier chapters presented as obvious. Postman’s inheritable auth is one row, and it is the row that makes `Auth::Inherit` the default variant and `resolve_auth` a request-to-folder-to-collection walk rather than a lookup. Postman’s dynamic variables are a row, and enumerating them is what turned `{{randomEmail}}` into a catalogue with a defined behaviour for unknown generators — an empty string, so nothing half-substituted reaches the wire — rather than an ad-hoc set someone added as they were noticed. The inventory is a design input, not a report.

The enumeration also sets the denominator, and setting it before the work is what keeps it honest. A denominator chosen after the fact is a denominator chosen to flatter: you count the behaviours you implemented and a few you know you skipped, and the percentage is a function of how generous you were feeling. A denominator fixed at the start by walking the predecessor is a number that can go against you. It is entirely possible for an honest inventory to reveal that a port is at half of its target, and a matrix that cannot deliver bad news is not a measurement. That the fleet’s published figures are conservative is a claim in Appendix K; that the method *permits* an unflattering figure is what makes the claim worth anything.

What “full” has to mean

The scoring vocabulary is three-valued — full, partial, missing — and the definitions do the real work. The fleet reserves `full` for a feature whose *behaviour and GUI both* match, which is a stricter bar than it first appears and is the single most important detail of the method. It means a behaviour perfectly implemented in the engine and not reachable from the interface scores `partial`, not `full`. It means “the engine can do it” is not a passing grade, which for an engine-centric architecture is exactly the discipline required, because an engine-centric architecture is the one most likely to mistake capability for delivery.

That bar is also what makes the number describe the *product* rather than the crate. A user does not experience

`zreq-core`'s dispatch; they experience whether the thing they did in Postman is a thing they can do in `zreq`. Scoring against the interface keeps the matrix pointed at that question, and it means the matrix cannot be improved by adding a command nobody can invoke. It has a cost — the score is dragged down by wiring work that is not intellectually interesting — and paying that cost is what separates a coverage number from a marketing number.

The `partial` category is where the method's honesty concentrates, for the reason Chapter 19 identified: partial is where dishonesty naturally hides, because rounding a “mostly” up to a “full” is a small act with no immediate consequence. Carrying partials as their own count and weighting them as partials rather than as wins means the published figure is conservative by construction, and it means the matrix is useful as work tracking rather than only as a claim — a list of nine partials is a list of nine specific things to finish, each one identified against the behaviour in the original that it does not yet fully match.

The formats are the objective test

Enumeration and scoring are judgement-based, and a method that stopped there would be a careful opinion. The fleet's second instrument is not a judgement at all: round-tripping the predecessor's own file formats. `zreq` imports a Postman Collection v2.1 and exports one; `zcite` imports and exports BibTeX, RIS, CSL-JSON, EndNote XML, MODS, MARCXML, and Zotero RDF; `zftp` reads Cyberduck's `.duck` bookmarks and FileZilla's XML.

The reason this is stronger evidence than a feature score is structural. A collection that survives `import_postman` followed by `export_postman` intact proves that the port's model carries every piece of information the original document contained — every folder, every body mode, every auth scheme, every script — because information the model failed to represent could not be written back out. That is a property a test asserts, not a score a human assigns, and it converts “the model is faithful” from a claim into a check that fails when it stops being true. Chapter 5 called import/export “the quiet proof that the model is faithful”; the methodological point is that it is the *only* part of the fidelity claim that is objective, and a port that skipped it would have nothing but its own scoring to stand on.

There is a second, subtler benefit. A format round-trip is a lower bound on model fidelity that keeps working as the port evolves. A feature matrix is a snapshot that goes stale unless someone re-scores it; a round-trip test runs on every commit and breaks the moment a refactor drops a field. For a solo author without a QA team (Chapter 19), the distinction between “an artifact I must maintain” and “a check that maintains itself” is the difference between a discipline that survives and one that decays, and it is why the formats do more of the fidelity work in practice than the matrix does.

What the matrix cannot score

A method owes an account of its blind spots, and this one has three worth naming.

The first is that a behaviour list flattens a workflow. Enumeration captures “supports OAuth 1.0a signing” and “supports collection-level variables” as independent rows, and users do not experience features independently — they experience sequences, where the value is in how naturally one step leads to the next. A port can score well on rows and still feel wrong if the sequence is awkward, and no amount of row-level scoring detects that. The fleet's partial answer is that the shared shell forces a consistent interaction model across the fleet (Chapter 12's R1 through R10), so at least the sequence-level behaviour is uniform and deliberate rather than accidental — but “consistent” is not “faithful to the predecessor's flow,” and the matrix does not measure the difference.

The second is that the inventory is only as complete as the walk that produced it. An 83-row enumeration of Postman is 83 rows someone found; a behaviour nobody thought to look for is not scored `missing`, it is simply absent from the denominator, which inflates the percentage in a way no amount of conservative scoring corrects. This is the most important limitation of the method and the hardest to mitigate, and the only real defence is the one the fleet uses elsewhere: publish the denominator and the list, so the enumeration is

auditable by someone who knows the original better than the author does. A number with a denominator can be argued with. That is the point of publishing it.

The third is deliberate scope exclusion, which the matrix records but cannot evaluate. `zreq` marks a full JavaScript test sandbox as missing and out of scope, because embedding a heavyweight JS engine would violate the durable-deps rule. That is a defensible engineering decision and it is also, for a user who scripts their tests in Postman, the feature that decides whether the port is usable. The matrix scores it `missing` and moves on; it has no vocabulary for “missing, and load-bearing for a particular user.” The honest reading of any coverage figure, therefore, is that it measures breadth and not fit, and the two named exclusions matter more to the reader they matter to than any percentage can express. Naming them by name, as Appendix K does, is what lets a reader do that evaluation for themselves — which is the most a measurement of a port can honestly offer.

Chapter 26 – Bringing Up the Eleventh Engine

Chapter 15 argued that the eleventh app is a template rather than a design, and left the claim at that. This chapter cashes it: the order of operations for standing up a new engine and its shell, what each step buys, and where the pattern makes a decision for you versus where the domain genuinely has to. The engine below is a placeholder — call the crate `zexample-core` with the prefix `zex_` — used to show the shape without implying a product. Everything in it is the pattern this book has already documented, assembled in the order it is actually assembled in.

Step one: the model and the dispatch, before anything else

The first code is not the FFI, not the plugin, and emphatically not the shell. It is the domain model and the one dispatch, in a crate with the mandatory dependency set and nothing else — `serde`, `serde_json`, `thiserror`, `dirs`, with everything platform- or network-touching left for a feature gate that does not exist yet.

```
// zexample-core/src/lib.rs - the only two things that must exist first.
pub struct Engine {
    model: Model,           // the domain, serde-derived
    sink: Option<EventSink>, // installed by the host, if it narrates
}

impl Engine {
    pub fn invoke(&mut self, cmd: &str, args: &Value)
        -> Result<Value, Error>
    {
        match cmd {
            "version" => Ok(json!({ "version": VERSION })),
            "item.list" => self.item_list(args),
            "item.add" => self.item_add(args),
            other      => Err(Error::UnknownCommand(other.into())),
        }
    }
}
```

Writing this first is not arbitrary ordering; it is what makes the rest of the steps mechanical. The `match` is the engine's public API (Appendix A), so naming its arms is the API design, and it is the only genuinely creative decision in the whole bring-up. The `Result<Value, Error>` is what every door will translate, so the error enum has to exist now, in the crate's `error` module, as a `thiserror` enum with a variant per failure kind and `#[from]` lifts for whatever lower-level errors the domain code will thread with `?`. The catch-all arm returning `UnknownCommand` is not a detail to add later — it is the failure contract's ground floor, the thing that makes a typo a clean rejection rather than a panic across an FFI seam.

The discipline that matters at this step is refusing dependencies. Every crate added here is a crate every consumer of the engine will carry, in every app that embeds it. The question to ask of each is the one `zreq-core` asked of a crypto crate and answered by writing six primitives instead: is this load-bearing domain plumbing nobody sane would rewrite, or is it a convenience worth a few hundred lines of owned code? `lopdf`,

`bollard`, `kube`, `opendal`, `russh`, and `boringtun` passed that test. MD5 and HMAC-SHA256 did not.

Step two: tests, while the crate is still pure

The tests come second because right now they are trivially possible and they will never be easier. The crate has no GUI, no network, no platform code — there is nothing to mock, no display to open, no event loop to spin. A test constructs an `Engine`, drives real command strings through the real dispatch, and asserts on the returned JSON, which is exactly what all three hosts will do later.

Two habits established here pay for themselves repeatedly. The first is driving the commands as *strings* rather than calling the internal helper functions directly, because the string dispatch is the thing that ships and a test that bypasses it will not notice a renamed arm — Chapter 4’s answer to the stringly-typed objection depends on this being true. The second is pointing the store at a temporary directory from the start, so the persistence round-trip is under test from the first commit and no test ever touches the user’s real data directory.

This is also the natural point to write the feature matrix of Chapter 25, if the engine is a port. The inventory of the predecessor’s behaviours is a design input while the model is still soft, and merely a retroactive audit once the model has hardened.

Step three: the store, and where its bytes go

Persistence is next, in a `store` module, following Chapter 17 exactly: `serde`-derived model to JSON, whole-file load and save, `#[serde(default)]` and `Option<T>` on everything so an older file loads in a newer build. The load path tolerates a missing file (first run) and a partial one (a shape that changed) rather than erroring; the save path serializes what is in memory.

The one decision the engine does not make is *where*, and the discipline is to take the directory as a parameter the shell supplies rather than resolving it in the crate. The convention is the bundle-id directory Chapter 17 describes, resolved by the shell through Tauri’s `app_data_dir()`. An engine that hard-coded a `dirs::data_dir()` join with its own product name would write outside its host’s identity the moment it was embedded, and would be untestable without touching the developer’s real data. Parameterizing the location is what makes the same store correct in three situations that look different and are not: a standalone app, an embedded guest inside a sibling, and a `cargo test`.

Credentials, if the domain has any, are the exception the same chapter carves out: a `creds` module behind a `keychain` feature, with the store persisting a reference and the secret going to the platform’s keychain. Getting that split right at bring-up is much cheaper than retrofitting it after a store has shipped with secrets in it.

Step four: the three doors, which are now mechanical

Only now does the boundary get written, and it is the least interesting part of the work — which is the entire point of having done the previous three steps first.

```
// zexample-core/src/ffi.rs - the C door: translate, lock, call, marshal.
#[no_mangle] pub extern "C"
fn zex_init() -> *mut c_char { /* OnceLock<Mutex<Engine>>; {"ok":true} */ }

#[no_mangle] pub unsafe extern "C"
fn zex_invoke(cmd: *const c_char, args: *const c_char) -> *mut c_char {
    // parse fallibly (never panic across the seam), lock, invoke,
    // serialize the Result - success or error - to a heap JSON string
}

#[no_mangle] pub unsafe extern "C"
fn zex_string_free(s: *mut c_char) { /* caller returns every string */ }
```

Three rules govern this file and none of them are domain decisions. Nothing may panic across the seam, so every parse is fallible and a bad command string or malformed args JSON becomes an error result. Failure is *in* the returned JSON, never a null or an error code, so a host reads success and failure the same way (Chapter 16). Every returned `char*` is heap-owned by the caller and freed with `zex_string_free`. If the domain narrates asynchronously, a fourth function registers the event callback and the engine's sink calls it; if it does not — as `zoffice-core` and `zphoto-core` do not — the function is simply absent, because the C-ABI function count is fitted to the domain rather than stamped uniformly (Appendix B's spectrum).

The Tauri plugin is smaller still: one `#[tauri::command] zex_invoke` behind the `tauri` feature that locks the global and calls `invoke`, plus a `setup` helper that installs a sink forwarding each event to `app_handle.emit("zex-event", event)`. A handful of lines and a wiring function, because the plugin is an adapter over the same body and not a second implementation.

The gates are declared here too, and the invariant to hold is the one Chapter 19 tests: `--no-default-features` must yield a pure, network-free, GUI-free library that still builds and still passes the bulk of the suite. If it does not, the domain logic and the platform reach have not actually been separated, and every downstream promise in this book — embeddability, testability, the C door being a translator — is weaker than it claims.

Step five: the view, then the shell

The mountable webui goes in the crate's `webui/` (Chapter 24): a view that resolves its transport at load, drives the engine through `zex_invoke`, listens for `"zex-event"`, falls back to an in-memory mock in a plain browser, and builds every piece of chrome from `window.ZGui.*`. Two constraints are worth restating because they are the ones a new author gets wrong. The view mounts into an element, never into the document — no `appShell`, no global hotkey binding, no `:root` write — because a view that owns the document is a view that cannot be embedded, and Chapter 14's `FORBIDDEN` list will turn the mistake into an exception the first time a sibling app mounts it. And no domain logic goes in the view; if the mock has to reimplement something to make the preview coherent, that something belonged in the engine.

The shell is last and is nearly all inherited. Its `main.js` mounts `ZGui.appShell`, relocates the engine's view into `shell.body`, and wires the engine's commands into the shell's palette and settings panel. Its `src-tauri/main.rs` registers the standard block of Chapter 18: the engine's one `zex_invoke`, the four `terminal_*` commands, the `fs_*` family, and whatever handful of genuinely platform-specific commands this app and no other needs — a window gesture, a trash call, the diagnostics trio. Its `tauri.conf.json` sets `frontendDist` and the `beforeBuildCommand` that vendors the shared web modules and the engine's `webui/` into the served directory.

Nothing in that paragraph is a design decision. Every part of it is a shape the fleet already settled, which is what Chapter 15 meant by “a reused design, not just reused code,” and it is why the honest description of building the eleventh app is that you build an engine and then perform a checklist.

What the template does not decide

Three things remain genuinely open, and pretending otherwise would make the template sound more powerful than it is.

The command vocabulary is the first and the most consequential. Whether an engine's surface is dotted namespaces (`docker.container.inspect`) or a flat operation list (`zpdf-core`'s `apply_redactions`, `linearize`, `certify`) is a domain judgement the pattern does not make, and it is effectively permanent once consumers exist — Chapter 13 noted that a shared engine's command names are public API and calcify accordingly. The observable pattern across the fleet is that domains with a natural object hierarchy took namespaces and domains that are a large flat set of transformations over one document took a flat list, but that is a description of what the engines did, not a rule the template enforces.

The second is whether the domain narrates at all. Seven engines carry an `EngineEvent` enum and three do not, and the difference is a fact about the domain rather than a completeness gap: a document, an image, and a PDF are computed on request and have no asynchronous life. Adding an event channel to an engine whose domain has nothing to say would be adding a mechanism with no messages, and the fleet's habit is to let the absence stand as information.

The third is where the engine stops. Chapter 1's rule says the engine does the pure computation and the shell absorbs the impure remainder, and applying it is a judgement each domain forces separately: `ztunnel-core` stops at a `PrivilegePlan` because the elevation is not its to hold; `zcontainer-core` stops at speaking the Engine API because running a hypervisor guest is not; `zftp-core` stops at `needs_transport` where a backend is not wired. The recurring resolution — compute the plan as inspectable data, return it, let the shell apply it — is the most transferable idea in the fleet and the one a new engine should reach for first when it finds itself wanting to do something impure. The template gives you the doors, the tests, the store, the failure contract, and the shell. Where to draw the line is still the work.

Chapter 27 — The Import Surface: Reading the Predecessor's Files

Every chapter about a port so far has argued fidelity in terms of *behaviour*: does the engine do what the original did, scored against an enumerated matrix. There is a second kind of fidelity that decides whether anyone ever gets far enough to evaluate the first kind, and it is the one nobody writes a chapter about because it is unglamorous. A user does not arrive at a reference manager empty-handed; they arrive with nine years of Zotero. A user does not arrive at a transfer client with no servers; they arrive with a Cyberduck bookmark file. The behaviour matrix measures what the port can do *once you are inside it*. The import surface measures whether you can get inside it at all.

This chapter reads the fleet's importers as one subsystem, because they were built one app at a time and they turn out to share a shape.

The switching cost is the real competitor

The competitor a faithful port actually loses to is not the predecessor's feature list; it is the user's accumulated state inside the predecessor. A port that matches ninety percent of Postman's behaviour and cannot read a Postman collection has, from the user's side, matched none of it, because the evaluation never starts. Every hour of configuration a user has invested in the original is an hour the port must either import or ask them to spend again, and asking them to spend it again is the same as asking them not to switch.

This reframes what an importer is for. It is not a convenience feature and it is not interoperability in the standards sense; it is the mechanism by which a port converts a competitor's installed base into its own. That is why the import commands sit in the engine rather than in the shell, why they are named in the same dotted vocabulary as the domain operations, and why they appear in every engine that replaces a product people have already configured — and, as we will see, are conspicuously absent from the engines that replace products people have not.

What the fleet actually reads

Appendix A's dispatch lists make the import surface countable. `zreq-core` reads four foreign shapes: `import.curl`, `import.postman`, `import.openapi`, and `import.har` — a shell command, a competitor's export, an API specification, and a browser's network capture. `zcite-core` reads seven bibliographic interchange formats — `import.bibtex`, `import.ris`, `import.csl_json`, `import.endnote_xml`, `import.mods`, `import.marcxml`, `import.zotero_rdf` — plus `import.file` and `inbox.import` for the file-level and watched-folder paths. `zemail-core` reads three mail-corpus shapes, `import.eml`, `import.maildir`, and `import.mbox`, and separately round-trips contacts through `contact.import_vcard` and `contact.export_vcard`. `zftp-core` reads two competitors' bookmark files: `bookmark.import` for Cyberduck's `.duck` plist and `bookmark.import_filezilla` for FileZilla's XML. `zgo-core` reads Alfred's `info.plist` through `workflow.import` and takes snippet collections through `snippet.import`. And `ztunnel-core` reads VPN configuration files through `config.import`, with `config.parse` and `config.wireguard_format` alongside it — the files a Tunnelblick user already has, since a VPN configuration is a document a provider handed them rather than something they authored in the app.

Read as a set, those are not arbitrary format choices. Each engine reads exactly the artifacts its predecessor's

users already have on disk, and the list per engine is a direct measure of how fragmented that predecessor's ecosystem is. `zcite-core` reads seven formats because bibliographic data has spent decades failing to converge on one; `zftp-core` reads two because desktop transfer clients each invented a bookmark file and never needed a third-party interchange; `zreq-core` reads four because an API client's input can arrive as a competitor's document, a spec, a capture, or a command someone pasted from documentation. The importer list is the predecessor's ecosystem, transcribed.

Import is broad, export is narrow

The asymmetry is the most interesting property of the surface, and it is visible without reading a line of Rust. `zreq-core` reads four formats and writes one: `export.postman` is the whole export list. `zftp-core` reads two bookmark formats and exports none. `zemail-core` reads three corpus formats and exports none of them, though it does round-trip vCard for contacts. Only `zcite-core` is symmetric: its `export.*` family names the same seven formats as its `import.*` family, and no other engine's export list mirrors its import list that way.

The asymmetry is not an oversight, and it is worth being explicit about what it means, because it cuts against the local-first argument the fleet makes elsewhere. A user's data is portable out of `zreq` in exactly one direction — back to Postman — and out of `zftp` in none, which is a weaker exit guarantee than the fleet's positioning implies. The mitigating fact is structural rather than rhetorical: the stores are `serde_json` documents in a known directory (Chapter 17), so the exit path from any engine is the store file itself, readable by anything that reads JSON. That is a real answer, but it is a different answer from a format export, and the honest reading is that the fleet optimizes hard for the inbound path and leaves the outbound path to the transparency of its own on-disk format.

`zcite-core`'s symmetry has an explanation the other engines do not share: for a reference manager, format export *is* a domain feature. A researcher exports BibTeX to hand to a co-author or a journal, not to leave the application. Where the format matters to the workflow, the engine writes it; where the format is only a migration path, the engine reads it and stops. Import breadth tracks the predecessor's ecosystem; export breadth tracks the domain's own interchange needs. Those are different pressures and the surface records both faithfully.

The importer is the purest code in the crate

Structurally, an importer is the ideal shape for this architecture, and the engines get that shape almost for free. An import is a pure function from bytes to model: give it a `.duck` plist and it yields bookmarks; give it a Postman collection and it yields a collection tree. It touches no network, holds no lock beyond the ordinary one, needs no privilege, and — critically — needs no feature gate. `zftp-core`'s bookmark importers live behind no gate at all, which is why Chapter 9 can say a `--no-default-features` build still gives you the bookmarks, the sessions, the queue, and the `.duck` importer with the transports stubbed. The importer survives the amputation of the network because it never depended on it.

That purity has a payoff in the testing story of Chapter 19 and in the fidelity argument of Chapter 25, and it is worth naming the difference between the two uses. A round-trip test — import a file, export it, compare — checks the engine against its own model. An import test against a file the *predecessor* produced checks the engine against the predecessor, which is the only automated check in the whole fleet that does not ultimately reduce to the author's reading of the original. Every other test asserts that the code does what the author believed the original did. An import of a real Alfred `info.plist` asserts that the code does what Alfred actually wrote. That is a stronger class of evidence, and it is available only where a predecessor emits files.

It also explains a scoring asymmetry in the coverage numbers. Behaviours that are only observable through a predecessor's live service — a server sync, an IMAP conversation — tend to stay `partial` in the matrix because there is no artifact to check against. Behaviours that the predecessor serializes to disk can be scored with more confidence, because the artifact is the specification. The import surface is therefore not just a migration path; it is the part of the port with the best evidence behind it.

Reading a format you did not design

There is one discipline the importers demand that the rest of the engine does not, and it is worth stating because it inverts a rule the fleet otherwise holds. Everywhere else, an engine's model is designed for its own domain: `zreq-core`'s `model` is the request/collection/environment shape the port wants, not the shape Postman happens to serialize. An importer, by definition, must implement somebody else's data model exactly, including its legacy fields and its version drift. `zgo-core`'s Script Filter parser (Chapter 8) auto-detects Alfred's JSON feedback versus the Alfred 2 legacy XML by first character, and carries both paths permanently, because both exist in the wild. That is not a design decision the port would make for itself; it is inherited liability accepted deliberately in exchange for the installed base.

The containment strategy is the same one the architecture uses everywhere: keep the foreign model at the boundary. The importer's job ends at producing the engine's own types, and nothing downstream of the importer knows which format the data arrived as. `import.bibtex` and `import.marcxml` converge on the same item; `bookmark.import` and `bookmark.import_filezilla` converge on the same bookmark. This is the same shape as the `Transport` trait converging sixteen protocols on one `fs.*` family — foreign variety absorbed at one seam so the interior stays uniform — and it means the cost of adding an eighth import format is one module, not a change to the model.

The one place the containment is imperfect is where a foreign format carries a concept the engine's model has no field for. The fleet's answer there is visible in `zgo-core`'s workflow graph: an `Object` carries an opaque `config: Value` rather than a typed per-object-kind struct, so an Alfred object type the port does not understand survives import, export, and the round trip without being decoded. Storing a foreign structure as an untyped value is a compromise — nothing downstream can reason about it — but it is the compromise that makes `workflow.import` followed by `workflow.export` non-lossy for object kinds the engine has never heard of. Fidelity to a format you do not control sometimes means declining to parse it.

What the import surface does not attempt

Four engines list no format importer at all: `zcontainer-core`, `zoffice-core`, `zpdf-core`, and `zphoto-core`. For three of them the reason is that the format *is* the domain — a PDF engine does not “import” a PDF, it opens one, and `open_pdf` is not an importer in any meaningful sense. The document engines consume their formats natively because parsing those containers directly is the entire product (Chapter 11), so the import/native distinction collapses.

`zcontainer-core` is the interesting exception, and it is the case that proves the chapter's thesis from the other side. Docker Desktop and Lens hold almost no user state worth migrating: your containers are the daemon's, your clusters are in a kubeconfig the operating system already owns, your Compose files are in your repositories. There is nothing to import, so there is no import surface, and the closest thing the engine has — `k8s.apply.yaml` — is a domain operation rather than a migration path. The engines with the broadest importers are the engines replacing products that hoard state, and the engine with none is the one replacing products that hoard nothing. The size of an app's import surface is a measurement of how much its predecessor was holding hostage.

Chapter 28 – Finding Things: Retrieval Models Under One Highlight

R7 says every filter in every app is fuzzy, matched by the one shared matcher, with matched characters highlighted in `<mark class="fzf-hl">`. Read quickly, that sounds like the fleet has one search. It does not. R7 is a constraint on *presentation* — one matcher, one highlight style — and underneath it sit several genuinely different retrieval architectures, chosen per domain, with different costs and different scalability ceilings. This chapter separates the two, because conflating them is how a reader ends up believing a design system solved a data-structures problem.

One highlight style is not one search

The distinction that matters is *where the candidate set comes from and where the matching happens*. `ZGui.fzf` is a JavaScript matcher operating on data the front end already has in hand. That is exactly right for a filter box over a table the shell just rendered: the rows are in the DOM, the query is a keystroke away, and round-tripping through the engine to filter a list you are already holding would be a boundary crossing for nothing. It is also, structurally, incapable of finding anything the front end has not already loaded.

That ceiling is invisible while a workspace is small and decisive when it is not. A filter over a modest request list is a client-side concern; a search over a mail store is not, because the shell cannot hold the corpus. So the fleet splits: everything that is “narrow a list I am already showing you” is `ZGui.fzf`, and everything that is “find things I have not loaded” is an engine command with its own index. The uniform highlight makes both look the same to the user, which is the point of R7 — and reading the command surface is the only way to tell which one you are actually using.

Where the search runs, per engine

Appendix A separates them cleanly. `zcite-core` exposes `search.quick` plus the `search.saved.list/add/remove/run` family, and carries `search` as a crate module alongside `identifier` — engine-side retrieval over a library the front end does not hold. `zemail-core` goes furthest, exposing both `gloda.search` and a second family `search.run/save/saved/run_saved`, backed by a `gloda` module of its own; it is the engine with the heaviest retrieval machinery in the fleet, which follows from having the largest corpus. `zgo-core` runs retrieval in the engine for a different reason: its candidates come from providers (`runningapps.search`, `bookmarks.search`, `contacts.search`, `spotlight.search`, `onepassword.search`, and `index.scan` with `index.search`) that only the Rust side can reach, so the matcher must live where the candidates do.

Three engines search *inside documents* rather than across a collection, which is a third model again: `zoffice-core`'s `writer.find` and `writer.replace`, its `calc.find` and `calc.replace`, and `zpdf-core`'s `find_replace_text`, `extract_text`, and `redact_search`. Here the corpus is one open document, the query is textual rather than fuzzy, and the operation frequently mutates as it searches — `redact_search` is a search whose output is an edit. Fuzzy subsequence matching would be actively wrong for these; a redaction that matched approximately would be a security defect.

And two engines whose domains obviously involve large collections expose no search command at all. `zftp-core`'s dispatch has no search family, and neither does `zcontainer-core`. In both cases the reason is that

the authoritative index is somewhere else — a remote server’s directory listing, a Kubernetes API server — and the honest architecture is to list and filter rather than to pretend to an index the engine does not own. When `zftp` needs content search over local files, the substrate provides it: `fs_grep` is a `zpw-r-file-browser` command, not a `zftp-core` one, which is the correct home for it by the Chapter 18 rule.

The two matchers, read as a boundary decision

Chapter 8 records that `zgo-core` has its own Rust matcher, `matcher::score`, distinct from `ZGui.fzf.fzfMatch`, and notes the two-matchers-two-jobs split. Read from this chapter’s angle it is a boundary decision rather than a duplication: the JavaScript matcher exists to highlight rows the shell is rendering, and the Rust matcher exists to rank candidates the shell will never see in full. A launcher’s provider set can propose far more candidates than it should ship across the boundary, so scoring must happen before the crossing, not after.

That is the general rule the fleet follows even where it is not stated: matching happens on the same side as the candidates. It also explains why the duplication is tolerable under a design system whose first principle is that divergence is a bug. The two matchers do not diverge on presentation — the highlight the user sees is `ZGui.fzf`’s in both cases, because `zgo`’s Rust score decides *order* and the front end decides *emphasis*. Ranking and highlighting are separable concerns, and separating them is what lets one highlight style survive two matchers. Chapter 8 also records that `zgo-core`’s matcher comments claim the *shape* of Alfred’s ranking rather than its constants; the shared JavaScript matcher makes no such claim about anything, because it is not ranking against a predecessor at all.

Saved searches: a query that outlives the session

Two engines let a corpus query become a durable object: `zcite-core`’s `search.saved.list/add/remove/run` and `zemail-core`’s `search.save`, `search.saved`, and `search.run_saved`. (`zgo-core`’s `search.add/search.remove` persist something different — a web-search keyword, the launcher’s ten-entry `BUILTINS` table extended by the user — not a query over a corpus the engine holds.) The shape is worth naming because it inverts the usual relationship between the store and the command surface — a saved search is a *query* persisted into the store and executed later, which means the store now holds something executable rather than something inert.

The consequences are the interesting part. A saved search must survive the data it queried: items get deleted, folders get renamed, and a stored query must degrade to an empty result rather than to an error, because a saved search that fails hard the first time a folder disappears is worse than useless. It must also survive the *engine*, in the sense of Chapter 17’s migration discipline: a query is a serialized structure like any other, so a query shape that gains a field must load an older saved query with `#[serde(default)]` the same way a model does. And it interacts with the two-phase deletion this book takes up in the next chapter — `zcite-core`’s `item.trash` is a state change rather than a removal, so a saved search must have a defined answer about whether trashed items match.

That these two engines and only these two grew saved searches is not arbitrary. They are the two engines whose corpora are large, long-lived, and accumulated rather than authored: a library and a mailbox. Where a user’s collection is small enough to see, a filter box suffices; where it is not, the query itself becomes an object worth keeping. Saved search is what a filter becomes when the corpus outgrows the window.

What R7 actually constrains

Pulling this together into the form a reader can use: R7 guarantees that every filter in every app is fuzzy and highlights identically, and it guarantees nothing about what is being filtered or where. That is a design-system requirement doing exactly what a design system should — making the interaction uniform — and it is not, and cannot be, an architectural guarantee about retrieval. The retrieval decision is per-engine and it is legible in

the command surface: an engine with a `search` module and a `search.*` family searches a corpus; an engine with neither is being filtered client-side; an engine with `find` and `replace` is searching one document; and an engine with no search at all is deferring to whoever owns the index.

The practical value of the split is that a reader can predict the ceiling. A surface filtered by `ZGui.fzf` behaves well up to what the shell can hold and then degrades in a way the design system will not warn you about, because the highlight looks identical at every scale. That is the cost of a uniform presentation over non-uniform mechanics: the UI stops being a reliable signal of what the machine is doing. The fleet's answer is that the command surface remains a reliable signal, which is one more argument for Appendix A being a real API document rather than a listing.

Chapter 29 – Time, Undo, and the Verb `clear`

An engine that changes state has to answer three questions the boundary contract does not cover: what does it remember, what can be taken back, and what can be forgotten on purpose. The fleet answers all three per-domain rather than uniformly, and the answers are legible in the command surface — which makes them auditable in a way an implementation-level undo stack would not be. This chapter reads those answers across ten engines and argues that the absence of a generic undo is the same decision as the absence of a generic cancel.

The fleet has no undo

Search the whole command surface and exactly one engine exposes undo: `zphoto-core`, whose `edit.undo`, `edit.redo`, and `edit.history` sit at the head of its dispatch. No other engine in the fleet has an undo command. `zreq-core` cannot un-send a request. `zftp-core` cannot un-transfer a file. `zcontainer-core` cannot un-delete a pod. `zoffice-core` cannot un-edit a cell, and `zpdf-core` — the engine with the largest operation catalogue in the fleet — cannot un-anything.

The first reading of that is that nine engines are missing a feature. The better reading is that undo is only definable where the engine owns a closed, in-memory document with a total ordering of edits, and `zphoto-core` is the only engine in that position. Its domain is a single image with a stack of non-destructive operations; every edit is a transformation of state the engine wholly controls, so an inverse is well-defined and the history is a data structure. Contrast every other engine: `zreq`'s “edit” escaped to a remote server, `zftp`'s wrote bytes on a host it does not own, `zcontainer`'s was executed by a daemon, and `ztunnel`'s changed the routing state of the operating system. Undo across a boundary the engine does not control is not undo; it is a second, possibly failing, forward operation with a hopeful name.

This is precisely the argument Appendix O makes about cancellation, arriving at the same conclusion from the other direction. There is no generic cancel because cancellation only means something against a domain entity, so the fleet models it as `transfer.cancel`, `unsubscribe`, `vpn.disconnect`. There is no generic undo because reversal only means something against a domain the engine owns, so the fleet models it as `edit.undo` in the one engine that owns one, and declines to fake it in the nine that do not. A uniform architecture is not one that gives every engine the same verbs; it is one that gives every engine the same *rule* for deciding which verbs it may honestly offer.

Three ways an engine remembers

Where the engines do model time, they do it in one of three shapes, and telling them apart matters because they have different retention, different failure modes, and different privacy exposure.

The first is the **append-only log**: a record of things that happened, kept for recall rather than for reversal. `zreq-core`'s `history.list`, `history.replay`, and `history.clear` is the clearest case — a request history the user browses and can re-send, where `history.replay` is explicitly a *new* request built from a past entry, not a rewind. `zgo-core` keeps three logs of this kind at once: `query.record`, `query.recent`, and `query.list` for what was typed; `clipboard.list`, `clipboard.add`, and `clipboard.search` for what was copied; and the frequency table behind `learn.record` and `learn.rank` for what was chosen. These logs

are not undo infrastructure; they are the raw material for features that would be impossible without memory — replay, clipboard history, and a launcher that ranks for you rather than for everyone.

The second is the **reversible edit stack**, which is `zphoto-core` alone, discussed above, and which is the only structure in the fleet where the engine holds enough to invert an operation.

The third is **two-phase deletion**, and `zcite-core` is its exemplar: `item.trash`, `item.restore`, and `item.delete` are three distinct commands, where the first is a reversible state change and the third is the irreversible one. This is not undo — a trashed item is not an undone edit, it is an item in a different state — but it delivers the property users actually want from undo at the granularity that matters for a reference library, and it does so with no history structure at all. It is worth noticing how cheap it is compared with the alternatives: a flag on the model, three commands, and no stack to bound, persist, or migrate.

Those three shapes account for every temporal structure in the fleet. Notably, none of them is shared code. There is no history crate, no undo trait, no common log type — three engines solved memory three ways and the architecture did not object, because the shapes have nothing in common but the word.

clear as a retention surface

The most under-read verb in Appendix A is `clear`, and collecting its occurrences turns it into a sub-system. In the fuller dispatch lists it appears as `history.clear` and `cookie.clear` in `zreq-core`; `creds.clear` and `vpn.clear_logs` in `ztunnel-core`; `clipboard.clear` and `query.clear` in `zgo-core`; and `creds.clear`, `session.clear_logs`, and `transfer.clear` in `zftp-core`. Four engines, and every one of those commands drops accumulated state in bulk.

The pattern underneath is sharp: `clear` exists exactly where an engine accumulates state the user did not deliberately author. Nobody creates a request history — it happens. Nobody authors a clipboard log, a query log, a cookie jar, a completed-transfer list, or a connection log; they are byproducts of use. The engines therefore pair every ambient accumulator with a command to drop it, and — this is the part worth stating plainly — they do *not* offer `clear` for authored state. There is no `collection.clear` in `zreq-core`, no `bookmark.clear` in `zftp-core`, no `library.clear` in `zcite-core`. Bulk-forget is offered for what the engine collected on its own and withheld for what the user made.

Read that way, the `clear` family is the fleet's retention and privacy surface, and it is the one place where a user can act on the byproducts of use as a class. It is also the surface most likely to be incomplete, because it grows one command at a time as each accumulator is added, and nothing in the architecture enforces the pairing. The discipline is a convention visible in the dispatch list rather than a rule enforced by a type, which means the audit is exactly the one performed above: enumerate what an engine accumulates, enumerate what it can drop, and compare the two lists.

The sub-family `clear_logs` — `vpn.clear_logs` and `session.clear_logs` — is a special case worth separating, because those two commands drop a *diagnostic* buffer rather than a user-facing one, and they belong to the two engines the chapter after next identifies as structural twins.

Irreversibility as a feature

The fleet's most interesting position on reversal is where it makes irreversibility explicit. `zpdf-core`'s catalogue contains a set of operations whose entire purpose is that they cannot be undone: `apply_redactions`, `flatten_annotations`, `flatten_form`, `flatten_layers`, and `sanitize`. Each takes a document with recoverable structure — content under a redaction rectangle, annotations layered over a page, form fields with values, optional-content layers, metadata — and produces one where that structure is gone rather than hidden.

This is the correct behaviour and it is the reason the two-command split exists at all. `redact` marks; `apply_redactions` destroys. A redaction that could be undone by the recipient is not a redaction, and the entire value of the operation is the irreversibility. The naming is unusually careful about it: each `flatten_*` says what happens to the structure, and `apply_redactions` is a separate verb from `redact` precisely so that

marking and destroying cannot be confused. In an engine with no undo anywhere, the operations that most need to be irreversible are the ones that say so in their names.

The general principle the fleet is following is that reversibility is a property of the domain, not a service the runtime provides. Where reversal is well-defined the engine offers it as a domain verb (`edit.undo`, `item.restore`). Where it is not, the engine offers no verb and no illusion. And where irreversibility is the *point*, the engine names it. What the fleet never does is offer a generic undo that works for the easy cases and silently fails for the rest — which is the shape that would have made an architecture-level undo possible, and would have been a lie in nine engines out of ten.

Chapter 30 — The Engine as Its Own Diagnostic

A `-core` crate is, by construction, a thing you cannot look at. It has no window, it holds its state behind a process-global mutex, it may be running as a guest inside another app's process, and its whole observable surface is one string-dispatched `invoke` plus an event sink. When something is wrong, the ordinary desktop debugging gestures — read the UI, click around, attach to the process — are either unavailable or answer questions about the shell rather than the engine. This chapter is about what the fleet does instead: a small set of commands by which an engine describes itself, two engines that keep an internal log, and three shell commands that put the artifacts in front of the user.

An engine you cannot attach a debugger to

The debugging problem an embedded engine creates is not difficulty; it is ambiguity. When `zcite` shows a cited paper's PDF through an embedded `zpdf-core` and the page renders wrong, the fault lies in one of at least four places: the host shell's mount, the embed contract, the engine's own logic, or the pointer the host has pinned that engine at. The first question is not “what went wrong” but “which of the components in this process is even running.” That question is answerable only if the engine can be asked directly.

The fleet's answer is that diagnosis goes through the same door as everything else. There is no side channel, no debug build with extra exports, no diagnostic FFI. What exists is a handful of ordinary commands in the ordinary dispatch, which means every diagnostic is available through all three doors — native, C ABI, and Tauri — and works identically when the engine is a guest. A diagnostic that only worked in the standalone app would be useless for exactly the case that needs it most.

The introspection commands, counted

Reading Appendix A's fuller dispatch for commands that describe the engine rather than operate on the domain yields a small and uneven set.

A bare `version` command is listed for four engines: `zreq-core`, `ztunnel-core`, `zgo-core`, and `zftp-core`. Given the version skew of Appendix D — where four apps run ahead of their cores — `version` answers the single most useful question about a suspected pointer problem, which is which engine build is actually loaded. That the remaining engines do not list one is the clearest gap in the fleet's diagnostic surface, and it is a gap that costs most in the embedded case, where the host's version tells you nothing about the guest's.

Three engines list `profile.get`: `ztunnel-core`, `zgo-core`, and `zftp-core` (the last with `profile.recent` alongside). Three list a settings family: `zreq-core` with `settings.get/update/reset/path`, and `ztunnel-core` and `zftp-core` with `settings.get/set`. `zreq-core`'s `settings.path` is the odd one out and the most diagnostically valuable of the group, because it makes the engine report *where its bytes are* — the one fact about Chapter 17's data directory that a user cannot otherwise discover, and the fact that immediately explains a class of “my data disappeared” confusions caused by an engine resolving a different bundle id than expected.

Two commands describe capability rather than state. `ztunnel-core`'s `feature.matrix` reports which of its backends are available in this build — the runtime read of the compile-time gates of Appendix J, which is the only

way a host can discover whether the `wireguard` feature was compiled in. `zftp-core`'s `transport.info` plays the same role for its protocol matrix, and pairs with the `is_wired` seam of Chapter 9: a host can ask which transports exist before offering a user a protocol that will answer `needs_transport`. `ztunnel-core`'s `platform.info` completes the set by reporting the platform-specific facts its `platform` module resolved.

The shape worth extracting is that all of these are *capability* queries rather than health checks. Nothing in the fleet's command surface is a liveness probe or a self-test for the engine itself. The two ping-shaped commands that exist — `zcontainer-core`'s `docker.ping` and `ztunnel-core`'s `servers.ping` — probe a daemon and a remote host respectively, which is to say they report on the environment, not on the engine asking. The engines describe what they can do and where they keep things; they do not assert that they are well.

Two engines keep a log; the rest do not

The most striking asymmetry in the diagnostic surface is that exactly two of the ten cores carry a `log` module: `ztunnel-core` and `zftp-core` (Appendix F). Those same two are the only engines with a `Log` event variant in Appendix H — `Log { config_id, level, message }` and `Log { bookmark_id, level, message }` — and the only two with a log-clearing command, `vpn.clear_logs` and `session.clear_logs`, and the only two with a log-reading command, `vpn.logs` and `session.logs`. Four independent pieces of the surface agree, which is not a coincidence: the two engines that keep a log are the two engines that supervise something they did not write.

That is the real criterion. `ztunnel-core` supervises an OpenVPN process line by line and a WireGuard tunnel; `zftp-core` supervises remote sessions over its protocol matrix. In both cases the interesting failures happen in a component the engine drives but does not implement, at a time the caller is not waiting, and the only faithful representation of “the far side said this” is a timestamped line. Every other engine either computes its result synchronously — where a failure is a return value, not a log line — or, in `zcontainer-core`'s case, hands the subordinate system's output to the caller as a stream rather than retaining it.

This also resolves an inconsistency worth stating directly, because the appendices disagreed about it. Appendix H's catalogue shows the `Error { tag, message }` event variant in `zreq-core`, `zcite-core`, `zgo-core`, `zemail-core`, and `zcontainer-core` — five of the seven event-bearing engines. The two without it are `ztunnel-core` and `zftp-core`, which is exactly the pair that has a `log` module. They do not lack error reporting; they route it through `Log { level, message }` instead, because for a supervising engine an error is one line among many at a severity, not a distinct kind of event. The two designs are coherent with their domains, and the honest count is five of seven.

The three commands the shell adds

The engines produce diagnostic material; the shell is what puts it in front of a human. Appendix D lists three app-side commands for this: `open_log_file`, `open_log_dir`, and `open_data_dir`. They are trivial — each reveals a path in the platform's file manager or editor — and they are the correct division of labour by the Chapter 1 split. Knowing where the data directory is, is the shell's job, because the shell resolved it. Opening a directory in the platform file manager is a platform gesture, which is the shell's job by definition. And the engine, which knows what it wrote but not where or how to show it, does neither.

The pairing with `settings.path` is the part worth noticing. The engine can *report* its path through its own dispatch; the shell can *open* it. Neither alone gives a user a route from “something is wrong” to “here are the bytes,” and together they do, without either component acquiring a capability that belongs to the other. It is a small instance of the whole architecture, and it is the least glamorous possible example of why the split earns its keep.

What the diagnostic surface does not have

The absences are as informative as the presences, and this chapter would be dishonest without them. There is no structured tracing subsystem: no span, no level filter, no runtime-configurable verbosity in the command surface. There is no metrics surface — `ztunnel-core` and `zftp-core` emit `Stats` events with byte counters, but those are domain telemetry for a UI, not engine instrumentation. There is no health command, no `version` on most of the fleet, and no command anywhere that reports which feature gates a build was compiled with except the two capability queries described above.

Each absence is defensible in isolation and the pattern is consistent with the durable-deps rule — a tracing framework is a dependency, a metrics exporter is a dependency, and neither is load-bearing for a desktop engine. But the honest summary is that the fleet's diagnostic story is thinner than its failure story (Chapter 16) or its persistence story (Chapter 17), and it is thinnest in exactly the place the architecture makes hardest: when an engine is a guest, there is no uniform way to ask it what it is. The obvious first closure is the cheap one, since `version` already exists in four engines and adding it to the others is a match arm each.

Chapter 31 – The Session Engine: An Unnamed Archetype

The book has so far described the fleet as ten engines instantiating one pattern, with each engine's internals shaped by its own domain. That is true, but it undersells a structure sitting in plain sight in Appendix F: two of the ten cores did not merely instantiate the pattern, they converged on the same skeleton, module for module, without any shared crate between them. `ztunnel-core` and `zftp-core` are structural twins. This chapter reads the resemblance, argues that it constitutes an archetype the fleet discovered rather than designed, and explains why it was correctly left un-factored.

Two crates that grew the same skeleton

Take Appendix F's module lists for the two crates and intersect them. `ztunnel-core` carries `config`, `creds`, `killswitch`, `log`, `manager`, `model`, `netmonitor`, `openvpn`, `platform`, `privileged`, `servers`, `stats`, `store`, `supervisor`, `wireguard`, and `wireguard_tunnel`. `zftp-core` carries `bookmark`, `creds`, `discovery`, `encoding`, `log`, `manager`, `model`, `protocol`, `stats`, `store`, `transfer`, and `transport`.

Six module names appear in both: `creds`, `log`, `manager`, `model`, `stats`, and `store`. Two of those — `model` and `store` — are fleet-wide commons and prove nothing. The interesting fact is the other four. `creds` appears in exactly these two crates and nowhere else in the fleet. So does `log`. So does `manager`. So does `stats`. Four module names, each unique to the intersection of two crates, in a tree of ten engines with no shared code between the two.

Two crates arriving independently at four identically-named modules is not naming coincidence; it is the same problem decomposed the same way twice. The problem is: hold a set of user-configured remote endpoints, authenticate to them with secrets that must not live in the store, establish and supervise long-lived connections to them, report throughput while they run, and keep a line-oriented record of what the far side said. Call that a session engine. `ztunnel-core` and `zftp-core` are the fleet's two instances, and everything else about them — WireGuard versus SFTP, tunnels versus transfers — is domain dressing over the same skeleton.

The event enums agree too

The resemblance is not confined to module names, which would be weak evidence on its own. Appendix H's event catalogue shows `ztunnel-core` emitting `Status { config_id, status }`, `Stats { config_id, bytes_in, bytes_out }`, and `Log { config_id, level, message }`, and `zftp-core` emitting `Status { bookmark_id, status }`, `Stats { bookmark_id, bytes_in, bytes_out }`, `Log { bookmark_id, level, message }`, and `Transfer { transfer_id, status, transferred_bytes, total_bytes }`.

Three of `zftp-core`'s four variants are `ztunnel-core`'s three with one field renamed — `config_id` becomes `bookmark_id` — and the field lists are otherwise identical, down to `bytes_in` and `bytes_out` on `Stats` and `level` and `message` on `Log`. These are the only two engines in the fleet with a `Status` variant, the only two with a `Stats` variant, and, as the previous chapter established, the only two with `Log`. They are also the only two event enums in the fleet with *no* `Error` variant, for the reason given there.

The command surface agrees a third time. Both engines expose `version`, `profile.get`, and `settings.get/set`; both expose `creds.set/clear`; both expose a connection lifecycle keyed on an entity id

(`vpn.connect/disconnect/status` against a config, `session.connect/disconnect/status` against a bookmark); both expose a logs pair (`vpn.logs` with `vpn.clear_logs`, `session.logs` with `session.clear_logs`). Three independent surfaces — module names, event variants, command families — agreeing on the same shape is about as much evidence of a real archetype as a source tree can offer.

Where the twins diverge, and why

The divergences are as instructive as the agreements, because they are all attributable to one difference in the domains: what the engine must obtain in order to act.

`ztunnel-core` needs *privilege*, and it grows modules the other does not: `privileged`, `killswitch`, and `platform` (with its `linux` and `macos` submodules), plus `supervisor` for driving OpenVPN as a child process. The `needs_privilege` status and the plan-as-data discipline of Chapter 7 all descend from that one requirement. `zftp-core` needs *reach*, and it grows the modules that correspond: `transport` and `protocol` for the protocol matrix, `encoding` for filename charsets, `discovery` for finding hosts on the local network, and `transfer` for a queue, since a transfer client's unit of work outlives the call that created it.

There is a pleasing symmetry in how the two honesty seams line up. `ztunnel-core` returns `needs_privilege` when the plan is computable but root is not held; `zftp-core` returns `needs_transport` when the plan is computable but the backend is not wired. Both are the typed non-errors of Appendix L, and both say the same thing in the same shape: the engine's half is done, the environment's half is not. The archetype has one failure mode by construction — the environment refused — and both instances name it rather than pretending.

The keyring version split

One divergence is not domain-driven and is worth flagging as the kind of drift a fleet accumulates. Appendix J records `ztunnel-core`'s `keychain` feature pulling `keyring` at version 4, and `zftp-core`'s `keychain` feature pulling `keyring` at version 3. Two crates, same feature name, same purpose, same architectural role in the twins' skeleton — and different major versions of the same dependency.

That is the durable-deps rule's blind spot stated concretely. The rule constrains *which* crates a core may depend on, and it says nothing about keeping versions aligned across cores, because each core is an independent crate with its own manifest and each was brought up at a different time. In the standalone apps this costs nothing: `ztunnel` links one, `zftp` links the other, neither knows. It becomes visible only in the composition case the whole architecture exists to enable — a host embedding both engines links two major versions of `keyring` into one binary. Cargo permits this and the build succeeds, so the cost is binary size and the mild hazard of two versions of a crate that talks to a single operating-system keychain, rather than a compile error that would force the question.

The general lesson is that the fleet's strongest property — engines compose — is exactly the property that turns per-crate dependency drift from invisible into structural. Nothing in the architecture surfaces it: there is no fleet-wide lockfile, and Appendix J is the ledger only because someone wrote it down.

Why the archetype was never factored out

The obvious suggestion, having found a repeated skeleton, is to extract it — a shared session-engine crate with the connection lifecycle, the credential seam, the log ring, and the stats counters, which both cores would consume. The fleet did not do this, and the reasoning available in the book's own principles says it was right not to.

Chapter 18 gives the criterion for factoring something out: extract what changes at the same rate and is reused by many. The session skeleton fails the second test outright — a population of two is not “many,” and the extraction would create a shared dependency at the bottom of two engines with all the calcification cost

Chapter 18 describes for the substrate crates, in exchange for deduplicating a modest amount of straightforward code. It probably fails the first test too: `ztunnel-core`'s lifecycle is converging on privilege escalation and platform-specific tunnel setup while `zftp-core`'s is converging on transport breadth and queue semantics. They started at the same place and are moving apart.

There is also a subtler cost. The twins' resemblance is currently a fact about the tree that a reader can verify and a maintainer can ignore. Extracting it would make the resemblance *load-bearing*: `zftp-core` would then be unable to change its `Status` shape without considering `ztunnel-core`, and the archetype would stop being a discovered pattern and start being a constraint. The fleet's position — one shared toolkit and one shared substrate, but no shared engine internals — keeps the ten cores independently evolvable, and paying a small duplication cost to preserve that is consistent with every other trade in this book.

What the archetype earns instead is predictive value. When the eleventh engine turns out to be a session engine — an SSH client, a database client, a message-queue console — its author already knows the shape it will grow: `model`, `store`, `creds` routed to the keychain, `manager` for the lifecycle, `log` for what the far side said, `stats` for throughput, a `Status/Stats/Log` event trio keyed on the entity id, and a typed non-error for the environmental precondition it cannot satisfy. Chapter 26 gives the order of operations for standing up any engine. For this class of engine, the archetype gives the module list too.

Chapter 32 – Feature Gates and the Dependency Ledger

Appendix J is the driest page in the book and it encodes more design decisions per line than any other. Every core's `[features]` table is a statement about what that engine is without its environment: which dependencies are essential to the domain and which are essential only to reaching the outside world. This chapter reads the gate tables as a design surface rather than as build configuration — what kinds of gate exist, what they imply about each other, what they cost an embedder, and what the two crates with no meaningful gates reveal about the rule.

Two kinds of gate

The gates fall into two categories that Cargo does not distinguish and the architecture very much does.

Most are **dependency gates**: turning the feature on adds crates. `zreq-core`'s `net` brings `reqwest`; its `grpc` brings `proton`, `prost`, `prost-types`, and `prost-reflect`. `zcore-core`'s `pdf` brings `lpdf`. `zftp-core`'s `net` brings `openssl`, `tokio`, `russh`, `russh-sftp`, and `encoding_rs`; its `discovery` brings `mdns-sd`. `zcontainer-core`'s `docker` brings `bollard` and `reqwest`, its `kube` brings `kube`, `k8s-openapi`, `base64`, `flate2`, and `serde_yaml`, and its `vm` brings the macOS `objc2/objc2-virtualization/block2/dispatch2` stack. For these, the gate's purpose is compile cost and dependency surface: a consumer that does not want Kubernetes does not compile `k8s-openapi`.

A minority are **marker gates**, which add no dependencies at all and exist purely to switch off a *capability* in code. Appendix J names two: `ztunnel-core`'s `net` is a marker, and `zgo-core`'s `exec` is a marker that gates process spawning. `zgo-core` is the sharper case, because it has no network dependencies whatever — its entire feature story is one decision about whether the engine may spawn a process. Turn `exec` off and, as Chapter 8 puts it, `zgo-core` becomes a feedback-only library that can parse and render Script Filter output without the ability to spawn a process.

The distinction matters because the two kinds answer different questions. A dependency gate answers “what do I have to compile?” A marker gate answers “what is this library permitted to do?” — which is a security property, and one the embedding story needs. `zgo-core` without `exec` is an engine a host can embed and reason about: it parses, matches, ranks, and renders, and it cannot execute anything. That guarantee is enforced by the compiler rather than by review, which is a materially stronger thing than a promise in a README, and it is available because someone chose to spend a feature flag on a capability rather than on a dependency.

Implication edges

Two gates in the fleet imply another, and both are recorded in Appendix J: `zreq-core`'s `grpc` implies `net`, and `ztunnel-core`'s `wireguard` implies `net` while also bringing `boringun`, `boringun/device`, and `libc`. The implications are correct — gRPC without a wire is meaningless, and a WireGuard data path is a network path — and they are the only two in the fleet.

The reason so few exist is worth extracting, because it is the same design principle the book has been describing in other registers. Gates are drawn around *capabilities the engine can be honest without*, and capabilities that are honest in isolation do not need each other. `zcore-core`'s `pdf` and `net` are independent

because extracting text from a local PDF and looking up a DOI are unrelated; `zcontainer-core`'s `docker` and `kube` are independent because they are two API clients that share nothing but the engine around them, which is why Appendix K can say a consumer pays for one client rather than both. Where implication does appear, it is because one feature is genuinely a specialization of another, not because the code got tangled. Two implication edges across ten crates is a measure of how cleanly the capability boundaries were drawn.

The two crates with no meaningful gates

Two cores stand outside the gate discipline, and each is instructive.

`zoffice-core` has no `[features]` table at all: every dependency is mandatory, and Appendix J notes there is not even a `tauri` gate in the core. That is defensible on the domain — `zip`, `quick-xml`, `flate2`, and the RustCrypto stack are not optional for a crate whose entire job is opening OOXML and ODF containers, and there is no meaningful office engine without the ability to unzip. It is also the crate the appendices flag as different in two other ways: Appendix D records it as the one engine on the 2024 edition, and Appendix B records it with the smallest C ABI of any engine that has one — two functions, with no `init`, because the engine is a stateless unit struct. `zoffice-core` is the fleet's outlier by three independent measures, and all three have the same root — it is a pure transformation library with no session, no state, and no optional environment.

`zpdf-core` gates only `tauri`; every other dependency is mandatory. That is the more consequential case, because Appendix I calls `zpdf-core` the fleet's most-embedded engine. The engine with the widest consumer set is the engine with the least configurable dependency surface: every host that embeds PDF rendering compiles the whole thing, including `boa_engine`, `lopdf`, `ttf-parser`, `subsetter`, and `tiny-skia`. There is no `--no-default-features` build of the PDF engine that gives you text extraction without a JavaScript interpreter and a rasterizer.

Whether that is a defect depends on what the alternative would cost. The counter-argument the crate's shape makes for itself is that PDF is not a layered format — its scripting, its fonts, and its rasterization are all inside the specification rather than beside it, so a PDF engine without JavaScript would be an engine that silently mis-renders documents whose behaviour depends on it. That is exactly the failure mode this fleet refuses elsewhere: a partial capability that does not announce itself. Better to compile the whole format than to ship an engine that opens a file and quietly does the wrong thing.

The JavaScript engine the fleet refused, and the one it accepted

Those facts sit next to another that looks like a contradiction until it is read carefully. Appendix K records that `zreq` scores its coverage against Postman with two behaviours marked missing, one of them a full JavaScript test sandbox, and Chapter 25 gives the reason: embedding a heavyweight JS engine would violate the durable-deps rule. Meanwhile Appendix I records `zpdf-core` depending on `boa_engine` for PDF JavaScript, ungated, in the most-embedded engine in the fleet.

The fleet refused a JavaScript engine in one crate and accepted one in another. The distinction is not about the dependency; it is about who the requirement belongs to. In `zreq`, a scripting sandbox is a *convenience the application would offer*: valuable to users who script their tests, entirely absent from the definition of “an HTTP request was sent correctly,” and therefore a dependency the port is choosing to take on. In `zpdf-core`, document-level JavaScript is *part of the format being implemented* — a PDF with a `set_doc_js` action or a form field with `set_field_calculation_js` has behaviour the specification defines in terms of JavaScript, and an engine that ignores it is not a lighter PDF engine, it is an incorrect one.

So the durable-deps rule, read precisely, is not “few dependencies.” It is: take dependencies the domain requires, and decline dependencies the feature list merely wants. `boa_engine` is load-bearing for format fidelity; a test sandbox is load-bearing for a convenience. The two decisions that look opposite are the same

decision applied to different questions, and the reason the fleet can be legible about it is that both outcomes are written down — one in a coverage matrix as a named missing behaviour, one in a dependency list — rather than left to inference.

What a gate costs the embedder

The gate tables are usually read from the engine's side, as compile-time configuration. Read from the *embedder's* side, they are something less comfortable: a set of build decisions a host has to make correctly on behalf of a guest engine it did not write.

Consider a host embedding `zcite-core` for citation formatting. It must decide whether to enable `pdf`, which determines whether `pdf.extract_text` works at all; and whether to enable `net`, which determines whether `identifier.lookup` can resolve a DOI. Neither decision is visible in the command surface — the dispatch list is identical either way — so a host that guesses wrong ships an app whose commands exist and fail. That is precisely the situation `ztunnel-core's feature.matrix` and `zftp-core's transport.info` were introduced to fix (Chapter 30): a runtime query that reports which compile-time gates were taken, so a shell can present a capability honestly instead of offering a button that cannot work. Two engines have such a query; the rest leave the host to know what it compiled.

There is a second, quieter cost. Because the dispatch surface is a string match rather than a typed API, a gated-off capability does not disappear from the engine's vocabulary; it becomes a command that returns an error the host must interpret. The alternative — removing the match arms under a `cfg` so the command is unrecognized — trades one confusing failure for another. Neither is wrong, and the honest statement is that the fleet's uniform string dispatch, which buys so much elsewhere (Chapter 4), gives up the one thing a typed API would have provided here: the compiler telling a host at build time that the capability it is calling was not compiled in.

The combinatorial ceiling

The last thing the gate tables imply is a testing burden nobody can pay in full. A crate with four independent features has sixteen build configurations; the fleet across ten crates has more configurations than a solo author will ever exercise. Chapter 19 describes the mitigation — put the bulk of the tests in the ungated core, test gated parts with their gate on — and that is the right allocation, but it is worth naming what it does not cover: interactions between two gates that are each individually tested, and the compile-time correctness of configurations nobody builds.

The structural mitigation is stronger than the testing one. The fleet's gates are drawn so that features are, with the two documented exceptions, independent — and independent features do not have interactions to test. The combinatorial explosion is a problem for crates whose features modify shared code paths; it is nearly a non-problem for crates whose features add self-contained modules behind a `cfg`. `zcontainer-core's docker` and `kube` can be tested separately with real confidence precisely because they share nothing but the engine's dispatch. Drawing gates around whole capabilities rather than around implementation details is what turns an untestable configuration matrix into a set of independent switches — which is the same argument as the module-level one, arriving in the build system.

Chapter 33 – The Palette as the Second Command Surface

R1 requires every app to ship a fuzzy command palette on Cmd/Ctrl+K, backed by the shared `command-palette.js`, and adds a clause that is easy to read past: every button and shortcut must *also* be palette-reachable. Chapter 12 treats the palette as a component in the design system, which is where it lives. This chapter treats it as something else — a second, human-facing projection of the same capability set the engine exposes as dotted command strings — and examines what it means that the fleet maintains two vocabularies for one set of capabilities with no mechanical link between them.

Two vocabularies for one set of capabilities

An engine's capabilities are enumerated once, canonically, in its `Engine::invoke` match: `request.send`, `transfer.cancel`, `vpn.connect`, `apply_redactions`. That vocabulary is designed for a machine caller — namespaced, terse, stable, greppable, and identical across the native, C, and Tauri doors.

The palette enumerates the same capabilities for a human: labelled, described, ordered by relevance, fuzzy-matched, and scoped to what is actionable right now. Appendix C's signature shows the shape the shell passes in — `palette: [{ label, hint, run }]` — where `run` is the closure that ultimately reaches the engine. Two lists, one underlying set, and the relationship between them is that a palette entry's `run` calls one or more engine commands.

Those two vocabularies are not in one-to-one correspondence and should not be. Many engine commands are not palette-worthy: `vars.resolve` is machinery the shell calls while rendering, not an action a user invokes. Some palette entries are not single commands: “duplicate this configuration and connect” is one human action over `config.duplicate` and `vpn.connect`. And some palette entries touch no engine at all — open the terminal, switch colorscheme, reveal the data directory. The palette is a projection with its own editorial judgment, which is exactly what makes it useful and exactly what makes it drift.

No engine can enumerate itself

Here is the mechanical fact underneath the drift: nothing in the fleet's command surface lets a caller ask an engine what commands it has. There is no command-listing command anywhere in Appendix A. `ztunnel-core`'s `feature.matrix` reports capabilities at the granularity of backends, not commands. `zgo-core`'s `actions.list` and `system.list` enumerate domain objects — workflow actions, system commands — not the engine's own vocabulary.

This is a direct consequence of the dotted-string dispatch that Chapter 4 defends. A `match` over string literals is not data; the compiler consumes it and nothing retains the list. A typed command enum, or a registry built from a macro, would have made the vocabulary introspectable at the cost of the properties Chapter 4 argues for. The fleet made that trade deliberately and the palette is where the bill arrives: the one place that most wants a list of every capability is the one place that cannot ask for it.

So every palette is hand-maintained. A capability added to an engine appears in the palette only when someone adds a corresponding entry, and there is no build step that fails when they do not. R1's completeness clause — every button and shortcut must be palette-reachable — is a requirement stated in a specification and enforced

by review, in a fleet that otherwise prefers enforcement to convention, with Chapter 14's **FORBIDDEN** list being the strongest counter-example: there the toolkit replaces global-seizing methods with throwing stubs rather than documenting that they should not be called.

What drift looks like

The failure mode is specific and worth describing, because it is invisible in the way that matters. A palette that has fallen behind its engine does not error; it simply does not offer something. The user's experience is not "this is broken" but "this app cannot do that," which is the worst possible failure for a feature that exists and is reachable through a menu the user did not think to open. The palette is the discovery surface, so drift in the palette is indistinguishable, from the outside, from an unimplemented feature.

The exposure scales with the size of the vocabulary, which is why it is worth raising for this fleet in particular. Appendix G describes **zpdf-core** as routing roughly three hundred operations through one match, and Appendix A puts **zphoto-core**'s filter family alone at around a hundred and sixty commands. No palette lists those in full, and no reasonable palette would — but "no palette lists them in full" and "the palette is missing the most recently added ones" are the same observation from the outside, and only the second is a defect. Without introspection there is no way to tell them apart mechanically.

R1's clause is written with the right instinct — it makes reachability a requirement rather than a nicety — but it is a requirement about a set nobody can enumerate on either side. Chapter 12 is candid that palette conformance is one of the places the roster is still converging, and this is the structural reason it is hard to finish: conformance to R1 is not checkable by a script the way R8's requirement that tables come from **ZGui.table** is checkable by grepping for a hand-rolled table.

The keyboard as the fleet's actual interface

The strategic reason to care is that the palette is not an accessory to the UI in these apps; for a certain kind of user it is the UI. The fleet's predecessors are largely keyboard-first tools — Alfred is a keyboard launcher by definition, Postman power users live in a request list they filter rather than browse, and a terminal-adjacent transfer client is used by people who type. R1 through R10 encode that: a palette on **Cmd/Ctrl+K**, a terminal on **Ctrl+backtick** and **Cmd+T**, fuzzy filters everywhere, keyboard nav in the file browser.

Which means the palette's completeness is not a polish issue; it is the interface's completeness. A capability that is not in the palette is, for a keyboard-first user, effectively unimplemented, however complete the engine's match arm is. This is the sharpest version of the argument the book makes elsewhere about the shell being thin but not unimportant: the engine can be perfect and the app can still be missing the feature, because the feature's existence and the feature's reachability are maintained in two different places by two different mechanisms with nothing tying them together.

The introspection command the fleet does not have

There is an obvious closure available, and it is worth stating precisely because the book's own honesty compact prefers naming a gap to leaving it implied. If each engine exposed one additional command — a self-description returning its command names, with a label and a hint per command — then the palette could be generated from the engine rather than maintained beside it, R1's completeness clause would become mechanically checkable, and the drift described above would become impossible rather than merely discouraged.

The cost is real and it is the reason this does not exist yet. The dispatch match would have to become data, or be accompanied by a table that itself can drift from the match — which trades an unenforced convention between the shell and the engine for an unenforced convention inside the engine, unless a macro generates both from one source. That is a meaningful change to the shape Chapter 4 defends, and it would land in ten crates. It is also the kind of change the fleet's own principles favour: the architecture consistently prefers

enforcement over convention, and this is the largest remaining place where a convention is doing a job a mechanism could do. Chapter 30 identifies the missing `version` command as the cheapest closure in the diagnostic surface; this is the most valuable one, and the most expensive.

Chapter 34 – The Process Model: One Instance, Two Windows, One Engine

Chapter 17 ends on a question it deliberately declines to answer. Having established that a whole-file store is last-writer-wins and that the bundle-id convention keeps different apps out of each other's directories, it observes that nothing in the persistence design prevents two instances of the *same* app from resolving the same directory, and it hands the problem off: "what prevents it is a shell-level concern about how many instances of an app may run, which lives on the shell side of Chapter 1's split." That hand-off is correct — process and window lifetime genuinely belong to the shell — but the book then never picks the question back up, and it is not a small one. Almost every property the earlier chapters establish is a property *per process*, and the number of processes is exactly the variable nobody has pinned down.

This chapter pins it down as far as the architecture allows, which turns out to be further than expected in one direction and not at all in the other.

Three singletons that are not the same singleton

The fleet contains at least three things called "single instance," and they are scoped differently. Confusing them is how a multi-window bug gets designed in.

The first is the engine. Appendix B's second uniform rule states it plainly: "the engine is a single process-global instance behind a `OnceLock<Mutex<Engine>>`, so there are no handles to track and calls are serialized by the mutex." The scope of that singleton is the *operating system process*. Every call from every source inside that process — the Tauri command, a native Rust caller, a C host through `<prefix>_invoke` — reaches the same `Engine` value and queues on the same lock.

The second is `appShell`. Appendix C records it as "single-instance, guarded by `data-zgui-shell-mounted`," and Chapter 12 gives the throw message: "appShell already mounted — only the top-level app mounts the shell; an embedded core must use buttons + scoped APIs." The scope of *that* singleton is a `document`. The guard is a dataset flag on `document.documentElement`, so it is one mount per document, and two documents in the same process each satisfy it independently.

The third is the app itself, and it has no enforcement mechanism recorded anywhere in the book. There is no lock file in the data directory, no named-mutex acquisition, and no single-instance plugin in the registered command surface of Appendix D. The engine's singleton is enforced by the type system; the shell's singleton is enforced by a DOM flag that throws; the *application's* singleton is enforced by whatever the platform does when a bundle is launched twice, which is not something the architecture has written down as a guarantee it depends on.

Line those three up and the interesting case falls out immediately: **two windows in one process share an engine; two processes do not**. Everything else in this chapter is a consequence of that sentence.

The two-window case, which mostly works

Suppose an app opens a second `WebView` in the same process — a separate Monaco window for an editor, which Chapter 4 already names as a real pattern, or a detached panel, or a document window in one of the document-suite apps. Each window is its own document, so each may mount its own `appShell` without

tripping the guard. Both windows' `invoke` calls land on the same registered Tauri command, which reaches the same `OnceLock<Mutex<Engine>>`.

What that buys is significant and mostly free. There is exactly one model, so two windows cannot disagree about it. A mutation made in one window is visible to the next read from the other, with no synchronization code, no cache invalidation, and no cross-window message passing — the mutex that Chapter 22 treats as a throughput constraint is here a correctness guarantee, and the whole-file save that Chapter 17 calls last-writer-wins has, within the process, exactly one writer no matter how many windows exist. A second window is not a second copy of anything.

What it costs is that the two windows are now coupled through that lock in a way neither of them can see. Chapter 22's discussion of a long operation blocking the host's next call is stated in terms of one window's UI stalling; with two windows the same stall is harder to attribute, because the operation that holds the lock was initiated in a window the user is not currently looking at. A transfer started in a background window makes a foreground window's unrelated command wait, and nothing in the boundary tells the foreground window why. The three escapes Chapter 22 describes still apply — they are what keeps this from being pathological — but the diagnostic story does not: the introspection commands of Chapter 30 report on the engine, not on who is currently holding it.

The event sink has a second-window question too. Chapter 4 describes the bridge as one sink whose events the shell forwards to a `WebView` emit. An engine with one sink and a process with two windows means either every event reaches both windows and each filters, or the shell picks a window and the other is deaf. The engine cannot make that choice for the shell, because the engine has no concept of a window — that is the zero-GUI rule working as designed — so the choice is the shell's, and it is one the architecture does not make for it. For the streaming case the ambiguity is smaller, because `StreamError { id, tag, message }` carries an id and a host with two panels can route by it; Chapter 16 introduces that id as the thing that tells a streaming host “which of several open panels to show it in,” and the same id is precisely what a two-window host would route on. That the streaming rail is addressable and the general `EngineEvent` rail is not is worth noticing: the fleet solved the routing problem for streams and left it open for everything else.

The two-process case, which does not work and is not meant to

Now suppose the same app is launched twice as two processes. Every guarantee above evaporates, because none of them was ever process-crossing.

Each process builds its own engine and takes its own lock, so the mutex serializes nothing between them. Each store module runs its own load path at startup and holds its own whole model in memory. Each save path serializes *its* model and replaces the file. Two processes that both load a bookmark set, each add one bookmark, and each save, leave one bookmark on disk — not because of a race in the narrow sense, but because the design holds the entire document in memory and the second save has no knowledge of the first. Chapter 17 states the last-writer-wins property; the process model is what determines whether it can ever be reached.

The engines whose stores are written most often are the ones where this matters most, and they are identifiable from Chapter 17's own list. `zgo`'s store holds “workflows, snippets, clipboard history, and frequency table” — two of those four are written as a side effect of the user simply using the machine, not of the user editing anything. Chapter 8 records a background clipboard daemon (`spawn_clipboard_daemon`) that polls the system clipboard, and it records `learn.record` driving the frequency memory. A store with a background writer is the store least suited to a second process, because the second process does not need a user to make a conflicting change; it makes them on its own.

The other resources partition differently, and the differences are instructive. The keychain is fine: `ztunnel-core` and `zftp-core` route secrets through the OS keychain behind the `keychain` gate, and an OS keychain is a multi-process store by construction — it is one of the few pieces of state in the fleet designed by someone else for exactly this case. The global hotkey is not fine: Chapter 8 records `zgo`'s window toggled by a global

Space through `tauri-plugin-global-shortcut`, and a global shortcut is a system-wide registration that two processes cannot both own in any meaningful sense. The privileged surfaces are worse than either. Chapter 1 records that `ztunnel`'s shell "installs and talks to the privileged helper that actually brings a tunnel up" and that `zcontainer`'s shell "manages the `dockerd` guest VM the engine only speaks to over an API." Both are single system-level resources supervised by an app-level process, and Chapter 7's note that `ztunnel-core`'s supervisor `Drop` exists so there are "no leaked processes, no orphaned secrets" is a per-process guarantee about a system-wide effect. A second `ztunnel` process does not double the tunnel; it produces two supervisors with two opinions about one tunnel's state.

Two window shapes, one architecture

The fleet is not one kind of app wearing eleven skins, and the process question lands differently on the two shapes it actually has.

The launcher shape is `zgo`: a menu-bar presence with a global hotkey, hiding on focus loss, with `zgo_hide` and `zgo_reveal` registered precisely because, as Chapter 1 puts it, "a launcher's window behaviour is a platform gesture the engine has no business knowing about." This shape is *inherently* single-instance. Its whole interaction model is one hotkey summoning one window, and a second copy is not a feature anyone would use; it is a bug that would fight for the hotkey. For this shape the architecture's assumption and the product's design agree, and there is nothing to reconcile.

The document shape is `zoffice`, `zpdf`, and `zphoto`: apps whose durable state is, in Chapter 17's words, "the *user's* file," which round-trips "through a format the user chose, at a path the user chose." This shape is the one where users expect multiple windows, because they expect to open two documents. And here the architecture and the expectation are in genuine tension, because the process-global engine has no handle model — Appendix B's second rule is explicit that "there are no handles to track" — so the engine's notion of the open document is whatever the single global instance last had loaded. Two documents at once is a *model* question, not a window question, and it is answered inside the engine or not at all.

There are two honest ways out and the book should say so rather than imply the case is handled. The first is that a document engine's commands carry the document as data rather than relying on ambient state, which is the direction Appendix G's operation catalogue already leans — an operation named for what it does to a document, given the document, is compatible with any number of open documents because the engine holds no "current" anything. The second is one process per document window, which restores per-document isolation at the cost of a full engine and a full WebView per window and which reopens every last-writer-wins concern for any app that also has a store. The three storeless document engines are, not coincidentally, the ones for which the second option is least dangerous: `zpdf-core`, `zoffice-core`, and `zphoto-core` persist nothing to an application data directory, so two processes of a document app have nothing to clobber. The same property Chapter 17 identifies as making them cheap guests — "a stateless guest is a cheap guest" — makes them cheap second processes.

That is the honest state of it. The engine singleton is a real simplification with real reach, the window singleton is enforced where it can be, and the process singleton is assumed rather than enforced. For the launcher shape the assumption is safe, for the storeless document engines it does not matter, and for the six engines with a store it is an assumption a user can violate with a double-click. Naming which of the three applies to which app seems better than a chapter implying the fleet has a process model when what it has is a process *assumption* that happens to hold.

Chapter 35 – Cold Start: The Cost of the First Frame

The book contains no startup budget. Chapter 23 accounts for the boundary's cost per call, Chapter 22 accounts for the lock's cost under contention, and Appendix K keeps an honest ledger of coverage — but nothing anywhere states what an app does between the moment the user launches it and the moment the interface is usable, or what that costs. This is a gap worth filling because the fleet's architectural choices have unusually legible startup consequences: nearly every decision described in the first twelve chapters either adds to or subtracts from the first frame, and the sum of them is a shape, not a number.

The chapter is structured as a walk down the launch path in order, because the architecture makes that path unusually easy to enumerate.

Everything the engine does before it answers

An engine's startup is short by construction, and the reason is the pattern itself. `<prefix>_init` “constructs the one instance,” in Appendix O's words, and Chapter 4 describes it as building the `OnceLock<Mutex<Engine>>`. A pure-Rust crate with, per Chapter 1, `serde`, `serde_json`, `thiserror`, and `dirs` as its mandatory dependencies has no runtime to boot, no connection to establish, and no plugin registry to populate. There is no dynamic dispatch table to build, because Chapter 4's dispatch is a match on a command string and the match is compiled.

The one real cost is the store, and Chapter 17 states its shape exactly: “read the file at startup, deserialize into the in-memory model, tolerate a missing or older file.” That is a whole-file read and a full `serde` deserialization on the launch path, and it scales with the user's data rather than with the code. For the shapes Chapter 17 names as the right fit — “a few hundred requests, a library of references, a list of bookmarks and servers, a workflow graph” — this is negligible. For the shapes Chapter 17 names as the edge cases it is not: `zemail`'s message store and `zcite`'s attachment index are called out as “growing, queryable data that a whole-file JSON model serves poorly,” and the word *poorly* is doing double duty. It means poorly at query time, which that chapter discusses, and it means poorly at *load* time, which it does not, because the load is unconditional. An engine that reads its whole model at startup pays for the largest thing the user ever saved, every launch, whether the user is going to look at it or not.

The four storeless engines pay none of this. `zcontainer-core` holds nothing across launches by design, and the three document engines' state is the user's file, which is not read until the user opens one. The persistence split and the startup split are the same split.

There is a second engine-side cost that exists only after an upgrade and is worth naming because it is invisible in the module lists: the migration. Chapter 17's “migrations without a migration framework” describes a load path built to tolerate old shapes through `#[serde(default)]` and `Option<T>`, with larger changes handled by reading the old shape explicitly and rewriting it on the next save. A rewrite-on-next-save migration is a *write* on the first save after an upgrade, not on the launch itself, which is the right side of the line to have put it on — the launch stays a read even when the schema moved.

Everything the frontend does before it paints

The frontend path is where the fleet's more distinctive choices show up, and two of them pull in opposite directions.

The first pulls hard toward fast. Appendix D records that the served frontend is vendored at build time — `copy-webui.mjs` `cpSyncs crates/<name>-core/frontend` into the served `frontend/` — and Chapter 12 records that `zgui-core` is “one global, no build step.” A frontend that is plain files on local disk, loaded by a WebView from the app bundle, has no bundler runtime, no module graph to resolve over a network, no hydration step, and no framework to initialize. Chapter 24 makes the related point that the served copy is disposable and regenerated; from the startup angle, the relevant property is that it is *already the artifact*. There is nothing between the file and the parse.

The second pulls the other way, and it is only visible if you read the token table closely. Appendix C records two bundled fonts — Orbitron for display and Share Tech Mono for body and terminal — with `font-display: block`. The `block` value is a deliberate choice about the invisible-text period: text styled with the font is not painted in a fallback while the font loads. For a *bundled* font read from local disk that window is short, and choosing `block` over `swap` is defensible precisely because the fonts are local rather than fetched — Chapter 12 makes the analogous point that the toolkit is loaded “at startup rather than over a network, which is why the fleet can accept it.” But it is still a rendering dependency on the launch path, and it is the only one in the fleet that can produce a frame with layout and no text. The splash (`ZGui.splash`) sits over exactly that window, which is a reasonable reading of why the fleet has a splash component at all: an app whose first paint is deliberately text-suppressed wants something to show.

Then `appShell` mounts, and Chapter 12 describes what that one call does: it reads its options object, builds the chrome, mounts the palette and shortcut surfaces, stamps a `baseline` set recording which family-baseline features the app wired, and toggles effects with body classes driven by `window.prefs`. This is DOM construction against a document that is already parsed — real work, but bounded by the size of the chrome rather than by the size of the user's data.

The order that matters

The two halves above are independent, and the interesting question is whether an app serializes them. Chapter 12 describes an app's `main.js` as “in large part, one `appShell` call plus the wiring of its engine's JSON into that handle,” and that phrasing describes a sequence: mount the shell, then ask the engine for data, then fill the handle. If it is a sequence, the store read happens *after* the chrome exists, which is the right order — the user sees the frame while the engine loads, and the app with the most data to read is the app whose frame appears at the same time as everyone else's.

That ordering is a property of how `main.js` happens to be written rather than something the architecture enforces, and it is worth stating as a rule rather than leaving as a habit, because the boundary makes the wrong order easy to write. `invoke` is a single uniform call and it is tempting to gather everything an app needs before drawing anything. The fleet's own structure argues against that: `setPaletteItems(items)` exists on the `appShell` handle specifically so the palette can be *refilled* after mount, which is the shape of a design that expects data to arrive after the chrome.

Eleven apps, eleven of everything

The last cost is the one the sharing story does not reduce, and it deserves to be stated plainly because the rest of the book is about how much the fleet shares.

The fleet shares *source*. Appendix D's submodule layout mounts the same `zgui-core` under `frontend/lib/` in every app that consumes it, the same `zpwir-embed-terminal` and `zpwir-file-browser` under `crates/`, and Chapter 13's matrix records who consumes what. What none of that changes is that each app is a separate

bundle with its own copy of the shared code, its own compiled engine, its own WebView, and its own process. Two apps running at once is two WebViews, two copies of `zgui-core` parsed into two JavaScript contexts, and two font loads. The submodule cascade of Chapter 15 makes a *fix* reach every app that pins it; it does not make those apps cheaper to run together.

Whether that matters depends entirely on how many are open at once, and the fleet's own shape suggests the answer is small. A launcher is resident by design; a document app is open while you are editing a document; a VPN client is open while you are configuring a tunnel and otherwise represented by a menu-bar presence. The design that would make this cheap — one process hosting many engines, which is precisely what the embed contract of Chapter 14 makes technically possible — is available and is not what the fleet does, because separately shipped apps is the product decision and one mega-app is not. Chapter 13's embed matrix is the compromise: the composition happens where a user actually needs two domains at once, such as a mail client that must preview an attachment, and not merely to save a process.

The honest summary is that the fleet's startup profile is good for structural reasons it did not have to work for — no runtime, no bundler, no network on the launch path, compiled dispatch, local fonts — and has exactly one input that grows without bound, which is the whole-file store read, in exactly the engines Chapter 17 already flags as having outgrown the substrate. There is no measurement in this book, and this chapter does not invent one. What it can say is which term in the sum is the one that will move.

Chapter 36 – The Idle Fleet: What Runs When You Are Not Looking

A desktop app is judged partly on what it does while nobody is using it, and this is a dimension the book has not examined. The command surfaces of Appendix A describe what an engine does when asked. The event catalogue of Appendix H describes what it says while working. Neither describes what is *running* in an app that is open, focused on nothing, and waiting — and the fleet has more of that than its request-response framing suggests.

The finding worth the chapter is that background work in this architecture is almost never in the engine, and tracing why is a good test of whether the zero-GUI rule holds under a kind of pressure the earlier chapters did not apply to it.

Four things that run without being asked

Reading the fleet for continuously-running work turns up four distinct patterns, and they sit at four different layers.

The first is a poller inside an engine, and there is one. Chapter 8 records that `zgo-core` runs “a background clipboard daemon (`spawn_clipboard_daemon`) that polls the system clipboard.” This is the exception to almost everything the pattern says: a pure engine that spawns a thread, reads a system resource on a timer, and mutates its own model with no caller. It is worth being precise about why it is not a violation of the zero-GUI rule — the system clipboard is not a GUI dependency in the sense the rule cares about, and Chapter 1’s rule is about not linking a windowing toolkit, not about never touching the operating system. But it *is* a departure from the request-response shape every other engine holds to, and the consequences follow from Chapter 34: it is the one engine whose model changes while no command is in flight, which makes it the one engine where a second process is dangerous without any user action, and the one engine whose store is written on a schedule rather than on an edit.

The second is a watcher in the shared substrate, exposed as a command. Chapter 18 records `fb_watcher_set` in the `fs_*` family as “a filesystem watcher that pushes change events,” and Chapter 21 counts it among the reasons the `fs_*` family is “the real amplifier” of the bridge’s authority. A watcher is background work by definition, and its placement is where the architecture says it should be: in a shared crate, registered as a command, pushing events rather than being polled. No app’s `*-core` owns it, which is why it is available to every app that mounts the file browser without any domain engine growing a thread.

The third is a supervisor, and it is the heaviest. `ztunnel-core` carries a `netmonitor` module that Chapter 7 describes as watching “for the path change that should trigger a reconnect,” alongside the OpenVPN supervision that reads the management socket line by line. This is not a poller waiting for a user; it is a control loop whose entire purpose is to notice something and act on it. Chapter 7’s note about the supervisor’s `Drop` — no leaked processes, no orphaned secrets — is the tell that this is long-lived work with ownership semantics, which is a different kind of thing from a command that returns.

The fourth is a server-push wait, which is the most efficient of the four and the least like the others. Chapter 11 records `imap.idle` and states the rationale directly: mail arrives “when mail lands rather than by polling.” An IDLE wait is a socket held open with essentially no local cost until the server says something. It is background work that consumes a connection rather than a CPU, and it is the pattern the other three would use if their sources supported it. The clipboard has no push notification, a network path change is an operating-system

event the engine reaches through its monitor, and a process's log is a pipe that must be read; only mail has a protocol that will call you back.

The pattern the four share

Set them side by side and a rule emerges that the book has not stated: **in this architecture, background work is a shell concern except where the domain makes it an engine concern, and the test is whether the work produces domain state or platform effects.**

Chapter 1's division of labour said the shell owns "streaming and long-lived channels — the PTY behind the embedded terminal, the subscription that follows a container's logs — which do not fit a pure `invoke(cmd, args) -> Value` and live in the shell's async runtime." That is a statement about *transport*: a stream of bytes going to a UI belongs to the shell. But the clipboard daemon and the netmonitor are not transports. They produce domain state — a clipboard history entry, a determination that a tunnel must reconnect — and domain state is the engine's, so the loop that produces it ended up in the engine even though a loop is the least engine-shaped thing there is. The rule is not "engines never run threads"; it is "engines never run threads whose output is pixels."

That rule also explains the placement of `fb_watcher_set`, which looks at first like a counterexample. A filesystem watcher produces domain state for a file browser, so by the rule above it should be engine-side — and it *is*, in the sense that `zpw-file-browser` is a Rust crate on the engine side of the bridge rather than JavaScript in the shell. It is simply not any particular app's **-core*, because the file browser is shared substrate rather than a domain engine. Chapter 18's argument that these are crates rather than per-app code covers the background case as cleanly as it covers the request case.

What the event contract does and does not carry

Background work needs a way to report, and the fleet has exactly one: the sink. Chapter 4 describes it as "a sink, two transports, no polling," and every one of the four patterns above ultimately arrives at the UI as an event.

This is the right mechanism, and it has one property worth drawing out because Chapter 22 approaches it from the opposite side. Chapter 22 identifies reentrancy as "the hazard the sink introduces" — an event delivered synchronously into a host that calls back into the engine while the lock is held. Read from the background-work side, that hazard is not occasional; it is the *normal* case for three of the four patterns, because a poller, a supervisor, and a watcher all emit events from threads the user did not start, at times the user did not choose, into a host that may be doing anything. The clipboard daemon does not wait for a quiet moment. The reentrancy discipline Chapter 22 describes is therefore load-bearing specifically for the background patterns, and an engine with no background work — which is most of them — could violate it for a long time without noticing.

The second property is that the sink has no notion of *rate*. An `EngineEvent` is a value pushed into a callback, and nothing in the contract coalesces, throttles, or drops. Chapter 23's per-event tax establishes that each event crosses the boundary as JSON, which means a background producer's frequency is a direct, unmediated cost on the host. For the four patterns the frequencies are wildly different: `imap.idle` fires when mail arrives, a netmonitor fires on a network path change, the clipboard daemon's poll produces an event only when the clipboard actually changed, and a filesystem watcher fires per filesystem change — which can be a great many during a build. Three of those four are naturally sparse. The filesystem watcher is the one that is not, and it is also the one shared by every app that mounts the file browser, which makes it the fleet's most likely source of an event storm and the place any coalescing would have to live.

The idle question the architecture cannot answer

There is one thing this analysis genuinely cannot determine from the material, and it should be said rather than guessed: whether a backgrounded, unfocused app reduces its work. The toolkit surface of Appendix C lists a `uiIdle` helper among its utilities, which indicates the frontend has *some* notion of idleness, but the book records nothing about what it drives, and nothing in any engine's command surface suggests an engine is told the app lost focus. The engine could not act on it anyway without breaking the zero-GUI rule — focus is a window property, and an engine that behaved differently when unfocused would be an engine that knows about windows.

So the honest position is: the shell knows about focus and could throttle what it drives; the engines do not know and do not throttle; and the one engine with a polling loop is therefore polling at the same rate whether or not you are looking at it. For a clipboard daemon that is arguably correct — a clipboard history that stopped recording when the app lost focus would miss essentially everything, since copying happens in *other* apps — and it is a good example of a case where the naive optimization is the wrong behaviour. That it is right for a reason nobody designed for is still worth writing down.

Chapter 37 – Durability: The Save That Is Not Fenced

Chapter 17 establishes where the bytes go, what shape they take, how they migrate, and who is allowed to write them. It does not establish what happens if the process dies during the write, and that omission is the subject here. The distinction matters because the two questions have opposite answers in this architecture: the *concurrency* story is strong within a process and explicitly handed to the shell across processes, while the *crash* story is not addressed at either scope.

What a whole-file save actually is

Chapter 17 describes the save path as “serialize the model, write it back,” and the last-writer-wins discussion sharpens it: “the save path serializes the in-memory model and replaces the file.” Replacing a file is not one operation. It is at minimum a truncate-and-write, and a truncate-and-write has a window during which the file on disk is neither the old model nor the new one but a prefix of the new one — which, for JSON, is a document with unbalanced braces and no valid parse.

The book records no atomic-replace discipline. There is no mention anywhere of writing to a temporary file and renaming it over the target, no mention of `fsync`, and no mention of keeping the previous good file. That absence is not evidence the code lacks them; it is evidence the *book* has never checked, and a reader deciding whether to adopt this pattern needs the question asked out loud, because it is the difference between a store that survives a power loss and one that does not.

The failure this exposes is more consequential here than it would be in a database-backed design, and the reason is the same property Chapter 17 celebrates. A whole-file store rewrites *everything* on every change. A database that loses a write loses a row; a whole-file JSON store interrupted mid-write loses the file, and the file is the entire model — every request in `zreq`’s collection, every reference in `zcite`’s library, every workflow in `zgo`’s set. The blast radius of a torn write is not proportional to the change that triggered it; it is proportional to everything the user has ever saved. This is the exact inverse of the property that makes whole-file JSON attractive, and it comes from the same source: the model is one value.

The mitigation the design already has, and which is worth crediting, is the tolerant load path. Chapter 17’s migration discussion describes a loader built to “tolerate a missing or older file,” with `#[serde(default)]` and `Option<T>` absorbing shapes that changed. A loader that tolerates a *missing* file is a loader that starts empty rather than failing, which converts a torn write from “the app will not launch” into “the app launched with nothing.” That is a meaningfully better failure than the alternative and it is a genuinely nice consequence of having designed the loader for migration. It is not data recovery.

Write amplification, and which engines feel it

The second consequence of whole-file saves is that the cost of a save is set by the size of the model rather than the size of the change, and Chapter 17’s phrase “written back on change” makes the frequency the other multiplier. Multiply the two and you get the engines where the substrate is working hardest, and they are identifiable without any measurement.

`zgo` is the extreme case in both terms. Chapter 17 lists its store as “workflows, snippets, clipboard history, and frequency table,” and Chapter 8 supplies the write drivers: `learn.record` updating the frequency memory,

`clipboard.list/add/search/clear` maintaining the history, and the background daemon feeding the latter with no user present. A frequency table is by nature updated on *every use* — that is what frequency means — so an engine that saves the whole model on change is rewriting its workflows and snippets every time the user launches anything. `zreq`'s history, described in Chapter 5 as “a first-class part of the engine,” has the same shape at lower frequency: every sent request appends to history, and every append rewrites the collections and environments alongside it.

The engines at the other end are the ones whose stores are edited rather than accumulated. `zftp`'s bookmarks and `ztunnel`'s configs and server lists change when a user changes them, which is rarely, and for those the substrate is exactly as correct as Chapter 17 says it is.

This gives a sharper version of Chapter 17's edge condition. That chapter identifies `zemail`'s message store and `zcite`'s attachment index as the data shapes that outgrow whole-file JSON, and the criterion it uses is size and queryability. The write-amplification criterion identifies a *different* pair — `zgo` and `zreq` — whose data is not particularly large but is appended to continuously. Two different pressures, two different lists, and the fleet has engines on both. An engine can outgrow whole-file JSON by getting big or by getting chatty, and only the first is currently named.

The asymmetry between the store and the keychain

Chapter 17's cleanest rule is that credentials do not live in the store: the `creds` modules route secrets to the OS keychain and “the store persists only a *reference* — which bookmark or config a secret belongs to — not the secret itself.” The chapter states the payoff exactly: “Delete the JSON store and you lose your bookmarks; you do not leak your passwords, because your passwords were never in it.”

Follow that through a crash and a second property appears that the chapter does not mention, because it only shows up when you ask about durability rather than about secrecy. The two halves have different lifetimes and no transaction between them. Creating a bookmark with a password means writing a secret to the keychain and writing a reference to the store, and those are two separate durable operations against two separate systems. If the second does not land — a torn write, a crash, or the app being killed between them — the keychain holds a secret that nothing references. It is not leaked in the sense the chapter cares about; it is unreachable through the app, invisible in the store, and it will outlive a reinstall, because a keychain entry is not in the data directory a user might delete.

The reverse case is benign by comparison. A reference in the store with no keychain entry behind it produces a credential lookup that fails, which is a normal error the fleet's taxonomy already handles — Chapter 16 puts a rejected credential squarely in the “fix the environment” branch, and the interface response is to ask for the password again.

So the asymmetry is: losing the store degrades gracefully, losing the correspondence between them accumulates orphans in a place the app cannot see. A design that wanted to close this would need the store to be reconcilable against the keychain — an operation that lists what the keychain holds for this app and drops what nothing references — and nothing in Appendix A's command surfaces is that operation. It is worth naming as a real gap because it is the kind that never announces itself: orphaned keychain entries do not break anything, they accumulate quietly for as long as the user runs the app.

Backup, restore, and the property that was free

There is one durability story the fleet gets for nothing, and it comes from the substrate choice rather than from any code.

Chapter 17 argues for JSON on the grounds that it is “simple, inspectable, diffable, and trivially correct,” and the argument is made about development and about migration. The durability consequence is separate and larger: a store that is a plain JSON file in a predictable directory is a store the *user* can back up with tools they already have. Copy the directory. Put it in version control. Sync it. Restore it by putting the file back and

relaunching — which works precisely because the load path is a whole-file read with no index to rebuild, no journal to replay, and no lock file to clear.

That is a real capability and it is worth stating that the fleet has it, because the alternative substrate does not. An embedded database's file generally cannot be copied safely while the app is running, cannot be diffed usefully, and cannot be restored by dropping an older copy in place without version considerations. The fleet's stores can, and the same property that makes a whole-file save catastrophic when interrupted — the file is the whole model — is what makes a copy of that file a complete backup.

The honest ledger entry, then, has both halves. The fleet's persistence has no transaction, no journal, no recorded atomic replace, and no reconciliation between the store and the keychain; and it has a backup and restore story that requires no code, no export format, and no cooperation from the app at all. Those are two consequences of one decision, and a reader evaluating the pattern should weigh them together rather than meeting only the flattering one.

Chapter 38 – Synchronous by Construction: Where the Fleet’s Async Actually Lives

Every engine in this fleet presents the same signature. Chapter 4 gives the native form — `invoke(cmd, args) -> Result<Value>` — and Appendix B gives the C form, `<prefix>_invoke` taking a command string and a JSON argument string and returning a JSON result string. Neither returns a future, a handle, or a promise. Neither takes a completion callback. The fleet’s entire domain surface is synchronous, and the book has never stated that as a design position, which means it has also never examined what the position costs and buys.

It buys more than it first appears, and the cost is concentrated in one place.

The runtime is on the other side of the boundary

Chapter 1 assigns the shell “streaming and long-lived channels — the PTY behind the embedded terminal, the subscription that follows a container’s logs — which do not fit a pure `invoke(cmd, args) -> Value` and live in the shell’s async runtime.” The phrase “the shell’s async runtime” is the whole architecture in four words: there is a runtime, and it is not the engine’s.

Appendix J’s dependency ledger is consistent with this in a way that is easy to skim past. Read the gate map looking only for an async runtime and the result is striking: across the ten crates it covers, `tokio` is named exactly twice. It appears under `zftp-core`’s `net` gate, alongside `opendal`, `russh`, `russh-sftp`, and `encoding_rs`; and it appears under `zphoto-core`’s `tauri` gate, which brings `tauri` and `tokio` in together. Eight of the ten engines name no runtime in any gate at all — including `zreq-core`, whose `net` gate brings `request` and nothing else, and `zemail-core`, whose `net` gate brings `imap`, `native-tls`, and `lettre`.

The second of the two appearances is the informative one, because of *which* gate it is under. `zphoto-core`’s runtime arrives with `tauri` — that is, with the gate that exists to make the engine speak to a shell — which is exactly where a runtime lives if the runtime belongs to the shell. An engine built `--no-default-features` for a C host, which Appendix J describes as “a pure, network-free, GUI-free library,” does not get a runtime at all, and that is not an oversight; it is the point. What the ledger does not do is tell you what drives the network-facing gates on the engines that name no runtime, and this book should not guess: the honest statement is that the ledger records the dependency, not the execution model beneath it.

This is why the C door can exist in the shape Appendix B describes. A C host calling `zrq_invoke` gets a blocking call that returns a string, which is an interface any language can consume — a JUCE plugin, a command-line tool, another Rust program, a binding in a language with no async story whatsoever. Had the engine’s domain surface been async, the C door would have needed a completion-callback protocol, and every embedder would have had to reason about which thread the callback arrives on and what it is allowed to do there. Chapter 1’s “two front doors to one engine” is only cheap because both doors are synchronous; an async engine has two doors that are not the same door.

The event callback is the exception that proves it. `<prefix>_set_event_callback` registers a `void (*)(const char *json, void *user)`, and that is a completion-callback protocol with exactly the thread questions above — which is why Chapter 22 has to treat reentrancy as a hazard at all. The fleet paid that cost once, for narration, and refused to pay it for the domain surface.

What a synchronous engine does with a slow operation

The obvious objection is that domains like these are full of waiting. A request goes over a network. A transfer moves bytes. A tunnel comes up. An image pulls. An engine that cannot await has to do something with that waiting, and the fleet's answer is distributed across three mechanisms that the earlier chapters introduce separately and that are, read together, one strategy.

The first is to not do the I/O. Chapter 16's `needs_transport` and `needs_privilege` are described there as “the non-error, honestly” — a successful call whose result says the impure remainder is pending. From the async angle they are something more specific: they are how a synchronous function returns from an operation it cannot complete without blocking. The engine computes everything computable, returns a value describing what remains, and the shell — which has a runtime — performs it. `ztunnel-core` “can compute the command line and parse the management socket but cannot hold root,” in Chapter 1's words, and the same sentence structure applies to waiting: it can compute the plan but cannot hold a thread.

The second is to put the I/O behind a gate and accept the block. Chapter 5 records that `zreq-core`'s send path “is gated behind `net`, and it is the sole function that touches `request`.” One function, one gate, and a call that blocks for the duration of a request. This is the pragmatic case, and it is defensible for exactly the reason Chapter 22 gives about long operations being domain entities: a sent request is a thing with a start and a finish that the engine narrates through `RequestStarted` and `RequestFinished`, so the caller is not staring at an opaque block.

The third is to move the whole operation out of the request-response shape. Chapter 4's “one exception: streaming through a second door” and `zcontainer`'s separately registered `zcontainer_subscribe/_unsubscribe/_stream_send` are the acknowledgement that some work does not fit and should stop pretending. A log stream is not a slow function; it is a channel, and the fleet gave it its own commands rather than deforming `invoke` around it.

Those three are a coherent strategy and they cover the fleet's actual operations. It is worth saying explicitly, though, that the strategy has a shape: it works when the impure remainder can be *described as data*. A plan the shell executes, a request the shell awaits, a stream the shell owns — each requires that the engine can hand off a complete description in one value. Where it would not work is an operation whose control flow interleaves computation and I/O many times over, because each round trip would be another `needs_transport` and the engine would become a state machine driven by its host. That the fleet has no such operation is a fact about these ten domains, not a general result.

The cost, which lands on the mutex

The one place the synchronous choice is genuinely expensive is where it meets the other global decision, and Chapter 22 names the mechanism without naming this cause. Every command is serialized by the process-global mutex, so a command that blocks holds the lock while it blocks. A synchronous engine with a global lock means *any* blocking I/O inside the engine stops the entire engine, not just the caller.

This makes the gating discipline load-bearing in a way that is easy to read as mere dependency hygiene. Chapter 5's note that the `net`-gated send is “the sole function that touches `request`” is usually read as a durable-deps statement: one place to audit, one dependency to justify. Read against the lock it is also a concurrency statement — it is an inventory of every place the engine can block, and keeping that inventory at one function per engine is what keeps the global-mutex simplification affordable. The feature gates of Chapter 32 turn out to double as a map of an engine's blocking surface, which is a use for the gate tables that Chapter 32 does not claim.

It also sets a bound on the pattern that a reader should weigh. This architecture — one global instance, one lock, synchronous domain calls — is correct for engines whose work is computation punctuated by a small, enumerable set of I/O points. An engine whose work is *mostly* I/O, with many operations in flight at once, would find the global mutex serializing things that have no reason to be serialized, and the fix would not be a better

lock; it would be abandoning the process-global instance for per-session state and paying the handle-tracking cost that Appendix B's second rule exists to avoid.

The fleet is on the right side of that bound because of what these domains are, and it is worth checking rather than assuming. A request client sends one request at a time from a user's hand. A citation manager formats. A launcher matches. A document engine transforms. The one domain that is genuinely many-operations-at-once is transfer, and `zftp-core` is — not coincidentally — the one engine whose *network* gate names `tokio` directly. The design kept the domain surface synchronous everywhere and let the transfer engine's plumbing be what its queue needs, which is a better outcome than a uniform answer in either direction.

Chapter 39 – Accessibility Below the Effects Layer

Chapter 12 has a section titled “Effects, splash, and accessibility toggles,” and it describes something real: the CRT scanlines, the neon glow, the bezel vignette, and the animations are centralized in `zgui-core` and switched off per body with `no-scanlines`, `no-vignette`, `no-neon-glow`, and `no-anim`, driven by `window.prefs`. The chapter’s claim for this is measured and correct — “a user who finds the scanlines distracting or needs reduced motion turns them off once and every component respects it.”

That is one axis of accessibility, and it is the axis a heavily-styled interface is most obviously exposed on. It is not the whole subject, and the book has not looked at the rest. This chapter does, using the same standard the rest of the book uses: what the architecture actually provides, what it makes cheap, and what it has not addressed — stated as a bound rather than implied away.

Contrast, computed from the token table

The one accessibility property that can be evaluated directly from material already in this book is contrast, because Appendix C publishes the exact token values. The core `:root` tokens give the backgrounds — `--bg-primary #05050a`, `--bg-card #0d0d1a` — and the text ramp `--text #e0f0ff`, `--text-dim #7a8ba8`, `--text-muted #3d4f6a`, along with the accent colours. Contrast ratios follow from those hex values by the standard relative luminance formula, and the thresholds in common use are 4.5:1 for normal-size text and 3:1 for large text and interface components.

Computed against `--bg-primary`, the default cyberpunk scheme lands like this:

token	value	ratio vs <code>--bg-primary</code>
<code>--text</code>	<code>#e0f0ff</code>	17.51
<code>--green</code>	<code>#39ff14</code>	15.00
<code>--cyan</code>	<code>#05d9e8</code>	11.72
<code>--text-dim</code>	<code>#7a8ba8</code>	5.89
<code>--accent</code>	<code>#ff2a6d</code>	5.62
<code>--red</code>	<code>#ff073a</code>	5.18
<code>--magenta</code>	<code>#d300c5</code>	4.43
<code>--text-muted</code>	<code>#3d4f6a</code>	2.45
<code>--border</code>	<code>#1a1a3e</code>	1.22

The results are more favourable than a neon-on-black palette’s reputation would suggest, and the reason is structural: the background is nearly black, so almost anything saturated clears the normal-text threshold comfortably. Body text at 17.51 is far above any requirement. Cyan and green are extremely high. The accent pink and the error red both clear 4.5:1 as normal text.

Two entries do not clear it, and they are worth separating because they mean different things. `--magenta` at 4.43 is a near miss — it passes as large text, fails as body text by a margin no user would perceive as a cliff, and the honest description is that it is fine for headings and marginal for small labels. `--text-muted` at 2.45 fails both thresholds, and unlike the magenta case it fails them by a wide margin. A token named *muted* is presumably used for what it sounds like — secondary metadata, hints, inactive states, timestamps — which is

small text, which is the case with the strictest requirement. `--border` at 1.22 is not a text token and the ratio is not a defect; a border that low against the background is a decision about how visible the chrome is, though it does mean the borders are doing very little work as *separators* for a user with reduced contrast sensitivity.

Ratios against `--bg-card` are marginally lower across the board, because that background is the lighter of the two, and the ordering does not change.

The useful thing about this analysis is not the verdict but the mechanism. Appendix C records that applying a scheme writes 24 `SCHEME_VAR_KEYS` and that the built-in schemes are `cyberpunk`, `midnight`, `matrix`, `ember`, `arctic`, `crimson`, `toxic`, and `vapor`, each with a dark and a light variant. That is a lot of colour pairs, and the *same* computation applies to every one of them. A fleet with a centralized token system and a fixed key set has the precondition for a contrast check that runs over all schemes mechanically — the values are in one file, the keys are enumerated, and the formula is arithmetic. Chapter 19 makes the argument that feature matrices become executable specifications; a token table is the same kind of artifact and is checkable in the same way. Nothing in the book says such a check exists, and the point here is that the architecture has already done the hard part, which is putting every colour in one place with a known name.

What the toolkit surface says about the rest

Beyond contrast and motion, accessibility is largely a question of whether the interface is navigable and describable without sight, and here the evidence in the book is thinner and mixed.

The favourable evidence is real and structural. This is a DOM interface, not a canvas one — Appendix C's component list is full of things built from markup (`table`, `dataTable`, `modal`, `statusBar`, `tooltip`, `contextMenu`), which means the platform's accessibility tree exists by default rather than having to be constructed. An interface drawn into a canvas starts at zero and must reimplement everything; this one starts wherever plain HTML puts it, which is a much better floor.

The keyboard story is better still, and it is the fleet's strongest accessibility asset by some distance. Chapter 33 argues that the palette is a second command surface and that the keyboard is the fleet's actual interface. R1 puts a Cmd/Ctrl+K palette in every app. Appendix C lists `shortcuts` among the shell-and-chrome components and `appShell` exposes `openShortcuts` alongside `openSettings`, with Chapter 3 naming `shortcuts.init` as the call that wires them. An application whose entire command vocabulary is reachable by typing is an application a keyboard-only user can drive, and the fleet built that for speed rather than for access — but the benefit does not care about the motivation.

One further entry in the component list is worth pulling out because it is the closest thing the toolkit has to an explicit statement on this subject: `keyboardNav`, grouped under embedding and infrastructure alongside `util`, `drag`, `fileDrag`, `uiIdle`, and `i18n`. A named, shared keyboard-navigation helper sitting in the infrastructure group rather than in any one component is the right *shape* — it is the factoring that would let focus movement behave identically in a table, a tree, and a launcher list — and its presence means the fleet did not treat keyboard traversal as each component's private business. What the book records is the name and the grouping, not the semantics, so this is an argument about structure rather than a claim about behaviour.

The unfavourable evidence is an absence. The component surface as recorded in Appendix C is described in terms of options and returned handles, and nothing in it mentions roles, labels, focus management, or live regions. The design-system conformance spec of Chapter 12 runs R1 through R10 and none of them is an accessibility requirement. Chapter 12's own description of the accessibility surface is scoped entirely to the effects toggles. It would be wrong to conclude from this that the components have no accessible markup — the book documents surfaces, not every attribute in every element — but it is fair to conclude that accessibility below the effects layer is not something the toolkit has made into a *contract*, in a book that is otherwise careful to make things into contracts.

That distinction is the substance of this chapter. The fleet has a mechanism for guaranteeing that every app shares a look (one token file), a mechanism for guaranteeing every app shares a command surface (R1 and the palette), a mechanism for guaranteeing every app shares a localization catalogue (Chapter 20), and a mechanism for guaranteeing an embedded core cannot seize the document (Chapter 14's forbidden globals,

which throw). It does not have a mechanism for guaranteeing that a component is announced correctly. The components are shared, so the *opportunity* is unusually good — fixing a table's roles once fixes every table in every consuming app, which is exactly the leverage Chapter 12 claims for everything else — but leverage is not the same as a guarantee, and an honest ledger entry distinguishes them.

Where the hard cases are

Two of the toolkit's components would be substantially harder than the rest, and naming them is more useful than a general recommendation.

The node graph is one. Chapter 8 records `prefs.js` building “the lane/node/bezier editor with `ZGui.nodeGraph`,” and Chapter 12 records it as the shared component behind `zgo`'s workflow editor. A directed graph edited by dragging is the canonical hard case for non-visual interaction; the underlying model — Chapter 8's “workflow graph: objects and connections” — is perfectly describable, but describing a *spatial editing gesture* is not, and the honest answer for a component like this is usually a parallel non-spatial editing path rather than an accessible drag.

The data table is the other, and it is the more important one because it is everywhere. Chapter 12 explains that `ZGui.table` is a set of engines — `columns`, `sort`, `batch`, `multiFilter` — with `dataTable` built on top of `table.sort` and `table.columns`. Sorted, filtered, multi-selectable tables are describable, and the accessibility work for them is well-understood; the reason it matters here is the factoring. Because `dataTable` calls the same `table.sort` and `table.columns` rather than forking them, sort state lives in one implementation, and one implementation is one place to announce it from. Chapter 12 presents that factoring as an argument about avoiding divergence. It is also, unclaimed, the thing that would make the fleet's tables fixable in a single edit.

Chapter 40 – Distribution: Signing, Notarization, and the Update Path

Appendix D is titled “Build, Packaging, and Versioning,” and it is precise about the build: the `beforeBuildCommand` that vendors the shared web modules, the `frontendDist` pointing at the served directory, the submodule layout, the crate types, the editions, the registered command surface, and the version table. What it does not describe is the step after the build, which is how an artifact gets from a machine that compiled it to a machine that runs it. That step has requirements the architecture is already shaped by, costs that multiply by the size of the fleet, and one gap the book should name rather than leave to a reader’s assumption.

The identity is already chosen

Distribution begins with identity, and the fleet made that decision long ago for a different reason. Chapter 17 records that every app persists under `~/Library/Application Support/com.menketotechnologies.<app>` on macOS, “resolved through Tauri’s `app_data_dir()` keyed by the app’s bundle identifier, not through a raw `dirs::data_dir()` join with a product name.”

That bundle identifier is not only a data-directory key. It is the name under which an application is signed, the name a platform uses to decide whether a new bundle is an update to an existing app or a different app entirely, and the name that permissions granted to an app attach to. Chapter 17 argues for the bundle-id convention on data-isolation grounds — one predictable directory per app identity, “no collision between an app and a core that happens to share a product string” — and the argument is correct on those grounds. What is worth adding is that the same choice fixes the identity for everything downstream: signing, update matching, and the operating system’s record of what the user has permitted this app to do.

The reverse implication is the one that bites. Because the data directory is keyed to the bundle id, changing a bundle id is not a rename; it is an app that has lost its data, because the new identity resolves a new directory and the old one is orphaned in place. The fleet’s uniform `com.menketotechnologies.<app>` scheme makes this unlikely to arise, and the coupling is worth stating precisely because it is invisible: a decision that looks like packaging metadata is load-bearing for persistence.

One fix, many signings

Chapter 15 develops the cascade — a fix in a shared submodule reaches every app that pins it, and the super-project records the pointers. Chapter 13’s matrix records who consumes what. The distribution consequence is a multiplication those chapters do not carry through, and it is the largest single cost distribution adds to this architecture.

Follow a `zgui-core` fix through. The toolkit is pinned as a submodule under `frontend/lib/` in every consuming app. Each of those apps must update its pointer, rebuild — which re-runs the vendoring step, since Appendix D records that the served frontend is a regenerated copy — and then produce a new signed artifact. For a fleet whose version table lists a row per app, a one-line toolkit fix is potentially a build, a signature, a notarization submission to a platform service that runs on its own schedule, and a published artifact, *per row*.

The comparison worth making is with the monolithic alternative. One application with many modes would have one signature and one notarization for the same fix. The fleet’s separate-apps decision — which Chapter 15

defends on other grounds, and which Chapter 35 notes is not free at runtime either — buys independence at the cost of a per-release constant that scales linearly with the number of apps. This is the same trade the version skew table already exposes from a different direction: Appendix D records `zgo`, `zpdf`, and `zcontainer` as “shell ahead” of their cores, `zterminal` as “shell far ahead,” and the rest in lockstep. Skew is what you get when release is expensive enough that you do not do it for every app every time, and the skew table is therefore a measurement of the distribution cost as much as of the development process.

There is one mitigation the architecture already provides and it is worth crediting. A shell-side fix — chrome, window behaviour, a menu-bar gesture — requires rebuilding only the apps that changed, and Chapter 15’s observation that minor versions of menu-bar and hotkey work over one stable engine version is the skew describes exactly this: the engine’s stability is what lets shells ship independently. The cascade is expensive when it starts at the bottom of the stack and cheap when it starts at the top, and the architecture puts the most-shared code at the bottom, which is where the cascade is most expensive. That is an unavoidable consequence of sharing rather than a defect, but a reader adopting this pattern should know that the shared-code leverage Chapter 12 celebrates and the release cost described here are the same fact with opposite signs.

The update path the book does not describe

Here is the gap stated plainly: nothing in this book describes how a user of one of these apps gets a newer version. There is no updater in the registered command surface of Appendix D, no update-related entry in any engine’s command surface in Appendix A, no update event in Appendix H, and no mention of an update mechanism anywhere in the architecture chapters. The book’s honesty compact requires saying that rather than assuming a reader will supply an answer.

Two of the fleet’s existing properties are relevant to what an absent updater implies.

The first is favourable. Chapter 17’s migration approach — “the load path is written to *tolerate* an older or partial document rather than to run a versioned migration pipeline,” with `#[serde(default)]` and `Option<T>` absorbing changes — is exactly the migration strategy that survives *skipped versions*. A pipeline of ordered migrations requires that every version be run in sequence, which requires an updater that steps a user through them; a forgiving deserializer does not care how old the file is, only whether the fields it needs are absent or defaulted. A fleet without an updater, where a user may go from a build many versions old directly to the current one, needs precisely the migration design the fleet has. Chapter 17 justifies that design on dependency-austerity grounds. It is also the correct choice for the distribution model, and the two reasons reinforce each other.

The second is less favourable. Without an in-app update path, the version a user is running is whatever they last installed, and the version skew table means the number they would report is the *shell’s* version — which Appendix D shows can be far ahead of the engine’s. A user reporting a bug in `zterminal` names a version well ahead of `ztmux-core 0.1.0`, and the mapping from that number to the engine actually running is a table in this book rather than something the app can state. Chapter 30 catalogues the introspection commands an engine offers for diagnosing itself; a version pair reported from inside the running process is the kind of thing that belongs in that catalogue and, per Chapter 30’s own accounting of what the diagnostic surface does not have, is not there.

What ships beyond the bundle

For most of the fleet, distribution is the bundle and nothing else, which is a direct consequence of the durable-deps rule: an engine whose mandatory dependencies are `serde`, `serde_json`, `thiserror`, and `dirs` compiles into the binary and needs nothing installed alongside it. The fonts are bundled, per Appendix C. The frontend is vendored into the bundle at build time. There is no runtime to install, no interpreter, and no shared library the user must have. That is the whole install footprint, and it is small for structural reasons.

Two apps are exceptions and both were introduced for other reasons. `ztunnel`'s shell, per Chapter 1, "installs and talks to the privileged helper that actually brings a tunnel up" — a privileged helper is a component installed *outside* the bundle, with its own lifetime, its own trust relationship with the operating system, and its own version. It is the one part of the fleet where installation is a stateful operation rather than a copy, and where removing the app is not sufficient to remove everything it put on the machine. `zcontainer`'s shell "manages the `dockerd` guest VM the engine only speaks to over an API," which is a substantially larger out-of-bundle dependency, and Chapter 10's platform split — `daemon`, `daemon_linux`, `daemon_windows`, `vm` — is where that complexity is organized.

Both exceptions share a shape. The engines stay pure — `ztunnel-core` computes a `PrivilegePlan` it cannot execute, `zcontainer-core` speaks to a daemon it does not own — and the messy, stateful, platform-specific installation lives in the shell. That is the same division Chapter 1 draws for privilege and pixels, extended to installation, and it means the distribution complexity is confined to two shells rather than smeared across ten engines. A reader looking for where this architecture pays off in an area the book had not yet examined can take that as the answer: the engines are trivial to distribute because everything hard about distribution is, by construction, on the other side of the boundary.

Appendix A — The Engine Command Surfaces

Every engine in the fleet routes its entire feature set through one string-dispatched `invoke(cmd, args)`, and the command names *are* the engine's public API — over the native Rust call, over the C ABI's `<prefix>_invoke`, and over the Tauri plugin's `<prefix>_invoke` alike. This appendix collects the real dispatch prefixes and representative command families, as a map of where each engine's surface lives.

- **zreq-core** (`zrq_`): `workspace.*`, `collection.*`, `request.get/add/update/remove/send`, `runner.run`, `environment.*`, `globals.get/set`, `vars.resolve`, `history.list/replay/clear`, `codegen.generate`, `import.curl/postman/openapi/har`, `export.postman`, `cookie.*`, `oauth2.token`, and (feature-gated) `ws.exchange`, `grpc.call`.
- **zcite-core** (`zct_`): `library.*`, `libraries.*`, `schema.*`, `item.*`, `collection.*`, `tag.*`, `search.quick`, `search.saved.*`, `bib.bibliography/citation/cite_key/csl`, `cite.document`, `import.*`, `export.*`, `duplicates.list/merge`, `identifier.detect/lookup/add`, `pdf.extract_text/recognize`, `webdav.*`, `zotero.*`.
- **ztunnel-core** (`ztn_`): `config.*`, `vpn.connect/disconnect/status/connections/logs`, `creds.set/clear`, `servers.list/fastest/ping`, `feature.matrix`, `vpn.killswitch_plan`, `platform.info`.
- **zgo-core** (`zgo_`): `workflow.*`, `scriptfilter.run`, `feedback.parse/render`, `match.filter`, `var.render`, `websearch.*`, `clipboard.*`, `snippet.*`, `trigger.*`, `calc.eval`, `learn.record/rank`, `system.*`, `index.*`, and the launcher's provider commands (`runningapps`, `bookmarks`, `music`, `contacts`, `dictionary`, `spotlight`, `onepassword`).
- **zftp-core** (`zftp_`): `bookmark.*` (including `bookmark.import` for `.duck`), `session.*`, `transfer.add/list/status/cancel/clear`, `fs.*`, `creds.*`, `discovery.scan`.
- **zcontainer-core** (`zcontainer_`): `docker.containers.*`, `docker.container.*`, `docker.compose.*`, `docker.daemon.*`, `k8s.contexts/namespaces/pods.*`, `k8s.deployments/services/nodes.*`, `k8s.pod.logs/exec/portforward`, `k8s.resource.*`, `k8s.apply.yaml`, `k8s.helm.*` — plus the streaming trio `subscribe/unsubscribe/stream_send`.
- **zoffice-core** (`zoffice_`): `writer.*`, `calc.open/eval/edit_cell/pivot_tables/charts`, `impress.*`, `draw.*`, `base.open/query/tables`, plus PDF export.
- **zemail-core** (`zml_`): `account.*`, `folder.*`, `message.*`, `imap.sync/search/idle`, `filter.*`, `junk.classify/train`, `contact.*`, `carddav.*`, `calendar.*`, `gloda.search`.
- **zpdf-core** (Rust/Tauri only, `zpdf_invoke`): a flat operation list — `add_note/add_ink/add_callout/add_markup/add_link`, `apply_redactions`, `certify`, `accessibility_check`, `add_qr_code`, `bates_number`, `flatten_annotations`, `downsample_images`, `linearize`, and the rest of the Acrobat/Preview surface.
- **zphoto-core** (`zphoto_`): `edit.undo/redo/copy/paste/history`, `fill.*`, `filter.*`, `brush.*`, `action.record/play/stop`, `comp.capture/apply`, and the `vec.*` vector family.

Fuller dispatch, by engine

The representative families above are the map; the fuller lists below are the territory, drawn from the actual `Engine::invoke` match arms so a reader can see the grain of each surface.

- **zreq-core** (`zrq_`): `version`; `workspace.get/save/list/`, `workspace.current/create/`, `workspace.switch/rename/delete`; `collection.list/add/get/`,

- collection.update/remove; request.get/add/update/, request.remove/send;
runner.run; oauth2.token; ws.exchange; grpc.call; environment.list/add/
remove/activate; globals.get/set; vars.resolve; settings.get/update/reset/path;
history.list/replay/clear; codegen.generate; import.curl/postman/openapi/har;
export.postman; cookie.list/set/clear.
- **zcite-core** (zct_): library.get/save/stats; libraries.list/active/create/
rename/switch/remove; schema.item_types/fields; item.list/get/add/update/ set_
field/set_reading/set_rating/set_favorite/trash/restore/delete/add_tag/ remove_
tag/add_note/add_attachment/update_note/remove_note/relate; attachment.store_
file/snapshot/index; annotation.add/list/update/remove; pdf.extract_
text/recognize; webdav.verify/upload/download/delete; zotero.verify/pull/push;
note.add/list/update/remove; collection.list/add/rename/remove/add_item/remove_
item; tag.list/set_color; search.quick, search.saved.list/add/remove/run;
bib.bibliography/citation/cite_key/csl/csl_document; cite.document;
import.bibtex/ris/csl_json/endnote_xml/mods/marcxml/zotero_rdf, import.file,
inbox.import; export.bibtex/ris/csl_json/endnote_xml/mods/marcxml/zotero_rdf;
duplicates.list/merge; identifier.detect/lookup/add.
- **ztunnel-core** (ztn_): version; profile.get; settings.get/set;
feature.matrix; platform.info; config.list/get/add/import/wireguard_
format/ remove/set_options/rename/duplicate/parse; creds.set/clear;
vpn.connect/disconnect/status/connections/autoconnect/network_changed/logs/
clear_logs/killswitch_plan; servers.list/add/remove/favorite/fastest/ping.
- **zgo-core** (zgo_): version; profile.get/save; workflow.list/get/add/remove,
workflow.import/export/run; scriptfilter.run, stryke.run, feedback.parse/render;
match.filter; var.render; websearch.list/url, search.add/remove/fallback;
clipboard.list/add/search/clear; snippet.list/expand/match/import/add/remove,
keystroke.type; theme.list/add/remove; calc.eval; learn.record/rank;
trigger.keyword/external/fileaction/snippet/fallback/hotkey;
utility.conditional/split/join/transform/replace/random/counter/file_
conditional/ json_config/junction/delay/expression; file.search/browse/filter,
list.filter; query.record/recent/list/clear; providers runningapps.search,
bookmarks.search, music.nowplaying/command, contacts.search, dictionary.define,
spotlight.search, onepassword.search, actions.list, system.list/run,
index.scan/search.
- **zftp-core** (zftp_): version; profile.get/recent; settings.get/set;
transport.info; bookmark.list/get/add/import/import_filezilla/update/remove/
set_options; creds.set/clear/store/load/delete; discovery.scan;
session.connect/disconnect/status, session.list/logs/clear_
logs; fs.list/mkdir/delete/rename/rename_plan/chmod/peek;
transfer.add/list/status/cancel/clear.
- **zcontainer-core** (zcontainer_): the Docker and Kubernetes vocabularies of Chapter 10 in full
— docker.containers.list, docker.container.*, docker.images.list, docker.image.*,
docker.hub.*, docker.compose.*, docker.networks/network.*, docker.volumes/volume.*,
docker.daemon.*, docker.engine.pause/resume, docker.builds.*, docker.system.*,
docker.ping, docker.events.follow, docker.topology; k8s.*.list for every workload
and resource kind, k8s.pod.*, k8s.top.*, k8s.resource.*, k8s.apply.yaml, k8s.helm.*;
vm.resources.get/set — plus subscribe/unsubscribe/stream_send.
- **zoffice-core** (zoffice_): writer.open/render/pdf/html/markdown/text/find/
replace/structure/runs/tables/images/links/notes/footnotes/fields/comments/
sections/style_definitions/revision_authors; calc.open/render/pdf/csv/eval/find/
replace/cells/edit_cell/formulas/tables/sort/charts/named_ranges/validations/
conditional_formats/pivot_tables/print_setups; impress.open/render/pdf/text/
shapes/tables/slide_notes/layouts/transitions/charts_render;
draw.open/render/svg/connectors; math.open/render/starmath;

- `base.open/tables/query.`
- **zemail-core** (`zml_`): `account.add/remove/`, `account.list/autoconfig`; `folder.list/create/`, `folder.rename/delete`; `message.list/get/move/delete/set_flags/threads/` `save_draft/add_label/remove_label/categorize/junk/pin/snooze/unsnooze/followup/` `set_aside/expire/unsubscribe`; `imap.sync/search/idle/folders/store_flags`, `smtp.send`, `pop3.fetch`; `filter.add/list/remove/run`; `junk.classify/train/run`; `contact.add/remove/list/import_vcard/export_vcard`, `carddav.fetch/put`; `calendar.parse_invite/rsvp`; `gloda.search`, `search.run/save/saved/run_saved`; `openpgp.*`, `smime.*`; and the power-user families `screeener.*`, `vacation.*`, `vip.*`, `list.*`, `template.*`, `outbox.*`, `thread.*`, `key.*`, `import.eml/maildir/mbox`.
- **zpdf-core** (Rust/Tauri only, `zpdf_invoke`): a flat ~300-operation list; see Appendix G for the catalogue.
- **zphoto-core** (`zphoto_`): `edit.*`, `image.*`, `layer.*`, `op.*`, `paint.*`, `select.*`, `fill.*`, `filter.*` (~160), `brush.*`, `action.*`, `comp.*`, `path.*`, `pattern.*`, `sampler.*`, `note.*`, `channel.*`, `text.*`, `project.*`, and the `vec.*` vector family.

Appendix B – The C-ABI Reference

Nine of the ten engines expose a C ABI over their single `Engine::invoke`; the tenth, `zpdf-core`, is Rust-and-Tauri only. This appendix is the reference: the function set per engine, the header files, and the one memory-and-threading contract they share.

The uniform contract

Every C-ABI engine obeys the same three rules, so a host learns them once. First, **strings are heap-owned by the caller**: any `char*` returned by `<prefix>_invoke` or `<prefix>_init` is a NUL-terminated UTF-8 buffer the caller must return to `<prefix>_string_free`. Second, **the engine is a single process-global instance** behind a `OnceLock<Mutex<Engine>>`, so there are no handles to track and calls are serialized by the mutex. Third, **arguments and results are JSON**: `<prefix>_invoke` takes a command string and a JSON argument string and returns a JSON result string, mirroring the native `invoke(cmd, args) -> Result<Value>`. Engines that emit events add a `<prefix>_set_event_callback(cb, user)` registering a `void (*)(const char *json, void *user)`; the engine serializes each `EngineEvent` to JSON and calls the callback. Each header lives hand-written in the crate's `include/` (not `cbindgen`), and most ship a header-only C++ RAI facade (`namespace <name> { class Core }` with static `init`, `invoke`, `onEvent`) alongside the `.h`.

The function set per engine

The count is fitted to the domain (Chapter 2's spectrum), not stamped uniformly:

- **zreq-core** — prefix `zrq_`, header `include/zreq_core.h (+ .hpp)`. Four: `zrq_set_event_callback`, `zrq_init`, `zrq_invoke`, `zrq_string_free`. Events: `RequestStarted`, `RequestFinished`, `Error`.
- **zcite-core** — prefix `zct_`, header `zcite_core.h (+ .hpp)`. Four: `zct_set_event_callback`, `zct_init`, `zct_invoke`, `zct_string_free`. Events: `LibraryChanged`, `Imported`, `LookupFinished`, `Error`.
- **ztunnel-core** — prefix `ztn_`, header `ztunnel_core.h (+ .hpp)`. Four: `ztn_set_event_callback`, `ztn_init`, `ztn_invoke`, `ztn_string_free`. Events: `Status`, `Stats`, `Log`.
- **zgo-core** — prefix `zgo_`, header `zgo_core.h (+ .hpp)`. Four: `zgo_set_event_callback`, `zgo_init`, `zgo_invoke`, `zgo_string_free`. Events: `ScriptFilterStarted`, `ScriptFilterFinished`, `Error`.
- **zftp-core** — prefix `zftp_`, header `zftp_core.h (+ .hpp)`. Four: `zftp_set_event_callback`, `zftp_init`, `zftp_invoke`, `zftp_string_free`. Events: `Status`, `Stats`, `Log`, `Transfer`.
- **zemail-core** — prefix `zml_`, header `zemail_core.h (+ .hpp)`. Four: `zml_set_event_callback`, `zml_init`, `zml_invoke`, `zml_string_free`. Events: `SyncStarted`, `SyncFinished`, `MessageSent`, `Error`.
- **zcontainer-core** — prefix `zcontainer_`, header `zcontainer_core.h`. Eight: the four above plus the streaming trio `zcontainer_subscribe`, `zcontainer_unsubscribe(unsigned long long id)`, `zcontainer_stream_send(id, data)`. Events: `Stream`, `StreamEnd`, `StreamError`, `Error`.
- **zphoto-core** — prefix `zphoto_`, header `zphoto_core.h`. Three: `zphoto_init`, `zphoto_invoke`, `zphoto_string_free`. No event callback (no `EngineEvent`).
- **zoffice-core** — prefix `zoffice_`, header `zoffice_core.h`. Two: `zoffice_invoke`, `zoffice_string_free`. No `init` (stateless unit-struct engine), no events; built on the 2024 edition with `#[unsafe(no_mangle)]`.

- **zpdf-core** — *no C ABI*. No `ffi.rs`, no `include/`, `crate-type` defaults to `rlib` only. Native Rust API and the Tauri command `zpdf_invoke` only.

A minimal C consumer

The sketch is the same for every four-function engine but for the prefix:

```
#include "zreq_core.h"
#include <stdio.h>

static void on_event(const char *json, void *user) {
    printf("event: %s\n", json); /* {"type":"request_finished","status":200,...} */
}

int main(void) {
    zrq_set_event_callback(on_event, NULL);
    char *status = zrq_init(); /* {"ok":true} */
    zrq_string_free(status);
    char *res = zrq_invoke("request.send", "{\"id\":\"r1\"}");
    printf("result: %s\n", res); zrq_string_free(res);
    return 0;
}
```

Appendix C – The Shared Component Surface

`zgui-core` hangs its whole toolkit off one global, `window.ZGui`. This appendix is a reference map of that surface and the tokens and schemes beneath it.

The `window.ZGui.*` namespace, grouped

- **Shell & chrome:** `appShell`, `header`, `logo`, `settings`, `shortcuts`, `prefsShell`, `launcher`, `statusBar`, `statusPill`, `helpOverlay`, `tooltip`, `contextMenu`, `floatingDock`, `dock`, `trayPopover`.
- **Command & search:** `palette` (`register/open/bindHotkey`; instance `create({items,scope})`), `fzf` (`fzfMatch`, `getMatchIndices`, `highlightMatch`, `highlightWithIndices`, `settingsPanel`), `searchBox`, `searchSelect`.
- **Data views:** `table` (`{columns,sort,batch,multiFilter}`), `dataTable`, `tree`, `jsonView`, `xmlView`, `csvView`, `tomlView`, `logView`, `markdown`, `codeEditor`, `richText`, `managerList`, `transferList`, `kvEditor`.
- **Layout:** `tabs`, `splitPane`, `dockPanel`, `resizable`, `drawer`, `accordion`, `collapsible`, `scrollArea`, `aspectRatio`, `segmented`, `breadcrumb`, `pager`.
- **Controls:** `button`, `buttonGroup`, `buttonBar`, `splitButton`, `textfield`, `checkbox`, `toggle`, `range`, `rangeSlider`, `select`, `radio`, `badge`, `chip`, `kbd`, `pinInput`, `datePicker`, `colorPicker`, `wizard`, `dropdownMenu`, `fab`, `speedDial`, `copyButton`, `strengthMeter`, `toggleGroup`.
- **Dashboard & HUD:** `tiles`, `tileGrid`, `statStrip`, `headlineStats`, `playingPill`, `heatColor`, `viz`, `donut`, `stackBar`, `statBars`, `metaGrid`, `progress`, `spinner`, `eta`, `timeAgo`, `countBadge`, `formatBadge`.
- **Node/graph & audio:** `nodeGraph`, `parametricEq`, `waveformPlayer`, plus the synth widgets (`knob`, `fader`, `xyPad`, `envelope`, `lfoShape`, `curveEditor`, `patchbay`).
- **Dialog & feedback:** `modal` (`open/confirm/prompt`), `modalDrag`, `toast`, `popover`, `banner`, `alert`, `emptyState`, `skeleton`, `loadingOverlay`.
- **Effects & theming:** `colorscheme`, `crt`, `neonFrame`, `neonGlow`, `splash`, `glitchText`, `marquee`, `fx`, `baseline`.
- **Embedding & infra:** `embed` (`view`, `FORBIDDEN`), `util`, `keyboardNav`, `drag`, `fileDrag`, `uiIdle`, `i18n`.

`appShell` at a glance

```
ZGui.appShell(root, opts) -> { el, body, filterInput, openSettings,
openShortcuts, setPaletteItems }. Options read include brand{glyph,title,subtitle},
filterPlaceholder, onFilter, palette, settings, settingsExtra, onColorscheme,
onOpenLog/onOpenLogDir/onOpenDataDir, actions, version, hotkeys, contextMenu, splash.
Single-instance, guarded by data-zgui-shell-mounted.
```

Tokens and schemes

Core :root tokens (from `cyberpunk.css`): `--accent #ff2a6d`, `--accent-light #ff6b9d`, `--cyan #05d9e8`, `--magenta #d300c5`, `--green #39ff14`, `--red #ff073a`; backgrounds `--bg-primary #05050a`, `--bg-secondary #0a0a14`, `--bg-card #0d0d1a`, `--bg-hover #12122a`; text `--text #e0f0ff`, `--text-dim #7a8ba8`, `--text-muted #3d4f6a`; `--border #1a1a3e`; glass `--glass-blur: blur(16px) saturate(165%)`, `--glass-fill`, `--glass-sheen`. Fonts: Orbitron (display, 400–900) and Share Tech Mono (body/terminal), bundled `@font-face`, `font-display: block`.

Built-in schemes (`ZGui.colorscheme`, each dark + light): `cyberpunk`, `midnight`, `matrix`, `ember`, `arctic`, `crimson`, `toxic`, `vapor`. Applying one writes the 24 `SCHEME_VAR_KEYS`: `--accent`, `--accent-light`, `--accent-glow`, `--cyan`, `--cyan-glow`, `--cyan-dim`, `--magenta`, `--magenta-glow`, `--green`, `--green-bg`, `--yellow`, `--yellow-glow`, `--orange`, `--orange-bg`, `--red`, `--text`, `--text-dim`, `--text-muted`, `--bg-primary`, `--bg-secondary`, `--bg-card`, `--bg-hover`, `--border`, `--border-glow`. Fuzzy matches are wrapped `<mark class="fzf-hl">...</mark>`.

Appendix D – Build, Packaging, and Versioning

The build

Every app is a Tauri v2 app with a static or workspace `frontend/`. `tauri.conf.json` sets `frontendDist` to that directory and a `beforeBuildCommand/beforeDevCommand` that vendors the shared web modules before packaging — e.g. `zgo` runs `node scripts/build-hooks.mjs && node scripts/copy-webui.mjs`, where `copy-webui.mjs` `cpSyncs crates/<name>-core/frontend` into the served `frontend/` (recursive, dereferencing symlinks, filtering `.git`). The copies are gitignored and regenerated; the served frontend is never hand-edited (edit `crates/<name>-core/frontend` instead). The same web modules run on a second substrate, JUCE, via a transport shim (`window.__TAURI__` vs `window.Juce` + `window.__JUCE__` `.backend`).

The submodule layout

Shared code is pinned as git submodules. The Rust pieces — the `<name>-core` engine and the shared Rust crates `zpwr-embed-terminal`, `zpwr-il8n`, `zpwr-file-browser`, `zpwr-hooks-editor` — mount under `crates/`; the JavaScript toolkits `zgui-core` and `zpwr-clip-engine` mount under `frontend/lib/`. A leaner app (`zpdf`) pins only `crates/zpdf-core` and `crates/zpwr-embed-terminal`; a fuller one (`zreq`) pins the whole set. `crate-type = ["rlib", "staticlib", "cdylib"]` on every engine but `zpdf-core` (rlib only); editions are 2021 except `zoffice-core` (2024).

The registered command surface

An app's `src-tauri invoke_handler` registers: the engine's one `<name>-core::tauri_plugin::<prefix>_invoke`; the four `terminal_spawn/` `write/` `_resize/` `kill`; the `fs_*` family from `zpwr-file-browser` (`fs_list_dir`, `fs_get_info`, `fs_grep`, `fs_hash`, `fs_compress`, `fs_extract`, `fs_read_file_base64`, `fs_git_status`, `fb_watcher_set`, and ~30 more); and a few app-specific commands (`zgo`'s `zgo_hide/zgo_reveal/zgo_trash`; diagnostics `open_log_file/open_log_dir/open_data_dir`). `zcontainer` also registers `zcontainer_subscribe/_unsubscribe/_stream_send`.

The version table

app	app	core	skew
zgo	0.2.0	zgo-core 0.1.0	shell ahead
zpdf	0.2.0	zpdf-core 0.1.0	shell ahead
zcontainer	0.3.0	zcontainer-core 0.2.0	shell ahead
zterminal	0.29.7	ztmux-core 0.1.0	shell far ahead
zreq	0.1.0	zreq-core 0.1.0	lockstep
zcite	0.1.0	zcite-core 0.1.0	lockstep

app	app	core	skew
ztunnel	0.1.0	ztunnel-core 0.1.0	lockstep
zftp	0.1.0	zftp-core 0.1.0	lockstep
zemail	0.1.0	zemail-core 0.1.0	lockstep
zoffice	0.2.0	zoffice-core 0.2.0	lockstep
zphoto	0.2.0	zphoto-core 0.2.0	lockstep

Appendix E — Glossary

***-core** — a pure-Rust engine crate with no GUI dependencies, exposing a native Rust API, (usually) a C ABI, and a Tauri plugin over one `invoke` dispatch. (Chapters 1, 2.)

`invoke(cmd, args)` — the single string-dispatched entry point every engine exposes; the command names are the engine's public API. (Chapter 2, Appendix A.)

C ABI — the four-function `extern "C" { ... }` skin (`<prefix>_init`, `_invoke`, `_string_free`, `_set_event_callback`) in a core's `src/ffi.rs`, with a hand-written header in `include/`. Nine of ten cores have one; `zpdf-core` is the exception. (Chapters 1, 2.)

EngineEvent / EventSink — the engine's asynchronous callback channel; events cross the boundary as a C callback or a Tauri WebView event ("`zrq-event`", "`ztn-event`"). (Chapter 2.)

Tauri v2 — the WebView-plus-Rust desktop runtime every app is built on; the front end calls the backend by command name through `window.__TAURI__`. (Chapter 3.)

zgui-core / window.ZGui.* — the shared cyberpunk GUI toolkit (~240 components on one global), consumed as a submodule; the single source of truth for all chrome. (Chapter 3.)

ZGui.appShell — the single-instance shell mount that gives an app its header, palette, settings, terminal, and chrome in one call. (Chapter 3.)

ZGui.embed.view(fn) — the embed contract; for the duration of `fn` a **FORBIDDEN** set of global-seizing **ZGui** methods are swapped for throwing stubs, so an embedded core feeds the host shell instead of fighting it. (Chapter 3.)

ZGui.fzf — the shared fuzzy matcher (`fzfMatch`, `highlightMatch`); one matcher, one highlight style, every filter in every app. (Chapter 3.)

cyberpunk.css — the design-token file: `--accent #ff2a6d`, `--cyan #05d9e8`, the near-black backgrounds, Orbitron and Share Tech Mono, consumed by every app and themeable via 24 keys. (Chapter 3.)

Embed matrix — the `COMPONENTS.md` consumption table recording which app consumes which shared component/engine, backed by each app's `.gitmodules`. (Chapter 13.)

Document plane — the union of `zoffice-core`, `zemail-core`, `zpdf-core`, and `zphoto-core`; the shared capability to render and edit any common file type inside any app. (Chapter 11.)

Pointer cascade — the sequence of submodule-pointer bumps that propagates an engine change up through each consuming app to the meta-repository. (Chapters 2, 13, 15.)

LaunchKind — `ztunnel-core`'s enum recording each VPN backend's data-path provenance (`Native`, `ManagedProcess`, `OsNative`, `Coordinated`) — inherited-versus-owned encoded in the type. (Chapter 7.)

Durable-deps rule — the discipline of depending only on a few durable crates (`serde`, `thiserror`, `dirs`) plus load-bearing domain crates, hand-rolling small things (crypto in `zreq-core`) rather than importing them. (Chapters 1, 5.)

Command bridge — the thin Tauri IPC layer: one `<prefix>_invoke` command per engine that forwards `(cmd, args)` JSON to `Engine::invoke`, plus the shared `terminal_*` and `fs_*` commands and a few app-specific platform commands. (Chapter 4.)

Subscription trio — `zcontainer-core`'s second door: `subscribe`, `unsubscribe`, `stream_send` (and their `zcontainer_*` C equivalents), for the streaming operations (log follow, exec, port-forward) that do not fit request/response. Stream output is delivered through the ordinary event sink as `Stream/StreamEnd/StreamError`. (Chapters 4, 10.)

C-ABI spectrum — the observation that the C door's function count is fitted to the domain: four for the network/session engines, eight for `zcontainer-core` (streaming), three for `zphoto-core` (no events), two for `zoffice-core` (stateless, no events), zero for `zpdf-core`. The invariant is the JSON-in/heap-JSON-out contract, not a fixed count. (Chapter 2, Appendix B.)

Two substrates — the two native runtimes the shared web modules run on unchanged: Tauri (`window.__TAURI__`) and JUCE (`window.Juce + window.__JUCE__.backend`), bridged by a transport shim. "Divergence across substrates is a bug too." (Chapter 3.)

SCHEME_VAR_KEYS — the 24 CSS custom properties a colorscheme writes onto `:root` (or a scoped element); applying a scheme is writing these keys, which is why every component recolours at once. (Chapters 12, Appendix C.)

FORBIDDEN — the hard-coded list of global-seizing `ZGui` paths (`appShell`, `palette.bindHotkey`, the document-writing `colorscheme` methods, `crt`, `splash`, `baseline.*`, ...) that `ZGui.embed.view` swaps for throwing stubs so an embedded core cannot corrupt its host. (Chapter 14.)

Plan-as-data — the recurring shape of computing an impure operation as an inspectable data structure the engine returns and the shell applies: `ztunnel`'s `PrivilegePlan/FirewallPlan`, `zftp`'s `rename_plan`, `zoffice`'s `DiffOp`. (Chapters 7, 9, 11.)

Mountable webui — the view a `-core` ships in its own `webui/` directory, copied into the app's served `frontend/` at build time, mounted into an element rather than a document so the same view serves the standalone app and every embedding host. (Chapters 5, 24.)

Transport auto-detection — the mountable view's load-time choice of how to reach its engine: Tauri's `invoke` when `window.__TAURI__` is present, the JUCE bridge on the other substrate, an in-memory mock in a plain browser. The one seam that makes two substrates and a backend-free preview the same decision. (Chapter 24.)

Feature matrix — the enumerated list of a predecessor's behaviours, each scored full, partial, or missing, with `full` reserved for a behaviour *and* GUI match; the artifact behind every coverage percentage in the book. (Chapters 19, 25.)

Critical section — the span of an `invoke`, during which the process-global engine mutex is held and every other command on that engine is serialized behind it; the reason long operations narrate through the sink or open a subscription rather than blocking in `invoke`. (Chapter 22.)

Sink reentrancy rule — the fourth clause of the boundary contract: a host's event callback may record, enqueue, or signal, but may not call `<prefix>_invoke` synchronously from inside the callback, because the engine may still hold its own lock. (Chapter 22.)

Base-field system — `zcite-core`'s schema layer that renames shared fields per reference type and maps them to CSL variables (`base_field_name`, `base_field_to_csl`), the bridge that lets one formatter handle all 37 types. (Chapter 6.)

Appendix F — A Source Map

Each engine is a crate whose modules are its structure. This is the module list of each `-core`, the fastest orientation to where a capability lives. Every crate also carries `ffi` (except `zpdf-core`), `error`, and a `tauri_plugin` behind the `tauri` feature; those scaffolding modules are omitted from the per-crate lists below except where notable.

- `zreq-core` — `auth`, `codegen`, `crypto`, `history`, `import`, `model`, `runner`, `script`, `settings`, `store`, `vars`, `http`, plus `grpc` and `ws` behind their features.
- `zcite-core` — `bib`, `csl`, `duplicates`, `export`, `identifier`, `import`, `model`, `pdf`, `schema`, `search`, `store`, `webdav`, `xml_formats`, `zotero`.
- `ztunnel-core` — `config`, `creds`, `killswitch`, `log`, `manager`, `model`, `netmonitor`, `openvpn`, `platform` (`linux`, `macos`), `privileged`, `servers`, `stats`, `store`, `supervisor`, `wireguard`, `wireguard_tunnel`.
- `zgo-core` — `actions`, `appindex`, `bookmarks`, `calc`, `clipboard`, `conditional`, `contacts`, `dictionary`, `dispatch`, `executor`, `fallback`, `filesearch`, `frecency`, `import`, `keystroke`, `matcher`, `model`, `music`, `onpassword`, `queryhistory`, `runningapps`, `scriptfilter`, `snippet`, `spotlight`, `store`, `syscommands`, `utilities`, `vars`, `websearch`.
- `zftp-core` — `bookmark`, `creds`, `discovery`, `encoding`, `log`, `manager`, `model`, `protocol`, `stats`, `store`, `transfer`, `transport` (`local`, `remote`, `scp`).
- `zcontainer-core` — `model`, `daemon`, `daemon_linux`, `daemon_windows`, `docker`, `kube`, `vm`.
- `zoffice-core` — `base`, `calc`, `crypto`, `crypto_odf`, `draw`, `dsig`, `format`, `impress`, `math`, `meta`, `ole2`, `pdf`, `render`, `writer` (plus the inline `DiffOp/diff` in `lib.rs`).
- `zemail-core` — `addressbook`, `calendar`, `carddav`, `filters`, `gloda`, `junk`, `mailapps`, `mime`, `model`, `net`, `openpgp`, `search`, `smime`, `store`.
- `zpdf-core` — `actions`, `annot`, `barcode`, `convert`, `doc`, `font`, `form`, `js`, `layout`, `linearize`, `media`, `ocr`, `page`, `render`, `security`, `sign`, `text` (`dispatch` lives in `tauri_plugin`; no `ffi`).
- `zphoto-core` — `b64`, `codec`, `fill`, `filter`, `model`, `ops`, `paint`, `text`, `vector`.

Appendix G – The zpdf Operation Catalogue

`zpdf-core` routes roughly three hundred operations through the `zpdf_invoke` match, a flat list rather than a dotted namespace. Grouped by area, a representative catalogue:

- **Documents & pages:** `open_pdf`, `save_pdf`, `pdf_info`, `insert_pages`, `insert_blank_page`, `delete_pages`, `duplicate_pages`, `replace_pages`, `reorder_pages`, `reverse_pages`, `rotate_page`, `flip_page`, `crop_page`, `resize_page`, `n_up`, `interleave`, `merge_file`, `overlay_page`, `to_single_page`, `split_by_bookmarks/count/size`, `split_odd_even`, `split_page_grid`, `repair`.
- **Text:** `extract_text`, `extract_text_in_region`, `page_text_runs`, `edit_text`, `move_text`, `restyle_text`, `find_replace_text`, `reflow`, `add_text`, `add_text_with_font`.
- **Annotations & markup:** `add_note`, `add_ink`, `add_callout`, `add_markup`, `highlight`, `underline`, `strikeout`, `squiggly`, `caret`, `freetext`, `line`, `square`, `circle`, `polygon`, `polyline`, `stamp`, `stamp_image`, `whiteout`, `list_annotations`, `flatten_annotations`, `set_annotation_color/contents/rect/ flags/opacity/author`.
- **Redaction & security:** `redact`, `redact_search`, `apply_redactions`, `encrypt`, `encrypt_aes256`, `encrypt_aes256_with_permissions`, `encrypt_pubkey`, `encrypt_pubkey_multi`, `decrypt`, `decrypt_pubkey`, `is_encrypted`, `permissions`, `sanitize`.
- **Signatures:** `sign`, `sign_image`, `sign_visible`, `add_typed_signature`, `certify`, `verify_signatures`, `verify_full`, `signature_count`, `is_signed`, `clear_signature`.
- **Forms:** `create_text_field/checkbox/radio_group/choice_field/push_button/signature_field`, `set_field`, `set_field_flags/appearance/calculation_js`, `form_fields`, `flatten_form`, `reset_form`, `recalculate_fields`, `validate_fields`, `has_acroform`, `has_xfa`, `xfa_datasets`.
- **Images:** `add_image`, `replace_image`, `extract_images`, `adjust_image`, `downsample_images`, `to_grayscale`, `invert_colors`, `create_from_images`, `detect_image_regions`, `watermark_image`, `split_scanned_images`.
- **Layers, links & navigation:** `create_layer`, `delete_layer`, `rename_layer`, `set_layer_visibility/usage`, `flatten_layers`, `add_link`, `add_goto_link`, `add_remote_goto_link`, `add_launch_link`, `links`, `auto_link_urls`, `add_bookmark`, `set_outline`, `named_destinations`, `set_page_mode/layout`.
- **OCR, accessibility & conversion:** `ocr_page`, `ocr_page_words`, `make_searchable`, `auto_tag`, `reading_order`, `accessibility_check`, `set_document_language`, `image_alt_texts`, `export_docx/pptx/xlsx/html/markdown`, `export_page_svg`, `export_png`, `to_pdf_a`, `to_pdf_x`, `linearize`, `optimize`, `preflight`.
- **Barcodes, media & scripting:** `add_qr_code`, `add_data_matrix`, `add_barcode`, `add_rich_media`, `add_3d`, `add_movie`, `add_sound`, `run_javascript`, `set_doc_js`, `apply_actions`, `bates_number`, `add_page_numbers`, `add_header_footer`, `watermark_text`.

Appendix H – The EngineEvent Catalogue

Seven engines narrate asynchronously through an `EventSink`; three do not. Each `EngineEvent` enum is `#[serde(rename_all = "snake_case", tag = "type")]`, so a variant crosses the boundary as `{"type":"...", ...}` to a C callback or a WebView event.

- **zreq-core** — `RequestStarted { method, url }, RequestFinished { status, time_ms }, Error { tag, message }`.
- **zcite-core** — `LibraryChanged { items }, Imported { count }, LookupFinished { id }, Error { tag, message }`.
- **ztunnel-core** — `Status { config_id, status }, Stats { config_id, bytes_in, bytes_out }, Log { config_id, level, message }`.
- **zgo-core** — `ScriptFilterStarted { bundleid, query }, ScriptFilterFinished { count }, Error { tag, message }`.
- **zftp-core** — `Status { bookmark_id, status }, Stats { bookmark_id, bytes_in, bytes_out }, Log { bookmark_id, level, message }, Transfer { transfer_id, status, transferred_bytes, total_bytes }`.
- **zemail-core** — `SyncStarted { account_id }, SyncFinished { account_id, fetched }, MessageSent { recipients }, Error { tag, message }`.
- **zcontainer-core** — `Stream { id, data }, StreamEnd { id }, StreamError { id, tag, message }, Error { tag, message }`.
- **zoffice-core, zpdf-core, zphoto-core** — no `EngineEvent`; request/response only, because a document, a PDF, and an image are computed on request and have no asynchronous life to narrate.

Appendix I – Predecessors, Dependencies, and the Claim

Each app credits a predecessor for its workflow and a set of dependencies for its plumbing, and claims something original only after both credits are paid. The table:

- **zreq** — predecessor Postman. Plumbing: [request](#) (behind [net](#)) for the wire; crypto hand-rolled in [crypto.rs](#). Original: a faithful, local-first Postman that shares engines with the rest of the fleet; 92.2% measured coverage.
- **zcite** — predecessor Zotero. Plumbing: [lopdf](#) for PDF text, [quick-xml](#) for the XML formats; CSL processor hand-written. Original: 37 typed reference types, a CSL 1.0 engine, and a citation service other apps embed; 95.0% coverage.
- **ztunnel** — predecessor Tunnelblick. Plumbing: [boringtun](#) (WireGuard Noise), [keyring](#) (keychain). Original: a native WireGuard data path Tunnelblick never had, and a privilege/firewall planner as data.
- **zgo** — predecessor Alfred. Plumbing: durable-deps only ([serde](#), [dirs](#)). Original: the Alfred workflow graph, Script Filter, and a frequency memory rebuilt in Rust, embeddable behind one [invoke](#).
- **zftp** — predecessor Cyberduck. Plumbing: [opendal](#) (the cloud/transfer matrix), [russh/russh-sftp](#) (SFTP/SCP), [encoding_rs](#) (filename charsets). Original: one transport trait over sixteen protocols, [.duck](#) and FileZilla import.
- **zcontainer** — predecessors Docker Desktop and Lens. Plumbing: [bollard](#) (Docker), [kube/k8s-openapi](#) (Kubernetes), [objc2-virtualization](#) (the macOS guest). Original: the first compiled-native desktop GUI for *both* Docker and Kubernetes.
- **zoffice** — predecessor Microsoft Office (LibreOffice-shaped scope). Plumbing: [zip](#) + [quick-xml](#) (OOXML/ODF), RustCrypto (encrypted packages). Original: the formats parsed and written directly, no office suite embedded.
- **zemail** — predecessor a Thunderbird-shaped desktop client. Plumbing: [imap](#), [lettre](#), [mail-parser](#), [pgp](#). Original: an owned, local-first mail client that previews the other three document engines' formats.
- **zpdf** — predecessors Adobe Acrobat Pro and macOS Preview. Plumbing: [lopdf](#) (object model), [boa_engine](#) (PDF JavaScript), [ttf-parser/subsetter](#) (fonts), [tiny-skia](#) (rasterizer). Original: ~300 operations over the raw object model, the fleet's most-embedded engine.
- **zphoto** — predecessors GIMP/Photoshop, with an Illustrator-style vector mode. Plumbing: [image](#) (codecs), [ab_glyph](#) (text). Original: a non-destructive raster editor with recorded actions and a vector stack that rasterizes into the same document.

Appendix J — The Feature-Gate Map

Every engine's platform reach is gated so a `--no-default-features` build is a pure, network-free, GUI-free library. The gates, per crate:

- **zreq-core** — default `["net"]`. `net` → `reqwest`; `tauri` → `tauri`; `ws` → `tungstenite`; `grpc` → `proton`, `prost`, `prost-types`, `prost-reflect` (implies `net`).
- **zcite-core** — default `["net", "pdf"]`. `net` → `reqwest`; `pdf` → `lopdf`; `tauri` → `tauri`. (`quick-xml` is mandatory.)
- **ztunnel-core** — default `["net"]`. `net` is a marker; `wireguard` → `borington` (+ `borington/device`), `libc` (implies `net`); `keychain` → `keyring` (v4); `tauri` → `tauri`.
- **zgo-core** — default `["exec"]`. `exec` is a marker (gates process spawning); `tauri` → `tauri`. No network deps at all.
- **zftp-core** — default `["net", "discovery", "keychain"]`. `net` → `opendal`, `tokio`, `russh`, `russh-sftp`, `encoding_rs`; `discovery` → `mdns-sd`; `keychain` → `keyring` (v3); `tauri` → `tauri`.
- **zcontainer-core** — default `["docker", "kube"]`. `docker` → `bollard`, `reqwest`; `kube` → `kube`, `k8s-openapi`, `base64`, `flate2`, `serde_yaml`; `vm` → the macOS `objc2/objc2-virtualization/block2/dispatch2` stack; `tauri` → `tauri`.
- **zoffice-core** — *no [features] table*; every dependency is mandatory (`zip`, `quick-xml`, `flate2`, the RustCrypto stack). No `tauri` gate in the core.
- **zemail-core** — default `["net"]`. `net` → `imap`, `native-tls`, `lettre`; `openpgp` → `pgp`, `rand`; `carddav` → `ureq`, `quick-xml`, `http`; `smime` → a pinned pre-release RustCrypto CMS stack; `tauri` → `tauri`. (`mail-parser` mandatory.)
- **zpdf-core** — only `tauri` → `tauri`; every other dependency is mandatory. No C ABI; `crate-type` defaults to `rlib`.
- **zphoto-core** — default `["codecs", "text"]`. `codecs` → `image`; `text` → `ab_glyph`; `tauri` → `tauri`, `tokio`.

Appendix K – The Honesty Ledger

The book marks what is provisional rather than rounding it up. Collected in one place, the honesty notes are:

- **zpdf-core has no C ABI.** No `ffi.rs`, no `include/`, `rlib`-only — because every consumer is a Rust/Tauri app that links it natively. The four-function skin gets added when a non-Rust host needs it. (Chapters 2, 11; Appendix B.)
- **The C-ABI is a spectrum.** Four functions is the norm, not an invariant: eight for `zcontainer-core`, three for `zphoto-core`, two for `zoffice-core`, zero for `zpdf-core`. (Chapter 2; Appendix B.)
- **Coverage numbers are conservative and named.** `zreq` scores 92.2% over 83 Postman behaviours (72 full, 9 partial, 2 missing: a full JS test sandbox, deliberately out of scope, and per-host client TLS certs); `zcite` scores 95.0% over 60 features (54 full, 6 partial, 0 missing). (Chapters 2, 5, 6.)
- **Network backends are phased and say so.** `zftp` reports `needs_transport` for an unwired backend; `ztunnel` reports `needs_privilege` when root is not held. Neither fakes a live connection. (Chapters 7, 9.)
- **The version skew is partial.** Four apps run ahead of their cores (`zgo`, `zpdf`, `zcontainer`, `zterminal`); many ship in lockstep. The skew is the mechanism working, not a rule that numbers must diverge. (Chapter 15; Appendix D.)
- **A crate's docs can lag its code.** `ztunnel_core.h`'s command-name comment trails the `Engine::invoke` match; the functions work, the comment is behind. (Chapter 7.)
- **zoffice's coverage is partial-by-design.** The formats are parsed directly rather than via an embedded suite, and the scope is honestly bounded. (Chapter 11.)
- **zcontainer's daemon spans three OSes at different maturities**, and the Kubernetes and Docker halves gate apart so a consumer pays for one client, not both. (Chapters 10, 11.)
- **The `zgo matcher` re-implements the shape of Alfred's ranking, not its private constants** — a distinction the source comments make and the book keeps. (Chapter 8.)

Appendix L – The Failure and Persistence Surfaces

Errors and persistence are the two cross-cutting subsystems of Chapters 16 and 17, and like the command surface they are worth collecting in one place as a reference map.

The error shape, by door

- **Native** — `Engine::invoke(cmd, args) -> Result<Value, <Name>Error>`, where `<Name>Error` is a `thiserror`-derived enum in the crate's `error` module, one variant per failure kind, with `#[from]` lifts for the lower-level errors (`serde_json`, `io`, and the feature-gated `request`, `imap`, `bollard`, and so on).
- **C ABI** — `<prefix>_invoke` always returns a heap JSON string; failure is *in* the JSON (an error object with the variant tag and message), never a null or an error code. The string is still owned by the caller and still freed with `<prefix>_string_free`.
- **Tauri** — `<prefix>_invoke` resolves to a JSON value; failure is the same in-band error object the C door returns, read by the same front-end code.
- **Asynchronous** — late failures ride the event sink as the `Error { tag, message }` variant (five of the seven event-bearing engines — `ztunnel-core` and `zftp-core` route late failures through `Log` instead), and streaming failures as `zcontainer-core`'s `StreamError { id, tag, message }`, keyed by subscription id.

The typed non-errors

Not every “cannot” is an error; some are honest data returned by a successful call:

- **zftp-core** — `needs_transport`: the bookmark is resolvable and the transfer is planned, but the transport backend for its protocol is not wired.
- **ztunnel-core** — `needs_privilege`: the command line and firewall plan are built, but root is not held, so the shell's privileged helper must complete the effect.
- **Plan-as-data** — `PrivilegePlan/FirewallPlan` (`ztunnel`), `rename_plan` (`zftp`), `DiffOp` (`zoffice`): an impure operation returned as an inspectable structure the shell applies, rather than performed inside the engine.

The persistence surface

- **store module** — present in `zreq-core`, `zcite-core`, `ztunnel-core`, `zgo-core`, `zftp-core`, `zemail-core` (Appendix F); owns the model's load/save round-trip over `serde` + `dirs`, pure enough to test against a temp directory.
- **Location** — `~/Library/Application Support/com.menketotechnologies.<app>` via Tauri's `app_data_dir()` (bundle-id keyed), the same directory for an app and every core it embeds; supplied to the engine by the shell, replaced by a temp dir under test.

- **Substrate** — JSON on disk by default (`serde_json`, already mandatory), whole-file load-and-save; the growing/queryable stores (`zemail`, `zcite`'s index/search) carry heavier structure where JSON-whole-file reaches its edge.
- **Credentials** — *not* in the store: `ztunnel-core`'s and `zftp-core`'s `creds` modules route secrets to the OS keychain behind the `keychain` feature (`keyring`), and the store persists only a reference to which secret belongs to which config/bookmark.
- **Migration** — no framework: `#[serde(default)]`, `Option<T>`, and defaulted collections absorb older shapes on load; larger shape changes are read-old/write-new in ordinary Rust in the `store` module.

Appendix M — The Shared Substrate Commands

The terminal and file browser of Chapter 18 are shared Rust crates whose commands every consuming app registers in its `src-tauri invoke_handler`. Collected here as the registration reference that complements Appendix D's per-app view.

- `zpwr-embed-terminal` (four commands, one PTY): `terminal_spawn`, `terminal_write`, `terminal_resize`, `terminal_kill`. Output streams back to the WebView as events the `zgui-core` terminal widget renders. The PTY is a long-lived, platform-specific, streaming resource — shell-side by the Chapter 1 split, never in a `-core`.
- `zpwr-file-browser` (the `fs_*` family, ~30 commands): `fs_list_dir`, `fs_get_info`, `fs_grep`, `fs_hash`, `fs_compress`, `fs_extract`, `fs_read_file_base64`, `fs_git_status`, `fb_watcher_set` (live-watch), and the rest — a full file manager's operation set, shared because browsing files is a desktop concern, not any one app's domain.
- **The standard registration block** — the “block of shared registrations” of Chapter 4: the engine's one `<prefix>_invoke`; the four `terminal_*`; the `fs_*` family; plus the app-specific platform commands that are the app's identity in the delta — `zgo_hide/zgo_reveal/zgo_trash` (`zgo`), the `subscribe/unsubscribe/stream_send` trio (`zcontainer`), `open_log_file/open_log_dir/open_data_dir` (diagnostics apps). Registering the block faithfully is how an app joins the shared frontend that calls those commands by name.

Appendix N — The Localization Surface

The shared i18n subsystem of Chapter 20, as a reference map alongside the command surface (Appendix A) and the component surface (Appendix C).

- **Catalogue** — `zpwr-i18n`, a shared submodule pinned into every localized app; per-locale JSON files (`i18n/<locale>.json`) keyed by namespaced, dotted keys, parsed by `serde` at runtime. No localization framework, no compiled catalogues — the same durable-deps, JSON-on-disk choice the stores make.
- **Namespacing** — shared chrome strings (the `zgui-core` shell, settings, shortcuts, common toasts) under shared namespaces, translated once for the whole fleet; app-specific strings under per-app namespaces. The namespace is what lets one file serve ten apps without content collision.
- **Lookup** — `ZGui.i18n` (Appendix C's infra surface): the front end's single lookup point in the active locale, with base-locale fallback so a partially-translated locale renders in its own language where it can and the base language where it cannot, never a raw key.
- **Runtime merge** — an app's effective catalogue is the shared `zpwr-i18n` strings as the floor with any app-specific strings layered atop; a store-backed app (the Audio-Haxor SQLite case) merges the shared JSON *underneath* its store so shared keys always resolve.
- **Concurrency** — concurrent additions to the same locale file are combined by a union-merge discipline rather than last-writer-wins, the i18n form of the pointer-cascade care: cheap to consume, careful to change.
- **Scope** — the catalogue localizes the *frame* (chrome and app UI strings), never the *data* (a document's contents, a citation's own language, log lines, filenames); and it is as complete as its locale files are filled — the mechanism makes fleet-wide localization cheap, and filling every locale is ongoing work the fleet marks rather than implies done.

Appendix O – The Boundary Contract in Full

Chapters 4, 16, 22, and 23 each specify part of what crosses between an engine and a host. Collected here as one contract, in the form a host implementor needs it — three clauses about mechanics, one about threading, and a note on cost.

The four clauses

1. **Strings are caller-owned.** Every `char*` returned by `<prefix>_invoke` or `<prefix>_init` is a heap-allocated, NUL-terminated UTF-8 buffer the caller must return to `<prefix>_string_free` — never `free()`, never left to the engine. On the Tauri and native doors the rule is invisible because `serde_json` owns and drops the values, but the discipline is identical: ownership crosses explicitly and the receiver releases.
2. **One process-global engine.** Each core builds its `Engine` behind a `OnceLock<Mutex<Engine>>`; `<prefix>_init` constructs the one instance and every `<prefix>_invoke` locks it. A host tracks no handles and cannot hold two divergent engines, and pays for that with the impossibility of two isolated instances in one process.
3. **JSON in, JSON out, failure in-band.** Arguments are a JSON object, results are JSON, and failure is *in* the result — an error object carrying the variant's tag and its message — never a null, an error code, or a separate channel. The same shape reaches a Rust caller as `Result<Value, Error>`, a C caller as a heap JSON string, and a JavaScript caller as a resolved value.
4. **Do not reenter from a callback.** A registered `EventCb`, or a Tauri sink handler, may record, enqueue, or signal; it may not call `<prefix>_invoke` synchronously from inside itself, because the engine may still hold the lock that call would need. Under Tauri the WebView round trip enforces this structurally; under the C ABI it is the host's obligation.

The asynchronous half

Late failures and progress ride the event sink rather than the return value: an operation that was dispatched and acknowledged narrates its completion or its `Error { tag, message }` through the same channel it would have narrated success through, keyed by whatever identity the domain gave it (a subscription id for `zcontainer-core`'s `Stream/StreamEnd/StreamError`, a transfer id for `zftp-core`'s `Transfer`). There is no third channel: a failure always arrives on the rail its success would have used.

Cancellation and phasing

There is no generic cancel at the boundary. Cancellation is modelled where it is meaningful, as a domain command over a domain entity — `transfer.cancel`, `unsubscribe`, `vpn.disconnect` — which mutates a status the slow work observes. Likewise, an operation that is computable but not yet performable is not an error: `needs_transport` and `needs_privilege` are successful results reporting exactly where the engine's half ended and the shell's begins.

The cost notes

- Small structured results are effectively free; the payload is data the `serde` derives produce anyway for the store and the tests.
- Bulk bytes are the expensive shape. Where they must cross as JSON they are base64, with the encoding named in the command (`fs_read_file_base64`); where a consumer is a Rust host, the native API returns the value directly rather than a serialized one — which is why the fleet's highest-bandwidth engine, `zpdf-core`, has no C door and needs none.
- Streams pay per emitted chunk, not per line: the granularity of `Stream` events is the granularity of the engine's reads, which is what lets the shared terminal widget consume an exec session without a bespoke fast path.

What the contract deliberately does not provide

Per-command permissioning (the bridge is one command per engine, so a framework-level permission decision is a decision about the whole engine); a retryability flag on errors (a host infers it from the variant); multi-instance engines; a binary framing for bulk results; and a generic cancellation token. Each absence is a trade the book argues in place — Chapters 21, 16, 4, 23, and 22 respectively — and each is recoverable later at a known cost, because in every case the thing that would have to change is the adapter, not the engine.

Appendix P – The Import and Export Surface

Chapter 27 reads the fleet's importers as one subsystem. This appendix is the reference behind it: which foreign formats each engine reads, which it writes, and which engines do neither. The commands are drawn from Appendix A's dispatch lists.

Reads

- **zreq-core** — `import.curl` (a shell command line), `import.postman` (the predecessor's own export), `import.openapi` (an API specification), `import.har` (a browser network capture).
- **zcite-core** — `import.bibtex`, `import.ris`, `import.csl_json`, `import.endnote_xml`, `import.mods`, `import.marxml`, `import.zotero_rdf`, plus `import.file` for the file-level path and `inbox.import` for the watched folder.
- **zemail-core** — `import.eml`, `import.maildir`, `import.mbox` for mail corpora; `contact.import_vcard` for contacts.
- **zftp-core** — `bookmark.import` for Cyberduck's `.duck` plist, sniffed and dispatched by `import_path`; `bookmark.import_filezilla` for FileZilla's XML.
- **zgo-core** — `workflow.import` for Alfred's `info.plist`; `snippet.import` for snippet collections.
- **ztunnel-core** — `config.import` for VPN configuration files, with `config.parse` and `config.wireguard_format` alongside it.
- **zcontainer-core**, **zoffice-core**, **zpdf-core**, **zphoto-core** — no format importer. For the three document engines the format is the domain and is opened natively; for **zcontainer-core** the state belongs to a daemon, a kubeconfig, and the user's repositories, so there is nothing to migrate.

Writes

- **zcite-core** — `export.bibtex`, `export.ris`, `export.csl_json`, `export.endnote_xml`, `export.mods`, `export.marxml`, `export.zotero_rdf`: the same format list as its importers, and the only engine whose `export.*` family mirrors its `import.*` family.
- **zreq-core** — `export.postman`, and nothing else.
- **zgo-core** — `workflow.export`, writing back the Alfred `info.plist` shape.
- **zemail-core** — `contact.export_vcard`; no corpus export.
- **zftp-core** — none; and **ztunnel-core** none, though `config.wireguard_format` renders a configuration back out.
- **The document engines** — export is a domain operation rather than a migration path: **zpdf-core**'s `export_docx/pptx/xlsx/html/markdown`, `export_page_svg`, `export_png`, `to_pdf_a`, `to_pdf_x`; **zoffice-core**'s `writer.pdf/html/markdown/text`, `calc.pdf/csv`, `impress.pdf`, `draw.svg`, `math.starmath`.

The two asymmetries

The first is direction: every engine that reads a foreign format reads more of them than it writes, except `zcite-core`, whose export list matches its import list because bibliographic export is a workflow feature rather than an exit path. The second is population: the engines with importers are the engines replacing products that hold user state, and the engine with neither importers nor exporters — `zcontainer-core` — is the one replacing products that hold none.

The exit path that is not a format

For an engine with no format export, the portable artifact is the store itself: a `serde_json` document under the app's data directory (Chapter 17), readable by anything that reads JSON, discoverable through `settings.path` where an engine exposes it and `open_data_dir` where a shell does. It is a weaker guarantee than an export command and a stronger one than nothing, and it is the honest answer to “how do I get my data out of an engine that has no exporter.”